Tor Vergata University of Rome

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

FACULTY OF ENGINEERING

COMPUTER SCIENCE, CONTROL AND GEOINFORMATION
CYCLE XXXVII

PhD Thesis

A UNIFIED SOFTWARE ARCHITECTURE FOR DISTRIBUTED AUTONOMOUS SYSTEMS

ADVISOR

Prof. Sergio Galeani

CANDIDATE

Roberto Masocco

COORDINATOR

Prof. Francesco Quaglia

A.Y. 2023/2024

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

COMPUTER SCIENCE, CONTROL AND GEOINFORMATION
CYCLE XXXVII

A UNIFIED SOFTWARE ARCHITECTURE FOR DISTRIBUTED AUTONOMOUS SYSTEMS

PHD THESIS

Advisor:
Prof. Sergio Galeani Signature:________________

Coordinator:
Prof. Francesco Quaglia Signature:________________

Candidate:
Roberto Masocco Signature:________________

A.Y. 2023/2024
Tor Vergata University of Rome, Faculty of Engineering
Via del Politecnico 1, Rome, Italy

Many times through a mistake
due to your own personality
or your own character
or interference that you get along the way
then you learn,
and the main thing is
to make sure you learn from your mistakes
and get better.

— Ayrton Senna

Acknowledgements

Some day, when this is finished, I will write something here…

… but it is not this day!

Rome, December 2024

The source code for this document is available at https://github.com/robmasocco/PhDThesis_RobertoMasocco (visited on 12/27/2024).

This work has been partially supported by the Italian Ministry of University and Research in the framework of the PRIN program (Progetti di Rilevante Interesse Nazionale), under the Grants no. 2017YKXYXJ and 2020RTWES4.

This work has been carried out within the framework of the EUROfusion Consortium, partially funded by the European Union via the Euratom Research and Training Programme (Grant Agreement no. 101052200 — EUROfusion).

The Swiss State Secretariat for Education, Research and Innovation (SERI) funded the Swiss contribution to this work.

Preface

This thesis is the summary of three years of research activity involving distributed systems programmed to perform specific, sophisticated tasks with different levels of autonomy and supervision. Since the basic subject is quite broad, the actual research work inevitably spanned multiple areas. During the three years spent at Tor Vergata University of Rome, my home institution, as well as during the six-months visit at the Swiss Plasma Center (SPC) of the École Polytechnique Fédérale de Lausanne (EPFL) in Switzerland, I had the opportunity to engage in both methodological and experimental research activities. Starting from addressing problems related to the design and implementation of control architectures for autonomous systems, as well as to the development of appropriate control laws, I then focused on the application of these methodologies in two scenarios: autonomous robotics and controlled nuclear fusion. However, two aspects allow one to see everything as a single journey. First, the leitmotif of the whole work is the intention to develop a common framework for control architectures involving distributed autonomous systems: once the problem and its basic requirements are contextualized, the solution is developed step by step, with the necessary tools and concepts being introduced as needed. Secondly, specific application scenarios are considered and new results are presented for each. Even if some might seem distant from the main topic, they are immediately reconciled considering that they are made possible by the presence of a distributed control architecture, which also adds to their own relevance. Thus, every part of this work follows the same structure: first, a problem is identified and analyzed, and then a solution is proposed and developed.

Contents

The presentation of the contents is organized to flow as much as possible from the general to the specific, giving all the necessary background on a topic before introducing new concepts, then practical scenarios, and finally the results obtained. Nonetheless, despite all the effort, in very few sections of the text it is inevitable to mention some concepts and items before they are properly introduced: each time this happens, the reader is referred to the appropriate section where a detailed description is provided. Furthermore, some of the solutions presented in this work, in particular those introduced in Chapter 4, are under active development. This book reflects their most up-to-date version at the time of writing, including some fixes and upgrades that are either ongoing or planned for the short term. Even if the possibility of drastic changes from what is hereby described is minimal, an occasional check at the proposed references might reveal that the actual implementations differ slightly from what is presented here: this is simply due to the ongoing design process and appropriate clarification will be given if requested. The full organization of the chapters is depicted in Figure 1.

Chapter 1 contextualizes the work and provides the first basic definitions of the most important concepts found throughout the whole book.

Chapter 2 defines the problem of interest in this work, introducing both basic requirements and practical scenarios that confirm their relevance. It also contains an overview of the related work, whose analysis results in a justification for the following developments.

Chapter 3 presents available technologies and tools that are of interest to this work, mainly because, if properly integrated and configured, they provide assistance in solving some parts of the main problem. Their description is detailed enough to allow the reader to understand their basic functioning as well as the features that make them valuable for this work.

The solution that this work proposes is presented in Chapter 4. The chapter offers a detailed description of its structure, implementation, and functionalities, leveraging on all the concepts introduced thus far.

At this point, the contents of the book follow two paths that stem from the framework developed previously, representing application scenarios for the proposed solutions and technologies. The first one is that of autonomous robotics, discussed in Chapter 5. The chapter presents the main adaptations and specific features of the proposed framework for this application scenario, as well as a detailed description of the first results that have been obtained.

The second scenario that has been considered is that of magnetic confinement nuclear fusion, introduced in Chapter 6. The chapter presents the experimental device that has been the subject of this part of the work together with its distributed control system. New results concerning the latter are also presented.

Working on the distributed control architecture of an experimental nuclear fusion device also highlighted possible improvements to its control system. Their introduction and motivation is the starting point of Chapter 7, which proceeds with the description of novel methodological results and concludes with their application to the development of a suitable solution. Experimental results are also presented and discussed towards the end of the chapter.

Finally, Chapter 8 summarizes the results obtained and discusses the possible future developments of the work presented in this book.

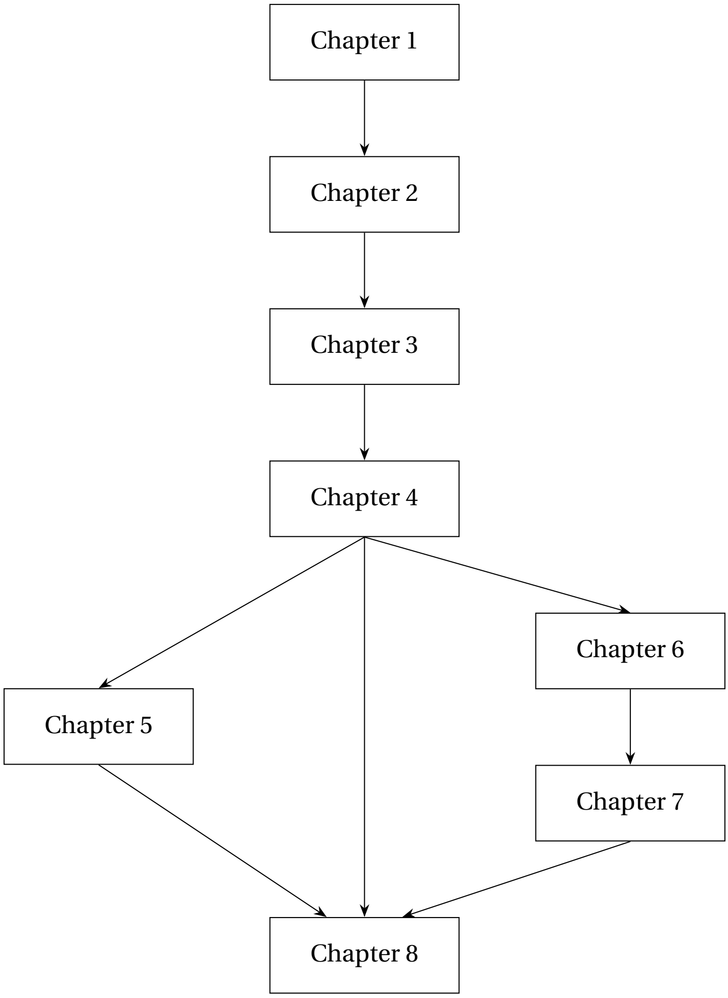

Figure 1: Organization of the chapters.

Published works

This book contains both new contributions and previously disseminated findings. The latter are divided into three categories: peer-reviewed papers (presented in ascending chronological order), posters, and open-source software (listed reflecting the order in which they are presented in the text). Proper references are provided to the reader where it is appropriate.

Peer-reviewed papers

- Galeani, S., Masocco, R., and Sassano, M. "Dynamic control allocation for sampled-data closed-loop systems". In: *Proc. of the IEEE 61st Conference on Decision and Control (CDC)*. IEEE, Dec. 2022, pp. 4206–4211

- Bianchi, L. et al. “A novel distributed architecture for unmanned aircraft systems based on Robot Operating System 2”. In: *IET Cyber-Systems and Robotics* 5.1 (2023), e12083

- Galeani, S. et al. “An optimal control perspective on the dynamic allocation problem for linear systems*”. In: *IFAC-PapersOnLine*. 22nd IFAC World Congress 56.2 (Jan. 2023), pp. 7492–7497

- Bianchi, L. et al. “Efficient visual sensor fusion for autonomous agents”. In: *Proc. of the 7th International Conference on Control, Automation and Diagnosis (ICCAD)*. IEEE, May 2023, pp. 01–06

- Oliva, F. et al. “A Trajectory Based Optimization Approach for Hybrid Observer Design”. In: *Proc. of the 62nd IEEE Conference on Decision and Control (CDC)*. Dec. 2023, pp. 1873–1878

- Masocco, R. et al. “A dynamic optimization approach to steady-state input allocation for nonlinear redundant systems”. In: *Proc. of the 10th International Conference on Control, Decision and Information Technologies (CoDIT)*. IEEE, July 2024, pp. 2833–2838

- Tenaglia, A. et al. “Dynamic steady-state coil current allocation for plasma shape control: a study on the TCV tokamak”. In: *Proc. of the 10th International Conference on Control, Decision and Information Technologies (CoDIT)*. IEEE, July 2024, pp. 278–283

- Tenaglia, A. et al. “A unified approach to plasma shape control with coil current allocation in magnetic confinement fusion”. In: *50th EPS Conference on Controlled Fusion and Plasma Physics*. Vol. P2.085. 2024

- Joffrin, E. et al. “Overview of the EUROfusion Tokamak Exploitation programme in support of ITER and DEMO”. in: *Nuclear Fusion* 64.11 (Aug. 2024), p. 112019

- Duval, B. P. et al. “Experimental research on the TCV tokamak”. In: *Nuclear Fusion* 64.11 (Oct. 2024). Publisher: IOP Publishing, p. 112023

Posters

- Masocco, R. et al. “A distributed real-time diagnostic and control network for the TCV tokamak based on the Data Distribution Service”. In: *33rd Symposium on Fusion Technology (SOFT 2024)*. Sept. 2024. DOI: 10.13140/RG.2.2.18403.16166

Open-source software

- Masocco, R. *dua-foundation*. URL: https://github.com/dotX-Automation/dua-foundation (visited on 11/16/2024)

- Masocco, R. *dotxautomation/dua-foundation - Docker Hub*. URL: https://hub.docker.com/r/dotxautomation/dua-foundation (visited on 11/20/2024)

- Masocco, R. *dua-template*. URL: https://github.com/dotX-Automation/dua-template (visited on 11/16/2024)
- Masocco, R. *dua-utils*. URL: https://github.com/dotX-Automation/dua-utils (visited on 11/16/2024)
- Masocco, R. *dua_app_management*. URL: https://github.com/dotX-Automation/dua_app_management (visited on 11/16/2024)
- Masocco, R. *dua_node*. URL: https://github.com/dotX-Automation/dua_node (visited on 11/16/2024)
- Masocco, R. *dua_qos*. URL: https://github.com/dotX-Automation/dua_qos (visited on 11/16/2024)
- Masocco, R. and Bianchi, L. *params_manager*. URL: https://github.com/dotX-Automation/params_manager (visited on 11/16/2024)
- Masocco, R. *simple_actionclient*. URL: https://github.com/dotX-Automation/simple_actionclient (visited on 11/16/2024)
- Masocco, R. *simple_serviceclient*. URL: https://github.com/dotX-Automation/simple_serviceclient (visited on 11/16/2024)
- Masocco, R. et al. *transitions_ros*. 2022. URL: https://github.com/dotX-Automation/transitions_ros (visited on 11/25/2024)
- Masocco, R. *zed_drivers*. URL: https://github.com/IntelligentSystemsLabUTV/zed_drivers (visited on 11/25/2024)
- Masocco, R. *flight_stack*. URL: https://github.com/IntelligentSystemsLabUTV/flight_stack (visited on 11/25/2024)

Notation

The following symbols, operators, mathematical items, and conventions are defined here and used throughout the text.

Sets

Given a set $\mathscr{U}$, $\text{int}(\mathscr{U})$ denotes its interior. The set $\mathbb{N}$ is defined as containing all the positive integer numbers, excluding 0, whereas $\mathbb{Z}$ denotes the set of all positive and negative integer numbers, including 0. The sets of real and complex numbers are denoted $\mathbb{R}$ and $\mathbb{C}$, respectively.

The imaginary unit is denoted j. Given a complex number $z \in \mathbb{C}$, $\mathrm{Re}(z)$ and $\mathrm{Im}(z)$ denote its real and imaginary parts, respectively. Consider the following sets:

$$\begin{aligned}
\mathbb{Z}_{\geq 0} &= \{k \in \mathbb{Z} : k \geq 0\},\\
\mathbb{R}_{\geq 0} &= \{a \in \mathbb{R} : a \geq 0\},\\
\mathbb{R}_{>0} &= \{a \in \mathbb{R} : a > 0\},\\
\mathbb{C}_{0} &= \{z \in \mathbb{C} : \mathrm{Re}(z) = 0\},\\
\mathbb{C}_{<0} &= \{z \in \mathbb{C} : \mathrm{Re}(z) < 0\},\\
\mathbb{C}_{\geq 0} &= \{z \in \mathbb{C} : \mathrm{Re}(z) \geq 0\},\\
\mathbb{C}_{<1} &= \{z \in \mathbb{C} : |z| < 1\}.
\end{aligned}$$

Moreover, $\mathscr{L}_2([a,b])$, $a, b \in \mathbb{R}$, denotes the set of all the functions that are square-integrable over the interval $[a,b]$.

Vectors and matrices

To define the following symbols and operators, given $n, m \in \mathbb{N}$, let

- $v \in \mathbb{R}^n$ be a vector,
- V be a vector space,
- $A, R, A_1, \dots, A_n \in \mathbb{R}^{n \times n}$ be square matrices,
- $B, B_1, B_2 \in \mathbb{R}^{n \times m}$ be rectangular matrices,
- $M \in \mathbb{R}^{n \times m}$ be a block-partitioned matrix.

The algebraic notation is summarized in Table 1.

Functions

Given a function $f(x)$, $f : \mathbb{R}^n \to \mathbb{R}$, ∇f denotes its gradient as a column vector, f_x denotes its gradient with respect to the variable x as a row vector, whereas $\frac{\partial f}{\partial x}$ denotes its jacobian. Given a set $\mathscr{X}$, $f(x)\big|_{x \in \mathscr{X}}$ denotes the restriction of $f(x)$ to the set $\mathscr{X}$.

For $T \in \mathbb{R}_{>0}$ and $t \in \mathbb{R}$, the notation $\mathrm{mod}(t, T)$ denotes the value of t modulo T.

$\|v\|_2$	Euclidean norm of v.
$\dim(V)$	Dimension of V.
e_i	i-th vector of the canonical basis of $\mathbb{R}^n$.
$\|v\|_R^2$	Product $v'Rv$.
$\mathrm{Im}(A)$	Image of A.
$\ker(A)$	Null space of A.
A'	Transpose of A.
$A^\dagger$	Moore-Penrose inverse of A.
$A^\perp$	Basis matrix for the null space of A.
$\mathrm{spec}(A)$	Spectrum of A.
$\mathrm{rank}(A)$	Rank of A.
$A \succ 0$	A is positive definite.
$A \odot B$	Hadamard, or entrywise, product of A and B.
$A \otimes B$	Kronecker product of A and B.
$\mathrm{diag}(A_1, \dots, A_n)$	Concatenation of $A_1, \dots, A_n$ on the diagonal.
$\mathrm{row}(A_1, \dots, A_n)$	Concatenation of $A_1, \dots, A_n$ on a row.
$\mathrm{vec}(B)$	Vectorization of B.
I_n	Identity matrix of order n (subscript is omitted when clear from context).
$0_{n \times m}$	Matrix of zeroes of dimensions n by m (subscript is omitted when clear from context).
$M_{[k,h]}$	$(k,h)-$th block of M.
$M_{[:,h]}$	$h-$th block column of M.
$M_{[:,1:h]}$	Sub-matrix formed by the first h block columns of M.

Table 1: Algebraic notation.

Dynamic systems

Let $k \in \mathbb{Z}_{\geq 0}$ be the discrete time variable. Let $t \in \mathbb{R}_{\geq 0}$ be the continuous time variable. Given a signal $v(t)$, v_k denotes its value at time $t = k\tau_s$, *i.e.*, $v_k = v(k\tau_s)$, where $\tau_s \in \mathbb{R}_{>0}$ is the *sampling time*. Consider the continuous-time dynamic system Σ defined by the equation

$$\dot{x}(t) = f(x(t)), \qquad t \in \mathbb{R}_{\geq 0},$$

where $x(t) \in \mathbb{R}^n$ is the state vector at time t and the function $f : \mathbb{R}^n \to \mathbb{R}^n$ is smooth. Let $x(0) = x_0$, $x_0 \in \mathcal{U}$. Then, for any $x_0 \in \mathcal{U}$, the solution of the system Σ is uniquely defined for $t \in [0, T)$, where $T = T(x_0)$, and it can be expressed by a function $x(t) = \Phi_t^f(x_0)$. The function $\Phi_t^f : \mathcal{U} \to \mathbb{R}^n$ is termed *flow* of f. Moreover, the system Σ is termed *complete* if and only if its solution exists for any $t \in \mathbb{R}_{\geq 0}$. Given a signal $v(t)$ generated by a dynamic system, the symbol $\tilde{(\cdot)}$ denotes the steady-state behavior of the signal, *i.e.*, $\tilde{v}(\cdot)$, provided that the latter exists. Also, when the clear from the context, the notation $v(\cdot)$ is replaced by v for the sake of simplicity.

Consider the continuous-time, linear time-invariant system

$$\begin{aligned} \dot{x}(t) &= Ax(t) + Bu(t), && t \in \mathbb{R}_{\geq 0}, \\ y(t) &= Cx(t) + Du(t), && t \in \mathbb{R}_{\geq 0}, \end{aligned}$$

where $x(t) \in \mathbb{R}^n$ is the state vector at time t, $u(t) \in \mathbb{R}^m$ is the input vector, $y(t)^p$ is the output vector, and $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times m}$, $C \in \mathbb{R}^{p\times n}$, $D \in \mathbb{R}^{p\times m}$ are constant matrices. For $h \in \mathbb{N}$, $O_h(C, A)$ denotes the *observability matrix of order h* of the pair (C, A), and $O(C, A)$ denotes the *observability matrix* of the pair (C, A), with $O(C, A) = O_n(C, A)$. Let s be a variable taking values in $\mathbb{C}$; the *transfer function* (or also, for multi-input multi-output (MIMO) systems, the *transfer matrix*) of the system from the input u to the output y is denoted

$$W_{yu}(s) = C(sI - A)^{-1}B + D.$$

Let $W(s)$ be a rational function of s; $\mathscr{Z}(W(s))$ and $\mathscr{P}(W(s))$ denote the set of zeroes and the set of poles of $W(s)$, respectively.

Contents

List of Figures

List of Tables

Listings

1 Introduction

In the history of humanity, many turning points have been marked by technological advancements. From the invention of the wheel to the steam engine, from the light bulb to the computer, technology has continuously changed how we live, work, and see the world. Its impact on society has always been profound and sometimes positive, sometimes negative. The changes it brought affected our daily habits, introducing new ways of doing things and sometimes even new ways of thinking. With the same spirit that drove our ancestors to develop *tools* made of stones and sticks to make their lives easier while harvesting food, we are now developing ever more powerful machines of all sorts to handle the challenges of the modern world, which can work either together with a human operator or in complete autonomy. Moreover, considering the recent developments of artificial intelligence, these tools have reached such a level of sophistication that they may soon reshape our thought processes.

The technological advancements that brought us these new solutions increased not only their capabilities but also their sophistication and, ultimately, their variety. An excellent example is the industrial manipulator robot: a machine that can perform various tasks, from simple pick-and-place operations to more sophisticated ones such as welding, painting, or even surgery. These machines have evolved from mechanical arms composed of multiple actuators, all connected to a single power and computational unit, to a *cyber-physical system*: a network of sensors and microcontrollers, grouped in subsystems all interconnected to a central unit that is, in turn, responsible for their coordination, supervision, and control, as well as for interfacing with human operators. In this case, the evolution has been driven by the need to increase the capabilities of the robot making it more versatile and adaptable to different tasks, as well as by the need to make it more autonomous and reliable, reducing the risk of accidents and downtime due to maintenance. Another suitable example is the modern airplane: considering even a small civilian aircraft originally equipped with an engine, control surfaces, and a few instruments, it has evolved into a complex system of systems composed of a multitude of sensors, probes, indicators, and avionics. In this scenario, the evolution has been driven by specific needs still related to safety and comfort that can ultimately be summarized in the need to ease the lives of both the passengers and the crew.

This design and development paradigm has recently become commonly denoted by the adjective *distributed*. The definitive characteristic of a distributed system is its decomposition into a network of subsystems, each being responsible for a specific task. All of them are interconnected and they all perform a piece of the job that the entire macroscopic system has been designed to do. This decomposition allows for more efficient use of the resources, better scalability, and higher reliability, as the failure of a single subsystem does not necessarily lead to the failure of the whole system. Moreover, the distribution of the subsystems allows for a better organization of the system's functionalities, as each subsystem can be designed and developed independently from the others, following a modular approach that eases the task of understanding, controlling, and maintaining the system as a whole. This approach has been widely adopted in many fields, from industrial automation to aerospace, automotive to robotics, telecommunications to energy. Another peculiar characteristic of a distributed system is the possibility, in many contexts, to deploy its parts in different locations: coming back to the manipulator example, it is common to have the sensors and the actuators distributed along the robot's arm and the control unit placed in a separate location. In the airplane example, the sensors and the avionics are distributed along the fuselage and the wings, while the cockpit is placed in the front of the aircraft. In pure software systems, the subsystems can be deployed in different servers, data centers, or even continents, with the additional advantage of partitioning the workload generated by the various regions among local computational nodes. These degrees of physical distribution allow great flexibility in the design but they also introduce new challenges in communication, synchronization, and fault tolerance.

All of this innovation comes at a price: when the multitude of subsystems, subcomponents, and submodules increases, new problems arise. The system's sophistication grows, and so does the difficulty of understanding, controlling, and maintaining it. The interactions between the different parts of the system become intricate. A deep understanding is needed, for which a good organization of the system *architecture* is of paramount importance. Moreover, a well-designed architecture may also be beneficial in two other stages: the design and development of the system and its maintenance. In the former, a clear and well-structured architecture naturally leads to a clear and well-structured design, following a top-down approach that starts from the system's requirements and ends with the definition of the subsystems and their interfaces. In the latter, a well-structured architecture allows for a clear identification of the subsystems and their interactions, easing diagnose and repair. Finally, a good architecture can also be a powerful resource for the evolution of the system as a whole, allowing for the addition of new functionalities or the replacement of old components without the need to redesign the whole system from scratch, reducing the time and the costs of such an operation.

The subject of this work is the development of a software architecture for distributed systems intended to control dynamic processes and autonomous agents. This setting is pretty broad: all of the above examples fall into these categories, representing topics of high interest in today's research and industrial environments. All the design and development problems listed above still apply to these cases, which also bring their own specific challenges addressed in the following. The main goal of this work is to provide a unified solution to these problems by defining a software architecture that can be used as a reference for the design and development of distributed systems in the contexts mentioned above. The proposed architecture is designed to be modular and flexible, and it is based on a set of standards and best practices that may guide the development of the subsystems and their interactions. Moreover, the architecture is designed to be adaptable to and deployable on different hardware platforms, to provide an abstraction layer for solutions ranging from embedded systems to general-purpose computers, and to easily integrate other software modules like third-party libraries. Finally, the architecture is built to be easily usable by developers, providing a unified set of tools and utilities that ease the task of developing, testing, and deploying new modules. The end goal of this work can be summarized as follows: building a standard development environment suitable for a single developer or an entire team, capable of replicating almost entirely the system configuration of a target platform on a development machine and enabling distributed applications to work and behave in the same way on all supported platforms, from build to run time. This way, the integration overhead is cut, allowing developers to focus on their most important task: research and development, streamlining the way to further technological advancements.

Just like our primordial ancestors faced the necessity of building tools to ease their lives, this work follows the same approach: identifying the problem, laying down the requirements, designing the solution, building it, and testing it. The requirements that drive the design of the architecture are identified and discussed in the following chapters, and the proposed solutions are based on the state of the art in the field of software engineering and distributed systems, as well as on some practical experience acquired during the years in which this work has been developed.

2 The importance of an architecture for autonomous systems

The development of an autonomous system is a process that involves various stages. Each stage is characterized by issues related to different aspects of the system ranging from the tasks it performs, to its construction and programming, and even to how the data it collects and that affect its operation should be presented to its operators. In general, most issues can be classified into a few high-level categories. These categories are not always addressed sequentially, although it may seem counter-intuitive from the following, nor are they entirely independent of each other. To address situations like these, various project management techniques have been developed in recent times, aimed at reducing the risk that an error made in an earlier phase compromises subsequent stages. In light of the observations made in Chapter 1 it is clear that, with the increasing sophistication of modern automated systems and of the tasks that they have to perform, these topics are more relevant than ever. In particular, new tools made available to developers and designers need to reflect these specificities, simplify the management of typical situations where possible, and highlight potential critical issues to aid in problem-solving. Practical experience shows that, despite the variety of cases and applications, the challenges faced are almost always, if not identical, very similar, as are the strategies to address them.

The aim of this chapter is to briefly present the issues typically encountered when designing an autonomous system, highlighting their unique characteristics without focusing on a particular application area, and then to examine some recent hardware and software solutions that improve the design, development, and testing activities. The discussion starts at a relatively high level of abstraction but becomes progressively more specific, forming the guiding thread that connects all subsequent chapters, finally highlighting the convenience of a well-structured software architecture to address the issues raised. The requirements that such an architecture should meet are then formalized and discussed in due detail.

Before moving further, it is instrumental to clarify the definition of an autonomous system in the context of this work. First of all, in this work the term *system* identifies a macroscopic entity, which can either be entirely hardware, or entirely software, or a mixture of both, that evolves in time and is characterized by a set of inputs and outputs. Its tasks are defined by the operations that it performs on the inputs to produce the desired outputs. The system can either be seen as a unitary entity or as a set of *subsystems*, *subcomponents*, and *submodules*, all terms that are used interchangeably in this work, each of which takes care of a part of the processing, thus contributing to a part of the tasks. For the system to work correctly, all the subsystems must be interconnected accordingly, both in terms of the data they share and of the resources they use. The term *autonomous* refers to the capability of the system to perform these tasks without human intervention, or with minimal human intervention, once it has been set up and started. This definition is intentionally broad and generic, as it is meant to encompass a wide range of applications and scenarios, *e.g.*, from mobile robots to drones, from power plants to industrial automation systems. Another class of systems that is included in this definition and that is of interest in this work is that of *dynamic systems*, which are systems whose state evolves in time according to a mathematical model that can be either deterministic or stochastic, and that can be either known or unknown. The evolution of the system's state, and thus that of its output, can be influenced by acting on its input, an operation that is commonly referred to as *controlling* the system or *applying a control law* to the system. In the context of autonomous systems this can be one of the tasks to perform, which is especially true for those systems that have a physical structure modeled as a dynamic system and that should evolve over time according to a specific desired behavior.

2.1 The design challenges of autonomous systems

2.1.1 Defining the tasks

The initial decisions in the design of an autonomous system concern the specifications of the operations that it performs. These specifications are fully defined from the onset down to the smallest detail possible at this stage, as they guide all subsequent choices, from hardware selection and construction to programming. During the breakdown of operational requirements into *tasks*, intended as discrete units of work to be executed, various types emerge. Without delving, for now, into the specifics of a particular scenario, common categories of tasks include the following ones.

- **Continuous tasks**, which begin at system startup and run uninterrupted until it is turned off or malfunctions.
- **Periodic tasks**, which are performed at regular intervals.
- **Asynchronous tasks**, which are executed only when a specific event occurs, which may or may not ever occur.

It is easy to see that most functionalities in modern autonomous systems can be broken down into tasks that fall within these categories. These include operations ranging from actuator control, to the collection of data and measurements required by control algorithms, to communication with operators and users. The system architecture should efficiently support the development of software modules and the integration of hardware solutions that can handle all these kinds of tasks.

2.1.2 Choosing the hardware

The choices made initially in this direction may not be definitive, because they concern both the construction of the initial prototypes and the final product: there likely are misjudgments or unforeseen issues to address. Nevertheless, the hardware should be selected carefully based on the operational, structural, and construction requirements defined so far. Decisions taken in the previous step are of critical importance at this stage: establishing the computational complexity of the operations that the system has to perform dictates the kind of resources required from the hardware, which may need to be equipped with discrete coprocessors to handle specific types of data or subdivided into multiple interconnected subsystems. The term *discrete* here refers to auxiliary hardware installed in a computer that cannot operate independently but is designed to be driven by the primary computer, providing specialized support for specific tasks.

A market survey is essential to assess whether pre-tested and ready-to-use components or SoCs (System-on-Chip) can be used rather than resorting to a proprietary solution, which generally entails significant development, testing, and certification costs. Another domain that adds its own level of complexity is that of lower-level hardware, which consists of:

- high-speed *microcontrollers*;
- actuators;
- sensors;
- interfaces and communication devices;
- data storage systems.

Nothing more can be said about these elements without reference to a specific case. However, practical scenarios discussed later in this work follow this design practice. From the multiplicity of devices mentioned and that may compose a modern autonomous system, it starts to become clear how the complexity of the system can grow quickly, as anticipated in Chapter 1. The need for a well-structured architecture becomes more evident.

2.1.3 Organizing the software

Except for those components that perform crucial and highly specific functions, such as electrical or electronic circuits, nearly all operations carried out by a modern autonomous system are encoded within an internal computer, which translates these instructions into commands for other components. Given the underlying complexity of the system itself, the software that controls it also has a layered and highly hierarchical organization. In this hierarchy, the lower levels are occupied by modules that interact with the smaller and faster subsystems (*e.g.*, the *firmware*) and these modules are developed with particular attention to their characteristics and purposes. Moving up in the hierarchy, more modules are found that encode sophisticated operations such as supervision, planning, and user communication, making them closer to the notion of *application* or *system software*.

When operating in this context, it is essential to rely on tools that simplify design and development as much as possible at all levels, assisting with the management of issues related to module organization, inter- and intra-process communication, data processing, and storage of various data types. The convenience of a good architecture to handle these challenges becomes increasingly evident.

2.1.4 Handling data

An autonomous system requires a nearly constant stream of information to complete its tasks. This information can be received in data packets managed by suitable communication interfaces, or it can be acquired directly through measurement devices commonly known as *sensors*. For both solutions, typical challenges that arise immediately include the integration of these devices with the rest of the apparatus and the processing of the data they produce. These issues are crucial, as the operation of the entire system is almost always contingent upon obtaining information about measurable quantities of interest and the system's own state (consider, *e.g.*, a feedback control algorithm).

It is therefore reasonable to expect that the hardware architecture of a modern autonomous system comprises numerous communication interfaces and sensors, the latter consisting of specific hardware designed to measure particular physical quantities of interest. The challenges mentioned above concern the integration of such hardware with the rest of the system and to the development of software modules capable of managing it, and of receiving and processing the data it generates.

2.1.5 Debugging and testing

As soon as the initial prototypes are finalized, testing should begin to verify functionality and confirm the validity of the design decisions that have been taken. Bearing in mind that a generic and universal testing procedure can not be defined, it can be expected that once hardware functionality is confirmed, attention shifts to software validation. Debugging software running on an autonomous system is in general more challenging than debugging common application software due to several specific factors, like the following ones.

- It is more difficult to trace program execution on an autonomous, potentially mobile device: direct access to the system may not be available, and the tools at hand are often limited or not very sophisticated.

- In the event of issues, both the software and hardware in use must be checked as the problem may be due to hardware faults or incorrect use.
- Conducting tests is not as simple as running a program and verifying its results: the safety of the apparatus and its operators must be ensured throughout the entire procedure, which can make even the organization of a test a challenging task.

Given the difficulty and importance of this phase, it is reasonable to employ tools and methodologies that can facilitate these tasks as much as possible.

2.2 Basic requirements

The challenges outlined in the previous section are not exhaustive, nor are they entirely independent of each other. They are, however, representative of the issues that typically arise when designing an autonomous system. Summarizing briefly the many aspects previously covered, suppose that the task at hand is that of developing an autonomous system capable of performing a series of operations. It is reasonable, in the context of this work, to focus on situations that meet the following assumptions.

- The system has a physical structure that can be modeled as a dynamic system, thus, one of the tasks consists in stabilizing and controlling this dynamic system, something that is commonly referred to in the following as *low-level control* or simply *control*.
- Digital programmable computers of different kinds are used to both develop and run all the algorithms, ranging from low-level control to task planning and supervision.

Note that the situations hereby described might span from mobile robots to drones and even power plants, all classes of systems that have been subject of this research work. Recalling Section 2.1, a series of choices regarding the following items have to be made:

- sensors and actuators to use;
- software tools and packages to employ and integrate;
- hardware platforms, *i.e.*, onboard electronics to adopt.

Note how these items might be strongly interconnected. Low-level hardware has to be chosen according to the control objective for the underlying dynamic system, while high-level hardware is selected usually weighing the software that it runs against the physical capabilities, as well as the costs, of the system that is going to host it. Finally, software tools and packages can be divided in two groups: that which has to be developed from scratch, possibly very specific to the system at hand, and that which can be reused from existing libraries or frameworks. The latter category, for example, includes solutions for data processing, communication, and visualization, as well as remote access and monitoring tools; when one wants to develop the prototype of an autonomous system, especially for research purposes, these things may be overlooked but they are crucial for the system to be usable and maintainable and the overhead that they might introduce can easily become significant if not properly managed early on.

There is a principle in computer science [207], dating back to the very early days of the discipline, stating that to do a proper job *no problem should ever be solved twice* or, in a historical fashion, *one must never reinvent the wheel* with the only exception of building a better wheel. One can see how this fits perfectly in the context of the development of autonomous systems: there is a wide range of complex foundational and integration problems to solve and present themselves each time a new system is developed. The scope of this work is to provide a solution that is *reusable*, *modular*, and *maintainable* and that can be used as a foundation for the development of innovative prototypes and products. With this in mind, the following requirements are formally identified.

(M) Modularity The architecture should constitute both a development and deployment framework for solutions ranging from single modules, meant to be easily integrated and reused, to entire autonomous systems prototypes. To guarantee this, the structure of the framework should suit both these use cases: single modules should carry with them all their dependencies, and it should be possible to easily *merge* single modules to build a *superproject* out of many *subprojects*.

(D) Development The architecture should support the development of single modules both as standalone packages and as parts of the software suite of a prototype. While working on a superproject that integrates a module, it should be possible to modify and update the module easily propagating the updates to every other superproject which integrates that module, unless the superproject has been specifically configured to use a different version of the module. The architecture should also ease testing and debugging activities, independently of the environment in which a module is developed.

(A) Abstraction The architecture should offer a *system abstraction layer*, *i.e.*, it should make it possible to carry on all the development activities without having to care too much about the underlying hardware on which a module or prototype software suite is deployed, apart from unavoidable specific differences, as well as to the target OS of such a platform. The architecture should ease deployment by offering a common framework across multiple different hardware platforms. In the end, the architecture shall offer very similar, if not equal, development and execution environments across different hardware platforms, requiring minimal effort to be replicated.

(O) Overhead The architecture should be as lightweight as possible, in terms of both computational and storage resources. This is to ensure that the overhead introduced by the framework itself is minimal, and that the resources of the hardware platforms on which the framework is deployed can be used to their full potential for the tasks that the system is meant to perform.

Note that these requirements, if examined by the perspective of subjects such as systems engineering, might not appear well-posed due to, *e.g.*, the lack of a clear and objective way to evaluate them. They are, instead, meant to guide the design of an architecture and not that of a specific system. Moreover, the design stage at which they are introduced is one in which many things still have to be evaluated such as, *e.g.*, hardware platforms to support and preexisting software tools to integrate, and thus the final definition of the architecture is the result of many trade-offs. They are high-level requirements meant to guide the design process, characterizing the solution from a broad and functional point of view.

2.3 Related work

This section presents some of the most relevant solutions that have been proposed in recent years to address the issues outlined in Section 2.1. The list, although short, is exhaustive: to the best of the author's knowledge at the time of writing no other such framework has been developed and proposed in the literature. Furthermore, a review of the works presented hereafter reveals that none of them fully meets the requirements identified in Section 2.2, especially those that deal with the independence from the underlying hardware and the construction of a replicable development environment. Commercial and proprietary solutions have been intentionally left out of this review: there exist commercial solutions that satisfy many requirements up to a certain point such as, *e.g.*, MathWorks's MATLAB [157] and Simulink [158]. Such solutions, however, limit the capabilities of the user to what their vendors decide to provide, also imposing a mandatory organization to the user works. As the following chapters clarify, among the aims of this work there is also that of providing to users a platform that follows its own set of standards and practices but at the same time does not enforce them in any way and does not require any proprietary tool, allowing instead the user to modify and expand any of its parts into what best suits their needs. Finally, to discuss the following, it is inevitable to mention solutions and tools which are the subject of subsequent chapters: they are presented in due detail in the following while proper references are provided here. At the end of this section, a brief comparison of the solutions presented hereafter is provided in Table 2.1 with respect to the requirements listed in Section 2.2.

2.3.1 RoboStack

The RoboStack framework [81], published in 2022 and maintained ever since, constitutes a first promising solution to the challenges outlined in Section 2.1. RoboStack is a collection of software packages built on the Conda package management system [10], which has become the de-facto standard in both industry and academia for the management of Python *environments*. This concept has gained particular interest in the scientific community in recent years thanks to the popularity of Python as a programming language in contexts like big data processing, statistics, machine learning, and artificial intelligence. The Conda package manager allows the user to specify a set of software modules to be collected and installed in an isolated location, together with their specific versions in case such a level of granularity becomes necessary. Once such a configuration is specified, it can be replicated with ease on any other machine and filesystem path, regardless of the operating system and many other factors, thus providing a way to ensure that the software environment in which a program is developed and runs is always the same. RoboStack builds on this concept, providing a wide set of software packages and libraries that are commonly used in the development of robotic systems, such as the Robot Operating System (ROS) and the Gazebo simulator illustrated in Section 3.1.3. The framework is designed to be used in conjunction with the Conda package manager, thus supporting a variety of operating systems ranging from Linux to Windows and macOS and it is meant to be installed on machines that are used for the development of robotic systems, deploying environments through the selection of specific sets of packages to install in each. This solution, although promising and already widely applicable, does not completely meet the requirements previously identified for three reasons. First, it is entirely dependent on Conda which, although a very powerful tool, may not be the best choice in terms of *isolation* of the software environment: while Conda allows one to install software packages in an isolated location, it does not provide a way to define a specific portion of system resources to offer to that software, a feature nowadays informally called *containerization* and illustrated later on. Secondly, every new piece of software must first be added to RoboStack, which means that the framework is not flexible because it is not possible to install any software package available from any source. The addition process requires a variable amount of work, particularly due to the many dependencies a software module may have, as explained in [81]. Lastly, RoboStack is dependent on Jupyter for development, which is not the best choice in terms of development environments. While Jupyter is a very powerful tool, its main purpose remains its original one: offering a way to write and run Python code multiple times in interactive notebooks displayed in a web browser, which is not really adherent to the scenario outlined at the beginning of this chapter which involves developing and testing in real time the software that runs on an autonomous system. Moreover, many plugins existing for other visualizers and IDEs, some of which are presented in the following sections, have to be reimplemented to be used in a Jupyter environment which is not always possible or convenient. These limitations, although not critical, make RoboStack not the best choice for the development of autonomous systems, especially when the requirements identified in Section 2.2 are taken into account.

2.3.2 eProsima Vulcanexus

The Vulcanexus software suite [76] developed by eProsima, originally released in early 2023, poses itself as a simple framework for the development of autonomous robotic systems. The suite is composed of a set of software packages that are meant to be installed either on a host operating system or inside a *container* managed by, *e.g.*, the Docker Engine presented in Section 3.2.1. The suite is fairly lightweight but nonetheless comprehensive: it provides the Robot Operating System 2 (ROS 2) middleware, together with the Fast DDS by eProsima [72] implementation of the Data Distribution Service (DDS) communication middleware presented in Section 3.1.1, plus some more software tools aimed at specific scenarios like, *e.g.*, that of microcontrollers. Moreover, the fact that it is shipped also as a Docker image compatible with different hardware platforms adds some flexibility to it in terms of isolation of both the development and execution environments with respect to, *e.g.*, the RoboStack framework. However, the major limitation of this framework lies in the fact that its features more or less end here. With respect to the requirements listed in Section 2.2, no effort is made towards the construction of a modular development and deployment environment, nor towards the support to a wide range of hardware platforms, especially in the case of SoCs aimed at robotic systems which, as explained later on, are gaining an increasing interest by the scientific community but require a fair amount of effort to be correctly set up. Furthermore, the restriction to the Fast DDS communication middleware for ROS 2, although evidently motivated by the fact that it is also developed by eProsima, hinders the flexibility of the framework in terms of communication protocols, an aspect that becomes critical in some operational scenarios where, *e.g.*, the network is subject to packet loss as explained in Section 3.1.2.

2.3.3 EPICS

EPICS (Experimental Physics and Industrial Control System) [55, 70] is a distributed software framework designed to meet the requirements of control systems in experimental physics, particularly for large-scale accelerators and other complex installations. Originally developed at the Los Alamos National Laboratory, U.S.A., and first published in 1991, the aim of EPICS is to address the challenges of managing data acquisition, control processes, and supervisory management in high-performance environments, where existing industry solutions at the time were inadequate. EPICS is built on a distributed architecture that utilizes a software communication bus, enabling the seamless integration of multiple functional subsystems. These subsystems include the Input/Output Controllers (IOCs), Display Manager, Alarm Manager, Archiver, and the Sequencer. Each subsystem is designed to operate in a distributed manner, ensuring that tasks can be easily distributed across nodes in a network, thereby scaling from small, localized test setups to expansive, highly distributed networks. The idea is that nodes in the network can represent either actual subsystems of the plant or users connected to workstations that act as terminals, on which EPICS also run to let them and their software interact with the control system. The Channel Access protocol serves as the communication backbone of EPICS, providing a "software bus" for communication between various components of the system. Channel Access supports operations like connecting, getting, putting, and monitoring data fields across distributed databases, using both TCP and UDP protocols. This protocol enables subsystems, including user-extended components, to communicate efficiently. One of the core features of EPICS is its distributed database, which is hosted on IOCs and provides functionalities such as data acquisition, data conversion, alarm detection, and closed-loop control. The IOCs run on hardware platforms such as VME or VXI with a real-time vxWorks kernel and are connected via Ethernet. This configuration allows efficient execution of control tasks and the ability to extend the system easily by adding more controllers. Other main components share data to and from the database in a distributed fashion. Among those there are multiple Managers, such as the Display Manager which provides a graphical interface to the current user connected to the workstation, and the Alarm Manager which keeps track of the state of all the alarms in the system and presents them in a hierarchical fashion to the rest of the architecture and to the users. EPICS also features a Sequencer, which uses a state notation language to manage the system through different operational states and handle exceptions. This sequencer allows for modal transitions of the IOCs, such as switching between operational modes or handling unexpected system events. EPICS has shown considerable extensibility, allowing new hardware and software modules to be integrated with minimal modifications. This extensibility is further bolstered by ongoing upgrades to both hardware and software components to improve the system's performance and ease of configuration. Among the related frameworks considered in this context, EPICS is without any doubt among the most relevant ones: it has been developed in a similar context to that of this work, *i.e.*, that of physics research and development, and evolved into an industry-standard. However, its scale is also what makes it not applicable to the context of this work. EPICS is designed to manage only large-scale installations and is not suitable for the development of small-scale prototypes or products, as it is too complex and resource-intensive, providing functionalities and services like, *e.g.*, slow remote control and supervision. Finally, EPICS is not designed to be used on a wide range of hardware platforms, as it is tightly coupled with the vxWorks kernel and similar proprietary solutions which do not fit the practical contexts at which this work is aimed.

2.3.4 BlockNet

BlockNet [33] is presented as a novel multi-platform control framework designed to address the limitations of existing control-oriented frameworks like, *e.g.*, Simulink and EPICS. Unlike these alternatives, BlockNet aims to provide an integrated solution that supports simulations, real-time control, and the integration of heterogeneous technologies while maintaining execution order, ensuring dynamism, and achieving portability across different computing environments. The primary goal of BlockNet is to enable the seamless creation of local or distributed control systems by breaking down computations into individual code blocks that can be linked to form a computational network, completely similar to what is the block diagram or Simulink diagram of a control system. The idea is exactly that of providing a framework for the development and deployment of control architectures starting from their modeling as block diagrams. This approach is highly beneficial in designing general-purpose control algorithms while maximizing code reusability. One of BlockNet's significant features is its capacity to execute these blocks in the correct order based on their input-output dependencies, thus ensuring deterministic behavior even in distributed systems. Moreover, BlockNet attempts to utilize all available computational resources by executing compatible blocks in parallel and incorporating asynchronous programming to improve overall efficiency. BlockNet is developed on top of the .NET framework, using the C# language to ensure compiled-level portability across multiple operating systems and architectures. The use of .NET also enables effective memory management and provides support for modular extensions of the framework, allowing users to add new functionalities easily. BlockNet introduces communication primitives made available to nodes based on a *publish-subscribe* paradigm but extends it to ensure deterministic execution, making it well-suited for real-time control applications. This paradigm allows easy distribution of data between blocks in a network or even across multiple machines without complex network configurations being required. BlockNet's distinguishing features include its emphasis on ease of use, dynamic adaptability, and extendibility. These characteristics make it suitable for a wide range of applications, from purely simulated environments to hardware-in-the-loop (HITL) systems. Unlike many existing frameworks, BlockNet enables the runtime modification of blocks and entire algorithms without the need for recompilation or system downtime, thereby enabling continuous integration and deployment scenarios in control engineering. The key architectural components of BlockNet range from timers and timing sources, to schedulers, data serializers, and mathematical libraries and solvers. BlockNet is indeed close to meet the requirements stated in Section 2.2, although in that section lies also the main reason why it cannot be considered a solution to the problem that this work addresses. Apart from the fact that it starts from the aim of conceptually modeling distributed systems as block diagrams, a choice that may pose its own limitations, BlockNet tries to provide by itself a solution to any problem that arises during the development of such a framework. This is a critical aspect, as it makes the framework less flexible and more complex, as well as prone to errors and design flaws and less open to contributions from the scientific community. In contrast, this work starts from a thorough review of existing solutions that may solve parts of the original problem, requiring only some integration work and the development of what is missing; the next Chapter 3 is entirely devoted to the detailed description of such tools. Moreover, the choice of the .NET framework, although it is very powerful and flexible, is not the best in terms of portability and integration with third-party software modules and libraries, as it is a proprietary solution that is not widely used in the scientific community compared to other programming languages and toolchains.

2.3.5 NASA F Prime

The F Prime framework [26], developed in the United States by the Jet Propulsion Laboratory (JPL) of the National Aeronautics and Space Administration (NASA) and published in 2018, is currently one of the solutions that better fit the requirements listed in Section 2.2. It is a lightweight, modular, and open-source framework for the development and deployment of software for small atmospheric and orbital vehicles like the CubeSats and the Mars Helicopter prototype. It provides the foundations for a software architecture oriented at flight control in which software modules, written in the C++ and Python programming languages, can be loaded and configured easily as *components* of the architecture itself. The framework, due to its organization, encourages the development of software packages meant to be published and reused among subsequent missions and prototype developments. Moreover, the framework offers ad-hoc communication primitives and services, as well as some thread scheduling capabilities, to ensure that software tasks in the architecture meet their real-time requirements, an aspect that is crucial in flight and space operations. Finally, thanks to its light organization and basic system requirements, it supports a wide variety of general-purpose and real-time operating systems (RTOS) and can easily be compiled on many hardware architectures. This framework is very promising and shows a lot of potential but still does not fully meet the above mentioned requirements, mainly because of its limited diffusion and specialized scope. Although it is open-source, it is aimed at a very specific scientific sector, that of space missions, thus receiving limited acclaim and support outside of such area. Moreover, a downside of its lightweight nature is that it offers limited support for the integration of third-party modules and libraries, a fact which limits the capability of the framework of being used in a wider range of scenarios comprising, *e.g.*, that of swarm aerial robotics, as well as the potential benefit of contributions from the scientific community. Finally, as declared in [26], in its current version it depends on commercial, licensed software for some of its functionalities, which is a limitation for users outside NASA and JPL.

2.3.6 MARTe2

The MARTe2 framework [87], standing for *Multi-threaded Application Real-Time executor 2* [138, 176], is another qualified framework that is presented in this section. Originally developed in 2005 by a team of nuclear fusion researchers at the Joint European Torus (JET) UK facility and then registered as a work of the Fusion For Energy (F4E) european organization, it has been under active development ever since and is currently employed in many nuclear fusion research facilities around the world [29, 80, 135, 136]. As its name suggests, it is a framework aimed at the development of real-time software control architectures similar to the EPICS framework described in Section 2.3.3 but on a smaller scale. It is written in C++ and primarily offers support for this programming language. Its organization is actually the closest to what is requested in Section 2.2 and thus deserves a deeper analysis. It is composed by a core set of libraries plus a set of optional *components* that add specific functionalities or support for particular hardware devices, algorithms, communication protocols, etc. As illustrated in Figure 2.1, the core libraries are divided in three distinct groups described as follows.

- `BareMetal`: a set of libraries that implement basic functionalities ranging from data types and hardware abstraction, to communication primitives and logging, and provide the definition of base classes that can be used to develop new components of an architecture.
- `FileSystem`: these libraries provide functionalities to read and write data from and to files, as well as to manage the configuration of the system and the logging of the data produced by the components.
- `Scheduler`: libraries in this group implement real-time thread schedulers and their algorithms, as well as models such as state machines that can be useful to keep track of the state of a software component or algorithm.

Moreover, note that libraries in each group follow a hierarchical organization, with those on the lower levels providing basic functionalities and those on the higher levels providing more sophisticated and specialized features. All of the framework and its components can be built using the `make` tool, as every MARTe2 software package comes with its own `Makefile` and the entire framework relies on a system of Makefiles that define environment variables and search paths for the compiler. This framework has now reached a maturity that makes it applicable also in contexts that are completely different from nuclear fusion, *e.g.*, industrial robotics. The vision guiding its structure is still that of its early days: implement real-time software control architectures for physics research plants. Thus, its scope has remained the same ever since: managing software units that run *algorithms* and exchange *data*

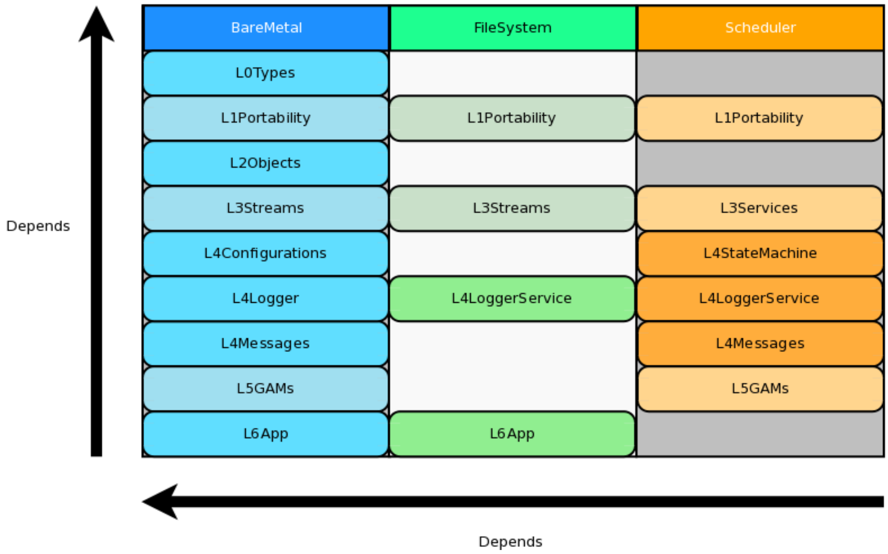

Figure 2.1: Organization of the core libraries of the MARTe2 framework [87].

organized in *signals*, which are collections of data of a specific type and dimension, modeling algebraic objects ranging from scalars to vectors and matrices. This idea reflects the main classes of software components that can be found and implemented in any MARTe2-based architecture, which are the following.

- `DataSource`: a component that exchanges data organized in signals with the external world, *e.g.*, a hardware device or another software component.
- `Broker`: a component that manages the exchange of signals between higher-level components and `DataSources`, implementing memory synchronization schemes if required.
- `GAM`: short for *Generic Application Module*, a component that implements an algorithm that processes input signals and produces output ones, *e.g.*, a control algorithm or a data filtering algorithm.

Finally, a MARTe2-based architecture is usually divided in one or more applications, intended as processes running on the host machine, which contain a number of components specified in a configuration file together with their configuration parameters. Such configuration files, informally called `cfgs`, have a specific syntax, allowing an application to be built by literally "assembling" portions of a configuration file that define the components and their parameters, and then running the application with a specific configuration file that specifies the components to be loaded at run time, plus the settings for the OS scheduler.

The attention that MARTe2 received over the years has led to its adoption in many fusion research facilities worldwide, as well as to the development of modules that integrate it with other similar control frameworks and tools such as the EPICS framework discussed in Section 2.3.3 [19, 139], and the Simulink commercial framework [137]. Among these integrations, many concern systems that handle experimental data storage and retrieval because, due to its signal-centric organization, a control architecture based on MARTe2 can easily generate great quantities of data for each experimental run. The most notable one is MDSplus [84, 85, 131, 134, 225]: a hierarchical, distributed data acquisition and management system originally developed for fusion research facilities through a collaboration between MIT (Massachusets Institute of Technology), Los Alamos National Laboratory, and CNR Padua (Consiglio Nazionale delle Ricerche). The system employs a tree-based architecture where experimental data, configuration settings, analysis results, and diagnostic information are organized in a comprehensive hierarchical structure similar to a file system. One of MDSplus's key features is its ability to handle both synchronous and asynchronous data acquisition tasks through a client-server architecture that supports multiple concurrent readers and writers. A particularly notable feature is its support for distributed operations over both Local Area Networks and the Internet, allowing remote diagnosticians and operators to access and manage experimental data. MDSplus also provides sophisticated tools for data visualization [130], experiment configuration, and diagnostic management, and has also been integrated with frameworks other than MARTe2 such as EPICS [132] and the commercial LabVIEW [133, 174].

Even if MARTe2 is currently the most promising solution to meet the requirements stated in Section 2.2, it still has some drawbacks. The first ones descend from its original scope: it has been developed for a very specific research field, that of nuclear fusion, and thus found little adoption outside of it; this translates to limited improvements over the original organization and lack of support for modern technologies, third-party libraries, and use cases. Its code organization and build system is functional and efficient but follows old practices, like the focus on signals rather than software components, and relies on tools such as `make` which, although standard and widely adopted, have been superseded by more modern solutions which are more suited to the management of large and modular codebases. Due to its almost exclusive focus on real-time software, MARTe2 also imposes some restrictions on the development of software modules, which have to be written in C++ and adhere to old language standards, lacking modern features and erecting a barrier against the integration of newer open-source libraries and tools. These facts, together with the effective but convoluted syntax of the `cfgs`, make MARTe2 a very powerful framework which may unfortunately scare away newcomers and make it difficult to upgrade and evolve the software architectures built with it. Finally, although its compilation expects only a few features from the host system, it requires a tailored OS kernel to fully exploit its real-time capabilities, limiting its application in practice to Linux-based systems if one wants the smoothest possible usage experience.

The scope for which this work has been developed is broader than that of MARTe2: instead of focusing only on real-time software for control, the inclusion of different modules written in other languages is allowed here, as well as the possibility of running the software on a wide range of hardware platforms including SoCs. Moreover, this work considers also the integration of external libraries and tools as a priority, to encourage the advancement of the proposed framework and of the solutions developed with it; this translates in the necessity to support newer language standards, build systems, and user interfaces. Nonetheless, MARTe2 stands as a solid framework for the development of real-time software and it provides the foundation on which the developments presented in Chapter 6 are based.

	Modularity	**Development**	**Abstraction**	**Overhead**
RoboStack	✓	×	+	✓
Vulcanexus	×	+	+	✓
EPICS	✓	✓	×	✓
BlockNet	✓	+	+	×
F Prime	✓	+	+	✓
MARTe2	✓	+	+	✓
DUA				

Table 2.1: Comparison of the solutions presented in Section 2.3 with respect to the requirements listed in Section 2.2; "✓" denotes a completely satisfied requirement, "+" indicates that the solutions provided do not fully satisfy a requirement, and "×" marks an unsatisfied requirement. Is DUA, the framework presented later on in this work, going to meet them?

3 Tools for the job

The previous chapters highlighted the need for a software architecture specifically aimed at distributed autonomous systems and provided a set of requirements that drive its design. Existing frameworks have been reviewed, some of which provided valuable insights and ideas about how to address some of the design challenges, but none ultimately offering the complete set of features required. Before presenting the design of the solution that this work proposes, it is important to delve further into a point that the previous discussion shed light onto: there are solutions available in the literature, in the industry, and in the open-source software community, that can be employed to address some of the key issues that the requirements identify. Following the computer science principle recalled in Section 2.2, that is, to not reinvent the wheel, it is wise to review these tools and see how they can be integrated in the proposed solution. This chapter provides such an overview: for each tool, its motivations are shown by illustrating a problem that is common in the development of distributed autonomous systems, and then the tool itself is presented along with a brief discussion of its features and limitations. Note that the aim of this work is to provide a general-purpose software architecture so not all of the solutions presented in this chapter, as well as all the features that the proposed architecture has, may be necessary or even beneficial in every practical context. However, the goal is to provide a solution that can be adapted to a wide range of applications as a good toolbox to have at hand, offering a set of features that can be configured and used as needed. Indeed, recalling that one of the requirements is also to develop an architecture that can be maintained over time, the intention is that the tools presented in this chapter are included because they represent valid solutions to the problems they address but each aspect of the proposed architecture can be changed in the future should the need arise.

3.1 Inter-process communication: the middleware

When developing distributed systems composed of many modules, especially software ones, the transmission of data among them becomes a topic of paramount importance. Note that this issue concerns both transmissions happening between software modules running on the same host and on different hosts. Moreover, alongside this topic there is that of *intra-process communication*, that is, the organization and regulation of data exchanges among different threads running in a same process. Examples of situations in which many software modules may coexist inside a single process, and why such a situation may be desirable, are presented and discussed in the following. Regarding this aspects, the issues to address are in general the following.

- **Data formatting**, *i.e.*, how information that should travel from one module to another should be encoded and structured in packets or similar entities.
- **Data serialization and deserialization**, *i.e.*, how packets should be transformed into a format that can be transmitted over a medium, which is usually some kind of point-to-point connection or a network.
- **Data transmission**, *i.e.*, how packets should be sent and received, and how the system should react to errors or delays in the transmission; this also includes things like reliability requirements, that is, whether the system should guarantee that a packet is delivered, and how many times it should try to deliver it in case of failures.

The points listed above are usually dealt with by employing a specific *protocol*, that is, a predefined set of rules that regulate the dynamics of the communication. All the software that composes the system should adopt such a protocol, which is usually implemented in a set of libraries that can be used in the code. However, dealing directly with networking protocols might slow down the development process: the host system configuration usually becomes very important, since all the operating systems involved must agree on the protocol implementation to support and use, and some work may become necessary to proficiently use all the protocol APIs to achieve all the desired communication features and behaviors. Remember that this is not the original task: the goal is to develop a distributed autonomous system and to write software for it, thus it is desirable to have a tool that abstracts all the networking details and provides a simple and easy-to-use API that makes data transmission in software modules a straightforward operation. This is where a particular kind of software, historically termed *middleware*, comes into play. To grasp its importance in the context of this work, a step back in time is necessary.

In 1968, a conference sponsored by the Science Committee of the North Atlantic Treaty Organization (NATO) was held in Garmisch, Germany, on the topic of software development. Although not widely discussed, it was a historically significant event in this field: it provided a comprehensive overview of the design practices and methodologies used by leading companies in the growing and highly competitive field of computing. The goal was to identify standard strategies for common problems. It should be noted that at that time and for at least another fifteen years there were virtually no standards in computer production and programming. Each company offered its own product developed largely from scratch, in both hardware and software. Standardization and collaboration among various manufacturers began only when information started to be exchanged between different computers,

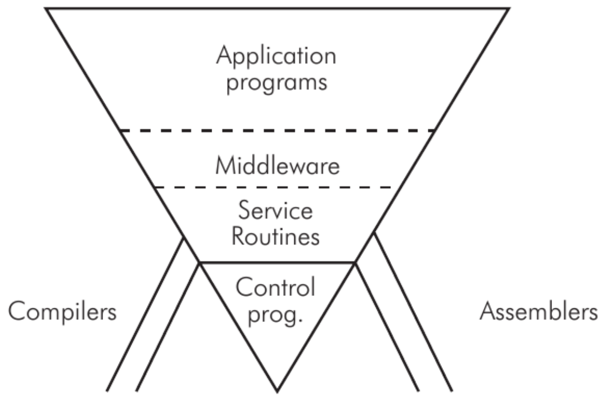

Figure 3.1: Software organization in a general-purpose computer, as presented at the 1968 NATO conference on software engineering [175].

first through storage media and later over the Internet, alongside a growing demand for new types of devices and standard formats to adopt when sharing data. A report of the conference can be found in [175]. It includes the first official mentions of several concepts well known today such as *component design* and *unit testing*. However, of particular relevance to this work is the representation of the hierarchical structure of software presented by one of the speakers. This structure, valid even today with certain extensions, is illustrated in the inverted pyramid scheme shown in Figure 3.1. The idea is that a sophisticated system is like an inverted pyramid: it can not remain operational without adequate support provided by secondary components. These might be completely invisible to the end user and thus may be considered "external", yet they are of vital importance for the correct functioning of the whole: *compilers* and *assemblers*, for instance, which are programs that translate code written in a given language first into the target architecture's assembly language and then into binary machine language, typically generating an executable file. These tools must be reliable and efficient as the implementation of everything else depends on them. Next, there are the layers of *Control Programs* and *Service Routines*: these are called today *operating system* and *device drivers*, respectively. They are typically developed with a strong dependence on the machine at hand, considering its specific characteristics, and heavily depend on the employed compilation tools. Their role is to provide an abstraction of the hardware that makes its use simple both for the end user, who interacts with applications, and for application developers, who interact directly with the hardware. The application software is precisely what is found in the topmost layer: programs that use the hardware more or less directly to provide various services to users. There is, however, an intermediate layer that is often overlooked: the *middleware*. Situated between system software and application software, middleware found its first definition and characterization at the conference mentioned above. Initially conceived merely as a connective layer between current and *legacy* software, middleware provides services and functionalities common to applications beyond what the operating system offers, building itself on top of it and abstracting its own implementation and protocol details. From the perspective offered by the topmost level of the inverted pyramid it appears as normal application software, often consisting of libraries. Middleware typically offers services to programmers aimed at, *e.g.*, data management, communication, and authentication. Therefore, middleware aids developers in building applications more efficiently by acting as a connective tissue between applications, data, and users. In distributed environments, middleware can facilitate the development and execution of large-scale applications. A typical service offered by middleware is a method for data exchange between applications, even across different devices, according to a common format referred to as an *interface*. In general, the services offered by middleware are specified only in terms of their dynamics but can be implemented on different architectures and devices, functioning consistently while keeping their implementation hidden from both the application programmer and the end user. For example, when middleware handles communication between applications, it might position itself within the network stack between the transport and application layers (considering the TCP/IP model of the network protocol stack). Today, middleware can be categorized into two types: that which operates in *human time*, designed for direct use by human users (*e.g.*, web services), and that which operates in *machine time*, offering an API that application developers can use to facilitate their work, benefiting from increased portability across a wide variety of devices. In light of all this, and considering the common difficulties faced when designing the software for a distributed autonomous system, it becomes clear how adopting communication middleware to solve the problem of data exchange among modules can simplify things: many functionalities that have to be implemented from scratch are instead offered by the middleware itself as services to be employed in the programs being developed.

The rest of this section introduces three specific middleware that are widely employed in the industry and in the open-source community, and that have become of pivotal importance in the two practical scenarios considered later in this work. The first middleware is the *Data Distribution Service* (DDS), which has been the first to provide a standard for data exchange in distributed systems, followed by the *Zenoh* middleware which addresses some of the limitations of DDS, and finally the *Robot Operating System* (ROS), which is a middleware specifically designed for robotics applications. All these three components are included in the architecture proposed in the following chapters.

3.1.1 Data Distribution Service

In 1989 eleven companies including Hewlett-Packard, IBM, Sun Microsystems, Apple Computer, and American Airlines, founded the *Object Management Group* (OMG), a consortium that is still active in the field of computing with the goal of defining standards for industrial software. This effort aims to foster the integration of solutions from different vendors into unified ecosystems, covering a variety of applications and technologies. The purpose of the OMG is to establish, for every new type of software under consideration, a common model to implement, thus enabling and promoting its use on any relevant platform. The group provides the industry with specifications only, not implementations. However, any team submitting a new specification has to provide a potential implementation before it is accepted, intuitively to avoid defining impractical or useless standards. OMG member companies encourage the development of solutions that can interoperate immediately without any limitations.

One of the standards defined by the OMG is the *Data Distribution Service* (DDS) [198, 216]. It specifies a particular type of middleware responsible for managing direct communications between computer systems, including the details of how data should be serialized, deserialized, transmitted, and routed, as well as how the API to invoke these operations should be structured. Its definition has been carried over with the goal of allowing implementations to satisfy real-time determinism guarantees whenever necessary. Moreover, the goal of the DDS is to enable technologies, platforms, and devices from different vendors, operating in distinct contexts and locations, to immediately communicate, exchanging data for which only the format and minimal other attributes have to be known beforehand. This facilitates the management of critical aspects, like performance and scalability of the various components and the network connecting them. Effectively, the DDS greatly simplifies what otherwise is for the programmer a complex task of configuring networks and organizing data transmissions which, as discussed, is one of the classic challenges faced during the development of a distributed autonomous system. As such, the DDS has become widely employed in industrial contexts ranging from manufacturing [79] to power grids [214].

The middleware model follows a *publish-subscribe* paradigm: entities involved in the communication, acting as either *publishers* or *subscribers*, declare their intent to transmit or receive packets of information and are instantiated as `DataWriters` or `DataReaders`, respectively. The publish-subscribe denomination arises from the way different transmissions are organized, from the programmer's perspective, into channels denoted as *topics*. Topics are defined by:

- a **name** that identifies the topic, generally encoded as a text string to be easy for programmers to remember and use;
- an **interface**, described using a specific language (*e.g.*, an Interface Description Language (IDL)), which specifies the content of a single data packet and how it is encoded.

Interfaces are intended as structured data types, *i.e.*, data structures containing ordered fields of different types, ranging from simple integers and floating-point numbers to more convoluted structures like arrays and strings, or other interface types as well. One key aspect to consider is that, given an interface specification, the middleware takes care of generating the code necessary to serialize and deserialize data packets according to that interface, as well as to transmit and receive them over the network. This is a significant advantage, as it allows the programmer to focus on the data itself and on the logic of the application, without having to worry about the details of the network communication. Another key aspect to consider, which also makes the DDS very versatile, is that communications are asynchronous by nature: `DataWriters` publish data on a topic regardless of the presence of any `DataReader` listening on that topic. `DataReaders`, on the other hand, can subscribe at any time to a given topic and start receiving messages from that point on. The OMG DDS standard provides APIs for both synchronously waiting for new messages, as well as asynchronous notification and handling via *callback* functions (recall that a callback, in a software program, is a routine that gets called asynchronously, *i.e.*, when an event occurs for which that callback is registered and either an event handler or a job scheduler invoke the callback to process the event).

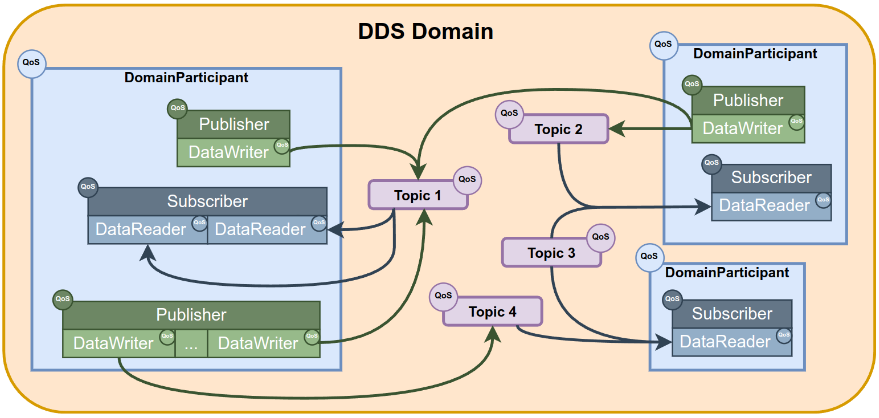

Figure 3.2: A network of applications using the DDS, each with its own `DomainParticipant` [71].

The behavior of any entity in a DDS network, and thus that of every transmission, can be configured by specifying a Quality of Service (QoS) policy for it [49]. A DDS QoS policy typically consists of items such as:

- the **history policy**, which indicates whether all packets transmitted on a specific topic should be delivered or rotating buffers of a given maximum size should be put in place and rotated in case of network congestion;
- the **history depth**, which is the size of the buffers optionally activated in the previous point;
- the **reliability policy**, indicating whether communications should be managed by the middleware to ensure that all subscribers of a topic receive all packets transmitted by publishers, or packet drop and loss are permitted, thus enabling *retransmissions* if necessary;
- the **durability policy**, allowing transmitted packets to be retained by publishers and delivered to subscribers that may connect later;
- the **deadline**, **lifespan**, **liveliness**, and **lease duration**, which are characteristic maximum times to determine when a packet has become outdated or an entity can be considered inactive.

Of course, there are default settings that are kept as consistent as possible across different implementations to ensure maximum compatibility. A complete scheme of a DDS network of applications can be found in Figure 3.2.

Even at this level, the significant simplification introduced by the DDS middleware becomes evident: with a relatively simple and quick configuration, issues such as defining a transport protocol and developing software modules to use it according to specific rules and formats can be immediately resolved. The DDS takes care of all the tasks necessary to organize communications: from the most basic, such as message transmission, to more organizational tasks like publisher advertising and node discovery within the network, as well as flow control and retransmissions. It also handles publisher redundancy and other pathological situations in a way that is completely transparent to the programmer. DDS, in effect, makes it very simple to build distributed applications or networks of applications, as it hides and simplifies many management tasks. Commercially available DDS implementations today offer API for various languages, such as Ada, C, C++, C#, Java, Python, Scala, Lua, Pharo, and Ruby, and since they are all built on the same standard, they are fully compatible and interoperable. There are also specifications concerning lower-level types of DDS, designed to be included in resource-constrained systems such as embedded systems while still maintaining compatibility with more comprehensive high-level implementations.

Communication modes, serialization, and deserialization of data are defined in the OMG standard called *Wire Protocol*. Despite the broad scope of this specification, it is instrumental to highlight two key parts of it for the benefit of the following discussion.

Discovery Module

This module of the DDS includes protocols that allow different DDS instances running on network-connected systems to discover each other and organize communications from that point on. There are two such protocols.

- **Participant Discovery Protocol (PDP)**, which allows DDS instances—each associated with a particular application running on a specific machine, hence called *participants*—to announce their presence to others while simultaneously obtaining information about any other reachable participant.

- **Endpoint Discovery Protocol (EDP)**, which comes into play immediately after PDP and relates to transmitting information about the *endpoints* owned by the various participants. These endpoints, as publishers or subscribers, are deemed compatible if they share the same topic and interface and if the QoS policies *offered* by publisher entities satisfy the *requests* of subscriber entities.

Implementations can choose to include, beyond the PDP and EDP protocols defined by the standard, their proprietary versions. If two instances initiating communication have at least one PDP and one EDP in common, they can exchange information and complete the discovery process. These protocols can be implemented in either *unicast* or *multicast*, depending on whether all participants are known in advance or not. In the former case, a preliminary configuration of the network is required to preset the addresses and port ranges of all the machines involved, while in the latter the protocols automatically manage everything as soon as the DDS instances are launched. The potentially vast operational scale of these solutions has made DDS attractive for mission-critical areas such as military and logistics although, for now, relying entirely on the Internet for such communications is not advisable. Unless predefined configurations are in place, it is necessary for every network device along the path between two participants to support multicast and not block such transmissions.

Platform Specific Model

This model establishes how data of different formats and encodings should be serialized, transmitted, and deserialized in a uniform manner across all possible implementations and platforms. In an IP network, all data packets are transported over UDP. Whether the endpoints are associated using multicast or unicast, the Discovery Module's protocol opens UDP *sockets* for them with well-defined port numbers, which depend on:

- the **domain ID**, a numerical constant that selects a range of UDP ports in the implementation, allowing applications using DDS to communicate simply by specifying the same domains; multiple domains can coexist even on a single machine;
- the **base range** of UDP ports available in the system;
- the **participant ID**, a unique identifier for an instance within a machine;
- any other **manual**, specific configurations.

One may wonder why UDP is used for data transport. The reasons are similar to those in other contexts where a deterministic behavior is desirable: UDP is a basic, best-effort protocol without any communication management features beyond simple transport between endpoints. As such, it is lightweight and supported by a wide variety of devices and platforms. This simplicity gives UDP implementations complete predictability, excluding the unpredictability of the network itself, and natural scalability. Therefore, it is the best choice for real-time systems as well as for the development of this type of software.

3.1.2 Zenoh

The Data Distribution Service middleware effectively solves the inter-process communication problem in distributed contexts, allowing applications to easily set up communication channels over a network of potentially large scale. As the previous section clarified, it is particularly user-friendly from the point of view of the application programmer also because of its automatic configuration capabilities: thanks to the Discovery Protocol, DDS participants and their related entities can automatically discover each other and connect, without any prior network configuration required. If all of this seems too good to be true, that is because it actually is. Coming back to the origins of the DDS specification, one can see how they date fairly back in time: the first drafts of the protocols were proposed in the late 90s, while the first officially published implementations are from the early 2000s. In the meantime, the world of computing has changed and situations that were only theorized before are currently the norm. A wide variety of new technologies, which are all somehow related to networking, have seen the light of day in the meantime. As an example, consider the advent and diffusion of various high-bandwidth wireless networks like WiFi or 5G, which then led to other advancements like cloud computing, Internet of Things (IoT) and edge computing. All these technologies are nowadays part of the daily life and they have all contributed to shape the current scenario in information technology: digital devices are everywhere, on every scale, all interconnected at the same time using a variety of technologies and protocols. That is also what made distributed autonomous systems possible but also changed the world into something much different than the one in which the DDS was born. When the OMG DDS standard was designed, networks were still made of cables and routers, with point-to-point links which finally achieved little to no packet loss and connected devices which were different declinations of the same concept: a big computer. Nowadays, the term "computer" has a much broader meaning and networks connect devices that operate on many different scales using multiple types of physical links. One of the few things that are common among all is that they all need to share large, if not huge, amounts of data among themselves. This is where the DDS starts to show its limitations: it was designed for a world where the network was a mostly reliable, predictable medium and connected devices were assumed to have a certain level of computational power and memory. This is not the case anymore, and the DDS is starting to show its age. The main limitations of the DDS are related to these exact contexts and came up in the last five years due to its wide adoption in new contexts like that of autonomous robotics, where having multiple heterogeneous devices connected over a wireless, lossy network is the norm. In this context, the adjective "lossy" denotes a packet switching network subject to *packet loss*, *i.e.*, the possibility that packets flying get dropped by the devices handling them because of network congestion or similar phenomena, in an attempt to preserve overall network functionality. The most limiting factor of the DDS is, in fact, its intrinsic reliance on a network layer which requires little to no retransmissions. If this assumption is not met, communications based on the DDS can become very slow or even fail completely due to network congestion. This is mainly due to the fact that the Discovery Protocol has not been designed with this situation in mind, so that it requires a number of multicast transmissions that grows exponentially with the number of participants in the network. When the network is not reliable, *e.g.*, in the case of WiFi, these transmissions may be lost leading to bursts of retransmissions that might clog the network altogether. A second important limitation of the DDS is its inability to optimize bandwidth usage and retransmissions when dealing with large data, *e.g.*, video streams, which obviously were not a concern back in the days in which the DDS standard was defined; this leads to even more risks of ending up with a congested, unusable network. Moreover, these problems become unsolvable if one attempts to route DDS traffic over a Wide Area Network (WAN) such as the Internet, whose routers and switching devices might apply traffic control policies that might throttle or even prevent DDS packets to travel safely; again, this scenario was not considered in the original specification, but has become of pivotal interest due to relatively new concepts such as IoT. Finally, note that all of the features and functionalities that the DDS offers come at the price of an overhead in terms of both resource usage and protocol complexity. A straightforward DDS implementation that has all the features specified in the standard typically ends up being quite a large piece of software, not so easy to adapt to resource-constrained environments and a DDS data packet usually has many control fields that increase the amount of data being transmitted.

This discussion highlights how the DDS middleware is useful in some contexts, mainly those in which the network is fully reliable and ease of configuration is of the essence, but inapplicable in others. This is where a new communication middleware, recently born as a successor of the DDS to address the challenges of the modern, interconnected world, comes into play. This middleware is called *Zenoh* [50], which is the crasis of “zero network overhead”.

The design of the Zenoh protocol starts from the following consideration: an application programmer needs communication APIs to handle three main types of data, which are:

- **data in motion**, *i.e.*, data that is being transmitted over the network among multiple devices and software modules, all active at the same time;
- **data at rest**, *i.e.*, data that is stored in a device and can be retrieved at any time by other devices;
- **data for computations**, *i.e.*, data that is used as input for computations requested from one network endpoint to some other(s) and that is also produced and returned as their output.

The publish-subscribe pattern was invented to handle data in motion, which eventually led to the definition of the DDS among many other protocols and to the diffusion of distributed systems, and is still the best solution to apply in this case. The last two sets of data, from a communication perspective, can be handled in the same way by implementing an API that allows to *query* network endpoints, formalizing both the interrogation phase and the answer phase. Zenoh is defined as a Pub/Sub/Query protocol, since it implements three fundamental kinds of objects: publishers, subscribers, and *queryables*. Each object, once instanced, can *declare* itself as one of the three kinds then exchange data according to the operations permitted by its type. This is very important because in many contexts in which the DDS is applied today, *e.g.*, that of autonomous robotics, there is in fact the need for a query-based or client-server type of communication which is not defined in the protocol, thus leading to its re-implementation on top of DDS transmissions and requiring at least two DDS topics and a control protocol; this contributes to the overheads and the issues mentioned above.

In Zenoh, each data point is organized as a pair of type *(key, value)*, where the value is the actual data and the key identifies the communication channel, or channels, which that value concerns. A key is typically an array of characters made of many substrings separated by the `/` delimiter, as in `home/kitchen/thermostat`; wildcards and regular expressions are also allowed in API calls to match multiple keys at once. Note how the use of the delimiter naturally enforces a partition of the space of keys, and thus of the data flowing in the network, into multiple subdomains. This is a key feature of Zenoh as it allows to optimize many phases in which this topology is involved like, *e.g.*, the discovery of network endpoints. As opposed to the DDS in which all endpoints have to know everything about each other to be able to exchange data, in Zenoh endpoints query for their fellows only when they have to share data that has a specific key and discovery-related transmissions are directed only to hosts and endpoints that handle that key, starting from its left-hand side going towards the right-hand one. This implies that when, *e.g.*, a new subscriber wants to receive data published with the `home/kitchen/thermostat` key, it first queries which endpoints handle the `home` subdomain, then ask them directly for endpoints handling the `kitchen` subdomain, and so on until the end of the key. This way, the number of transmissions required to discover and connect endpoints is effectively reduced, since these transmissions are triggered only when necessary and happen among only the hosts and applications that are actually involved. To achieve this huge improvement over the DDS, however, Zenoh abandons the automatic discovery feature: there is no such automatic discovery functionality in Zenoh and all applications or hosts have to be configured manually to know which other such entities are present in the network and how to reach them. Multiple network topologies are possible, as explained in [50] and illustrated in Figure 3.3:

- **clique**, where all endpoints are peers and can communicate with each other;
- **brokered**, where communications are routed among endpoints by a central router application, which is implemented as a *daemon* running in background;
- **routed**, in which many routers enable communication among multiple groups of hosts and applications in brokered mode;
- **mesh**, a hybrid situation in which endpoints may share partial network structure information or route data among themselves.

This may seem a step back from the DDS specification but if one recalls that the Discovery Protocol is among the first causes of trouble when the DDS is used in relevant contexts, such as that of autonomous robotics, this has rightfully been deemed a reasonable price to pay to have a reliable and efficient communication.

Finally, another positive aspect of Zenoh is that it is implemented with the additional goal in mind of having the lowest possible resource and memory footprints, which implies that it can run on a variety of different hardware platforms ranging from servers down to microcontrollers. For example, the control fields in its data packets can take up to as little as 5 bytes, as opposed to the at least 56 bytes required by the DDS. Moreover, it is almost completely agnostic of the underlying network layer, positioning itself in the protocol stack eventually as low as on top of the data link layer as showed in Figure 3.4, thus supporting a wide variety of protocols and physical links including UART, Bluetooth, LoRa, Unix Sockets, TCP/IP, UDP/IP, QUIC, WebSockets, and CANbus, offering the same API and functionalities in all cases. Thanks to this level of versatility, Zenoh is a very good candidate to be used in the development of distributed autonomous systems, especially in those contexts where the DDS is not applicable due to the reasons discussed above, and it has already been successfully employed in industrial contexts achieving good performance in terms of both throughput and latency [18, 101] despite being relatively new, as its version `1.0` has been released in October 2024. For these reasons, it is included in the proposed architecture in its current state alongside some DDS implementations, to provide a complete set of communication tools that can be used in a variety of contexts, platforms, and applications.

3.1.3 Robot Operating System 2

The previous sections presented a class of software solutions whose objectives align with those that emerge during the design of a distributed system and provide real-time communication services. With

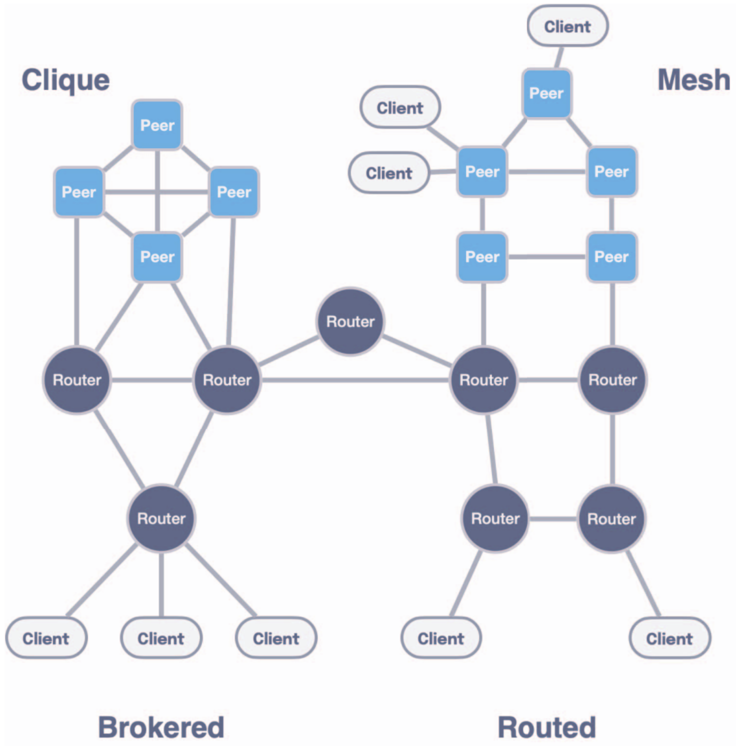

Figure 3.3: Possible network topologies in Zenoh [50].

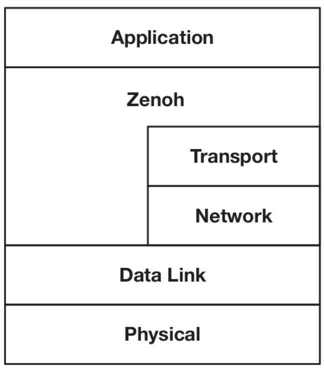

Figure 3.4: Placement of the Zenoh protocol in the network stack [50].

all these elements in place, the next solution to be described is a middleware specifically designed for robotics applications, which are a particular case of distributed systems, that is also widely employed in the industry and in the open-source community. Since autonomous robotics is one of the applications for which the proposed architecture has been developed and in which it has been tested and employed, this particular middleware is of paramount importance for the rest of this work and is described in the following in due detail.

ROS 2 [126] is the second version of the *Robot Operating System* middleware [188], originally developed

Figure 3.5: ROS 2 logo [188].

in 2007 by Willow Garage [205], a spin-off of the Stanford Artificial Intelligence Laboratory. Although its name might give a different idea, ROS is not an operating system in the traditional sense of process management and scheduling; rather, it started as a structured communications layer above the host operating systems of a heterogenous compute cluster. Then, ROS has evolved into a middleware that offers robotic and automation programmers services such as hardware abstraction, low-level device control, data exchange and inter-process communication, support for creating client-server modular architectures, and software package management. The entire project consists of a collection of software libraries and tools. It has always been open-source with a community-based development model and rigorous review processes: the source code of each version, ready for compilation and installation, is organized into various repositories hosted on GitHub, and documentation is published online alongside numerous *design documents* that explain and comment on the design choices that are made and on the new features that are introduced. Fixes and innovations are tested in a *Rolling* [210] release and subsequently introduced in stable releases, which follow an alphabetical naming scheme similar to that adopted by Canonical for the Linux distribution *Ubuntu* [35] and are thus often referred to as "distributions". Currently, the project is maintained and steered by the Open Source Robotics Foundation [211]. Its open nature led to a steady increase in popularity over the years and has contributed immensely to a worldwide renewal of the interest in robotics thanks to code sharing and open scientific contributions, both in a theoretical and in a practical sense, favoring the birth of a large number of communities, research projects, frameworks, laboratories and startups all over the world, sparking in the world of robotics a revolution similar to that happened in the software engineering world in the 90s with the advent of open-source software. Today if, *e.g.*, a research team publishes a novel spatial mapping algorithm, it is very likely that the implementation of that algorithm is made available as a ROS package and that it is tested and reviewed by the community, thus becoming a standard tool for all roboticists to use in their projects.

By its documentation, ROS 2 supports the Linux, Windows, and macOS operating systems but the reality is that support levels are defined by three *tiers* that define support quality and frequency of updates. Currently, the only Tier 1 platform is Ubuntu Linux, in which a ROS 2 distribution can be installed either from binary packages or compiled from source; the latter installation technique can also be applied to different Linux distributions. As for Windows and especially macOS, since their closed nature with respect to Linux usually adds additional difficulties when developing robotic software, they are not so widely adopted and thus the community puts less effort into updating ROS for these platforms, also because of the lack of developers and testers that actually have these platforms available to develop on together with the expertise required to manage and integrate the whole ROS codebase.

For a summary of the history of ROS and an overview of its design goals, see [94]. From that document, it is evident that the project soon proved potentially suitable for complex use cases, such as distributed and real-time management of collaborative swarms of robots or the homogeneous integration of different types of hardware and communication interfaces. A more technical and detailed description of the new features of the version 2, which introduces many of the most popular features, can be found in [230].

Historically, the first service offered by ROS was inter-process communication: in ROS 1, that was implemented by custom protocols built on top of TCP and UDP. The most significant difference between ROS 2 and the first version, and currently its greatest strength, is the fact that instead if reimplementing such protocols it sits directly above a layer, denoted as the RMW or ROSMiddleWare layer, which houses support for various communication middleware implementations aimed at distributed systems, such as the DDS and Zenoh. The first communication middleware to be integrated with ROS was the DDS [245]: the entire middleware was indeed built on top of the DDS, whose specific implementation is left to the user and is fully interchangeable, to implement the basic inter-process communication and data exchange services. This means that all the benefits of the DDS mentioned earlier such as ease of use and interface definition, Quality of Service, transmission configuration through discovery, and serialization, are immediately available to robotic system developers, solving quite a few problems but introducing new ones as previously discussed. Developers can also take advantage of ROS's standard libraries and of the vast open-source ecosystem of packages based on ROS that are publicly available, as well as contribute their own work to advance the community of engineers and developers who use it.

From its origins, each version of ROS supports the following two programming languages: C++ and Python. This choice has been motivated by the high diffusion of both on nearly every platform, as well as by their intrinsically complementary nature. C++ is the most used and recommended for performance-critical as well as production applications, while Python is more suitable for rapid prototyping and for the development of scripts that interact with the system, as well as for high-level software modules that have no strict performance requirement but whose development and maintenance may benefit from a higher-level language. Moreover, Python is the de-facto standard programming language in contexts like machine learning and artificial intelligence, both fields that are having a disruptive impact on robotics. The ROS 2 project has also introduced support for other languages, such as Rust, Java, and plain C, and has developed a set of tools to facilitate the integration of these languages with the middleware, thus making it easier to develop software for robots using the language that best suits the task at hand. This is a significant improvement over the first version, which was limited to C++ and Python, and it is a clear sign of the project's commitment to keeping up with the times and with the needs of the community.

To provide an idea of the main services offered by this middleware relevant to this work, the key features of ROS 2 are now described following a top-down approach, eventually reviewing some auxiliary but very useful tools and frameworks; thus, many minor utilities are excluded for the sake of clarity but the reader is referred to the technical documentation [189] and basic tutorials [191] for a deeper overview.

Build system

In the ROS ecosystem as in many similar contexts, software is naturally divided into units called *packages*. The idea is that each package constitutes a standalone software module, designed to fulfill a specific function within the system's overall architecture, potentially interacting with others at various levels and using specific services offered by the system itself through APIs or specialized hardware. Nevertheless, a robot presents a different scenario from those typical of software development and it is not uncommon for a developer to work on multiple packages simultaneously. Adding to this complexity is the fact that if different modules need to interact, their corresponding packages are inevitably interdependent. Therefore, a common system is needed to simplify the creation of a *workspace* in which to develop, execute and test the software contained in multiple packages. The solution provided by ROS 2 consists of a universal build system applicable to any context where this middleware is employed and divided into two modules, with the reasons for this approach described in [232].

Initially, there is *colcon* [234]. It was originally intended to be a tool that automates and simplifies the compilation and installation of Python or C++ code packages via CMake [116] (a cross-platform free software for build scripting and automation, whose name stands for "cross-platform `make`") or Setuptools [68] for use with ROS 2 but it soon became a more general project. Today, it is used even outside of the robotics world to manage large projects and codebases: it is not uncommon to find software libraries and tools whose source code can be managed and built using colcon. ROS 2 relies on colcon to organize every aspect of a workspace, intended as a collection of packages either in source code or binary form. Essentially, colcon creates a workspace by defining some basic directories in the file system where both source code and build artifacts reside, and then manages compilation and installation according to either automatic or specified rules resolving dependencies automatically, *i.e.*, ensuring that all external pieces of software that are required are actually present on the system and correctly configured. Each package, to be manageable by colcon, must include a *package manifest* in the form of an XML file (XML stands for eXtensible Markup Language, based on a syntactic mechanism that allows the definition and control of the meaning of elements contained within a document or text) in which all the characteristics of a software module, such as its name or dependencies, are listed. Installation may result in the generation of an executable file or a shared library. The many command-line options of colcon also allow interaction, if necessary, with lower-level tools used at various stages of compilation and build.

In addition to the typical challenges related to building interdependent packages, it is also necessary to address those concerning integration with preexisting libraries and interfaces that compose a ROS 2 installation. To address this and many other issues, the second module is used: *ament* [231]. It is a collection of Python scripts invoked automatically by colcon that complete the build process by resolving dependencies with other ROS 2 packages and ensure that paths, environment variables, and scripts are correctly set up so that the software can later be executed without issues. This system proves effective: when having multiple packages in a workspace, they can all be built together with a single command.

Interfaces

In ROS 2, like in the DDS, it is possible to specify the format of the data packets exchanged among software modules, which are called *messages*, using special text files called *interface files* [233]. In these files, similar to DDS IDL files, the structure of the packet is defined by specifying each field on a separate line entering its name and data type. Data types range from integers or floating-point numbers to strings and even variable-length arrays. It is considered good practice to group these files into packages consisting only of interface definitions, which are then built and installed using the same build system: the result in this case is a collection of Python classes, C++ header files, and shared libraries that define the types of the data packets as well as their serialization and deserialization methods, which can then be included and linked in other modules that need to exchange data in such formats. In practice, a ROS 2 developer never needs to concern themselves with anything related to format specification, serialization, transmission, reception, or deserialization: all of this is automatically managed by ROS 2 via its RMW layer.

Unlike a standard DDS, however, ROS 2 also offers other two communication primitives, each with its own interface file format, which extend the communication services beyond the classic publish-subscribe paradigm. All the communication primitives are listed in the following.

- **Messages**: they are the most common and are used to exchange data packets among software modules in a publish-subscribe fashion; they are specified in `.msg` files. An illustrative scheme is provided in Figure 3.6, and the definition of the `Image` message from the `std_msgs` library is provided as an example in Listing 3.1.

- **Services**: they are used to request a computation to be performed by another software module and to receive the result, enabling a client-server, *stateless* communication; they are specified in `.srv` files. Note that in this context, the term *stateless* identifies a kind of communication that has no intermediate states between its beginning and end: in a service call, a *client* requests a computation to the *server*, which performs it and returns the result to the client, wihtout any further interaction between the two being possible and usually in a short time. This is useful for simple transmissions like quick parameter setting or data retrieval but not for complex interactions that require multiple steps, long computation times, or many possible outcomes.

An illustrative scheme is provided in Figure 3.7, and the definition of the `AddTwoInts` service, which allows to perform the addition between two integers and is widely employed in tutorials, is provided as an example in Listing 3.2: note how the two request and result messages are delimited in the file by the three dashes.

- **Actions**: they are used to request a computation to be performed by another software module and to receive the result, enabling a client-server, *stateful* communication; they are specified in `.action` files. In this context, the term *stateful* identifies a kind of communication that has intermediate states between its beginning and end: in an action call, a *client* requests a computation to the *server*, which performs it and returns the result to the client but may also send feedback to the client during the computation. The client may even request the cancellation of the ongoing operation to the server. Moreover, the server may reply to the client at the end of the operation sending back two items: the actual output value of the computation, if any, and various codes that describe nearly any possible outcome. This is the most flexible and powerful communication primitive offered by ROS 2, usually employed in software modules that perform sophisticated operations like navigation or tracking where the client needs to know the status of the computation and possibly to interact with the server during its execution. An illustrative scheme is provided in Figure 3.8, the state machine of an action goal is portrayed in Figure 3.9 and the definition of the `Fibonacci` action, which in many tutorials expresses the computation of the Fibonacci sequence up to a given order, is provided as an example in Listing 3.3: note how the three request, result, and feedback messages are again delimited in the file by the three dashes.

All of these are implemented on top of the RMW layer. In the case of DDS, they are all implemented using topics: a service maps to two topics, one for the request and one for the reply, while an action is implemented with a *feedback topic* and two services: the *goal service* to place the request for a computation and the *result service* to request its output. In the case of other middleware that natively supports client-server communication, like Zenoh, the implementation is more straightforward and efficient as the middleware itself manages the stateful communication and the client-server interaction, thus reducing the overhead and the complexity of the system.

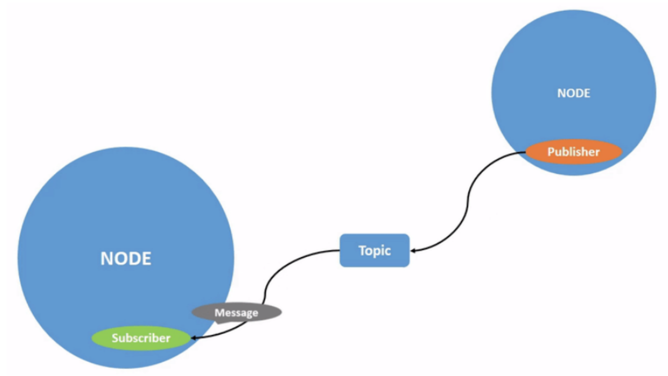

Figure 3.6: Scheme of the message communication primitive [192].

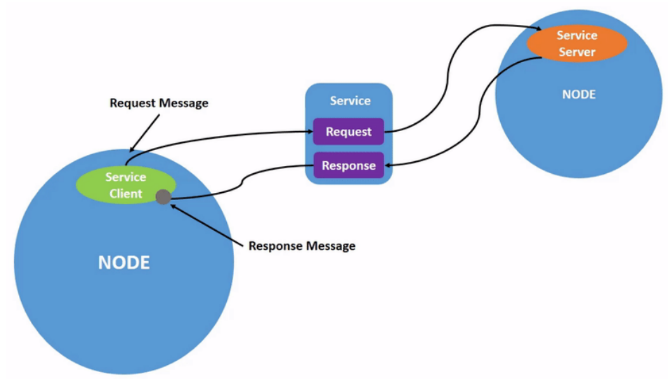

Figure 3.7: Scheme of the service communication primitive [194].

```
# This message contains an uncompressed image
# (0, 0) is at top-left corner of image

Header header # Header timestamp should be acquisition time of image
              # Header frame_id should be optical frame of camera
              # origin of frame should be optical center of camera
              # +x should point to the right in the image
              # +y should point down in the image
              # +z should point into to plane of the image
              # If the frame_id here and the frame_id of the CameraInfo
              # message associated with the image conflict
              # the behavior is undefined

uint32 height # image height, that is, number of rows
uint32 width  # image width, that is, number of columns

string encoding # Encoding of pixels -- channel meaning, ordering, size
                # Taken from the list of strings in
                # include/sensor_msgs/image_encodings.h

uint8 is_bigendian # Is this data big-endian?
uint32 step        # Full row length in bytes
uint8[] data       # Actual matrix data, size is (step * rows)
```

Listing 3.1: Definition of the `sensor_msgs/Image` message.

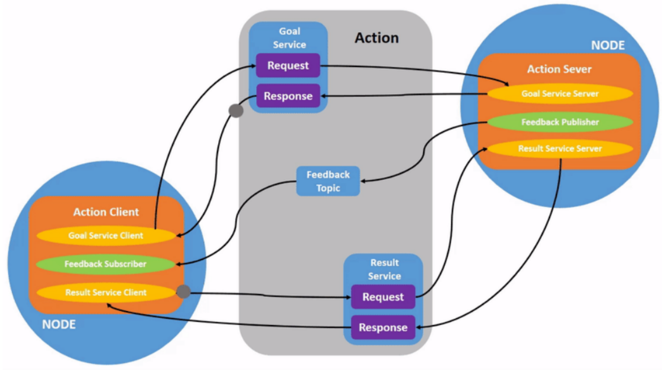

Figure 3.8: Scheme of the action communication primitive [193].

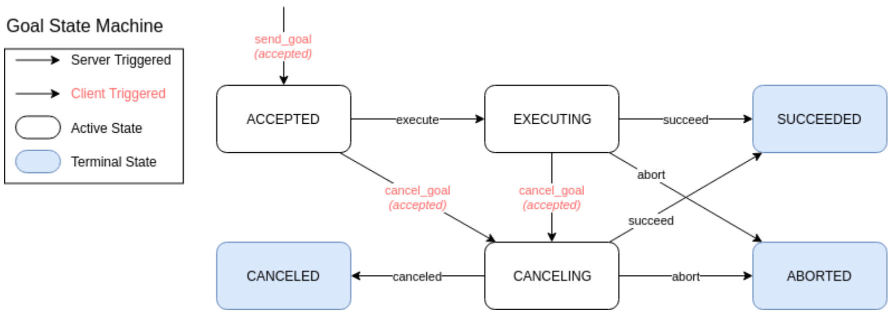

Figure 3.9: State machine of an action goal [24].

```
int64 a
int64 b
---
int64 sum # = a + b
```

Listing 3.2: Definition of the `example_interfaces/AddTwoInts` service.

```
# Goal
int32 order
---
# Result
int32[] sequence # Variable-length array of 32-bit integers
---
# Feedback
int32[] sequence # Variable-length array of 32-bit integers
```

Listing 3.3: Definition of the `example_interfaces/Fibonacci` action.

Core API organization

To give an idea of how an architecture based on ROS for a distributed autonomous system may work, it is instrumental to present briefly the organization of the core API of ROS 2, which is shown in Figure 3.10. The core API is the set of libraries and tools that implement the basic functionalities offered by the ROS 2 middleware, intuitively starting from the communication layer and going up to the services offered to the application layer. Below the core API lies the communication middleware layer, which hosts the selected implementation(s) of the communication middleware to employ. This layer is represented in Figure 3.10 by the blue blocks at the bottom. On top of it all, there are code and applications that use the services offered by ROS 2 and everything else above them: these levels are represented by the top gray block in Figure 3.10. The core API, laying in the middle, is divided into several modules, each of which is responsible for a specific aspect of the ROS 2 middleware. The most important levels are, from top to bottom, are the following.

- `rmw` **level**: the ROSMiddleWare level hosts a set of C libraries that abstract different implementations of communication middlewares like, *e.g.*, many DDS implementations and Zenoh, offering a common interface to upper levels to achieve basic communication functionalities.
- `rcl` **level**: the ROS Client Support Library level hosts a C library that implements basic entities like *nodes*, discussed below, *publishers*, and *subscribers*, service and action *clients* and *servers*, and *timers*.
- **ROS Client Library level**: at this level, various libraries are implemented to offer support for programming languages other than C, like Python and C++. The entities defined in the lower levels are extended here, adding language-specific features and abstractions. Moreover, another core entity is defined at this level: the *executor*, which manages the execution of ROS-related jobs in dedicated OS threads.

Nodes and executors

In ROS 2 applications, the most important entity is the *node*. Conceptually, a ROS 2 node is an operational unit of the distributed system capable of performing a specific task for which it has been designed. In the code, regardless of the programming language employed, a ROS 2 node is an object of a class that usually expands the base `Node` class of the specific client library used, adding features that are specific for the operations that the node has to perform. Primarily, and thanks to class members and methods they inherit, nodes act as access points towards the middleware that enable application code to use ROS 2 features. For example, a node can create publishers and subscribers to exchange messages, service servers or clients, and even action servers and clients. Nodes have *parameters*, which are variables that can be set from the outside to configure the behavior of the node using a very versatile API: a node parameter is identified by a name and a type and can be set at the startup of the node or at run time; in the context of an autonomous robot a ROS 2 node parameter may be, *e.g.*, a

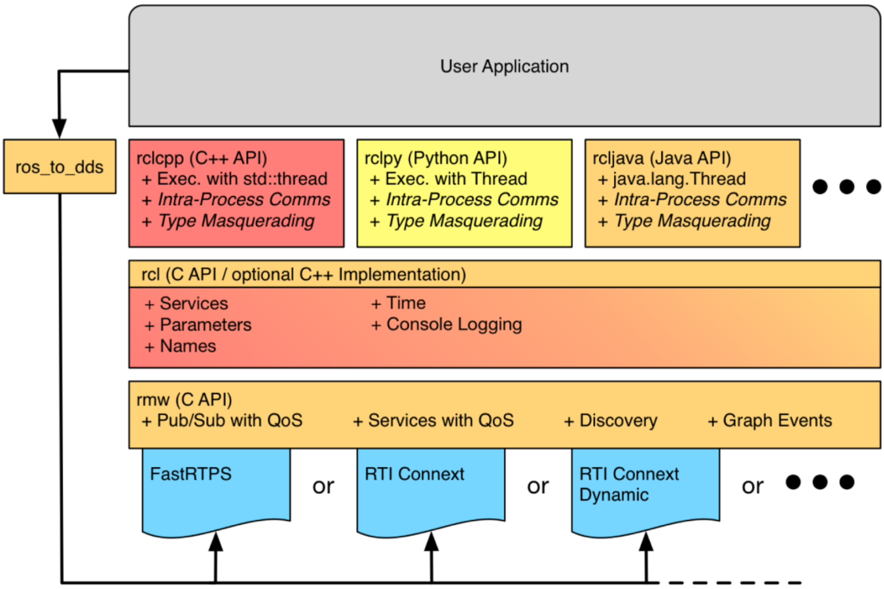

Figure 3.10: Core ROS 2 API organization [128].

gain for a control law being applied to the underlying physical system. Parameters can be queried and updated from both users and other software modules, *i.e.*, other nodes in the architecture, possibly deployed on different hosts in a distributed fashion.

The main idea is always that, to describe and design a distributed system, the node is a unit that performs computations that are aimed at performing a specific task, implementing a specific subsystem of the overall architecture. To achieve this, and to fully comply with the distributed paradigm even in real-time contexts, nodes generate computational *jobs* when specific events occur. Such events must be handled accordingly, in a way that is orchestrated by the middleware. The situation at hand is described in Figure 3.11, which shows the event-based execution model of ROS 2. In this model, aside from synchronous computations that are part of the user code, part of the processing is handled asynchronously when the following specific events occur:

- a subscriber receives one or more messages;
- a timer expires;
- a service request is received;
- an action goal or result request is received;
- a parameter is updated.

Apart from timers, the parameter subsystem is implemented using services but for the sake of clarity Figure 3.11 highlights the most important event groups. Each of those can be processed by a callback

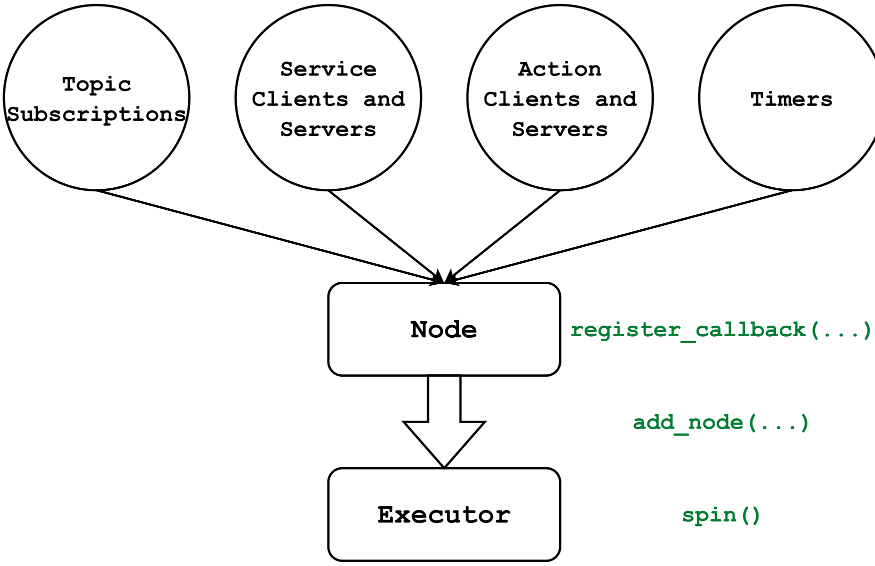

Figure 3.11: ROS 2 event-based execution model.

that is registered as part of the node configuration. Then, nodes carrying their asynchronous workload are added to an executor, which handles callbacks in groups over one or more OS threads; node that executors can handle multiple nodes at the same time. To set things in motion, hence the name, in any ROS 2 application the executor is then started by calling its `spin` method.

Over the years, the real-world performance of this paradigm has been thoroughly evaluated [20, 119, 141, 203]. What emerged, especially in the early versions of the framework, is that although conceptually correct it was affected by some design flaws. First of all, while in its ideal contexts the DDS performs well with little to no real-time overheads, the RMW abstraction introduces some unwanted latencies. Moreover, the ROS 2 executor, which is the entity that manages the execution of the asynchronous jobs, was not designed to be real-time capable: its algorithm lacks any notion of priority and it seems to favor timers over any other kind of event, to empirically maximize responsiveness. If one thinks that commercial robots with ROS stacks are not only available but employed in the industry since almost a decade, it is clear that the ROS 2 executor is not a showstopper: in many contexts in which ROS 2 is employed things work fine because that level of strict real-time requirements is not needed and as long as the system resources are not saturated both throughput and latencies remain acceptable even with such primitive algorithms. Nonetheless, the community has got the message: many solutions to these issues have been put in place and the core ROS 2 libraries listed in Section 3.1.3 are constantly updated and improved. The interested reader can find references to some of these solutions in [20], while a novel priority-based executor suited for real-time applications is presented in [222]. Finally, problems induced by the DDS, as discussed in Section 3.1.2, are addressed by the ongoing adoption of Zenoh as a communication middleware in ROS 2 [185], which is expected to be completed in the next few years. Another solution that deserves a mention is the *node composition* [128], which is similar to the way in which software modules are handled in the MARTe2 framework presented in Section 2.3.6. In ROS 2, C++ nodes can be compiled in two ways: either as parts of executables or as standalone shared libraries, denoted as *components*. Components can either be loaded and unloaded at run time, inside a process that hosts executors for their jobs acting as a *component container*, or they can be statically linked to an executable. The thing is that, in either of these ways, multiple nodes end up running inside the same process, thus sharing the same memory and address space. This allows to avoid the overhead of inter-process communication, completely bypassing the RMW layer and anything underneath, leading to very low latencies and high throughput. This is a very powerful feature that can be used to increase the overall efficiency of the architecture, as shown in [128], provided that nodes are stable enough to run in the same process: in case one of them crashes, the whole process is terminated, and all the other nodes are stopped. This is a trade-off that has to be be carefully evaluated when designing a distributed system based on ROS 2 and the usual way of running each node in its own process remains the suggested one for the early stages of development and testing.

Launch system

A ROS 2 executable, or a group of such, has to be launched using specific middleware commands to ensure that all necessary daemons and logging subsystems are activated. The middleware subsystem responsible for this is called the *launch system*, and the action of starting up multiple modules composing an architecture is called *launching*.

There are two ways to launch modules: either by using command-line tools to start each module individually, or by running scripts that specify the whole set of modules to run, together with their parameters and configuration options. These scripts are called *launch files* and they are usually added as part of any package that contains executable code, and to any architecture once all the modules have been collected, to start everything up with a single command. A launch file specifies, according to their format [186], the configuration with which the executable should be launched, including details ranging from input arguments to how logging should be managed.

Data recording and playback

For various purposes, such as testing or post-analysis, it may be necessary to capture all the messages that were handled by the middleware during a specific time interval and on particular topics. ROS 2 provides this service through a system called *bagging* [187]: it is possible to record messages sent on a given set of topics into a file, with configurable format and compression algorithms. This file, which also includes information about the message interfaces it contains, can be processed by other software to review the data. Bags can also be replayed from the beginning, creating ad-hoc publishers that re-publish the recorded messages at an original or different rate. This way, it is possible to develop, test, debug, and even tune algorithms offline, without the need to perform an experiment with the actual robot each time, just by replaying the recorded data, avoiding potentially risky or costly operations.

This feature is so popular that the vendors of many proprietary scientific software like, *e.g.*, MATLAB [157] or LabVIEW [174], have developed plugins to read and write ROS 2 bags and many open-source tools have been developed to parse and process them, some of which are part of the ROS 2 ecosystem themselves. However, this system offers no way to organize the data: there is no way to inspect the messages or to compare them across multiple bags without unwrapping the bag files first. Some ideas on how to address this issue, relying on established solutions such as MDSplus presented in Section 2.3.6, are outlined in Chapter 8.

Visualization and simulation

A complete installation of ROS 2 includes several useful tools for debugging, such as commands for inspecting active topics, examining their interfaces, measuring the publication rate, and launching publishers as needed. There are also software tools that are not required for the operation of a robot and, in fact, are superfluous in that context but extremely useful during the development phase. It is appropriate to mention at least the following ones.

- **Gazebo** [209]: this is an open-source simulator natively compatible with ROS, capable of interacting with middleware topics to send and receive data. It allows for the creation of 3D models of robots—complete with sensors and actuators—as well as the environments in which they operate using an XML-based syntax, and provides a platform to test control and perception algorithms. Gazebo offers a 3D visualization of the simulated world and it can be used to simulate the behavior of a robot in a virtual environment thanks to its embedded real-time physics engine. Moreover, thanks to its open-source nature, Gazebo can be extended with plugins that add new features or modify existing ones, like implement simulated sensors and scenario elements, and it can be integrated with other software tools like MATLAB [157], Simulink [158], or LabVIEW [174].

- **RViz** [190]: this is a 3D visualization tool that can be used to display the state of a robot and its environment, as well as the data that is exchanged among the various modules of the system. It is a very useful tool for debugging and testing, as it allows to visualize the data that the robot is processing in a way that is easy to understand and to interact with the system by sending commands and receiving feedback. RViz can be used to display the robot's position and orientation, the data from its sensors, its map of the environment, and the planned path, among many things. It can also be used to display the data exchanged among the various modules of the system, like the messages sent on the topics, the services called, and the actions performed. RViz can be used to visualize the data in 3D, 2D, or even in a tabular form, and it can be configured to display only the data that is relevant to the task at hand. As Gazebo, it is open-source and extensible with plugins that users can develop themselves.

An example of the capabilities of these two tools together is shown in Figure 3.12.

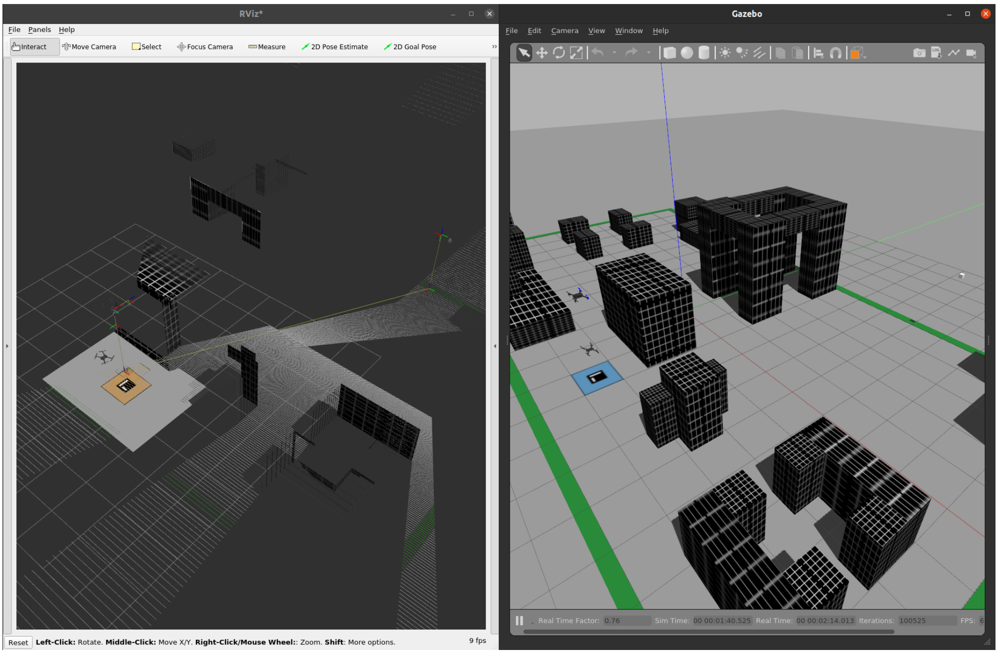

Figure 3.12: Simulation of an autonomous drone with imaging and depth sensors in Gazebo (right) and visualization of the data in RViz (left).

Applications and related frameworks

The ROS 2 ecosystem is vast and varied, and it is used in many different application areas, from industrial automation to service robotics, from autonomous vehicles to drones, from medical robotics to space exploration. The ROS 2 middleware is also used in many research projects, both in academia and in the industry, and it is the basis for many open-source software libraries and tools that are widely used in the robotics community. Its features, discussed so far, clearly defined its relevance in the context of the present work, and it is included in the proposed architecture as illustrated in the following chapters. To conclude the presentation of ROS 2, some further references are in order to clarify its diffusion in both the research and industrial communities, as well as the variety of projects that are based on it.

One of the areas in which ROS has historically been employed is that of mobile, terrestrial robotics. In this field, there are a variety of problems to solve, ranging from localization to mapping, from path planning to obstacle avoidance, from perception to manipulation. One of the most famous projects related to this setting is Nav2 [125, 127, 129, 165, 184], the official ROS navigation stack for mobile robots. This package, originally developed to expose APIs for autonomous movement of mobile, planar robots, has evolved into a vast framework covering nearly every aspect of autonomous navigation: environment mapping and reconstruction, path planning and smoothing, obstacle avoidance, localization, and even human-robot interaction. A variety of algorithms for each case is available, and the system is designed to be modular and extensible in each of its parts, including visualization plugins for RViz and simulation elements for Gazebo. Operations carried out by the many modules of the framework are orchestrated by automata such as behavior trees [107].

Another area that robots revolutionized is that of industrial automation, in which manipulators of different sizes and shapes are employed to perform repetitive tasks with high precision and reliability. In this context, ROS 2 is used to control the robots and to interface them with other systems, like sensors and actuators, and to manage the data that is exchanged among them. One of the most famous projects in this field is MoveIt [30, 46, 98, 200], the official ROS manipulation stack for industrial robots. As Nav2 for mobile robotics, MoveIt is very large in terms of the functionalities that it offers, all of which are built on ROS 2 tools and libraries. First, it allows the developer to define a physical and kinematic model of the robot using the Unified Robot Description Format (URDF), which is a particular kind of XML file in which the geometrical and dynamic properties of a robot can be specified: masses, inertias, joints, links, and even sensors. Then, it provides extensible software modules to interface with the robot hardware, which is usually made of custom microcontrollers with their own communication protocols and semantics. Finally, it allows the specification of states and movement operations thanks to many path planning and tracking algorithms and it offers plugins to turn the RViz visualizer into a Graphical User Interface (GUI) for the robot.

One setting which is still relatively young is that of aerial robotics. Autonomous drones and aircraft can be used in a variety of applications thanks to their ability to fly, from surveillance to search and rescue, from agriculture to entertainment. The capabilities of the ROS 2 framework in this context are still being explored by the robotics community, particularly because a flying platform is more complex to control than a terrestrial one, and because the environment in which it operates is more dynamic and less predictable. Some results which led to the development of this work and can be considered its embryonic stage are presented in Section 5.3 [22, 23].

Finally, it is important to asses the kind of hardware platforms on which ROS 2 has been designed to run. If one recalls the discussions held in the previous sections, it appears clear that the lowest tier of hardware on which ROS 2-based software can run is a Single-Board Computer (SBC), which is usually the highest grade of hardware that can be found on autonomous robots. However, as previously remarked, robots are sophisticated systems that usually also include specialized hardware like micro-controllers to drive both sensors and actuators. It is desirable to be able to organize the code for such devices in a similar way to that based on ROS 2, even leveraging similar communication paradigms and abstractions. This is the scope of the micro-ROS project [75]. Developed by eProsima, one of the major developers of DDS implementations, micro-ROS is a reduced software stack implemented mostly in C, which offers a subset of the core ROS 2 functionalities with a minimal memory footprint and computational overhead. Its software stack, shown in Figure 3.13, is designed to run on a variety of Real-Time Operating Systems (RTOS) supporting their task scheduling and memory allocation API and it is developed to preallocate or statically allocate memory to avoid dynamic memory allocation and deallocation. The micro-ROS stack is designed to be used in conjunction with the full ROS 2 stack, exchanging data over topics in both directions thanks to a bridge put in place by the Micro XRCE-DDS [74] implementation, composed by an agent application running on a PC or SBC which

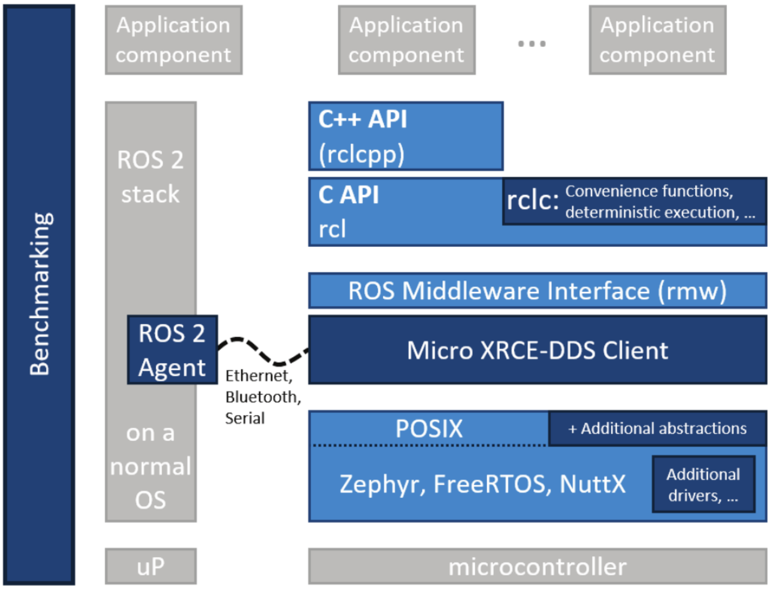

Figure 3.13: micro-ROS architecture [75].

talks with a client running on the microcontroller. Supported physical links include Ethernet, Wi-Fi, and serial connections. As every solution presented up to this point, micro-ROS is open-source and its community is growing as is the number of experimental evaluations, projects, and supported microcontrollers like, *e.g.*, the STM32 series from STMicroelectronics [226] or the ESP32 series from Espressif Systems [78].

3.2 Containerization

Recalling the Modularity (M) requirement, suppose that a software module requires external dependencies, *e.g.*, libraries or tools to be built and run. These external dependencies may not be made part of the module sources but have to be set up in the systems that host it. Setting up a development and execution environment addressing situations like this is a common but usually also quite complex task. Multiplying the complexities by the number of different hardware platforms that the software module has to run on, one may get a set of problems that are hard to manage. This is the motivation of the Abstraction (A) requirement: the architecture should provide a way to abstract the software modules from the hardware platforms they run on, to allow one to develop software that is portable across different platforms and to simplify the setup of the development and execution environments. Historically, the technology that allowed this kind of abstraction is called *virtualization*, which in general comes at the price of a significant overhead in terms of computational resources and performance. Preventing this is the reason why, in Section 2.2, the Overhead (O) requirement is also stated, which requires the architecture to be as lightweight as possible to avoid any unnecessary computational overheads that may affect the performance of the software modules. The technology that addresses all these requirements, forming a base layer of the proposed architecture, is called *containerization*.

Containerization represents a transformative approach to software deployment that addresses fundamental challenges in application portability and environmental consistency [197], while remaining distinct from traditional virtualization technologies [64]. While virtualization employs hypervisors to create virtual machines with independent operating systems, containerization achieves isolation by leveraging the host operating system kernel to establish separate environments that share the underlying OS. This architectural difference results in containers exhibiting substantially lower overhead compared to virtual machines, enabling higher resource density and improved efficiency. The technology effectively resolves the challenges of inconsistent behavior across deployment environments by bundling applications with their dependencies, ensuring uniform operation across diverse platforms. This capability proves particularly valuable in distributed software architectures where deployment consistency and scalability are crucial for operational success, while the reduced resource overhead makes containerization an optimal choice for scenarios requiring isolated environments without necessitating complete operating system separation.

The origins of containerization can be traced back to early innovations in Unix-like operating systems. In the early 2000s, technologies such as FreeBSD Jails (2000) and Solaris Containers (2004) introduced the concept of partitioning a single operating system's resources into distinct, isolated instances. FreeBSD Jails enabled administrators to create secure, isolated environments capable of independently running services, while Solaris Containers provided an integrated method for managing multiple isolated application environments on a single instance of Solaris. Concurrently, the Linux ecosystem began to develop foundational features that later form the bedrock of modern containerization technologies. Notably, Linux introduced kernel features such as cgroups (control groups) and namespaces—both essential to containerization's core principles of isolation and resource control. Linux namespaces provide process isolation, determining the visibility of system components like the filesystem, network interfaces, and other processes. In parallel, cgroups regulate the allocation of hardware resources—such as CPU, memory, and I/O—ensuring that containers do not monopolize or starve system resources, thus enabling fair resource sharing and predictable performance.

These features enabled the creation of isolated environments while maintaining efficient resource sharing, ultimately leading to the development of contemporary *container engines*. The efficiency of containerization manifests in reduced overhead compared to virtual machines, as containers eliminate the need for multiple operating systems. This characteristic makes containers particularly suitable for microservices architectures, where applications are decomposed into independently deployable components. The container architecture ensures consistent operation across different infrastructure configurations while allowing potentially conflicting dependencies to coexist without interference.

The operational distinctions between virtualization and containerization significantly influence their respective application domains. Virtualization maintains relevance in scenarios requiring complete operating system isolation, particularly in multi-tenant environments where security and compliance demands necessitate strict workload separation. However, in contexts where such comprehensive

Figure 3.14: Docker logo [60].

isolation proves unnecessary, containerization offers notable advantages in scalability, deployment speed, and resource efficiency. The lightweight nature of containers facilitates rapid provisioning and horizontal scaling, making them particularly advantageous for cloud-native applications and environments requiring dynamic resource allocation.

In the context of this work, containerization can effectively help to build a common architectural ground that abstracts the software modules from the platforms they run on, at the same time packaging with them every external and third-party dependency they require to be built and run. This way, the software modules can be developed and tested in a controlled environment, and then deployed on any platform that supports the containerization technology, a condition that is not restrictive since today container engines are supported on nearly every commercial platform that the proposed solution may target. For this reason, modules and projects based on the proposed architecture are managed through containers: the technical details of this process are postponed to the next chapters, where the architecture is described in detail. To conclude this dissertation, the chosen containerization technology is illustrated in the following section.

3.2.1 Docker Engine

The release of Docker [60] in 2013 was a watershed moment in the evolution and widespread adoption of containerization [163, 238]. Docker unified the previously fragmented ecosystem of container technologies, incorporating existing Linux kernel features into a cohesive, user-friendly tool that simplified the process of container creation, deployment, and management. Docker's innovation lays in its ability to abstract the underlying complexity of containerization, providing a standardized interface and image format that made the technology accessible to a broader audience of developers and IT professionals. This accessibility facilitated a shift in how software is developed, tested, and deployed.

Docker Hub [61], an associated public *registry* of container *images*, further accelerated adoption by providing a central repository from which developers share and reuse container images, promoting community collaboration and reducing redundancy. Moreover, since its birth as a Linux-only tool, Docker has evolved to support also the Windows and macOS operating systems through its Docker

Desktop release [59], although with a limited set of features available on these platforms, which also require a thin virtualization layer to work. This revolution has also impacted the research world outside of the cloud computing and software development industries, as it has been adopted in many scientific fields to manage the software tools and libraries that are used to analyze data and run simulations, as well as to distribute the results of the research and novel algorithms and procedures in a way that ensures reproducibility, transparency, and accessibility [28].

The first key component of Docker's utility is the Dockerfile, which serves as the blueprint for creating Docker images. A Dockerfile is a simple text file that contains a series of instructions on how to assemble a particular Docker image. These instructions include the base image to use, any software dependencies to be installed, environment variables, and commands to be executed when building the image. When a Dockerfile is processed by the Docker engine, it creates a Docker image, which is a read-only template containing the application and its environment. Docker images can then be instantiated as running containers.

Another fundamental feature of Docker is its use of union filesystems [65], which are specialized filesystems that merge multiple layers into a single coherent view. Union filesystems are particularly useful in containerization as they allow multiple layers to be stacked, where each layer represents a set of *filesystem changes*. This approach enables efficient storage as new layers can be added or modified without altering the underlying base layers, thereby saving storage space and allowing for rapid image creation and reuse. Specifically, Docker employs OverlayFS by default to create image layers [168]. Docker images are built in layers, with each layer representing a set of filesystem changes such as installing a package or adding a configuration file. The union filesystem allows Docker to stack these layers on top of each other, presenting them as a single coherent filesystem. This approach provides significant efficiencies in both storage and resource usage, as multiple images can share the same underlying layers. For example, if multiple images are based on the same base operating system image, Docker stores only one copy of that base layer. OverlayFS, for example, enables efficient management of these layers by allowing new changes to be made in an uppermost layer without altering the lower layers, enforcing a copy-on-write mechanism (a paradigm according to which stored data is shared among multiple entities keeping a single copy, until one of such entities has to modify it). This layered approach not only saves disk space but also accelerates the building of new images by reusing existing layers.

Docker Compose [62] is another powerful tool within the Docker ecosystem that facilitates the management of multi-container applications. With Docker Compose, developers can define and run multi-container Docker applications using a YAML file (Yet Another Markup Language, a popular markup language for configuration and text files) to configure the services, networks, and volumes required by the application. This capability is particularly useful for orchestrating the deployment of complex applications that require multiple components—such as databases, backend services, and front-end web servers—to work together seamlessly. Docker Compose allows these components to be defined as individual services, each in its own container, and specifies how they interact. This simplifies the process of setting up development environments and ensures that the entire application stack can be easily replicated and deployed.

Another crucial feature of Docker are *volumes* and *volume mounts*, which are of paramount importance for data persistence. Containers are ephemeral by nature: when a container is stopped or deleted, all data within it is lost. To overcome this limitation, Docker uses volumes, which are directories or files stored outside the container's writable layer. Volumes can be mounted into one or more containers, allowing data to persist even if a container is removed or updated. This capability is essential for stateful applications, such as databases, which require durable storage that outlives the individual container instances. By using volume mounts, developers can also share data between containers, enabling efficient communication and data exchange between different parts of an application or of a distributed software architecture.

Networking is another key area where Docker provides significant capabilities, enabling containers to communicate with each other and with the outside world. The networking model of Docker includes several types of networks hereby only simply mentioned such as *bridge*, *host*, and *overlay*, each suited to different use cases. Bridge networks are the default and are typically used to enable communication between containers running on the same host. Host networking, on the other hand, allows a container to share the network stack of the host, providing greater network performance and simplifying access to services running on the host. This mode is particularly useful in scenarios where the containerized service needs to avoid the overhead of NAT (Network Address Translation) and requires direct access to the network interfaces available on the host.

Docker also provides mechanisms to control the resources allocated to containers, such as CPU and RAM using, *e.g.*, cgroups on Linux. Resource limiting is critical to ensure that no single container can consume disproportionate system resources, which negatively affects other containers running on the same host. Docker allows users to specify limits on the CPU shares and memory usage for each container, ensuring predictable performance and efficient utilization of system resources. By configuring these limits, administrators can create a balanced environment where all containers receive their fair share of resources, preventing resource contention and ensuring stable operation.

Finally, Docker can use the Linux capabilities subsystem to manage hardware and system call access in a granular manner. Instead of giving containers full root privileges, which poses security risks, Docker allows specific capabilities to be granted as needed. For instance, a container may need the capability to bind to low-numbered network ports or change the system time but it does not need broader privileges that further compromise the host. By selectively enabling only the necessary capabilities, Docker minimizes the attack surface of containers, enhancing security while maintaining the required functionality for each application.

This section presented and described the Docker Engine containerization solution alongside many of its features. The choice of the functionalities to discuss here was not coincidental: each of the features described is essential to the architecture and to fully employ Docker as the solution that addresses the Abstraction (A) requirement. Thanks to the features outlined so far, applications that run inside Docker containers may do so within a simple partition of the host operating system, thus with no additional overhead with respect to, *e.g.*, applications that run on the same system but outside containers. Some configuration on the host is required to achieve this, as the next chapters illustrate, but once everything works this constitutes also a solution to the Overhead (O) requirement. The Docker Engine is used in this work to build and manage hardware and system abstraction layers that establish a common environment for both the development and the execution of software modules and projects among many different platforms.

3.3 Version control

Recall the Modularity (M) and the Development (D) requirements. In essence, the architecture has to be modular in the sense that it has to enable the development of software modules that can be easily maintained as standalone units but at the same time it should provide a straightforward integration process when a larger-scale project, like a prototype, requires to set up many of these modules at the same time. Furthermore, when working in any of these two situations, the architecture should provide easy means of maintaining the single units composing a larger project and of tracking the changes made to them. Moreover, it should be possible to issue updates to these modules and propagate them to other projects that integrate the same modules, in case they do not require a specific version of them. The technology that addresses these requirements is called *version control*.

Version control [249] is an essential element of contemporary software design and development, serving as a systematic mechanism for managing changes to source code, documentation, and other artifacts associated with a project over time. Version control involves the practice of tracking and managing modifications to software artifacts, thereby allowing developers to maintain a comprehensive history of changes, collaborate efficiently, and ensure consistent software quality. Version control systems (VCSs) enable concurrent development by multiple contributors without conflicts, thus mitigating issues such as overwritten code and conflicting updates. Furthermore, VCS facilitate the exploration of new features or bug fixes through branching, which isolates development efforts into independent streams, and merging, which integrates these branches into a cohesive codebase. This adaptability is crucial for modern software teams, who have to concurrently manage multiple development lines, such as feature development, bug resolution, and experimental changes.

For these reasons, it is clear that the adoption of a version control system as a basic element of the architecture provides the means to satisfy the (M) and (D) requirements: as for the Modularity (M) requirement, the version control system enables to maintain the software ranging from modules to larger projects as standalone repositories under version control, eventually linked among each other. Furthermore, thanks to the branching and merging capabilities of the version control system, the development process is streamlined and the integration of the modules into a larger project is facilitated. The version control system also provides the means to track the changes made to the software, to issue updates to the modules, and to propagate them to other projects that integrate them, thus satisfying the Development (D) requirement. The inner workings of these processes are described in the following section, which introduces the VCS of choice for the proposed architecture.

Figure 3.15: Git logo [236].

3.3.1 Git

Git [236] is one of the most influential version control systems in contemporary software development [25, 124], recognized for its robustness, efficiency, and distributed architecture. Its inception dates back to 2005 when Linus Torvalds, the creator of the Linux operating system [237], sought an effective and flexible version control system for managing the Linux kernel's development. At that time, the Linux kernel project was experiencing significant growth, with thousands of developers from around the globe contributing code. Initially, the project relied on a proprietary version control system called BitKeeper which, although effective, became unsuitable for the open-source nature of the kernel due to licensing conflicts. This prompted Torvalds to develop Git, a system specifically tailored to meet the unique requirements of the kernel project, particularly distributed collaboration, speed, and support for non-linear workflows. There is no official explanation of its name but some well-known theories suggest that it may be a play on the word "get" or some underground slang term in an unidentified language meaning "simple" or "stupid", also since Torvalds himself refers to it as *the stupid content tracker* [236].

The development of Git emerged from specific requirements identified during the Linux kernel development process, with an emphasis on supporting distributed workflows and non-linear development patterns. The system architecture reflects these requirements through a distributed model that enables each developer to maintain a complete repository clone, including the full history of changes [212]. This approach differs fundamentally from centralized version control systems by eliminating dependency on a central repository, thus enhancing resilience and enabling offline development capabilities. The distributed nature of Git serves multiple critical functions: it provides safeguards against data loss, facilitates independent experimentation without compromising project integrity, and enables efficient handling of large-scale contributions. In the context of open-source development, where contributors operate across different geographical locations and time zones, this architecture promotes autonomous progress while maintaining synchronization capabilities. The system supports peer-to-peer collaboration through direct exchange of changes between developers, creating a robust and scalable development workflow that accommodates the extensive branching and merging operations characteristic of large-scale software projects.

Git submodules represent an advanced feature that enables repositories to incorporate and track content from other repositories, facilitating complex dependency management in large-scale software projects. This capability proves particularly valuable when different components of a system undergo independent development cycles, as submodules allow teams to maintain specific versions of external libraries while ensuring compatibility and stability. The modular approach enables distinct teams to work autonomously on separate components while preserving overall project coherence through controlled integration mechanisms. Each submodule links to a specific commit in its respective repository, providing precise version control and enabling deliberate, tested updates that maintain system stability. This architecture extends beyond basic dependency management to create an efficient development ecosystem, as submodules facilitate code reuse across projects without duplication while ensuring that all components maintain version-specific compatibility. The result is a scalable development process that balances component autonomy with system-wide consistency, particularly beneficial in projects involving multiple dependencies and development teams.

A similar and complementary feature to submodules is that of subtrees. Subtrees and submodules are both mechanisms for managing dependencies in a Git repository but they offer distinct workflows and advantages. Subtrees allow developers to embed the content of an external repository into a subdirectory of the main repository. Unlike submodules, subtrees do not require additional configuration or extra repositories to be manually initialized when cloning the main repository. This makes subtrees more straightforward for new collaborators, as the embedded content becomes a natural part of the repository's history and structure, without any extra setup steps. On the other hand, submodules are more suited for maintaining clear separations between distinct modules or components that need to evolve independently while remaining linked to a specific version in the main project.

Given the many functional features of Git mentioned above, the design of the proposed architecture starts from the organization of every work it supports as a Git repository, as illustrated in the following chapters. The architecture also includes support for both submodules and subtrees as they are both useful in different contexts and for different purposes. While submodules are suited to allow for the combined integration and development of single software units into larger projects, they require a more complex setup and management that is not always necessary. Subtrees, on the other hand, are more straightforward to use and are more suited in scenarios in which it is not expected that active development on the embedded content is carried out but rather that it is used as a dependency and eventually only updated to newer versions. The architecture is designed to support both mechanisms.

4 The proposed Distributed Unified Architecture

This chapter presents the core contribution of this work. The introductory chapters defined the problem setting, outlining a situation in which distributed autonomous systems are becoming ubiquitous in many aspects of everyday life but at the same time increasingly complex to develop and manage. A versatile framework to work on such systems is needed but once some specific yet broad requirements are stated one finds that the current state of the art does not provide suitable solutions. Still, if the trends from the last decade in software engineering and computer science are examined, one can find that many tools have been developed that address specific portions of the original problem.

Recalling once again the leading principle of this entire work, it is now possible to build a new type of wheel without starting completely from scratch: some good tools are ready at hand but most importantly some pieces are already available, whereas some others are still missing. However, it is important to further clarify the actual setting in which this work has been developed, in particular with respect to the Abstraction (A) requirement from Section 2.2 and to supported operating systems: the scope of the design process has been to build a framework that can be adapted to many target platforms but naturally, due to availability and time constraints, the actual implementation at the time of writing has been tested on a limited yet representative set of them. This process has been driven by the need to provide a proof of concept for the framework and, at the same time, to demonstrate its feasibility and effectiveness in real-world scenarios over the years in which this work has been carried out. Nonetheless, the chosen set of target platforms is broad enough that the resulting framework works on all of them and can be adapted, with minimal effort and specific changes, to other platforms as well, basically repeating the same setup process that has been followed for the current set. Furthermore, as clarified in the following, other assumptions stand regarding the configuration of the host operating system: for simplicity and to benefit from the existing solutions presented previously, the framework has been designed to work on systems that support the solutions described in Sections 3.2 and 3.3. Having defined this starting point, the framework follows by integrating what is ready and then developing what is missing, to first provide a system abstraction layer as a solid foundation and then to build an architectural template that can be used to develop and deploy software modules. Additional tools can subsequently be developed and integrated to further enhance the framework, addressing issues that are specific to some use cases like, *e.g.*, the ones described in subsequent chapters.

The organization of this chapter reflects this description, which is nothing but the recollection of the design and development process that led to the *Distributed Unified Architecture* (DUA). After the discussion of the implementation of the framework, some concluding remarks are provided also with respect to the other solutions presented in Section 2.3.

4.1 Assumptions

Before delving into the details of the system abstraction layer offered by DUA, it is instrumental to describe the host system configuration that it expects. Recalling what has been said at the beginning of this chapter, the following brief set of assumptions holds mainly for the sake of simplicity: satisfying them allows to employ solutions that are readily available, represent current industry standards in their respective fields, and help to solve significant portions of the problem at hand. What follows from them constitutes a working proof of concept but nothing prevents the framework from being adapted to different configurations should any requirement change, or should the need arise to support different host system configurations.

For clarity, the following definition is in order here: the term *system configuration*, in this context, refers to a combination of hardware and software that identifies a specific class of systems; each class shall support a defined set of functionalities, hardware components, and operating systems. When a *host system configuration* is mentioned, it refers to a system configuration on which Docker is also present and on which containers are run, *i.e.*, not the one that is found inside containers built and managed by DUA.

Hardware platform type The framework is designed to work on systems that range from single-board computers to more powerful general-purpose computers of any architecture, provided that they support the solutions described in Sections 3.2 and 3.3. This assumption excludes lower-end systems like, *e.g.*, microcontrollers, which do play important roles in the field of autonomous systems. This choice has been motivated by two facts: first, microcontrollers and similar systems usually have very limited and constrained sets of resources, which make them incompatible with the software solutions required as of now to build and manage the proposed framework. Secondly, the variety of microcontroller architectures available today is so vast and each of them is so different from the others that extending the proposed approach to this class of systems requires a significant effort. This is not to say that the proposed framework can not be adapted to work with microcontrollers but that is left as future work since their integration requires a dedicated design process.

Support for version control The host system should support a version control system, as described in Section 3.3. This is a very important requirement, as the framework relies on version control to manage the software modules that are developed and deployed on the components of the distributed system. The choice of Git as the version control system is motivated by its many features that make it highly efficient when working with projects that embed many modules, as discussed in Section 3.3.1.

Support for containerization The host system should support containerization, as described in Section 3.2. This is another very important requirement, as the framework relies on containerization to isolate and manage software components. The choice of Docker as the containerization solution is motivated by its widespread adoption and the functionalities that it offers, enabling it to build replicable environments on a multitude of platforms, as described in Section 3.2.1.

Operating system support The framework is designed to natively work on the Linux operating system, as it is the most widely used operating system in the field of distributed autonomous systems. This choice is motivated by the fact that Linux, in its many distributions, offers the widest support for hardware devices and software tools. Moreover, it is built to be developed onto, being natively open-source and offering a stable and fully equipped environment right from the start for the development of software of any kind, especially that which is intended to directly access any hardware resource connected to the host machine. Furthermore, Linux is the operating system that best supports both version control and containerization solutions, whose importance in this project has been highlighted in the previous points. For these same reasons, when an SBC is released to the market, it happens frequently that its natively supported operating system is a Linux distribution, so to offer a consistent environment across all platforms supported by DUA it is natural to assume that the host configurations share this level of similarity. Among the many possible Linux distributions to choose from this work does not impose any specific one, but Ubuntu Linux in one of its latest Long-Term Support (LTS) versions available at the time of writing (22.04, 24.04) has been employed for the development and testing of the framework and is used as the reference for the system abstraction layer described later on. Again, this choice is motivated by similar reasons: Ubuntu is the most popular and easy-to-use Linux distribution and it is widely supported by many software tools and packages as well as by many SBC vendors. As in any other such case, the use of a different distribution is possible provided that one manages to obtain a working environment with Git and Docker. To conclude, note that this assumption is not a strict requirement since the framework can be adapted to work also on Windows and macOS operating systems thanks to Docker but it is recommended to use a Linux distribution since some features are not supported on those OSes due to their different support for containerization which, as explained in Section 3.2.1, always involves a layer of virtualization. Such features are presented and described in the following sections together with their role in the proposed framework but for now, since they mainly concern containerization and map to specific Docker functionalities, a summary table is provided in Table 4.1 in which each feature appears with the name of the corresponding Docker functionality. For completeness, note that while macOS offers a good support to containers running a Linux-based image thanks to

Docker Desktop, the best way to run them on Windows is currently to use Docker Desktop in conjunction with the Windows Subsystem for Linux (WSL) [167].

	Linux	Windows	macOS
Volume mounts	✓	✓	✓
Host networking	✓	×	×
Graphical applications	✓	✓	×
Access to hardware devices	✓	×	×
Capabilities	✓	×	×
IPC namespaces	✓	×	×

Table 4.1: Docker features relevant for the DUA framework and their support on different operating systems.

4.2 System abstraction layer: dua-foundation

The most important problem solved by DUA is also the first that it addresses: providing a similar environment across multiple system configurations. Achieving this is not a trivial task: the hardware platforms and operating systems that are of interest are very different from each other; yet, the aim is to provide an environment that offers the same OS API, libraries, and tools across all supported platforms. Nonetheless, DUA operates within the limits of what is possible without unnecessarily increasing any overhead: some aspects of the underlying hardware are not abstracted on purpose since it is not necessary to provide a consistent experience. This is the case of, *e.g.*, the hardware architecture: as illustrated in the following, currently supported platforms include solutions based on both the x86-64 and ARM64 architectures and the framework is designed to work on both of them without hiding them or their differences under an abstraction layer. This is due to multiple factors: first, there is no practical necessity to do so, and secondly, the developer using the framework can benefit of the peculiarities and specific features of each platform on which they want to deploy their software. Moreover, abstracting the hardware architecture implies using hardware emulators such as QEMU [21], which are very powerful but also introduce significant overhead in the execution of any application, they are not always available on any platform, and they complicate access to system resources such as hardware devices which are instead of paramount importance in the practical contexts of this work, as shown in Table 4.1.

This first part of DUA is called `dua-foundation` [144] and is in essence a collection of Dockerfiles. The basic intuition is that environments defined by a set of OS API and software tools can be easily replicated and managed in the form of Docker images, as described in Section 3.2.1. Building a Docker image tailored to every supported platform allows the developer to account for the specific characteristics of each one, providing a consistent system configuration inside containers that use such images. Each researcher or developer, *i.e.*, each *user* of the DUA framework, has then to add their own software dependencies and packages to work on their specific modules. A user-developed software module based on DUA is termed a *unit*. To define the system configuration that a unit requires, the

Docker image is still the best tool available for the same reasons. For the sake of consistency along the whole chain, Docker images related to units have to start from the same set of base images as explained in Section 3.2.1, which are the ones built from the Dockerfiles found in `dua-foundation`; the specific one is chosen depending on the hardware platform to work on. Let us then refer to images built from `dua-foundation` as to *base units*.

The way a base unit is created differs even significantly for each but the setup process they follow is the same for all. They all start from a platform-specific base image available on public registries like Docker Hub. Then, this image is enriched by a set of system packages which represent common dependencies like, *e.g.*, compilation toolchains. Finally, some more libraries are installed that are specific to the currently evaluated application areas of DUA: even if they may not be required by all units, they are included in the base units due to the frequency of their use in real-world scenarios, *i.e.*, because it is very likely to find different units that require them. At the end of the build process, each image is pushed on Docker Hub [143] to be available to all users of the framework.

To conclude, note that each supported hardware platform has its own base unit mainly to adapt the abstraction layer to the specific characteristics of the platform but also to leverage and expose to the user the specific features of the platform itself. In fact, the aim of DUA base units is to provide *similar* but not *equivalent* environments: it is clarified in the following, when the currently supported platforms are presented, that some have specific resources that others lack. The framework is designed to work on all of them but it is also designed to take advantage of the specific features of each one and to expose those to the user in a consistent way. This is the reason why the setup process for each base unit is made of the same steps but performed differently each time and why the resulting images are different from each other. It is then up to the user to develop their units on top of the base units that offer the best environment and features for their specific needs, depending on the specific requirements of the software contained in the unit.

The next sections explain the contents of `dua-foundation`: first, the set of software dependencies to be installed in every base unit is discussed, to describe what is the final system configuration achieved by every base unit. Then, currently supported hardware platforms are briefly illustrated, to show the variety of hardware that DUA integrates but also to provide a first hint of the differences that the framework deals with. Finally, currently available base units are explained in more detail, together with the setup process that they follow.

4.2.1 Common software dependencies

Providing here a full list of common software dependencies that are provided in every base unit is a daunting task. The interested reader can directly inspect the Dockerfiles in [144]. However, to give a general idea of the environment that base units provide, as well as to facilitate the inspection of their Dockerfiles, a summary of the most important software dependencies is provided here.

Each base unit starts from a set of system packages, that is, software packages that can be installed using the package manager of the Linux distribution that the base unit is built on, and provide basic functionalities. Since the distribution of choice for every base image is Ubuntu Linux, the package manager is `apt`; the choice of Ubuntu as first layer of every base unit is motivated by the same reasons given in Section 4.1 and by the fact that an image containing a basic Ubuntu environment is relatively small in size, currently around 30 MB [34]. The first set of installed packages includes a GCC compilation toolchain [86], a Python system environment [240], a Java development kit [196], shell utilities, and a set of libraries that are required in the subsequent steps and that may depend on the specific base unit as explained later on. These are installed now, all in a single command, to reduce the number of layers in the Docker image and improve its size, a well-established practice when building Docker images.

Next comes the middleware that is specific to the current main application areas of DUA, which are also subjects of this work and discussed in the next chapters. Usually, the first one to be installed is the ROS 2 middleware presented in Section 3.1.3. The amount of ROS 2 packages installed depends on the target platform of a base unit: if the target is a development PC, a wide variety of packages is installed including simulators and visualizers, whereas if the target is an SBC, a reduced set of packages is installed skipping, *e.g.*, visualization tools. Alongside a ROS 2 installation, DUA base units include both the eProsima Fast DDS [72] and the Eclipse Cyclone DDS [69] implementations of the DDS middleware presented in Section 3.1.1, as well as the Zenoh middleware illustrated in Section 3.1.2.

Then is the time for the OpenCV computer vision library [54, 195] and the Eigen linear algebra library [109]. The former is currently the de-facto standard in open-source computer vision, originally developed by Intel in the late 90s and then expanded by a vast team of developers, it now offers a wide range of functionalities and the implementation of thousands of algorithms ranging from image processing to mathematical operations. Eigen is a C++ template library for linear algebra, which is widely used in the field of robotics and computer vision for its efficiency and ease of use. Finally, the Gazebo simulator presented in Section 3.1.3 is installed in base units for development systems.

The setup of a base unit concludes with specific tools that vary from time to time, depending on the evolution of application scenarios and supported platforms. Among these, the most important is currently the `dua-utils` collection, which is part of DUA and is illustrated in Section 5.1. Their position at the end of the setup process is intended: these are the last stages of each Dockerfile contained in [144], hence they correspond to the last layers in the image. This is done to easily change these stages in case some tools are not needed or some new ones are to be added, without having to rebuild the whole image from scratch but only the last layers. This is another common practice in Docker image building, made possible by the ability of Docker to cache image layers among subsequent builds and it is used here to reduce the time required to build a base unit.

4.2.2 Currently supported hardware platforms

To develop a framework that is applicable to a wide variety of practical cases, a good variety of hardware platforms has been considered for the initial implementation. The goal of this section is to present them in due detail, to show the variety of hardware that DUA integrates but also provide a first hint of the differences that the framework deals with. The categorization of the platforms made here is based on their system resources and intended use: it is not a strict classification but a way to group them to better understand the differences among them. Note that, for simplicity, only 64-bit architectures are currently supported by DUA, as they have become the standard in the last decade and are now common in the microprocessor market. A real example is illustrated for almost all categories: such example items have also been used to test the framework and implement real autonomous systems prototypes during the development of this work. At the end of this section, Table 4.2 summarizes the most important system specifications of the example items, while some of the platforms that are supported by DUA, either fully or with some limitations as discussed above, are illustrated in Figure 4.1.

General-purpose PCs

In the context of this work, this is the simplest category to address. The first base units, as well as the other pieces of the DUA framework, have been put together and tested on this kind of platforms to ensure that the main concepts work. The expression *general-purpose PC* denotes here a computer that is not specifically designed for a particular task but that can be used for a wide variety of applications. This category includes desktop computers, laptops, and servers. In this context, it is assumed that among this category of systems, the ones that are of interest for this work are based on either the x86-64 or the ARM64 hardware architectures. In the practical and applicative contexts relevant to the DUA framework, these systems are used for development and testing purposes, as well as for running simulations and visualizations, or intensive real-time computations. The software tools that are used on these systems are the same as the ones that are used on the other platforms but the hardware resources available on them are usually more powerful: many CPU cores and much RAM and storage space are generally available. Dedicated coprocessors like, *e.g.*, GPUs (Graphic Processing Unit) may also be available on these platforms.

Single-board computers

In the field of autonomous systems and especially in robotics, SBCs are very common. They are used in many applications, from small robots to drones, and they are also used in many research projects as fast prototyping platforms. The main reason for their popularity is that they are very cheap, small, and easy to use and integrate in larger systems, also thanks to their multiple communication interfaces. They are also very versatile, as they can be used for many different tasks. In the context of this work, SBCs are intended to be used to run the software modules that make an autonomous system perform its tasks. The hardware resources available here are generally more limited than the previous case and GPUs and similar dedicated solutions may also not be available. This is also what makes SBCs so versatile in this field: their variety allows the system designer to choose the best solution that fits all the requirements, from software support, to mechanical constraints and energy consumption. In this work and for the explanatory purposes of this section, three classes of single-board computers are considered, making broad distinctions based on their hardware architecture and intended use.

The first class consists of single-board computers based on the ARM64 architecture. They are probably the most common and versatile type of SBC, due to some renowned features of the ARM architectures. First, ARM architectures usually have a very low power consumption, which makes them ideal for battery-powered applications. Secondly, their open design allows for a wide variety of implementations, which makes them very versatile and has paved the way for some very small but powerful implementations, perfect for SBCs and embedded systems. Lastly, ARM architectures have a very tidy instruction set, which contributes to both their low power consumption and high efficiency. The most famous ARM64 SBC is probably the Raspberry Pi series [206] and DUA has been tested on the model 5.

The second class involves SBCs based on the x86-64 architecture. These are generally less common and more expensive than their ARM64 counterparts, since x86 processors are usually more powerful and have more features. They are usually employed for applications where more computational power is needed, or where the software that is going to be run on them is already available for x86 architectures. With respect to ARM-based SBCs, x86 ones usually have higher single-core frequencies and more cache memory but at the price of higher power consumption. The selection of communication interfaces is also usually wider and the support for peripherals is usually more complete but each item on the board requires more power to operate. Moreover, the x86 architecture has been expanded over the years with many specific instruction sets, such as vector instruction sets like MMX, SSE, and AVX, which are very useful in many applications that require fast computations over large datasets. DUA has been tested on the UP Squared and Xtreme models of UP series [1].

The last class is made by Nvidia Jetson SBCs [180]. These are ARM64-based solutions for edge computing on robotic systems, which also include a GPU, all integrated inside a Tegra SoC (System-on-Chip). The GPU is the most important feature of these SBCs, since it enables a wide variety of applications that require fast parallel computations ranging from image processing to object recognition and artificial intelligence models [221]. Many models and series have been released over the years, with models within a series being distinguished by their different set of system resources and price, so that customers can choose the solution that best suits their needs. The host system can be easily set up using the JetPack distribution by Nvidia [179] which includes an OS installation based on Ubuntu, device drivers, and many Nvidia toolkits and SDKs[I]. DUA has been tested on the Jetson Nano, TX2, Xavier NX, Xavier AGX, Orin NX, and Orin AGX models; as illustrated by this list, the employment of DUA on Jetson spanned over four series and six different models, showing the versatility of the framework. Different series, however, require different base units, due to both the host system configuration and the base image that is used to build the base unit: the setup process for each of them is described in the following. Note that, as shown in the following section, a specific JetPack version is used in every

[I] Software Development Kits.

Jetson base unit image: to ensure compatibility with the hardware and software tools that are available on the Jetson platform, the JetPack version is chosen to be the one that is recommended by Nvidia for the specific model that the base unit is built for and has to be the same that is actually installed on the host Jetson system.

SBC	CPU	RAM	GPU
Raspberry Pi 5	4c/4t Cortex-A76 @ 2.4 GHz	8 GB	VideoCore VII
Up^2	4c/4t Pentium N4200 @ 2.5 GHz	8 GB	Intel HD Graphics
Up Xtreme	4c/8t Core i7-8665UE @ 4.4 GHz	16 GB	Intel UHD Graphics
Jetson Nano	4c/4t Cortex-A57 @ 1.43 GHz	4 GB	NVIDIA Maxwell 128 CUDA cores
Jetson TX2	4c/4t Cortex-A57 + 2c/2t NVIDIA Denver 2 @ 1.95 GHz	8 GB	NVIDIA Pascal 256 CUDA cores
Jetson Xavier NX	6c/6t NVIDIA Carmel ARM64 v8.2 @ 1.9 GHz	16 GB	NVIDIA Volta 384 CUDA cores
Jetson Xavier AGX	8c/8t NVIDIA Carmel ARM64 v8.2 @ 2.2 GHz	64 GB	NVIDIA Volta 512 CUDA cores
Jetson Orin NX	8c/8t Cortex-A78AE ARM64 v8.2 @ 2.0 GHz	16 GB	NVIDIA Ampere 1024 CUDA cores
Jetson Orin AGX	12c/12t Cortex-A78AE ARM64 v8.2 @ 2.2 GHz	64 GB	NVIDIA Ampere 2048 CUDA cores

Table 4.2: Summary of the technical specifications of some of the single-board computers currently supported by DUA.

4.2.3 Base units

Section 4.2.1 and Section 4.2.2 have presented the concept of base units, which are the building blocks of the DUA framework. They are Docker images that provide a consistent system configuration across all supported platforms, preserving the specific characteristics and features of the host platform wherever possible. Software modules, *i.e.*, units built with DUA are then developed within containers whose images are built on top of these base units. Section 4.2.2 has illustrated the currently supported hardware platforms, giving a general overview of their variety. It is now time to complete the description of the base units contained in `dua-foundation` by illustrating the setup process that each one follows in more detail, as well as the specific features that they offer to the user. The description of each base unit can now be more detailed but not so long, since much of the setup process is common to all of them and has already been described in Section 4.2.1. Moreover, some base units are obtained by

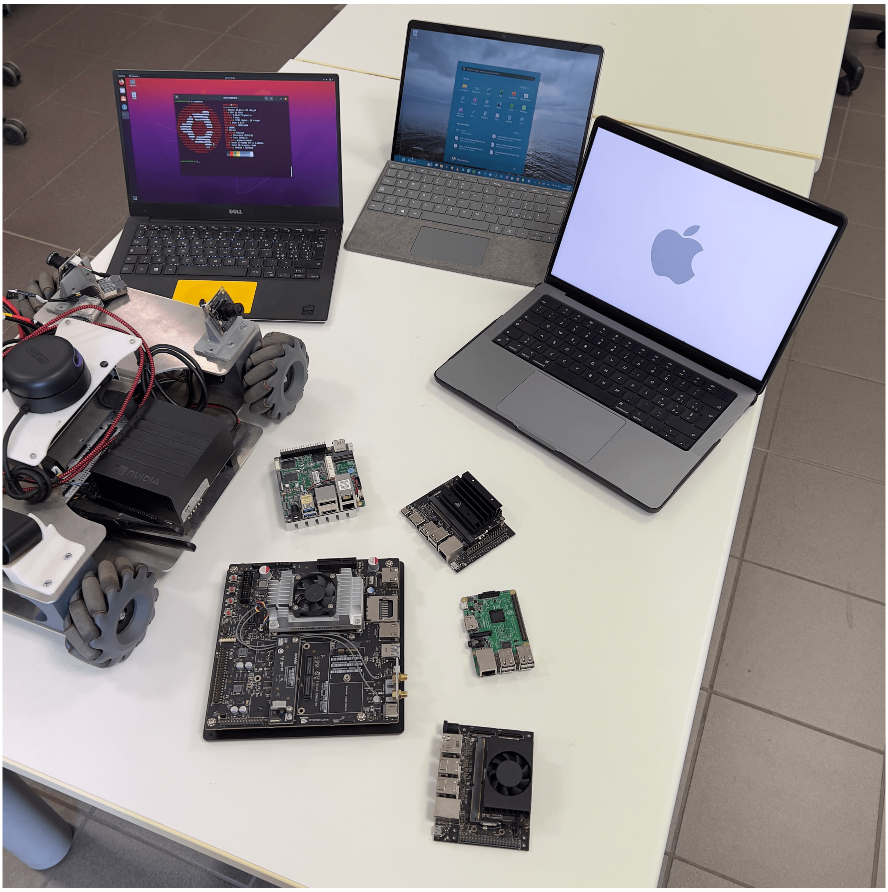

Figure 4.1: Some of the platforms currently supported by DUA, either with full support or with some limitations as in Table 4.1; from top left to bottom right: a Dell XPS 13 laptop, a Microsoft Surface tablet, a MacBook Pro, an Nvidia Jetson Xavier AGX mounted on a mobile robot prototype, an Up Squared, an Nvidia Jetson Nano, an Nvidia Jetson TX2 development kit, a Raspberry Pi, and an Nvidia Jetson Xavier NX.

extending others, so the setup process for the former is a subset of the latter; this is the case of, *e.g.*, base units for x86-64 systems and generic ARM64 systems, which can start with a configuration that suits a SBC, and then be evolved into a configuration that suits a development PC by adding more software tools, as anticipated in Section 4.2.1. One last thing to note is that, even if these particular base units could be built on top of each other by, *e.g.*, obtaining the x86-64 development image starting from the x86-64 SBC image, this feature has not been employed and each base unit is built separately. This decision has been taken considering each build step that is performed in every base unit, which led to this conclusion: installing some development tools for, *e.g.*, ROS 2 or similar middleware after their core implementation has been installed in the SBC portion of the image forces Docker to go back to work on layers that have been already built, which slows down the build process and complicates the management of the image layers. Instead, building the base units separately allows to install the right set of packages in the right order and in single build steps, optimizing the configuration of the image layers.

Before discussing each base unit in detail, a few words can be spent on the final sizes of the corresponding Docker images. Table 4.3 offers a summary of the sizes once the layers are compressed and stored on Docker Hub. As one can see, images can be quite large, reaching up to 10 GB for the most complex development ones, with the ones for the SBCs being smaller and the ones for the Jetson systems being in the middle. The size of the images is not a concern for the framework, since they are not meant to be used directly by the user but only as a base for the development of software modules: as explained in Section 3.2.1, even if multiple DUA images are built on a same host system, Docker reuses layers from the base unit since they are common to all images used on a host system. Regarding the build steps, the Dockerfiles in [144] show that care has been taken to reduce the size of the layers as much as possible by, *e.g.*, installing packages using the minimum number of commands that is possible and removing temporary files and build artifacts. The final size of the images is then a compromise between the need to provide a complete environment to the user and the need to keep the images as small as possible to reduce the time required to build them and the space required to store them. Development images are larger than the others because they include more tools, many of which have graphical elements, and Jetson ones are also the largest among those dedicated to SBCs since they include many specific libraries by Nvidia and third parties, as discussed below. Recall that the aim of this framework, in its current version and state, is to provide a complete, easy- and ready-to-use environment for research, development, and prototyping purposes: the size of the images as well of the amount of software tools that are installed in them are a consequence of this aim, in order to provide a complete, generic, and versatile environment.

`x86-base`

The `x86-base` base unit is the simplest one, and it is intended to be used on SBCs based on the x86-64 architecture, as well as to deploy modules on general-purpose PCs when there is no need for additional development tools. It has also been the first one to be developed to test the framework. It starts from the official Ubuntu base image and installs a set of system packages that are common to all base units, as described in Section 4.2.1. Since there is no need for additional development tools, the setup process ends here.

`x86-dev`

The `x86-dev` base unit is intended to be used on general-purpose PCs and it is the one that is used to develop software modules that are intended to run on the x86-64 architecture. It starts from the same build stages of the `x86-base` base unit, adding to some of them development tools that are required to build software modules that are based on the ROS 2 middleware. Afterwards, the Gazebo simulator is also installed. Since this base unit is intended for a simple host platform, the setup process ends here.

`x86-cudev`

This is the first image that accounts for peculiar characteristics of the host platform as well as for the specific features of the base image that is used to build the base unit. The `x86-cudev` base unit is intended to be used on general-purpose PCs based on the x86-64 architecture that include an Nvidia GPU. They require a specific set of device drivers and libraries to be installed on the host system, as well as a specific Docker runtime to give them access to Nvidia GPUs on the host. The main difference is the base image: instead of a plain Ubuntu image, the `x86-cudev` base unit starts from the Nvidia CUDA+cuDNN base image, which is a Docker image based on Ubuntu that includes a CUDA Toolkit [178] and the cuDNN library and development files [177]. This is done to allow the user to develop software modules that can take advantage of the GPU on the host system, as well as many libraries that offer APIs ranging from parallel computations to machine learning. After the Nvidia ones, other libraries and tools aimed at machine learning and artificial intelligence are also installed, such as PyTorch [204], Keras [42], ONNX [122], TensorFlow [97], and Ultralytics YOLO [239]. The installation of these and other Python libraries, here as well as in all the other base units, takes place inside a Python environment specifically created for DUA and that extends the system environment. The rest of the setup process is the same as the one for the `x86-dev` base unit, with the addition of some compilation options for libraries built from source such as, *e.g.*, OpenCV, to enable GPU support.

`armv8-base`

The `armv8-base` base unit is the simplest one for ARM64-based SBCs and it is intended to be used on them when there is no need for additional development tools. It starts from the official Ubuntu base image and installs a set of system packages that are common to all base units as described in Section 4.2.1. The main difference of this base unit with respect to the `x86-base` one is that the build processes of libraries built from source such as, again, OpenCV are configured to take advantage of the ARM64 hardware architecture.

`armv8-dev`

The `armv8-dev` base unit, like its `x86-dev` counterpart, is intended to be used on general-purpose PCs running on ARM processors such as, *e.g.*, Apple MacBook laptops with the recent Silicon series of SoCs. It starts from the same build stages of the `armv8-base` base unit, adding in some of them development tools that are required to build software modules that are based on the ROS 2 middleware. Afterwards, the Gazebo simulator is also installed. Source builds are again configured to take advantage of the ARM64 hardware architecture.

`jetsonnano`

This is the first of the series of base units aimed at Jetson systems. Since they are all SBCs not particularly suited for on-board development, all the `jetson` base units starting from this one do not include development tools. The Jetson Nano is today an old platform and its last compatible JetPack version is 4.6.2. At that time, Nvidia did not provide a base image for Docker, so the `jetsonnano` base unit starts from the official Ubuntu 20.04 base image and performs a series of steps that transform it into a JetPack installation corresponding to that version. The choice of Ubuntu 20.04 is motivated by the need to maintain compatibility with the Linux kernel version running on the Nano, which is 4.9 since JetPack 4.6.2 is based on Ubuntu 18.04. A JetPack installation is obtained by configuring environment variables and configuration files and adding Nvidia repositories to `apt` from which Nvidia packages are then downloaded. Then, the setup process proceeds mostly as in the `armv8-base` base unit, with the addition of some Nvidia-specific packages and libraries that are required to run GPU-accelerated software on the Jetson and to perform the source build of ROS 2. In this image and the next `jetson` ones, building ROS 2 from source is required to keep its version consistent with the other base units: to use the most recent ROS 2 version available, in `x86` base units which are based on Ubuntu 22.04, the ROS 2 Debian packages are used to install the Humble Hawksbill version, while in `jetson` base units which are based on Ubuntu 20.04 the only way to install the same version is to build it from source. This is not a trivial task at all and requires many dependencies, build options and paths to be set correctly but it is the only way to ensure that the same version of ROS 2 is used across all base units, which is a key requirement to ensure consistency of the environment provided by DUA.

`jetsontx2`

The `jetsontx2` base unit is intended to be used on Jetson TX2 systems, and its setup process is almost identical to the one of the `jetsonnano` base unit. The main difference is that the JetPack version is 4.6.3, which is the last one compatible with the Jetson TX2.

`jetson5`

This is the last of the series of base units aimed at Jetson systems and thanks to recent support to Docker by Nvidia is one of the most simple to set up. It is meant to run on Jetson systems running a minor version of JetPack SDK version 5, a category which includes Xavier NX and AGX, and Orin NX and AGX. The base image is the official Nvidia JetPack SDK Docker image for machine learning, called `l4t-ml`, which is available from the Nvidia Container Registry [181]. This image is based on Ubuntu 20.04 and includes a CUDA toolkit, cuDNN, and a set of libraries and tools for machine learning and artificial intelligence similar to that included in the `x86-cudev` base unit. Thus, the installation begins already as a JetPack environment, and what is left to install is the same set of tools and middleware as in the previous base units, set up in a similar way as in the other `jetson` base units.

Base unit	Docker image compressed size
`armv8-base`	1.7 GB
`armv8-dev`	1.93 GB
`jetson5`	9.04 GB
`jetsonnano`	3.6 GB
`jetsontx2`	3.6 GB
`x86-base`	1.69 GB
`x86-cudev`	13.11 GB
`x86-dev`	2.96 GB

Table 4.3: Sizes of the compressed Docker images of the base units currently available.

4.3 Unit template: dua-template

The previous section showed how the consistent environment provided by DUA is designed and built to support an already wide variety of hardware platforms and host system configurations. Recalling that the intended user of this framework is a researcher or a developer tasked with the design of a prototype for a distributed autonomous system, what is left to do is to provide them with a development environment based on the base units discussed before. In other words, since frameworks define a standard way of developing and organizing software as shown by the other solutions presented in Section 2.3, DUA has to provide a *template* for the development of a unit. This is the purpose of the next core DUA module: `dua-template` [145].

The `dua-template` is implemented as a GitHub template repository, *i.e.* a special type of Git code repository hosted on GitHub [95] which exists only to be *forked*, *i.e.*, copied to generate new projects which start with its contents. It contains a set of files that make up a structural template for the development of software modules for distributed systems as DUA units. Once appropriately forked on GitHub, it can be configured as the starting point for a new project as hereby described. The main goal of this project is to offer a common, shared work environment that reduces the time and effort required to set up prototyping and experimenting of new solutions, as well as to provide a common framework for teams that can be versatile enough to support any development workflow. It aims at achieving so by providing three main features: workspace organization for both development and deployment, hardware and operating system abstraction via the base units and their contents, and a modular structure for easy integration of multiple components into a single macro-project or distributed architecture.

A key component of a software development process is today a good Integrated Development Environment (IDE). During the development of this work, the IDE of choice has been Visual Studio Code (VS Code) by Microsoft [166]. Visual Studio Code is a free, open-source, cross-platform IDE that can be used to develop software in many different languages. It is particularly useful for this use case, where it can be used to develop code in C/C++, Python, and other languages, and it can also be used to manage Git repositories and Docker containers. It is also very customizable and can be extended with plugins that add support for many different languages and tools. As the following sections show, this template

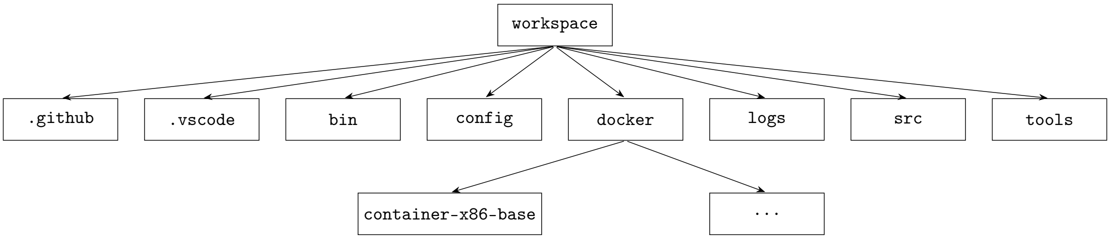

Figure 4.2: Directory tree of a DUA unit based on `dua-template` [145].

is configured to work with VS Code out of the box but it is just a suggestion: it is possible to use other IDEs, or none at all, by simply managing differently the files and directories that are included in the template according to the specific needs of the project at hand.

The description of the functionalities offered by this template starts from the explanation of the project layout that it provides, hereby informally referred to as the *filesystem layout* of a unit. Then, it can be explained how to set up a DUA unit on top of one or more base units and how to eventually set up a bigger project by integrating multiple units. The last part of this section is dedicated to the discussion of a GitHub workflow that is used to keep units and projects hosted on GitHub up to date with respect to the main `dua-template` version.

4.3.1 Unit filesystem layout

The filesystem layout of a DUA unit is designed to be as simple and intuitive as possible, to allow the user to quickly understand the structure of the project and to easily find the files that they are looking for. The layout is also designed to be consistent across all units, to allow the user to quickly understand the structure of a new unit even if they have never seen it before. The layout is also designed to be flexible, to allow the user to easily add new files and directories to the project as needed. However, a standard set of directories is provided to help the user organize their project in a way that is consistent with the DUA framework. The layout is also designed to be compatible with the structure of the base units, to allow the user to easily integrate their software modules with the base units and to quickly deploy them on the target platform. A visual inspection of the layout of [145], which is also depicted in Figure 4.2, shows that it resembles the organization of a colcon workspace discussed in Section 3.1.3: this is due to the fact that the template is designed to be used also to develop software based on ROS 2 and the colcon workspace is the standard way to organize ROS 2 projects. Nonetheless, a set of countermeasures are put in place to avoid adding build artifacts and similar contents to version control: almost every directory among those listed below contains an appropriate `.gitignore` file that prevents known build artifact file types to be added to Git and the user is free to modify them as they see fit to customize this behavior. The best way to understand the `dua-template` filesystem layout is to examine the purpose and contents of each directory it contains.

`.github`

This directory contains the GitHub workflow files that are used to keep the unit up to date with respect to the main `dua-template` repository. The workflow is based on GitHub Actions, a feature of GitHub that allows to run automated tasks when certain events occur in a repository. It is contained in the file `sync-dua-template.yaml` in the `workflows` subdirectory, and it is triggered when a new commit is pushed to the repository or at a specific time of the day. It is presented in Section 4.3.3.

`.vscode`

This directory contains configuration files for Visual Studio Code. They are present to provide each user with a good starting point for their development environment and they can be customized as needed. They are applicable since, as discussed later on, Docker containers based on DUA base units for development are also designed to be capable of being opened and managed with VS Code and when this is done the settings for the IDE are automatically loaded from this directory. The files are the following.

- `c_cpp_properties.json`: this file contains the configuration for the C/C++ extension for VS Code, which is used to develop C/C++ code.
- `launch.json`: this file contains configurations to run software modules from a VS Code instance.
- `settings.json`: this file contains the general settings for VS Code, such as the theme and the font size, as well as system paths to be used by the IDE.
- `tasks.json`: this file contains the tasks that can be run from VS Code, such as building the software modules or running tests.

`bin`

This directory is intended to contain any sort of standalone executables and scripts concerning a project. As for any other directory that has an intended purpose, the user is encouraged to put such files here but as many other directories in the template, this is just a suggestion and the user is free to organize their project as they see fit. In the template, only the `dua_setup.sh` script is present, which is used to set up a DUA-based project as explained in the following sections.

`config`

This directory is intended to contain configuration files that concern and entire unit or project. Again, this is just a suggestion and for the moment the template does not require any file to be present from the start in this directory.

`docker`

This directory starts empty, but contains the configuration files for the Docker containers that are used to develop and run the software modules. As explained previously, there may be a Docker container for each base unit that a project is intended to run on, so this directory may have multiple subdirectories, one for each base unit. This aspect is also explained later on, since the contents of this directory are also set up by the `dua_setup.sh` script.

`logs`

This directory is also empty at the beginning, because it is intended to contain log files generated by the software modules when they are run, including sampled data of experimental runs. To ensure that such large files are not added to the version control system by mistake, a `.gitignore` file is present in the directory that excludes all files in it.

`src`

This is probably the most important and most used directory of the entire template. It is intended to contain the source code of the software modules that are developed for the unit, as well as subdirectories containing units that are added to a larger project. This directory is also meant to be found and managed by colcon, in case it is used as a build system for the project.

`tools`

This last directory also starts as an empty directory and it is also meant to contain source code and software modules which are under active development within the context of the unit or project at hand. The difference with respect to the `src` directory is that the files in this directory are not meant to be built using colcon, which in fact ignores this directory thanks to the `COLCON_IGNORE` file that is present in it.

4.3.2 Configuring a new unit

The setup of a new unit or project is a process that is meant to be as simple as possible, to allow the user to quickly start developing software modules for a distributed system. The setup process is performed by running the `dua_setup.sh` script that is present in the `bin` directory of the template. The script is designed to be run from the root directory of the unit, in one of the ways shown in Listing 4.1.

```
1 dua_setup.sh create [-a UNIT1,UNIT2,...] NAME BASE_UNIT
2 dua_setup.sh modify [-a UNIT1,UNIT2,...] [-r UNIT1,UNIT2,...] BASE_UNIT
3 dua_setup.sh delete BASE_UNIT
4 dua_setup.sh clear BASE_UNIT
```

Listing 4.1: Possible invocations of the `dua_setup.sh` script.

An explanation of all the possible invocations of the script is given in the following sections. Before going further, another definition is in order here: the term *target* identifies a single Docker image that is used in a project and that is built from a single Dockerfile pair plus many other configuration files discussed later on. A target can be used to run a single unit, or multiple ones, or even a whole control architecture; the name resembles the meaning of the term "target" in the context of a Makefile and is used to refer to the same concept in this context: each unit or architecture can have multiple targets, *i.e.*, Docker images, each one based on a different base unit. Thanks to the base units, the software that makes up a unit or project is ensured to run appropriately and similarly across all supported targets.

Creating a target

The command at line 1 of Listing 4.1 creates a new target, *i.e.*, a Docker image for a unit or project, plus a set of configuration files. The `-a` option can be used to specify a list of units that have to be integrated in the new target, a feature that is explained later on, and the NAME argument is the name of the project. The `BASE_UNIT` argument is the name of the base unit to use. A password is required for all projects since the container's internal user runs with elevated privileges and has full access to the host's network stack and hardware devices. It is be asked for during the creation of the target, and is be stored in hashed form in the target's Dockerfile.

`create` creates a new `container-BASE_UNIT` directory inside `docker/`, and copies and configures all the template files stored in `bin/dua-templates`. These include the `Dockerfile` and the `docker-compose.yaml` files, as well as many shell configuration scripts and other configuration files that are used to build and run the image. Once copied, they can be modified to fully support the specific features of the unit or project at hand. The most notable ones are the following.

- `aliases.sh` is meant to contain shell aliases.
- `commands.sh` is meant to contain shell functions that execute commands that involve the software package that make up the unit or project at hand.
- `ros2.sh` contains configuration options and functions for the ROS 2 installation that is provided in every DUA image.
- Other files configure the shells and other basic programs like text editors.

Once a target has been created, it can be managed using Docker or VS Code extensions interchangeably.

Containers are meant to be run in detached mode, *i.e.*, with the `-d` option, and can be accessed using the `docker-compose exec` command as shown in Listing 4.2.

```
docker-compose exec NAME-BASE_UNIT zsh
```

Listing 4.2: Docker Compose command to open an instance of the Zsh shell inside a DUA container.

This is due to how DUA containers are brought up, which is written at the end of the most important file among those generated by the setup script in this invocation: the `docker-compose.yaml` file. This file is meant to be used with Docker Compose (see Section 3.2.1), either directly or through VS Code, to start the container specifying the many configuration options for the Docker Engine that would instead result in an unreasonably long command line for `docker run`. Each base unit requires specific configurations, which is why the setup script copies and configures the right files for the right base unit from the template ones stored in `bin/dua-templates`. The contents of this file can be modified accordingly once the target configuration has been generated. The settings specified in the `docker-compose.yaml` file can be summarized as follows.

- Build context and other useful options for the container build process.
- Environment variables that allow terminals and graphical applications to work properly.
- Host networking, shared memory and IPC namespace, to provide to applications running in the container full communication between the container and the host, as well as with remote hosts.
- Access to GPUs, if present.
- Access to all hardware devices with full privileges and `/dev` and `/sys` mounts.
- Access to the unit or project files from the host filesystem: the root directory of the unit is mounted in the container at `~/workspace` and all the configuration files for the container respect this convention.

These settings, apart from those exclusively dedicated to the image build, make up the core of the integration of Docker in DUA as a tool to build and deploy replicable system environments: applications running inside a container configured in such a way have access to the host system in pretty much the same way they do if they run directly within it, with direct access to any hardware device and driver ranging from the network stack to communication interfaces and even GPUs. The advantage is that the entire installation and dependency configuration is performed inside a Docker image, avoiding the risk of tainting the host configuration or of compromising it, leading to a reinstall that is a timely process especially on SBCs. Furthermore, a Docker image can be simply packed and downloaded to any number of hosts once it is finalized, making the replication of a system environment a matter of seconds compared to traditional solutions for, *e.g.*, SBCs, that involves flashing the board with a new OS image and then configuring it from scratch, manually or using scripts. Moreover, a Docker image constitutes a snapshot of the system environment at a given time, including the version of the software packages and libraries that are installed in it, which can be useful to reproduce experiments or to ensure that a system is always running the same software version, guaranteeing stability and reproducibility. The only overhead is due to the size of the images as in Table 4.3 but considering that once the containers are started there is no more overhead, due to the fact that the unit files are mounted from the host filesystem, the advantages of using Docker in this context are clear. Nonetheless, these options deactivate by default many security features that Docker enforces to achieve full isolation of a process running inside a container with respect to the rest of the host system. Note that this work, as similar solutions that are starting to leverage Docker in the field of autonomous systems, does not intend to offer that kind of isolation for two main reasons. First, the end goal is that of easily replicating system configurations across multiple hosts, allowing applications to access the full set of resources offered by the host system: this implies that only a subset of the functionalities of Docker are necessary by the definition of the original problem. Moreover, the context of interest is that of a development environment for research and prototyping in which things have to be made to work and tested quickly first, and then eventually robustified and gradually isolated. This is why the default settings of the Docker containers are set to allow full access to the host system but nonetheless they are fully exposed to the user of the framework, who can modify them as they see fit to achieve a configuration in which containers are properly isolated, being exposed only to a subset of the features and resources of the host system that are necessary for their applications to run. Some of these options have not been considered for simplicity, such as configuration of the *secure computing mode* (`seccomp`) module to restrict access to specific system calls but given the complete set of configuration files offered by the template, the user can easily add them to the container configuration.

Finally, an item that deserves a thorough explanation is the `command` item, which specifies the command that is run when the container is started. This is set by default to what is shown in Listing 4.3, which is essentially a Bash shell process that sleeps indefinitely, being woken up only by a `TERM` signal that triggers a graceful exit. This is done to keep the container running and to allow the user to access it at any time and in any way they see fit, *e.g.*, opening within it multiple shells, running graphical applications, accessing server applications running within it from remote hosts, connecting to it using a VS Code instance, etc. These containers are meant to be deployed on autonomous systems, so they should stay active with an arbitrary set of processes running within them. This is again not the originally intended use of Docker, *i.e.*, that of running single microservices inside each container but is instead a new way of using Docker to provide a complete, versatile, and consistent environment, which is currently being exploited also by other frameworks in similar contexts.

```
/bin/bash -c "trap 'exit 0' TERM; sleep infinity & wait"
```

Listing 4.3: Default command of a DUA container: an idle shell process.

Modifying a target and integrating units

The remaining core functionalities of `dua-template` concern the configuration of a unit to run the software modules that it contains. In this context, the main aim of DUA is not only to provide a stable and reproducible way to do so but also a mechanism to simply integrate a unit alongside multiple other ones in a bigger project, in a modular and simple fashion.

Since units are software modules in essence, the system configurations they require usually boil down to two categories: system settings, like environment variables and configuration files, and dependencies, such as packages to install. Recalling how base units are set up as explained in Section 4.2, these steps should be somehow added to the image build process on top of what the base unit already provides. This is the purpose of the `Dockerfile` file, generated by the `dua_setup.sh` script and placed in the directory of every configured target.

The interested reader can find templates for the `Dockerfile` in [145]. As for the previous discussion, this section aims to guide the inspection of such templates by detailing their contents. The operations performed by the `Dockerfile` are the following, in order.

1. Start from the target's base unit image.
2. Perform unit-specific operations.
3. Configure the internal user of the container.
4. Create mount points for configuration files and the workspace.
5. Configure the shell.

While most steps are standard and should not be altered in general, although the user is given complete control over how the framework works, the second is the one where manual intervention is encouraged. This is because, initially, that step is nonexistent, as illustrated by Listing 4.4.

```
# NAME SETUP START #
# NAME SETUP END #
```

Listing 4.4: Unit configuration step in a Dockerfile as generated by the template.

`NAME` is, of course, replaced by the name given to the unit by the user as in line 1 of Listing 4.1. Between those two lines, the user can add any Dockerfile directive they require to configure the system environment of the container to support the software that makes up the unit. Once built, the container has all the features and functionalities of its base unit plus what the user added to support their own software module. This section of the `Dockerfile` and the way it is delimited are instrumental to achieve the desired level of modularity, making the integration of multiple units in a single project again a matter of a few commands, listed in the remaining lines of Listing 4.1.

Suppose that a unit should be integrated in a project. The first step consists in adding the unit codebase to the one of the project, *i.e.*, copying it in the `src/` directory. It is assumed that all codebases are managed by the Git version control system, as stated in Section 4.1, which offers two functionalities meant to this purpose: submodules and subtrees, illustrated in Section 3.3.1. They are both capable of linking the unit's repository to the project one but they are meant for different use cases.

Submodules are the preferred way to include units in a DUA project when the user plans to actively work on the units themselves, while working on the project. A submodule is a Git repository that is included in another Git repository as a subdirectory. This means that the submodule is a separate repository, with its own history, branches, and tags, that is included in the parent repository as a subdirectory. This is particularly useful when the submodule is actively developed and when changes to it have to be pushed back to the original repository. The main drawback of submodules is that they require additional commands to be managed and that special attention must be paid to the point of the Git history in which the submodule is found before making any changes inside it, or worse, committing them. Moreover, the reference commit of the submodule in the parent repository always has to be updated manually. Nonetheless, submodules are a very powerful tool that, when used correctly, can make the simultaneous development of multiple projects much easier. For this reason, given the degree of modularity that DUA aims to achieve, they are supported by `dua-template` and their usage is encouraged as the preferred way to include units in a DUA project in any case.

Subtrees, on the other hand, are the preferred way to include units in a DUA project when the user does not plan to actively work on the units themselves. Essentially, integrating another Git repository as a subtree still ensures the possibility of specifying the commit, branch, or tag of the repository to be integrated but requiring a simpler management with respect to submodules, since files and Git histories are merged with those of the parent repository, with little to no additional commands required to keep them in sync resulting in a single repository that contains all the files and histories of its subtrees. However, they have one major drawback: they may make Git history management more computationally difficult, since the history of the subtree is merged in the one of the parent repository. This often leads to Git having to go through the whole history of the subtree to find the changes that are relevant to the parent repository, to ensure that a coherent history is maintained; this easily becomes a slow process, which is why the use of subtrees is discouraged when the subtree is actively developed.

Then, as for every DUA codebase, a target for the project must be configured. What follows can be applied to any number of targets a project shall support. The `modify` command of the `dua_setup.sh` script, at line 2 of Listing 4.1, is intended for this purpose. The `-a` option is used to integrate one or more units into the specified target, while the dual `-r` option is used to remove one or more units from the target. To ensure that all the system configurations that the supplied units require are performed at the same time within that target, what the script does in case of an addition is simply to look for the relevant units in the `src` directory and then for their corresponding target directory; in case the target at hand is, *e.g.*, `x86-base` and the unit is called `sensor-driver`, the script looks for the directory `src/sensor-driver/docker/container-x86-base`. It the search is successful, the script proceeds to copy the unit configuration section of the `Dockerfile`, delimited as in Listing 4.4, within the target's

`Dockerfile` and into its own unit configuration section. This way, the user can be sure that all the system configurations that are required to run the software module are performed at the same time, and in the same way, for all the units that are integrated in the target. The same operation is performed in case of a removal but in reverse: the script looks for the unit configuration section in the target's `Dockerfile` and removes it if it is found. If some units need to be set up in a specific order, it is up to the user to specify them in the desired order in the `-a` option of the script. This description also clarifies the remaining reasons behind the filesystem layout of a unit and the presence of the `Dockerfile` in the subdirectories of the `docker/` directory, as illustrated previously in Section 4.3.1.

The commands at line 3 and 4 of Listing 4.1 are pretty straightforward and are provided for convenience. The `delete` command can be used to quickly and easily remove a target from a unit or project, effectively deleting its subdirectory from the `docker/` directory. The `clear` command can be used to remove all the lines between the delimiters of the unit configuration section of a target's `Dockerfile`, effectively resetting the target to its base unit configuration.

4.3.3 Updating dua-template in units and projects

As every software component, the `dua-template` is subject to updates. They can be of various types, ranging from bug fixes to new features, and they are meant to be applied to all units and projects that are based on the template. To ensure that all units and projects are up to date with respect to the main `dua-template` repository, a GitHub workflow is provided in the `.github` directory of the template as explained in Section 4.3.1. Every GitHub repository forked from the main `dua-template` repository has this workflow enabled by default and it is triggered when a new commit is pushed to the `master` branch of such repository or at a specific time of the day. As for any feature of DUA, the user can modify or disable this workflow as they see fit.

The interested reader can find the code of the workflow at either [145] or in Listing A.1 in Appendix A. The workflow, meant to run on virtual machines called *runners* provided by GitHub, clones both the main `dua-template` repository and the repository where it is running and then it compares the two repositories to find out if any file in the main repository has been updated, *i.e.*, either modified or created or even deleted, with respect to the forked one. The files that the workflow compares are only those that are part of `dua-template`, *i.e.*, it does not compare nor modify files belonging to targets that have already been created and configured. This behavior is intended by design and due to two reasons: first, it is not possible to deterministically distinguish user-applied modifications to target files from those that are part of the template, and second, the user always has the last word on how their project is configured, having the ability to modify every template file as they see fit to ensure that their software works correctly. Thus, the user is the only one that can decide whether and how to apply the update, either manually reconfiguring or regenerating the targets as explained in Section 4.3.2 using the updated template files. To provide a convenient way of doing so, the workflow creates a pull request in the forked repository with the changes to the template portion of the codebase so that the user can then review it, update their unit files, and merge the pull request to apply the changes.

4.4 Remarks

Recalling Table 2.1, it is now possible to compare DUA against the other frameworks presented in Section 2.3. The comparison is summarized in Table 4.4.

As one can see, the Modularity (M) requirement is satisfied by almost all frameworks. This is a common feature of modern software engineering practices and in the context of distributed and autonomous systems it clearly improves the development process and the overall system maintainability. The Overhead (O) requirement is also satisfied by most of the frameworks, which is a good indicator of the efficiency of the solutions presented. Using the same framework for both development and deployment is clearly a good intention, thus the overhead that it introduces should be minimal. In this context, it cannot be said that DUA fully satisfies the Overhead (O) requirement, due to the reliance on Docker containers and the presence of the Docker Engine on SBCs. Although applications running in containers configured as explained above experience no overhead, the presence of the Docker Engine itself may take up some resources on the host system, even if in a negligible amount. Nonetheless, the configuration of the base units targeted at SBCs provided an overview of what a desirable system configuration for those platforms should contain; some ideas on how to follow this same approach with solutions suitable to embedded systems are presented in Chapter 8.

DUA is the only framework to completely satisfy the Abstraction (A) requirement. This is a key feature of DUA, as it allows the developer to focus on the high-level aspects of the system while the framework takes care of the low-level details. This is a clear advantage of DUA over the other frameworks, as the developer who uses it can focus on the functionality of the system, rather than on the system configuration. The key feature of the implementation of DUA that allows this is the use of the base units discussed in Section 4.2.

The only requirement that DUA does not fully satisfy is the Development (D) requirement, which is also an issue for other frameworks. Because of how the requirement is stated, there are a lot of aspects that may be considered while evaluating it. The issue that DUA still does not solve is that of facilitating testing and debugging processes: the system abstraction layer of `dua-foundation` does a great job at simplifying the development process over different platforms but it does nothing to help the developer while testing their modules among the many different targets on which they may run. Moreover, there is currently no support for cross-compilation or network-based debugging. These aspects have been identified as future work.

	Modularity	Development	Abstraction	Overhead
RoboStack	✓	×	+	✓
Vulcanexus	×	+	+	✓
EPICS	✓	✓	×	✓
BlockNet	✓	+	+	×
F Prime	✓	+	+	✓
MARTe2	✓	+	+	✓
DUA	✓	+	✓	+

Table 4.4: Comparison of DUA against the solutions presented in Section 2.3, with respect to the requirements listed in Section 2.2; "✓" denotes a completely satisfied requirement, "+" indicates that the solutions provided do not fully satisfy a requirement, and "×" marks an unsatisfied requirement.

5 The case of autonomous robots

The previous chapter presented the Distributed Unified Architecture, the novel framework proposed in this work. Its design and implementation have been described, from the system abstraction layer to the unit template structure. The various features and functionalities of the framework, provided by either existing tools presented in Chapter 3 or custom-made components, have been described in detail and discussed.

This work is the result of research activities spanning some years in the field of autonomous and distributed systems, as illustrated at the very beginning of this book. The Distributed Unified Architecture is the materialization of multiple attempts and ideas whose goal was to simplify research work involving autonomous systems and control architectures, as well as the activities of single developers or small teams. Many of the ideas, concepts, and requirements on which DUA is based have stemmed from long and fruitful discussions with colleagues and friends, usually held after a project was completed, to assess what went well and what could be improved during the development process. This chapter and the next tell about the fruits of these practical experiences, showing some first applications of the DUA framework and the technologies that it integrates in real-world scenarios.

As discussed in Chapters 2 and 3, autonomous robots today are intrinsically distributed systems, from both the hardware and the software perspectives. DUA has been designed to offer a unified approach to the development of such systems, providing a common ground for the integration of heterogeneous components and tools and enabling the development of sophisticated behaviors and functionalities in a modular and scalable way.

The contents of this chapter summarize the additional features that the DUA framework provides to the autonomous robotics domain, as well as the first results obtained by applying it and the technologies on which it is based to a real-world scenario. First, some robotic software packages developed for and integrated into DUA are presented. Then, to provide a concise yet explanatory example of the capabilities of the framework, two software modules developed with DUA are briefly presented: their design and development process and the challenges they posed were among the main motivations for the creation of the framework. Finally, some results obtained by applying the technologies and solutions on which DUA is based to real-world scenarios are illustrated.

5.1 Software utilities for ROS 2: dua-utils

With the system abstraction layer and the unit template finalized, a single problem is left to solve. It is of practical nature, arises only when one starts to use the framework and the middleware tools integrated into it for robotic software development, such as ROS 2 (see Section 3.1.3). The features of the ROS 2 middleware compose a set of services that the robotic software developer can use to build applications and modules of various degrees of sophistication and the open-source nature of the framework has allowed a wide community to improve it over the years, reviewing existing functionalities and adding new ones. Nonetheless, the ROS 2 middleware is a complex system and its many features can be overwhelming for a beginner or a developer who is not familiar with all of its functionalities. Moreover, parts of its API are not intuitive.

The ROS 2 community is aware of these issues and the development effort is improving the quality of the software every day. However, over some years of experience and active development of ROS 2 modules, the necessity arose to create a set of utilities that simplify some parts of the framework, wrapping its complexity and providing a more intuitive interface to the developer. While waiting to propose the same set of changes to the community and have them integrated into the official ROS 2 distribution, a process that can happen but also take a long time, the choice has been made to develop the same set of utilities as packages that are integrated in the system configuration provided by every DUA base unit, so that they are immediately distributed and used in the development of robotic software modules based on DUA. Note that these software modules, as shown later on, are maintained as independent projects with no specific ties to DUA, so that every published work that uses them can be easily integrated into other systems, frameworks, or architectures for the benefit of the scientific community.

This led to the creation of the third basic pillar of DUA: the software collection called `dua-utils` [146]. Note that, although at the time of writing it is a collection of utilities for ROS 2 and robotic software development, it may host different kinds of third-party software packages that can be useful in the development of distributed systems. This can be seen as a limitation but it is only due to how it is currently integrated into DUA base units: at the very end of their setup process, the last stage clones the repository into the image filesystem and builds the packages contained in it, cleaning up build artifacts afterwards. This process has two main advantages: first, when the modules are built in this way they end up being part of the DUA base unit image, so that they can be used in the development of software modules based on DUA without the need to install them manually; secondly, the process is automated but performed at the end of every base unit, so if one of the utilities in `dua-utils` is updated the base unit image can be rebuilt by performing only this last step, reusing previous cached layers and saving time and bandwidth. What has been described so far may of course change in the future, should the necessity arise to include in `dua-utils` software packages that cannot be built with colcon or that simply concern other domains of distributed systems development. The motivating idea is that `dua-utils` is a way to customize the very base layer of the DUA framework which is provided to and intended for its users.

To give an idea of the set of improvements that DUA provides to the ROS 2 middleware, as well as a comprehensive overview of the work that has been done so far, the following sections introduce some very representative utilities that are part of the `dua-utils` collection.

5.1.1 Managing ROS 2 processes: dua_app_management

`dua_app_management` is a collection of software modules and source files for the management of applications and processes in the Distributed Unified Architecture. This package contains modules, both compiled and source-only, to be included in DUA-based projects. It is particularly aimed at the development of ROS 2 applications, with the aim of simplifying some common tasks and wrapping "boilerplate" code (this term refers to long code portions that has to be written each time in the same way, to perform what is usually a simple and frequent operation), especially that which involves the lifecycle of a process and requires calling the OS.

The package currently contains the `ros2_app_manager` C++ package, which provides two libraries and the implementation of a component container (see Section 3.1.3). The first goes by the name of the package itself and it is a header-only library that implements a wrapper object for the usual main thread code of ROS 2 applications. The convenience of this library comes from the fact that, if one wants to write the `main` function for a ROS 2 application, one finds after some time that the same code is written over and over again, with only the node name and the node class changing. Operations performed by this code usually include initializing the ROS MiddleWare context, which is the entrypoint towards the underlying communication middleware used by ROS 2, configuring options for the ROS 2 node, creating a proper executor for the application, and finally adding nodes to it and spinning it. The lifecycle of such an application ends when an asynchronous event, such a signal, occurs, stopping the executor and destroying all such objects in reverse order to correctly release resources and terminate middleware instances. The `ros2_app_manager` library provides a simple API to perform all these operations as follows.

- The constructor of the `ROS2AppManager` class does all the aforementioned initializations, as well as disabling I/O streams buffering for the current process to reduce latencies, which is a common practice in ROS 2 codes.

- The `ROS2AppManager::run` method spins the executor and returns only when a signal is caught, which is the usual way to handle the termination of a ROS 2 application.

- The `ROS2AppManager::shutdown` method stops the executor and releases all resources as previously described.

The class is implemented as a template class whose template parameter is the ROS 2 executor class to use. The library also includes the relevant system headers.

Since signals are the most common way to request the termination of a ROS 2 process, the user benefits from a finer-grained level of control over their action. To clarify the description of what follows, recall briefly the concept of POSIX signals. In operating systems design and programming and in particular for systems that follow the POSIX standard, *signals* are a way to notify asynchronous events to userspace programs, a sort of software interrupt that can be delivered to a process by the kernel either on its own behalf or on behalf of another process. Different signals are identified by numerical codes and represent different situations and errors. For example, when Ctrl-C is pressed to stop a program running in a terminal, the terminal tracks that key combination and sends the `SIGINT` signal to the process it is running, which is the default signal for interrupting a process. The process can then catch the signal and perform some cleanup operations before terminating, provided that it includes a handler for that signal in its code.

In ROS 2 applications, signals are not extensively used for two reasons: first, applications running on a robot should usually run as long as the robot is on, and secondly, they require a signal handler to be installed in the application code and some knowledge of the OS API. For this reason, ROS 2 usually handles signals by itself, which is good and handy but limiting in some cases. A first example scenario is if debugging is being performed to track a bug that crashes the applications: appropriately coded signal handlers have access to information concerning what was the memory state when the crash occurred, what kind of error occurred, what was the state of the application and of buses it was accessing, and so on. A second scenario is that of a prototype robot which is being tested, a situation in which applications are usually started and stopped via remote shells opened by users. Suppose that the robot is, *e.g.*, a flying drone, and that such applications allow it to fly safely: it is not desirable if, *e.g.*, the WiFi link crashes and the drone follows as well because the shells that the applications were running within close abruptly; in this case, ignoring some signals like `SIGHUP` or `SIGPIPE` is a good idea.

The second library of this package, called `ros2_signal_handler`, provides a POSIX-compliant signal handler module for ROS 2 applications, which replaces the default signal handler installed by the ROS 2 middleware. The library provides a class called `SignalHandler` implemented as a singleton. It uses the `sigaction` API to manage handlers, allowing fine-grained control over the signals that are caught and the actions that are subsequently performed and can be given access to the executor running in the application to stop it when a signal is caught. The library also includes the relevant system headers. The following methods are provided.

- `SignalHandler::get_global_signal_handler` returns a reference to the singleton instance of the class.
- `SignalHandler::init` installs the signal handler, which is a static method that has to be called before any other method of the class. This handler can call user-defined functions when a signal is caught and can be configured to ignore some signals.

- `SignalHandler::install` to install a handler for a specific signal.
- `SignalHandler::ignore` to ignore a specific signal.
- `SignalHandler::uninstall` to restore the default handler for a specific signal.
- `SignalHandler::fini` to reset the `SignalHandler` instance to its initial state and restore the default signal handlers for all signals.

All these features are actually useful only when a ROS 2 application is run as a standalone process, which is not the case for all robotics applications. In reality, what happens is that nodes may also be loaded into a process at run time: such a process acts as a component container and nodes are denoted as components, as explained in Section 3.1.3. In this case, the container application is usually already compiled and provided as part of the ROS 2 installation. This is where the third part of this package comes in: the `dua_component_container_mt` application. It is essentially a component container based on the `MultiThreadedExecutor` executor class, just like the standard `component_container_mt` provided by ROS 2 but with the addition of the `ros2_app_manager` and `ros2_signal_handler` libraries. The application is intended to be used while testing ROS 2 components which may be dynamically loaded and unloaded at run time on real robot prototypes, thus accounting for the potential issues such as the one described in the example scenario above. For this reason, this container application uses the `SignalHandler` object to handle many signals and ignoring some of them, and the `ROS2AppManager` object to manage the lifecycle of the application.

5.1.2 Handling Quality of Service correctly: dua_qos

In Sections 3.1.1 and 3.1.3, the concept of Quality of Service (QoS) has been presented as a way to configure the behavior of the communication middleware in a distributed system. In ROS 2, QoS policies are used to configure the behavior of the underlying communication middleware, being it either the DDS or Zenoh (see Section 3.1.2) or something else and are applied to topics to specify the reliability, durability, and other properties of the communication. The ROS 2 middleware provides a set of predefined QoS profiles that can be used to configure the communication but it also allows the user to define custom QoS profiles to meet specific requirements. This system is very flexible and powerful but often used in an inconsistent way. First, note that ROS 2 libraries provide default QoS profiles, whose names range from `qos_profile_default` to `sensor_data`. However, these profiles as well as the rationales behind them come from the ROS 1 age, in which the communication was implemented by the middleware itself on top of UDP and TCP. Since there were mostly only these two protocols to choose from, one had almost always to distinguish between a reliable and an unreliable communication, which led to the establishment of a set of rules of thumb like, *e.g.*, the common belief that data produced continuously by a sensor should be sent with a best-effort policy, since packet loss is always allowed. This is the case unless algorithms processing that data may also account for late packets, in which case a reliable policy should be preferred. Moreover, with the many options that DDS QoS offers, these binary distinctions do not apply anymore. The ROS 2 API is quite flexible and allows the user to define custom QoS profiles but the user is left alone to decide which profile to use and how to configure it. This can lead to inconsistent configurations across different nodes and topics, which can result in unexpected behavior and performance issues.

More configuration options imply that a finer control over the transmission policies can be achieved, accounting for an issue that ROS 1 struggled with: intrinsic difference in data types being transferred. For example, an inertial sensor may produce many samples every second but they all correspond to small packets since they are made of a few numeric fields. Conversely, a LiDAR scan may have a lower frequency but produce large packets, since each scan is made of many points; a similar situation may arise with cameras. Reliability is not the only issue to consider: the frequency of the data, the size of the packets, the importance of the data, and the network conditions are all factors that can influence the choice of the QoS profile to use.

The `dua_qos` library [149] provides APIs that return QoS profiles that account for these issues, distinguishing first between the desired reliability of the communication and then between the data type being transferred. The repository contains two software libraries: `dua_qos_cpp` for the C++ language and `dua_qos_py` for the Python language. Apart from their actual implementation, whose details concern only the syntax of each language, the functionalities they offer are the same. However, DUA does not enforce the use of these QoS profiles in any way nor place: they are only included as a suggestion for the user to simplify the development of ROS 2 applications and are also designed to be compatible as much as possible with the default QoS policies that are both provided by the ROS 2 middleware and used in the majority of the ROS 2 software available today. Lastly, the profiles are intended to be used only with topics. One can modify QoS for internal communications of services and actions too but that is usually not recommended: those primitives are synchronous and reliable by nature and, in the case of actions, require specific QoS policies to achieve stateful communication. For these reasons, the `dua_qos` libraries provide only QoS profiles for topics.

The first two categories are `Reliable` and `BestEffort`: these map to two distinct libraries in both packages, whose contents are almost the same. The difference they enforce is in the reliability policy termed `ReliabilityPolicyKind`, *i.e.*, the reliability part of the QoS policies provided by each library. There is no additional distinction between the two categories since it is actually up to the user to decide which one to use: today, the `Reliable` category applies to practically every transmission since network congestion and bandwidth optimization are managed independently by the communication middleware used by ROS 2, mainly thanks to the `HistoryPolicy`. This policy is set to `KEEP_LAST` by default, which means that only a fixed-size buffer of samples is kept in memory to be either sent or received, and that the rest is discarded, independently of the configured reliability which comes into play shortly after, when samples are actually being transmitted; this mechanism, although not sophisticated, acts as a sort of congestion control that works well in most cases. All APIs in the `dua_qos` libraries allow to set the depth of such buffers but provide a default value that is usually good enough for most cases.

For each package, both `Reliable` and `BestEffort` libraries then provide QoS profiles via the following

API.

- `get_datum_qos`: returns a QoS profile for generic data, *i.e.*, data that is not particularly large in size or requires special handling on the QoS size. This is due to the above reasoning, thanks to which both commands, setpoints, and sensor data are, *e.g.*, the same thing.
- `get_scan_qos`: returns a QoS profile for LiDAR scans, spatial scans, or similar data. The peculiarity of this data is that while it may have relatively high or low frequencies, it gets easily very large in size, so a low history depth is suggested here to reduce the risk of network congestion and memory exhaustion.
- `get_image_qos`: returns a QoS profile for images, whose size may vary from small to very large but whose frequency rarely exceeds a few tens of Hertz. The history depth default value is not as low as for scans but still lower than for generic data and configurable by the user.

The only difference among the two libraries is that the `Reliable` one also provides another method named `get_persistent_qos`, a particular type of policy intended for reliable transmissions that may be resumed at a later point in time. This is possible thanks to a feature of the DDS standard called the `DurabilityPolicy`, discussed in Section 3.1.1, which allows a publisher to store all samples in a persistent storage and send them again them when a new subscriber appears. This is useful for data that may not be produced continuously but is still important, like configuration data or state information, and is used in this library together with a higher history depth to ensure that all samples are stored and sent.

5.1.3 Improving node development: params_manager, dua_node

Among the many ROS 2 features (see Section 3.1.3), there is and important one concerning the nodes: the possibility of specifying configurable variables for them that can be inspected and set at any point in time, known as *node parameters*. These parameters are usually set at launch time, either by the user or by the launch system via configuration files, and are then read by the node to configure its behavior. The ROS 2 middleware provides an API to manage these parameters, whose task is that of building and storing in each node a database of the parameters that it has. The keys used in this database are strings that contain the name of a parameter. Each parameter is then associated with its type, which can be a string, an integer, a floating-point number, a boolean, or a list of any of these types, and its current value. There are also other flags and fields, identifying whether a parameter is read-only or, in case it is of numeric type, its valid ranges.

ROS 2 configuration files allow the user to specify only parameter values, meaning that all the aforementioned information has to be provided when the parameter is *declared*. In ROS 2 code, a *parameter declaration* is the act of calling an API that creates an entry into the parameter database of the node, writing all such information which is passed as arguments to the API. One has to write quite the quantity of code to correctly declare parameters passing all the correct and relevant information. This is not a problem when the number of parameters is small but it can become inconvenient when the number of parameters grows, especially if they are of different types and have different constraints. As mentioned earlier, this is a classic example of "boilerplate" code.

Another issue concerns parameter updates and their usage in the actual code of the node. Each time a parameter has to be read, the node code has to include an API call that retrieves the parameter from the node's database and then casts it to the correct type. This is not a great concern if the parameter is read only once or with a low frequency but when it is, *e.g.*, the gain of a control law that must be evaluated at each iteration of a fast control loop, the overhead caused by the database queries might become significant. Assuming that the parameter type is never going to change, it is preferable to store the parameter value directly in the node and update it only when the parameter is updated. This way, parameter read operations are much faster. To achieve this, a callback has to be registered to be called when an update is requested for a set of parameters: when this happens, some checks like the range of the parameter have already been performed but other ones like the type of the parameter have not.

The amount of code that one has to write to achieve the aforementioned functionalities, in every ROS 2 software and every time in the same way, grows uncontrollably with respect to the amount of parameters that each node has. The code that manages parameters in a ROS 2 node becomes unnecessarily long and complex, difficult to manage and debug. This is the motivation behind the development of the `params_manager` package [154], which provides a set of utilities to simplify the management of parameters in ROS 2 nodes. This package currently contains only an implementation for C++ packages.

The first component of the `params_manager` package is a Python script that can be used to automatically generate the code that declares the parameters in the ROS 2 node class. This script is automatically called by a CMake directive that extends the ament installation (see Section 3.1.3). Thus, the first step, as shown in the documentation of [154], is to add a YAML file to the source code of the node that, with a specific syntax shown in Listing 5.1, contains the list of parameters that the node has to manage together with all their metadata. The script then parses this file and generate a C++ source file that is added to the node build process, containing a routine called `init_parameters` that declares all parameters in the node and registers the update callback.

```yaml
header_include_path: dir/node_header.hpp
namespace: node_namespace # This is optional
manager_name: manager # This is optional
node_class_name: MyNode

params:
  camera_offset:
    type: double
    default_value: 0.1
    min_value: 0.0
    max_value: 0.5
    step: 0.0
    description: "Distance between camera and drone center."
    constraints: "Must be in meters."
    read_only: false
    validator: function_name3 # This is optional

  hsv_from:
    type: integer_array
    default_value:
      - 0
      - 1
    min_value: 0
    max_value: 255
    step: 1
    description: "HSV color lower range."
    constraints: "."
    read_only: false
    var_name: hsv_from_ # This is optional
```

Listing 5.1: Example of `params_manager` YAML configuration file [154].

Listing 5.1 shows also two other useful features of the `params_manager` package. The first is the possibility to specify either a *validator* routine or a variable name: the autogenerated parameter update callback checks the name and type of a parameter and, if a match is found, may perform other operations before approving the update. One of these is to perform an additional check, whose outcome decides if the parameter update is approved, which may be coded by the user in a function informally referred to as a *validator*. Alternatively, to account for the necessity to quickly store the new value directly inside the node as explained above, the user can specify the name of a variable member of the node class that the callback accesses and overwrites with the new parameter value. Of course, the two things can be done at the same time: it is up to the user in the first case to code the variable member update inside the validator routine. To keep track of each parameter's metadata together with these two additional values, the `params_manager` library uses the `std::set` data structure from the `<set>` C++ standard library header, which is implemented as a red-black tree for maximum efficiency [102].

After the `params_manager` library has been implemented and successfully tested, the question arose if it is possible to eliminate the necessity of the user to write the five lines of code that instantiate the `Manager` object, call the `init_parameters` function in the node constructor, and then clear the `Manager` at the end. Hiding the `Manager` would be possible in case the base ROS 2 node class that each node is derived from already included an object of the `Manager` class as a variable member. In the `dua_node` package [148], an alternative ROS 2 base node class is provided that includes the `Manager` object and handles its lifecycle together with that of the node. This way, the only necessary lines of code to manage parameters in a ROS 2 node are those that declare and call the `init_parameters` function; this is due to the fact that, by the current implementation, `init_parameters` is a member function of the derived class and can only be called by its constructor, after the base class one has run. The `dua_node` package is intended to be used in conjunction with the `params_manager` package, and is included in the `dua-utils` collection.

5.1.4 Wrapping clients: simple_serviceclient, simple_actionclient

Section 3.1.3 presented ROS as a project aimed at providing a middleware to robotic software developers that offers many services, the most important of which is distributed communication. To this end, ROS 2 provides three communication primitives: topics, services, and actions. The last two are synchronous communication primitives, meant to implement client-server communication over a short time span in the former case, and a possibly longer one in the latter case, with the additional feature of being stateful. The ROS 2 middleware provides APIs to create and manage services and actions, which are grouped in the server and client sides. Servers are usually simple to implement: the user instantiates the server object and defines the relevant callbacks for it. In the case of actions there are usually multiple callbacks involved to account for every possible asynchronous event requested by the client or terminal state (see Figures 3.8 and 3.9) but the developer can choose which ones to implement based on which features of the communication their code shall support.

For both services and actions, the client side is completely different. In both C++ and Python, the client-side API is based on asynchronous programming, which is a programming paradigm that allows one to perform multiple tasks concurrently without the need to wait for the completion of one before starting the next. This is achieved by using callbacks together with *futures*, which can be essentially described as variables whose value becomes available at a later point in time.

The developer writing code making use of a service client has to first instantiate such an object, and then send a service request through it. The result of this API call is a future, which eventually holds the value of the service response. The developer can act in three ways here, all made possible by the ROS 2 API: they can block and synchronously wait for the future to get a value, they can attach a callback to the future that is called when the value is available, or they can poll the future at any time they want in the code and check for its value. The API to manage each of these cases is not particularly complex but it can get quite verbose. As for all the previous sections describing packages of `dua_utils`, the next one is motivated by a convenience that arises when one writes actual ROS 2 software: in the vast majority of cases, if a service is called, the response data is necessary right after the call. This is why the first eventuality among the three described above is the most common but it still requires the developer to write quite some code to manage the future, put the node in a synchronous wait state by spinning an executor, and then get the value from the future. This is the reason why the simple_serviceclient package [152] has been developed.

This package offers two equivalent libraries for both C++ and Python, which offer the same methods in both languages. The libraries implement a client class that wraps the original ROS 2 service client, extending its functionalities. Since the other situations are better handled with the official ROS 2 API, the simple_serviceclient libraries offer only the following two methods.

- call_sync: this method sends a service request and waits for the response, returning it when it is available. The method is blocking and takes a flag argument telling whether the node has to be added to an executor or not; this is to support the situation in which the node is already added to an executor and callbacks execute in separate threads, so waiting for the future consists in blocking the calling thread waiting for it to get a value using the future object API. In both scenarios, the only code that can execute while the future is being waited for is that of node callbacks.
- call_async: this method sends a service request and returns a future that holds the response when it is available. The method is non-blocking and provided for convenience, so that this client object can be used in all scenarios.

The C++ library contains a template class whose template parameter is the type of service, so that the code can automatically handle the service request and response types.

The wider variety of cases that the stateful communication provided by actions defines is the main reason behind the complexity of its client side. Essentially, each operation that the client requests to the server is implemented as a service call and must be treated in the code in a way similar to what has been described before, *i.e.*, using futures and eventually callbacks. Note that even the most desirable outcome for an action from the perspective of the client, *i.e.*, completing a goal, requires two service calls: one to request the acceptance of the goal and one to request and get the result (see, again, Figure 3.9). This easily leads again not only to "boilerplate" code but also to complex code that is difficult to manage and debug. Moreover, there is an additional problem: the ROS 2 action API is not as well organized as the service API, due to its history. It has been one of the last features to be ported, regarding the communication services, from ROS 1 to ROS 2 and its implementation is as such split among two sets of libraries: the language client library, that is rclcpp for C++ and rclpy for Python, and the rclcpp_action and rclpy.action libraries. This dichotomy, still unresolved, has led to a situation in which the style of the client API might differ even significantly between its different parts, which reduces clarity and increases the learning curve for the developer. One of the most evident examples of this is the numerous types that are involved in client-side action programming, which model goals, their handles, futures, feedbacks, etc.. The API is very powerful and allows one to achieve sophisticated asynchronous, event-based behaviors to handle scenarios of various degrees of complexity but the problem is the same as before. In reality, there are mostly two ways in which actions are called: either the client sends a goal and waits immediately after for the result, or the client sends a goal and then either polls for the result or requests a goal cancellation. The simple_actionclient package [151] has been developed to address these issues, providing an action client class that acts as a wrapper for the original action client. The package contains two libraries, one for C++ and one for Python, whose contents and implementations are the same. For simplicity, the following explanation refers only to the C++ library, which implements a template class.

The first big improvement of this class over the original API is that it completely hides all the type definitions that are required to handle an action client: the only template parameter is the action type. This leads to a very simple instantiation of an action client, where all type definitions are handled internally by the class. Then, the class implements three sets of methods that implement the most important moments of the lifecycle of an action call. For each, this library also provides both synchronous and asynchronous variants, which counts to six methods in total, as follows.

- `send_goal`: sends a goal to the server and returns a future that holds the result of the goal request.
- `send_goal_sync`: sends a goal to the server and returns the result of the goal request.
- `cancel`: cancels the goal that is passed as argument and returns a future that holds the result of the goal cancellation.
- `cancel_sync`: cancels the goal that is passed as argument and returns the result of the goal cancellation.
- `get_result`: requests the result of the goal that is passed as argument and returns a future that holds the result of the goal.
- `get_result_sync`: requests the result of the goal that is passed as argument and returns the result of the goal.

Finally, the `simple_actionclient` library also provides a method to handle the entire lifecycle of an action call in one go, from the goal request to the reception of the result: the `call_sync` method. The signature of this method for the C++ library is shown in Listing 5.2. The first thing to note about it are its arguments: apart from the goal to send, it takes a flag and a series of timeouts. The flag tells the method whether to cancel the goal if a timeout occurs, and the timeouts are the maximum time that the method waits for the goal request to be sent, for the result to be received, and for the goal to be canceled, respectively. The method returns a tuple, implemented in C++ with the `std::tuple` standard library class, with three elements:

- a boolean flag that tells if the goal has been accepted or not by the server;
- a `ResultCode` object, which is an enumeration defined in the `rclcpp_action` library that tells the result of the goal request, *e.g.*, if the goal has been accepted, rejected, or canceled;
- a shared pointer to the result of the goal, which can be valid or not depending on whether the server actually returned a result.

Internally, the method performs each step by requesting each operation to the server and then synchronously waiting for the response, spinning the node. As the call progresses, the method checks for timeouts handling all possible outcomes and return values. When the return value of this method is accessed, by the way it is designed as a tuple holding the three values discussed above, the caller can immediately determine how the action call went since those three values together encode every possible outcome of the sequence of operations that the method performs. This method is the most powerful and flexible of the library and is intended to be used in the vast majority of cases where an action is called. The other methods are provided for convenience and to allow the user to handle each step of the action call lifecycle separately if needed.

```
std::tuple<bool,rclcpp_action::ResultCode,std::shared_ptr<ActionResultT>>
  call_sync(
    const typename ActionT::Goal & goal_msg,
    bool cancel_on_timeout = false,
    int64_t send_goal_timeout_msec = 0,
    int64_t get_result_timeout_msec = 0,
    int64_t cancel_timeout_msec = 0);
```

Listing 5.2: Signature of the `Client::call_sync` class method from the C++ action client wrapper library `simple_actionclient_cpp` [151].

5.1.5 Implementing ROS 2 automata: transitions_ros

As illustrated in Section 2.1, the software that operates a robot typically follows a hierarchical organization. The lower levels usually consist of firmware, controllers, and device drivers. More advanced modules rely on the abstractions provided by these lower layers to perform sophisticated operations. At the top, a single module typically acts as a coordinator, ensuring the proper functioning of the entire system and offering interfaces to human operators. This module usually requires complete access to the software stack of the robot for control purposes and its design is sophisticated enough to handle the diverse situations that might arise during the execution of the prescribed tasks. The higher the level of autonomy required, the more complex this kind of software is going to be.

Finite-state machines (FSMs) are a renowned tool in computer science [242]: their mathematical foundation describes a system whose behavior is represented by a sequence of discrete configurations within a finite set, called *states*. These states are interconnected by a collection of *transitions*, which may lead to another state or loop over the same one. Transitions occur when specific conditions are met, which alter the current state of the machine. FSMs have been widely utilized in modeling various hardware and software systems, ranging from control algorithms [113] and communication protocols [32] to digital control systems [202, 220] and distributed systems [217].

It is now essential to clarify the type of FSM relevant to this use case. In software engineering, two main categories of FSMs are recognized. The first category, often referred to as a *passive* FSM, is the more commonly used of the two. This type mainly serves as an internal representation of the state of an object or algorithm, acting as a data structure that holds information about the current state without performing any actions. Methods that interact with a passive FSM are called externally and their primary role is to keep a record of the execution state of the program based on the information encoded within the FSM. The module presented in this section focuses on the second category of FSM, known as an *active* FSM. Unlike its passive counterpart, an active FSM is an autonomous entity that retains state information and actively executes operations on data and interfaces with other system components by calling appropriate APIs. An active FSM plays a crucial role in autonomous robotics by governing the behavior of the robot. This includes decision-making processes, executing movement commands, and interpreting data from its various subsystems. It is one of the most widely used paradigms for developing supervisory software for autonomous robots, as it provides a structured approach to managing the robot's behavior.

This discussion clarifies that, when looking forward to develop a supervisory software for an autonomous robot that can be modeled as a finite-state machine, an active FSM is an appropriate choice. Moreover, if the software running on the robot is based on ROS 2, having an active FSM implementation which is designed to directly support ROS 2 communication primitives and other middleware functionalities is highly beneficial. The `transitions_ros` package [142] has been developed to address this need: it provides the implementation of an active FSM capable of seamless interaction with ROS 2 software, thereby enabling sophisticated autonomous control functionalities. Such an active FSM is intended to facilitate high-level autonomy in robotic systems, allowing them to operate effectively in dynamic environments by integrating sensory inputs, executing control logic, and interfacing with other modules cohesively.

When dealing with a ROS 2-based robot, one can broadly isolate three types of tasks that an automaton handles.

- **Asynchronous:** the robot reacts to some external events that do not alter its state but require data processing, *e.g.*, detecting a target.
- **Synchronous:** the robot executes operations specific to its current state, *e.g.*, arming, takeoff, landing, or exploring the environment.
- **Collaborative:** the robot performs sophisticated operations requiring real-time coordination with another agent or human operator.

Given the three communication primitives offered by ROS 2 (see Section 3.1.3), a logical approach for implementing a ROS 2-enabled FSM is to handle asynchronous tasks through appropriate topic subscription callbacks. In contrast, services and actions are ideal to manage synchronous and collaborative tasks. The FSM implementation should thus include a reference to a ROS 2 node object acting as a communication endpoint towards the middleware. To ensure the FSM is active in the sense described above, each state should be associated with a specific *routine*, *i.e.*, a function executed upon entry to the state, whose return value determines the subsequent transition.

The `transitions` [246] Python library is one of the most famous and versatile open-source software libraries that implements a finite-state machine. It has many standard features of software FSM implementations, such as support for hierarchical state machines and guard conditions evaluation. However, the only crucial feature here is the possibility of easily defining states and transition tables. Strings identify both states and transitions by name and the library supports regular expressions to make some definitions quicker such as, *e.g.*, catch-all transitions to an error state. With respect to the other use cases described so far in previous sections, the choice of this library which is based on the Python programming language has been motivated by its high-level nature, which reduces the time and code complexity required to encode and test even sophisticated routines, deeming it appropriate for a supervisor software module whose implementation should be as straightforward as possible.

The proposed `transitions_ros` library [142] starts by extending the `State` object of the `transitions` library: each state holds a reference to a ROS 2 node object and a function to call as their state routine by the new `routine` method of the `State` class. Such a routine has to be defined elsewhere in a software module that uses this library to build a specific automaton, takes a ROS 2 node as argument to interact with the rest of the software architecture, and returns a string that identifies the next transition that must occur based on the outcome of the routine itself; such a string should be empty in the case of a terminal state. An example of transitions and state tables for the `transitions_ros` library is given in Listing 5.3. Next, the library extends the `Machine` object in a similar way: a machine is created as an independent object that holds information about states and transitions, as well as a `run` method that sets things in motion. The `run` method, when called, executes the routine of the initial state, performs subsequent transitions, and calls new state routines based on their string return value, stopping only when an empty string is returned by some routine which indicates that the automaton execution has to stop. Finally, to receive and process the incoming data and requests of the node while the automaton is, *e.g.*, synchronously waiting for some task to finish, the `Machine` object offers the `wait_spinning` method that adds the node to an executor and then spins it up to a configurable timeout.

```
transitions_table = [
    ['START', 'INIT', 'WAIT_FOR'],
    ['READY', 'WAIT_FOR', 'FIRST_TAKEOFF'],
    ['AIRBORNE', 'FIRST_TAKEOFF', 'EXPLORE'],
    # ...
    ['MISSION_DONE', 'DONE', 'RTH'],
    ['AT_HOME', 'RTH', 'COMPLETED'],
    ['ERROR', '*', 'ERROR']
]

states_table = [
    {'name': 'INIT', 'routine': init_routine},
    {'name': 'RTH', 'routine': rth_routine},
    {'name': 'COMPLETED', 'routine': completed_routine},
    {'name': 'ERROR', 'routine': error_routine}
]
```

Listing 5.3: Example of transitions and state tables in the `transitions_ros` library [142].

The code of a ROS 2 Python application using this library has to, in order, define a ROS 2 node class with all the required interfaces, specify states and transitions tables, define routines for each state, then create both the node and machine objects and call the `Machine.run` method in a dedicated thread. The proposed implementation keeps the code concisely organized by design, allowing each of the definitions to be performed in a dedicated source file. To conclude, it is instrumental to observe how this FSM implementation handles the classes of tasks mentioned before.

- **Asynchronous:** their triggering or processing can be encoded within topic subscription callbacks of the node, which may be executed during a call to `wait_spinning` or in a similar busy-wait context and may modify data that influences the automaton execution flow.
- **Synchronous:** these can be performed entirely within state routines by calling, *e.g.*, service or action clients of the node that invoke specific operations in the underlying software modules, and inspecting their results.
- **Collaborative:** these usually require a mixture of the two previous approaches, since the task execution might be synchronously coordinated by all parties but some of its conditions might arise asynchronously, *e.g.*, the mutual triggering of the task itself among multiple autonomous robots or the human operator.

Finally, note that the design of this active FSM implementation integrates very well with the client wrappers for services and actions presented in Section 5.1.4.

5.2 Developing robotic software with DUA

With the introduction of the `dua-utils` software collection in Section 5.1, the description of the Distributed Unified Architecture implementation and features is complete. Before discussing some results that led to its development, it is instrumental to see this framework in action. This section presents two software modules that are some of the most representative of the issues that DUA aims to solve, as well as of the functionalities that it provides to the developer to tackle them. Their choice is also motivated by the fact that they have been successfully employed in real-world scenarios during the course of this work, some of which are described in the following sections. In both cases, the aim is mainly to show how the DUA framework has been used to develop them and solve the integration issues that they presented; thus, an introductory description of their purpose and functionalities is given, followed by an explanation of how they have been configured as DUA units.

5.2.1 Autonomous flight: flight_stack

This project is a collection of software modules for autonomous drone flight and low-level control interfacing. The term *autonomous drone*, in this context, identifies a flying robot: its structure can either be that of a multicopter, or a fixed-wing aircraft, or anything in between that flies, but it has no human operator for most of its operating time. Instead, the flight control and stabilization as well as planning tasks are entirely handled by onboard electronics, *i.e.*, integrated computers and microcontrollers. The drone is supposed to be equipped with sensors of various kinds, such as inertial measurement units (IMUs), global navigation satellite systems (GNSS), LiDARs, and cameras, and actuators such as servos and motors.

Among the many kinds of autonomous robots, drones are one of the most difficult to develop and operate. This is due to their nature, which requires them to operate in three-dimensional space, and to the fact that they are used in scenarios where the environment may not be entirely known. The greatest complexity is due to the fact that drones are unstable systems, which means that they require continuous control to remain in the air and to perform the tasks they are assigned. Such control is usually achieved by means of a control loop that reads sensor data, computes the necessary corrections to the drone's attitude and position, and sends commands to the actuators in the form of torques and thrusts. The control loop runs at a frequency usually in the order of hundreds of Hertz, to ensure that the drone remains stable and responsive to external disturbances; for this to work, the most important source of feedback for an autonomous drone is the IMU, which provides the drone's attitude and angular velocity at a frequency that can get as high as 1 kHz. This tight timing requirement implies that basic flight stabilization and control is handled by a dedicated microcontroller, on which it runs as a task of some real-time operating system (RTOS) that ensures that the control loop runs at the required frequency.

The Pixhawk project [160, 161] represents a significant advancement in open-source flight control systems, establishing comprehensive hardware and software standards for autonomous aerial vehicles. The project implements a modular hardware architecture that supports multiple communication protocols and enables seamless integration of diverse sensors and actuators. This hardware framework operates in conjunction with the PX4 firmware [159], an open-source flight control software developed at ETH Zurich and subsequently expanded under the Dronecode project [63]. PX4 provides a modular control architecture that incorporates autonomous navigation, sensor fusion, and failsafe mechanisms, making it suitable for applications ranging from hobbyist projects to industrial systems. The combination of Pixhawk hardware standards and PX4 firmware capabilities creates a robust ecosystem that supports diverse autonomous operations, including agricultural monitoring, environmental data collection, and search and rescue missions. The system architecture emphasizes modularity, allowing individual components to be enabled or disabled based on mission requirements while maintaining efficient operation on resource-constrained hardware. Through sophisticated attitude estimation, position control, and advanced sensor fusion algorithms, this integrated platform delivers reliable state estimation even in challenging environments, establishing itself as a leading solution in both research and industrial applications.

Even if PX4 has evolved into a very powerful and flexible platform, there are some tasks that are still beyond its capabilities. To correctly perform its main task of basic flight stabilization, it has to remain a real-time software that runs on dedicated hardware. This means that it cannot be used to perform

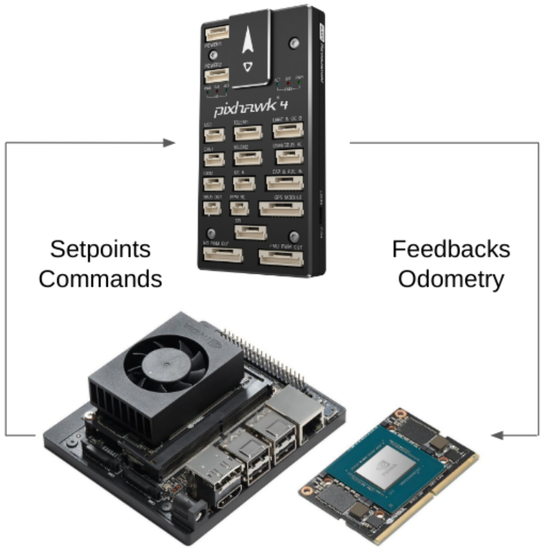

Figure 5.1: Functional diagram of a hardware architecture for autonomous flight: a Pixhawk 4 flight management unit (FMU) (top), and an Nvidia Jetson Xavier NX SBC (bottom).

high-level tasks that require more computational power, such as computer vision, path planning, or decision-making. This is why the most recent control architectures for autonomous drones usually include a *companion computer, i.e.*, an SBC that runs a full operating system and that is connected to the flight controller through a serial or Ethernet link. The companion computer is responsible for all the high-level tasks that the drone performs, and it communicates with the flight controller to send to it the necessary commands. This way, the drone can perform sophisticated operations such as autonomous navigation, obstacle avoidance, or target tracking, while still being able to fly and stabilize itself thanks to the flight controller. This situation is depicted in Figure 5.1.

The flight management unit (FMU) is usually capable of passing all the necessary data to the companion computer, such as the drone's attitude, position, and velocity, as well as the status of the sensors and the actuators. The companion computer can then use this data to perform its tasks and send the commands back to the FMU. From the point of view of the companion computer, the drone is essentially the FMU: there is no distinction, since all the actuation and sensing technicalities are addressed by the FMU. Recalling the introductory discussion made in Section 2.1, since an autonomous drone can be viewed as an autonomous robot, it is reasonable to suppose that its software architecture still follows a hierarchical organization. The FMU offers just a set of communication channels over which data can be sent but all of the technicalities involved in parsing this data, or sending the right commands and correctly parsing their feedback to request even a simple operation remain unsolved. Demanding this task to every single software module in the higher levels of the drone's control architecture leads to a seriously flawed design, just for a simple issue: if every module can talk to the FMU at the same time, who is actually controlling the drone at any given time? It appears clear at this point that, in such a sophisticated system, an entire layer of the architecture is reserved to modules that are in charge of interfacing with the FMU, abstracting the complexities of the communication protocols and the data formats, and providing a clear and consistent interface to the higher-level modules while handling all the necessary synchronization and arbitration tasks. This is the role of the `flight_stack` [150] DUA unit, which offers these functionalities thanks to the ROS 2 middleware and the PX4 firmware. Currently, it supports only multicopters but support for different types of aircraft is an ongoing development, and is based on version 1.13.3 of PX4. The `flight_stack` repository contains five ROS 2 packages, the main three of which are called `px4_msgs`, `micrortps_agent`, and `flight_control`.

The `px4_msgs` package comes first since it contains a set of message definitions that are used to communicate with the FMU in both directions. These definitions correspond both in terms of fields and data types with the data packets that the modules of the PX4 firmware exchange internally among themselves. Thanks to this similarity, a module of the PX4 firmware easily serializes and deserializes packets coming from a serial UART link that connects the FMU with the companion computer. This module is called the `micrortps_client`, and it is one of the real-time tasks of PX4; its peculiarity is that it follows the same real-time data transmission protocol that is implemented as part of the DDS standard presented in Section 3.1.1. The other end of this communication is a task that runs on the companion computer, implemented as a ROS 2 node in the `micrortps_agent` package. The operational task that this node performs is that of managing the serial interface connected to the FMU and interfacing it with the ROS 2 domain. For every different function of the PX4 firmware, there is a dedicated pair of message type and ROS 2 topic that the FMU either publishes or subscribes to. To make this possible, the `micrortps_agent` node publishes or subscribes to the same ROS 2 topics that it offers as communication interfaces towards the FMU to other ROS 2 nodes running on the companion computer, handling FMU data serialization and deserialization as well as managing the serial UART link. To efficiently interface itself with the physical layer, as well as with ROS 2, the `micrortps_agent` makes use of both ROS 2 RMW and bare DDS APIs at the same time, the latter of which are provided by the eProsima Fast DDS implementation [72].

Having available a set of ROS 2 topics that expose all the functionalities of the PX4 firmware solves only the first half of the problem: abstracting the serial communication link and its technicalities, like message queueing and flow control, and translating it into a set of publish-subscribe communication primitives. What remains to be addressed is the abstraction of the whole drone, offering to the higher layers of the control architecture only a set of communication primitives which may even be more sophisticated than simple message topics but still hides a lot of complications concerning the execution of elementary tasks that the drone performs such as, *e.g.*, takeoff and movement. This is the purpose of the `flight_control` package, which is the core of the `flight_stack` DUA unit.

In the control architecture of an autonomous drone, the `flight_control` node is the first one that does not directly interface any hardware: to it, thanks to the first abstraction provided by the `micrortps_agent`, the drone appears as another node in the architecture that communicates over a set of topics. The first, basic functionalities that the `flight_control` offers are in fact based on this communication primitive: other nodes are expected to exchange data directly with the `flight_control` using standard formats and it then translates the data into formats that PX4 expects and forwards the packets to the micrortps_agent, which is in charge of sending them to the FMU on the serial link. Such first basic functionalities, all implemented with topics, include the following.

- Position control, which is implemented by forwarding local position setpoints to PX4: nodes can send a setpoint in the form of a ROS 2 message, and the flight_control translates it into a PX4 message and forward it to the FMU; note that this is also how hovering of a multicopter is achieved, since the flight_control retains and republishes the most recent position setpoint at a frequency that avoids triggering PX4 failsafes.
- Velocity control, which is implemented by forwarding twist setpoints in body frame to PX4.
- Local pose data forwarding, for navigation in GNSS-denied environments or when an additional source of local position and orientation data is available and provided to PX4. This data has first to be transformed in a NED (North-East-Down) frame and then forwarded to PX4.
- PX4 local pose estimate republishing.
- Battery monitoring.
- PX4 log message monitoring.

More sophisticated functions are implemented using services and actions, that other nodes in the architecture can call acting as clients to request them and wait for their completion, receiving feedbacks in the meantime. These functions include the following.

- Arming.
- Disarming.
- Takeoff.
- Landing.
- Basic movement to a specific position.
- Turn to a desired heading at the current position.
- FMU reboot.

For each of these functions, the flight_control action and service servers exchange data with specific topics handled by the micrortps_agent, gathering feedback and handling all possible outcomes and error conditions. The node also internally keeps track of the current operational state of the drone, *e.g.*, if it is armed or not, to allow every client to request only the operations that are possible in the current state. The flight_control node is also in charge of handling the synchronization of the requests that it receives, to avoid that multiple clients request conflicting operations at the same time, and to ensure that the drone is always in a consistent state.

The relevant portion of the `flight_stack` within the context of this work is its configuration as a DUA unit. It currently supports, and has been tested with, multiple targets ranging from SBC ones such as `jetson5`, to development ones such as `x86-cudev`. The unit configuration section of the Dockerfile for the `jetson5` target, intended for Nvidia Jetson SBCs presented in Section 4.2.2, is shown in Listing 5.4. Since no firmware development is performed on the SBC that is to be installed on the drone, the only configurations required concern the dependencies that are necessary for the `micrortps_agent` to work. These include some Python packages, as well as an installation of the Fast-DDS-Gen tool by eProsima [73]: this is required to generate IDL files that define messages for both the ROS and the serial side of the communications that the `micrortps_agent` handles.

Development targets add to the aforementioned dependencies additional setup steps that configure what is required to build the PX4 firmware. For reference, Listing B.1 in Appendix B reports the unit configuration of the `x86-cudev` target. Alongside the same steps performed in SBC targets, more dependencies are installed which are necessary to build the NuttX RTOS [13], which is the preferred real-time operating system to run the PX4 firmware on Pixhawk boards. Furthermore, since a cross-compiler is required to build the firmware in an x86 environment, version 9 of GCC [86] for ARM is downloaded and decompressed. Finally, version 2.10.1 of eProsima Fast DDS and its dependencies are downloaded and built from source, to provide support for the compilation of the PX4-side of the communication protocol that the `micrortps_agent` handles.

The `flight_stack` DUA unit is a clear example of how DUA can be used to solve the integration issues that arise when developing software for autonomous systems. Focusing on the development targets, the initial dependencies are easily installed since they are distributed as packages for the system package manager and thus may be made part of the host system installation. The latter tools, however, are built from source and then installed system-wide, a process that may only be scripted at best. In contrast, DUA automates these integration and setup steps in a way that is transparent to the developer who, to use this unit, only needs to add it configuring their project targets as described in Section 4.3.2.

```
# flight_stack START #
# Install PX4 general dependencies required by microRTPS Bridge
RUN apt-get update && apt-get install -y --no-install-recommends \
  astyle \
  libxml2-dev \
  libgstreamer-plugins-base1.0-dev \
  rsync && \
  rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*

# Install PX4 Python dependencies required by microRTPS Bridge
RUN . /opt/dua-venv/bin/activate && yes | pip3 install -U \
  future \
  jinja2>=2.8 \
  jsonschema \
  kconfiglib \
  lxml \
  psutil \
  pygments \
  pymavlink \
  pyulog

# Build and install Fast-DDS-Gen (required by microRTPS Bridge)
WORKDIR /opt
RUN git clone --recursive --single-branch --branch 'v2.4.0' \
  https://github.com/eProsima/Fast-DDS-Gen.git && \
  cd Fast-DDS-Gen && \
  ./gradlew assemble && \
  ln -s /opt/Fast-DDS-Gen/scripts/fastddsgen scripts/fastrtpsgen && \
  cd .. && \
  chgrp -R internal Fast-DDS-Gen && \
  chmod -R g+rw Fast-DDS-Gen
WORKDIR /root
ENV PATH=/opt/Fast-DDS-Gen/scripts:${PATH}
# flight_stack END #
```

Listing 5.4: Unit configuration of the `jetson5` target of the `flight_stack` DUA unit [150] as of November 26, 2024.

5.2.2 Driving a sensor: zed_drivers

One of the hardest problems that autonomous robots face is that of localization: determining continuously its position and orientation (*i.e.*, its *pose*) within the environment. Robots are equipped with sensors, whose variety ranges from inertial measurement units to cameras, LiDARs, and GNSS receivers. These devices provide data that can later be used, either directly or by appropriate post-processing algorithms, to obtain an estimate of the current pose of the robot. The most common approach is that of sensor fusion, which consists of combining the data from multiple sensors to obtain an estimate of the robot's pose.

Among the many classes of sensors, RGB (Red-Green-Blue) cameras are one of the most versatile and widely used in robotics. They can range from cheap and lightweight solutions including, *e.g.*, a single optic and producing only a video stream, to more expensive but sophisticated ones that include more optics and can provide additional information, such as depth maps. One of the most famous series of cameras aimed at robotics applications is the ZED series by Stereolabs [223]. The ZED cameras are stereoscopic cameras, *i.e.*, they mimic the human eye by housing two identical optics that are separated by a fixed, known distance. The sensors on each optic are synchronized, thus the frames they produce can be used to compute a depth map of the scene that the camera is viewing. This depth map can be then be expressed in a variety of formats, from grayscale images to point clouds, and can be employed in a variety of contexts, from obstacle avoidance to 3D environment mapping. ZED cameras have emerged as a solution also due to their software support offered by Stereolabs: the ZED SDK [224]. It is a versatile software development kit that enables programmers to easily interface with a ZED camera and get its data. One of the most famous disadvantages of working with images in real-time is the high cost in terms of computational power that is demanded to the host system: processing pictures usually requires a lot of memory and CPU power, and the more complex is the processing, the more resources are needed. The ZED SDK, however, has been designed to take advantage of Nvidia GPU devices present on the host, being it either a desktop computer or an SBC: thanks to the CUDA parallel computing platform, the internal algorithms of the ZED SDK can run in real-time and produce high-quality results even on low-power SBCs and similar devices. Thanks to this feature, the ZED SDK integrates a positional tracking module that, if activated, provides a real-time estimate of the pose of the camera in the environment, with respect to a reference frame that is placed at the camera's starting position. This estimate is obtained internally by the SDK algorithms by fusing the data from the camera's optics and the IMU.

In the context of a robotic software architecture based on ROS 2, the integration of a sensor like a ZED camera requires the development of a specific module that abstracts it and publishes its data to the ROS network in standard formats. Such a module is usually called, in the ROS world, a *driver node*: note that it is not a device driver in the sense that this term usually implies in the operating systems world but rather a software module that interfaces with a device and exposes its functionalities to the rest of the software architecture. It is called *driver* because its task is that of driving the device, *i.e.*, of making it work and of providing a clear and consistent interface to the rest of the nodes in the architecture, who should not be concerned with the technicalities of the device itself. The code of the driver node is based on the SDK of the device itself, if any, which is the ZED SDK in the case of the package discussed in this section: the `zed_drivers` package [153].

Like the other software modules discussed so far, `zed_drivers` is a DUA unit that contains multiple ROS 2 software packages. The main package is called `zed_driver` and it is the subject of the current discussion. To briefly introduce it, consider first the task that the corresponding node has to perform: a ZED camera is a hardware device that must first be opened, specifying a set of configuration parameters for its various functions, then started, and finally its data may be read, processed, and published to the ROS 2 network. This is, in essence, a summary of the structure of the `zed_driver` node implemented by the package.

At startup, the ROS 2 node parses its parameters: these specify many configuration options of the supported functionalities of the camera, ranging from resolution and framerate, to depth sensing mode and depth map size, to IMU sampling rate. The node then initializes the ZED camera, setting it up according to the options that the parameters specified, and starts it. The node then enters a loop where it reads the data produced by the camera. The data can be divided in four main categories: RGB frames, depth maps, positional tracking data, and IMU samples. Each of these four sets of data has to be processed before it can be published over the ROS 2 network, mainly because it has to be copied from the program memory to properly-formatted ROS 2 messages that encode standard sensor data types. To streamline execution, the `zed_driver` node runs four parallel threads that handle each type of data separately at the same time, optimizing the use of system resources to reduce latencies and ensure that the data is published before the camera produces the next frames, so that the node can keep up with the camera's framerate; the IMU thread, in particular, runs at a higher frequency sampling the camera IMU independently of the optics, since it runs at a higher rate. RGB frames have to be converted to ROS 2 image messages, depth maps to point cloud messages, positional tracking data to pose messages and transformations, and IMU samples to IMU messages. The node then publishes these messages over appropriate topics.

To summarize the node's capabilities, here are the ZED SDK functionalities that it supports, each with a complete set of configuration options exposed as node parameters.

- RGB video stream.
- Depth maps and point clouds computation.
- IMU sampling.
- Pose estimation with the positional tracking module.
- Raw camera data logging in SVO files.
- Real-time video stream over the network with NVENC encoding [182].

To achieve this, the code of the node makes wide use of the ZED SDK C++ API. Coming back to the scope of this work, this means that a compatible installation of the ZED SDK has to be present on the host system to which the camera is connected. According to the official documentation, setting it up is easy since it is distributed as a script that first installs the dependencies and then the SDK itself. The system on which the script is run has to meet a certain set of requirements: it has to be a Ubuntu [35] system with some dependencies already in place and it must have a Nvidia GPU with a version of the CUDA toolkit [177] installed which is also compatible with the version of the ZED SDK that one is installing. In the case of Jetson SBCs, things get more complicated because additional libraries, distributed by Nvidia via the JetPack SDK installation, are required. The `zed_drivers` DUA unit automates all these setup steps, making the installation of the ZED SDK and the configuration of the project repository a matter of minutes.

Before delving into two example unit configuration Dockerfiles, recall that the system environments offered by DUA base units are stable and configured in a way to offer a good set of system packages that are ensured to be compatible with each other. Still, to install an SDK that is distributed as a self-installing script, one has to do some level of work on each target to ensure that the installation runs smoothly in a reproducible way and that it actually results in a usable SDK.

The `zed_drivers` unit supports only targets aimed at systems that may have an Nvidia GPU. Listing 5.5 shows the unit configuration for the `jetson5` target. In this case, the setup is pretty straightforward: after an initial set of system dependencies that the SDK installer necessarily expects, the ZED SDK installation script is downloaded and executed. Note that, due to a compatibility issue with an Nvidia library, the version that is downloaded supports a higher JetPack version than the one on which the base unit is based: this is one of the many kinds of integration issues that usually happen in this field and that, to be solved, might require many reinstallation or board reflashes. The DUA framework, on the other hand, grants the developer the possibility to make many quick subsequent attempts without the need to reflash the Jetson board each time, by simply modifying the Dockerfile and rebuilding the image. The only critical configuration step is then the creation of a fake `/etc/nv_tegra_release` file that the SDK installer expects to find to correctly identify the system on which it is running, which apperas to be missing in Nvidia JetPack Docker images.

The unit configuration for the `x86-cudev` target is shown in Listing 5.6: surprisingly, this is more intricate than the one for the Jetson boards. First, due to other incompatibilities with the contents of the official Nvidia image, cuDNN [178] and cuBLAS libraries have to be upgraded to the latest version, the latter of which even requires a manual installation from the contents of its system package. Then, finally, the ZED SDK can be installed by downloading its installer script and running it, requiring no additional steps.

At the end of this section, after these units have been discussed as examples of how the DUA framework can help solve integration issues and provide replicable and reliable development environments, it is possible to note that thanks to the contents of these units a working prototype of an autonomous system such as an autonomous drone can be set up in a matter of minutes: first, a DUA project has to be created, adding the units to it and configuring supported targets as illustrated in Section 4.3.2. Then, the project can be deployed onto a compatible SBC to be installed on the prototype and what is left to be done is just to download the base unit image and build the target image on top of it. A development image is then also set up to build and flash the microcontroller firmware. The necessary software modules are then ready to be built, configured, and run on the SBC, and the prototype may be tested right away.

```
# zed_drivers START #
# Install ZED SDK dependencies
# (Nvidia stuff should already be installed in the base unit)
RUN apt-get update && \
  apt-get install -y --no-install-recommends \
  apt-transport-https \
  udev \
  zstd && \
  rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*

# Install the ZED SDK
# We currently support version 4.1
# Do some more black magic here to enable streaming features
# on Jetson inside a container.
# The syntax for the echo is:
# R${L4T_MAJOR_VERSION} (release), \
#   REVISION: ${L4T_MINOR_VERSION}.${L4T_PATCH_VERSION}
# NOTE: We install an SDK version for a newer version of JetPack
# than that of this base image, due to an incompatibility with the
# deprecated libnvbuf.
RUN echo "# R35 (release), REVISION: 2.1" > /etc/nv_tegra_release && \
  wget --no-check-certificate -nc -O zed_sdk.run \
  https://download.stereolabs.com/zedsdk/4.1/l4t35.4/jetsons && \
  chmod +x zed_sdk.run && \
  ./zed_sdk.run -- silent skip_python && \
  rm zed_sdk.run && \
  ln -sf /usr/lib/aarch64-linux-gnu/tegra/libv4l2.so.0 \
  /usr/lib/aarch64-linux-gnu/libv4l2.so && \
  rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/* && \
  chgrp -R internal /usr/local/zed && \
  chmod -R g+rwx /usr/local/zed
ENV LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/usr/local/zed/lib
ENV LOGNAME=neo
# zed_drivers END #
```

Listing 5.5: Unit configuration of the `jetson5` target of the `zed_drivers` DUA unit [153] as of November 26, 2024.

```
# zed_drivers START #
# Install ZED SDK dependencies
# NOTE: Here we require an upgrade to make sure
# we have the latest Nvidia libraries.
RUN apt-get update && apt-get install -y \
  --no-install-recommends --allow-change-held-packages \
  libcudnn8 \
  libcudnn8-dev \
  zstd && \
  rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*

# Install libcublas-12-0 libraries manually
# to fix some incompatibilities in ZED SDK dependencies
WORKDIR /tmp
RUN apt-get update && \
  apt-get download libcublas-12-0 && \
  mkdir contents && \
  dpkg-deb -xv libcublas-12-0_12.0.2.224-1_amd64.deb contents/ && \
  mv contents/usr/local/cuda-12.0/targets/x86_64-linux/lib/* \
  /usr/local/cuda/lib64/ && \
  rm -rf contents libcublas-12-0_12.0.2.224-1_amd64.deb && \
  ldconfig && \
  rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*
WORKDIR /root

# Install the ZED SDK
# We currently support version 4.1
RUN wget -nc -O zed_sdk.run \
  https://download.stereolabs.com/zedsdk/4.1/cu118/ubuntu22 && \
  chmod +x zed_sdk.run && \
  ./zed_sdk.run -- silent skip_python && \
  rm zed_sdk.run && \
  rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/* && \
  chgrp -R internal /usr/local/zed && \
  chmod -R g+rwx /usr/local/zed
ENV LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/usr/local/zed/lib
# zed_drivers END #
```

Listing 5.6: Unit configuration of the `x86-cudev` target of the `zed_drivers` DUA unit [153] as of November 26, 2024.

5.3 Practical applications

The DUA framework and the technologies presented so far have led to the development of some prototypes during the course of this work. These prototypes have been developed either to perform specific tasks in real-world scenarios, or to test novel solutions in fields like robot autonomy and localization and mapping. The common ground of these prototypes is the use of the DUA framework in their development and deployment, which proved the effectiveness of the proposed solutions. This section presents some published results concerning the application of the technologies on which DUA is based to the field of autonomous robotics. Their success has brought to the formalization of the DUA framework as presented in Chapter 4, as well as to the solutions presented in the previous Section 5.1.

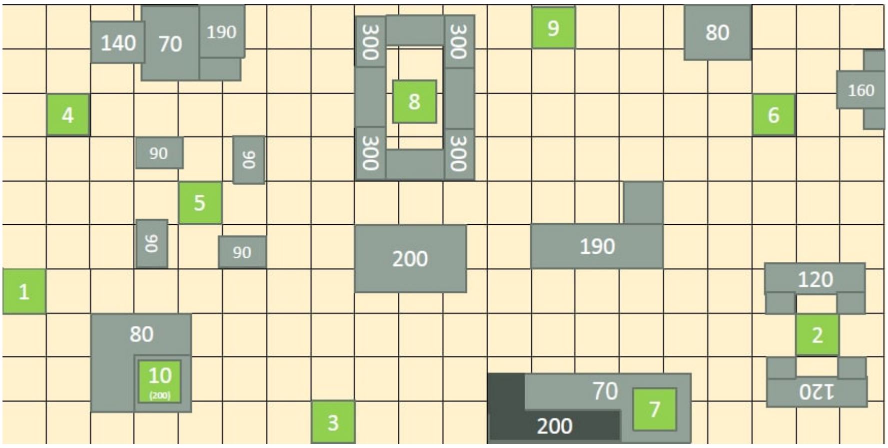

Figure 5.2: The 2021 Leonardo Drone Contest field layout.

5.3.1 A ROS 2 distributed architecture for unmanned aircraft

In [22], a novel distributed architecture is designed and proposed for an autonomous drone, tasked with the execution of a target reconnaissance and precision landing mission in an indoor, GNSS-denied environment. The mission was part of the 2021 Edition of the Leonardo Drone Contest, a robotics competition hosted by Leonardo S.p.A. in which custom-built drones have to perform a series of complex tasks autonomously, ranging from localization and flight control to environment mapping and autonomous navigation.

The research team of the Tor Vergata University of Rome participated in the 2021 Leonardo Drone Contest, which was conducted in Turin, Italy in September 2021. This competition featured six Italian universities deploying autonomous drones to execute a sophisticated mission within a structured indoor environment. The operational field was meticulously configured with various elements, including 10 distinct landing pads and 3 mobile ground robots, with all participants receiving comprehensive details about the field layout prior to the competition. The visual markers demarcating the landing pads provided additional navigational guidance. A depiction of the 2021 field layout is shown in Figure 5.2: landing pads are in green and marked with their numerical ID, while known obstacles are in gray and their height in centimeters is reported.

The mission objectives for each drone were structured as follows.

1. Initiate take-off from a designated pad.
2. Conduct environmental reconnaissance to locate one of the three ground-based mobile robots.
3. Capture photographic documentation of the identified robot and transmit the imagery to an operator, who subsequently communicates a specific landing sequence.
4. Execute precision navigation between pads, performing sequential landings and take-offs ac-

cording to the prescribed sequence.

Participants were allowed to use a Ground Control Station (GCS) for mission initialization and continuous drone monitoring. The landing sequence was strategically determined by an operator at the GCS to optimize the cumulative score, which was calculated through the summation of points assigned to each successful landing on a given pad. The point distribution was uniquely encoded on the ground-based robot, so it required initial identification.

The competition imposed rigorous technical constraints on all participating teams. Specifically, the use of GNSS platforms was prohibited and Light Detection and Ranging (LiDAR) sensors were expressly forbidden. Additionally, teams were precluded from deploying any fixed ground-based sensors or motion capture systems, thereby challenging participants to develop innovative autonomous navigation strategies using the few remaining sensors which were not only allowed, but also capable of being mounted on a small UAS.

To facilitate the comprehensive mission objectives, a sophisticated hardware and software architecture was designed and implemented on advanced hardware. The autonomous quadcopter developed for the Leonardo Drone Contest 2021 represents a sophisticated system that not only meets the specified requirements but also serves as a valid test framework for the methodologies introduced in Chapter 3. The architectural design strategically partitions both hardware and software components across multiple hierarchical levels, addressing the distinct critical challenges inherent to each layer.

Implementation details

The hardware architecture was composed by three distinct systems: a Holybro Pixhawk 4 flight controller (see Section 5.2.1), an Nvidia Jetson Xavier NX companion computer (see Section 4.2.2), and a PC acting as the Ground Control Station. The two embedded systems were mounted on the prototype and interconnected as in Figure 5.1, which also depicts a functional diagram for the hardware architecture: the Pixhawk 4 flight controller is responsible for low-level flight control tasks, while the Nvidia Jetson Xavier NX companion computer executes high-level algorithms and mission-critical tasks and interfaces with the former to send commands and receive feedbacks and status data. A summary of the technical specifications of these two embedded systems is given in Table 5.1.

Figure 5.3 illustrates the comprehensive organization of software modules across functional layers.

- **Lowest Layer:** Composed of real-time modules critical for executing fundamental flight operations, as well as device drivers.
- **Middle Layers:** Hosts higher-level, computationally intensive algorithms responsible for executing individual mission tasks.
- **Highest Layer:** Dedicated to supervision logic, implemented as a Finite-State Machine (FSM), and the Ground Control Station (GCS) operator interface.

Nvidia Jetson Xavier NX	
CPU	NVIDIA Carmel ARM v8.2 (6C/6T, up to 1.4 GHz)
RAM	8 GB LPDDR4x
GPU	NVIDIA Volta (384 CUDA Cores, 48 Tensor Cores)
OS	Ubuntu Linux 20.04
Holybro Pixhawk 4	
CPU	STM32F765 (ARM Cortex-M7, up to 216 MHz)
RAM	512 KB
OS	NuttX 10.0.0

Table 5.1: Main technical specifications of the SBCs used in the implementation of [22].

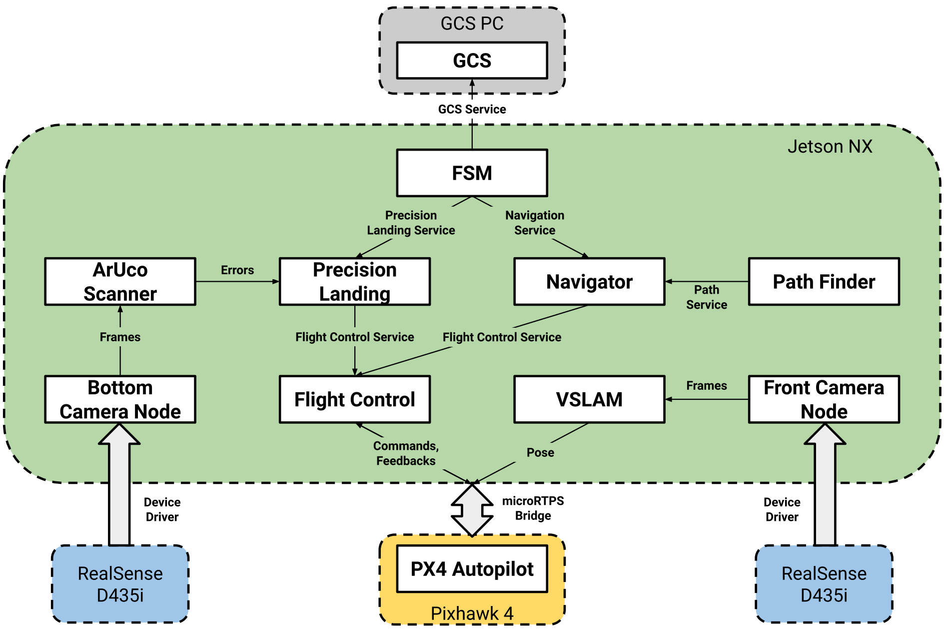

Figure 5.3: Software architecture of the LDC21 autonomous drone: thin arrows indicate communications established with ROS 2 interfaces, thick arrows represent dedicated software interfaces, white boxes represent ROS 2 nodes.

The architectural objective was to develop a cohesive set of ROS 2 modules that leverage interdependent functionality. This modular approach ensures vertical abstraction, enabling higher-level modules to operate without direct concern for lower-level implementation details such as take-off, landing, and navigational maneuvers.

Regarding the set of software modules running on the UAS onboard computers, the lowest layer was composed of the early version of what has become the `flight_stack` DUA unit presented in Section 5.2.1. This prototype was in fact running the very first version of this software and it has provided a testbed for the development of a bridged DDS-based communication among the two levels of the hardware architecture.

The remaining layers contain modules that concern the following specific tasks: perception, navigation and exploration, and mission supervision. Regarding perception, the prototype was equipped with two Intel RealSense D435i depth cameras: one was pointed horizontally and used for localization, while the other was pointed downward and used for target detection. Both mobile robots and landing pads were equipped with ArUco markers [93, 213], a particular kind of fiducial markers with limited information aimed at improving detection probability and pose estimation capabilities. The `ArUco Scanner` module was responsible for detecting these markers and transmitting detection data to the `FSM` module, that ran a state machine to manage the mission phases and the drone behavior.

To understand the environment and get a consistent estimate of its position and orientation within it, the drone ran a VSLAM (Visual Simultaneous Localization And Mapping) algorithm. The SLAM problem is notoriously complex and computationally intensive to solve, requiring sophisticated solutions from both the mathematical and implementation perspectives [183]. Nonetheless, it was deemed as the only solution in this case due to the Contest rules: no sensors other than cameras were reliable enough to provide the required information and due to accumulated error it was not possible to rely on odometry alone. The chosen VSLAM algorithm for this implementation was the ORB-SLAM2 system [172], a state-of-the-art solution for monocular, stereo, and RGB-D cameras that provides real-time performance and high accuracy in both localization and mapping tasks.

The `VSLAM` module was responsible for running the algorithm and providing the estimated pose of the drone directly to the PX4 firmware. The ORB-SLAM2 system employs binary feature extraction from input frames to construct a local map while addressing bundle adjustment optimization problems. To increase computational efficiency and improve memory utilization, these features are consistently employed throughout the entire system. The algorithm designates certain frames as keyframes, which serve as critical reference points for the local map structure. These keyframes are instrumental in computing the current local pose through the construction of a *covisibility graph*, which facilitates both localization and environmental mapping functions. As the map increases in scale, the system generates an additional structural representation known as the *essential graph*, which enables loop detection capabilities. Upon loop detection, the system executes an automatic loop closure procedure wherein all graphs undergo analysis and updating. During this process, redundant keyframes and nodes are eliminated, and the local map undergoes revision incorporating recent measurements to minimize systematic errors.

The architecture of the ORB-SLAM2 system comprises four distinct operational phases: tracking, local mapping, loop closing, and global adjustment. Each phase executes in a dedicated thread. The tracking phase monitors scene features, while local mapping utilizes these same features to construct the local map in relation to the established keyframes. The loop closing phase identifies potential loops and communicates this information to the global adjustment thread, which performs global bundle adjustment across the entire map to enhance overall accuracy.

ORB-SLAM2 supports multiple input modes: monocular, stereo, and RGB-D. The implementation used in this work introduces an additional mode that uses infrared and depth images, called IRD mode. This modification addresses specific technical limitations of Intel RealSense cameras, particularly the temporal misalignment between RGB and depth frames, while maintaining proper synchronization with infrared frames. Significant modifications to the ORB-SLAM2 algorithm have been implemented to expand parameter control capabilities, incorporate ARM processor compatibility, and integrate NVIDIA CUDA GPU support. The adaptation for the ARM architecture required the disabling of x86-specific optimizations, which particularly affected performance in feature extraction and computation phases that previously leveraged AVX/SSE vector instruction sets. To address these performance implications, CUDA GPU support became essential. Within the distributed architecture shown in Figure 5.3, this algorithm operates as a ROS 2 node, consuming camera topic frames and publishing both the current pose estimation and system state on separate topics.

The next task to address has been that of autonomous navigation and exploration of the environment, to find ground targets and landing zones. The `Navigator` module has been implemented for this purpose, with the aim of providing a high-level interface to navigation operations. These operations were exposed as ROS 2 synchronous communication primitives, *i.e.*, in the form of services. The `Navigator` module was responsible for sending commands to the lower levels to move the drone from its current position to a given target position, provided that such a position was reachable and that there existed a path to reach it. This additional task of computing and returning paths was left to the `Path Finder` module: its primary objective was to determine an trajectory between current and target positions that simultaneously minimizes distance while ensuring collision avoidance. The implemented solution approaches path planning through graph construction representing traversable space within the environment, applying the A* search algorithm [103].

The algorithm initially constructs a graph representation directly from environmental point cloud data acquired via ORB-SLAM2. This process begins by discretizing the environment into a three-dimensional grid of variable-resolution cubic cells, each identified by its centroid. The interconnection network of traversable cells constitutes the graph structure. Boundary cells are automatically designated as non-traversable to constrain the drone operational space. Environmental obstacle identification proceeds through point cloud preprocessing, beginning with the removal of out-of-bounds points. The algorithm associates points with their containing cells, designating cells exceeding a predetermined point density threshold as non-traversable. The remaining cells form the graph vertex set, followed by edge construction. For each cell, the algorithm identifies neighboring cells in eight coplanar directions at equal altitude, plus immediate vertical neighbors. Edge insertion between adjacent cells depends on a corridor analysis based on the physical dimensions of the drone: an edge is established if and only if the point density within the corridor falls below a specified threshold. This methodology enables variable-resolution environmental discretization while maintaining consideration for the physical footprint of the drone during transitions.

Valid corridors result in edge insertion with costs equal to the Euclidean distance between cell centroids. This iterative process yields a graph representation of the operational space, computed offline. The designated `Path Finder` ROS 2 node implements a computationally improved A* algorithm to determine shortest paths between arbitrary nodes upon request, carried over a service interface. Path finding operations begin with origin and target cube identification and validation. Upon confirming

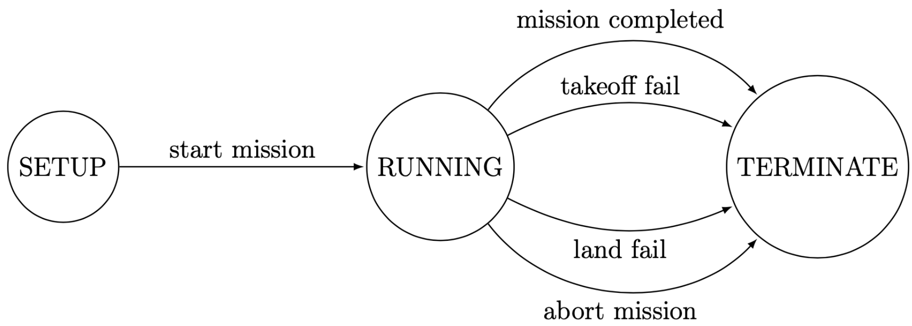

Figure 5.4: High-level finite-state machine for the Leonardo Drone Contest 2021 mission.

traversability, the algorithm initiates shortest path computation, returning a sequence of waypoints leading to the target position. The selection of A* is motivated by its mathematical properties of admissibility, completeness, and optimality.

Supervision

The top level of the software architecture is a finite-state machine that implements high-level supervision logic. This FSM presents the drone as an automaton executing complete missions within the Leonardo Drone Contest parameters while managing contingencies such as hardware failures, battery depletion, and localization system degradation or complete failure. State transitions trigger formulation of requests to subordinate modules, with transitions conditioned upon both current state and intervening events.

The implementation integrates with the distributed architecture through a ROS 2 node that facilitates bidirectional communication with other modules via service calls and alert reception. System initialization comprises a comprehensive status verification of all modules and mission execution begins only upon confirmation of full operational readiness across all subsystems.

The structure of the FSM mirrors mission dynamics as specified in the Contest rules. The implementation consolidates all abstraction levels into a single routine where state transitions are invoked based on the operational conditions of the drone following each execution step. Error handling leverages the meta-machine concept: primary mission phases execute within states comprising a subordinate FSM, while maintaining guaranteed transition paths to termination states under all circumstances. The high-level and low-level FSMs are shown in Figures 5.4 and 5.5.

Experimental results

The system hereby described has been tested, in both its single parts and its operational capabilities as a whole, also to evaluate the robustness of the proposed FSM. Laboratory tests allowed to perform

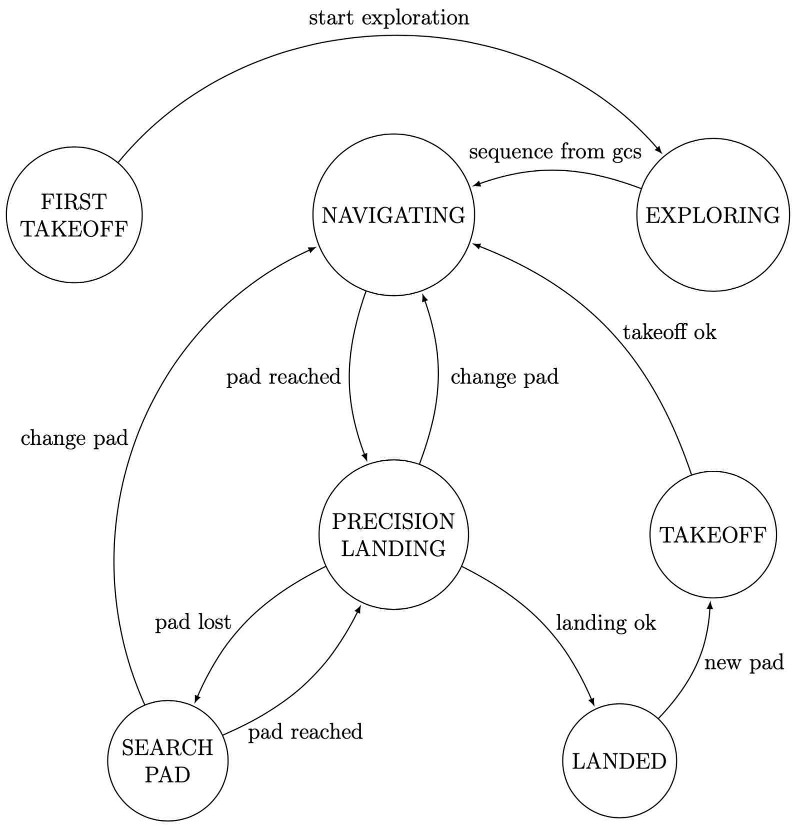

Figure 5.5: Low-level finite-state machine for the Leonardo Drone Contest 2021 mission.

partial evaluations, especially regarding the possible outcomes of the various phases of the mission. The complete verification of the functioning of all the modules and the FSM was performed during the 2021 Leonardo Drone Contest.

The first tests performed in the laboratory were aimed at assessing the capabilities of the VSLAM algorithm implementation. Performance evaluation of the custom implementation was conducted in an indoor office environment under variable illumination conditions to assess trajectory estimation accuracy. The evaluation methodology compared the estimated trajectory against more reliable, "ground truth" measurements. Due to limitations in acquiring continuous reliable data, the assessment procedure involved five discrete reference points measured using a laser sensor with a precision of ± 2 mm. The experimental hardware matched that of the UAS prototype as detailed previously. Figure 5.6 presents the experiment, depicting the ORB-SLAM2 estimated trajectory alongside the reference trajectory and ground waypoints where Root Mean Square Error (RMSE) calculations were performed. Table 5.2 quantifies the RMSE between ORB-SLAM2 estimated positions and corresponding reliable measurements at the five reference waypoints. The analysis demonstrates that ORB-SLAM2 maintains functional performance on this novel platform architecture, despite its intrinsic differences from the original target platform, while indicating potential areas for further improvement.

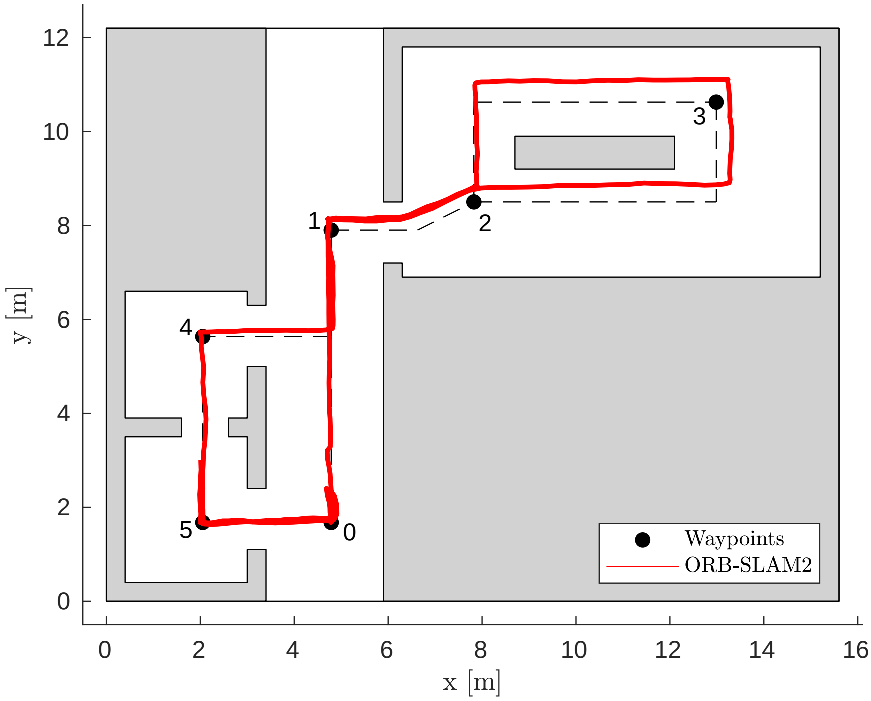

Figure 5.6: VSLAM algorithm performance evaluation test.

Waypoint position (x, y) [m]	ORB-SLAM2 position (x, y) [m]
(4.788, 7.901)	(4.718, 8.158)
(7.826, 8.501)	(7.925, 8.871)
(12.986, 10.626)	(13.329, 11.146)
(2.053, 5.634)	(1.980, 5.729)
(2.053, 1.676)	(2.034, 1.665)
RMSE	0.352 m

Table 5.2: RMSE between the ORB-SLAM2 estimated trajectory and the ground waypoints.

Next, laboratory validation testing was conducted to evaluate the performance of the full system. The test sequence began with an exploration and perception phase, during which the system constructed an environmental map while localizing itself within it. An ArUco marker fixed to the ground, serving as a simulated robot target, was successfully identified during this phase. Upon completion of exploration, the mission phase commenced, comprising navigation and landing operations targeting two designated pads. To validate the FSM error handling capabilities, the first target pad was intentionally removed, which triggered the execution by the FSM of a proximity search routine at the expected location. Failing to locate the first pad, the system proceeded to the second landing position, where the pad was successfully identified and a precision landing was performed on it. Figure 5.7 illustrates the complete flight trajectory. Despite spatial constraints of the laboratory environment, the facility proved adequate for comprehensive algorithm validation prior to Contest deployment. The test execution spanned 7 minutes and 33 seconds, covering 52 meters at a maximum velocity of 1.7 km/h.

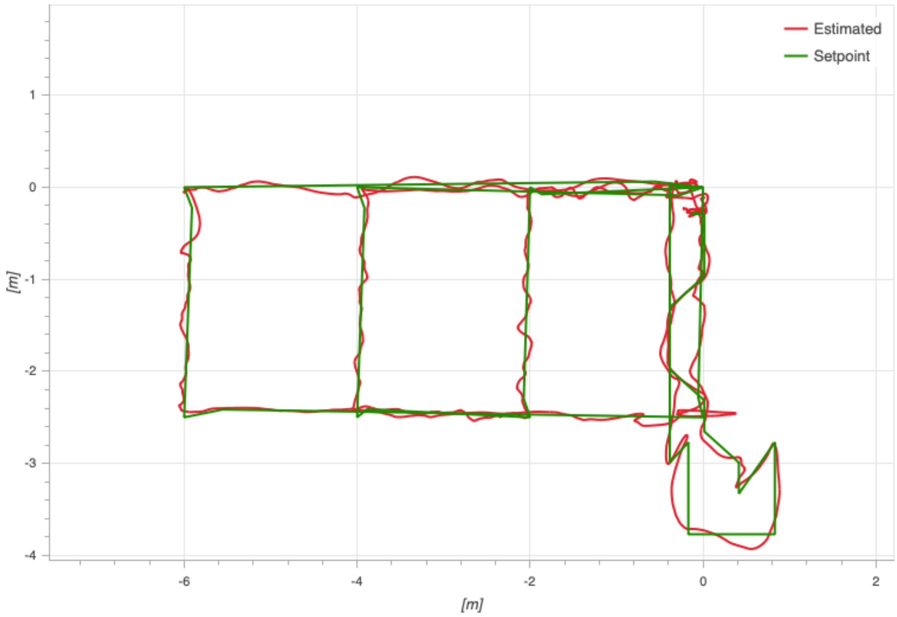

Figure 5.7: Laboratory test of the 2021 Leonardo Drone Contest mission; the green path represents the setpoints while the red one represents the estimated trajectory.

RMSE$_x$ [m]	**RMSE$_y$ [m]**	**RMSE$_z$ [m]**
0.37	0.47	0.29

Table 5.3: Laboratory test RMSE along each axis.

The trajectory demonstrates high precision with one notable exception: a significant oscillation near the conclusion of the initial exploration loop, attributed to computational resource saturation on the Xavier NX companion computer; an issue subsequently resolved. Quantitative performance evaluation of the navigation subsystems was conducted through Root Mean Squared Error (RMSE) analysis along each spatial axis, with results presented in Table 5.3.

The Contest environment required extended flights to complete full mission profiles, with the quadcopter demonstrating 14-minute endurance capabilities while covering a cumulative distance of 118 meters. The presented data corresponds to the fourth of the four Contest runs. This particular execution involved an abbreviated exploration phase, transitioning to mission operations immediately upon robot detection to begin the search for the landing pads. Figure 5.8 illustrates the flight trajectory executed during the Contest run, during which five successful precision landings were achieved. Quantitative performance analysis yielded RMSE measurements across the trajectory as detailed in Table 5.4. The (x, y) planar error demonstrates good performance compared to laboratory testing, attributed to the execution of the initial loop enabling ORB-SLAM2 to construct a reliable map with reduced temporal overhead. Conversely, the increased RMSE along the z axis reflects the increased magnitude of vertical maneuvers required during Contest operations.

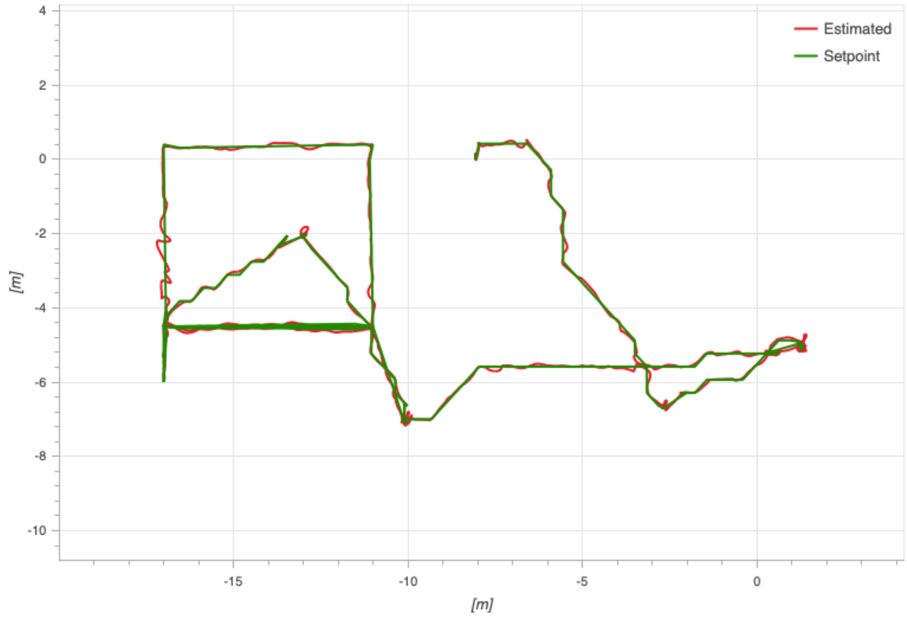

Figure 5.8: 2021 Leonardo Drone Contest mission run; the green path represents the setpoints while the red one represents the estimated trajectory.

RMSE$_x$ [m]	**RMSE$_y$ [m]**	**RMSE$_z$ [m]**
0.21	0.24	0.38

Table 5.4: 2021 Leonardo Drone Contest run RMSE along each axis.

Regarding the overall performance of the UAS prototype also in terms of its full software architecture, the implementation has been a success. The FSM has been able to manage the mission phases and the drone behavior, correctly transmitting data back to the GCS, which was an essential requirement for the Contest. Despite some technical issues that compromised the first two out of four runs, the team was able to sort those out and the UAS prototype completed its mission in the last two runs. Thanks to extensive press coverage offered by Leonardo in 2021, Figure 5.9 shows the moments right before and after some of the most successful and exciting precision landings performed by the UAS prototype during the Contest. It is interesting to note how, once the procedure was started, the prototype always brought it to completion landing right within the central black portion of each pad, correcting localization errors by aligning its center with the ArUco marker on the ground. The Tor Vergata team has been awarded the second prize in 2021, paving the way for a successful streak in the following years: the 2022 edition has seen the team winning the second prize again, while the third prize has been won in 2023 together with the jury prize. In 2024, the team has finally won the first prize demonstrating also the effectiveness of the proposed DUA architecture, which in the meantime has been developed and adopted in this project.

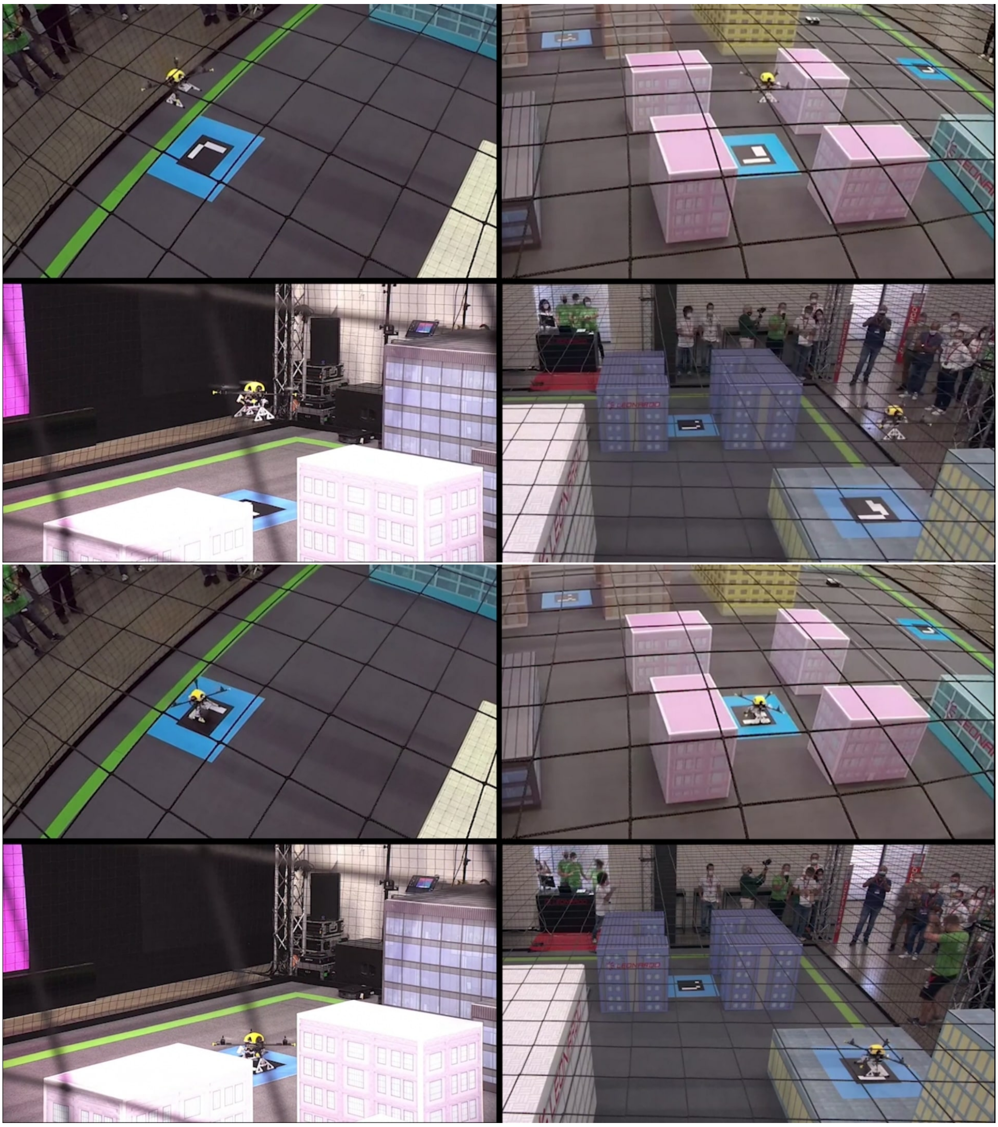

Figure 5.9: Successful precision landings performed on the ArUco landing pads during the 2021 Leonardo Drone Contest.

5.3.2 Efficient sensor fusion for autonomous agents

In Section 5.2.2, the problem of pose estimation for an autonomous robot has been presented. Its complexity has been outlined, together with the variety of sensors that might be used to achieve a reliable and accurate estimate of the position and orientation of a robot. The concept of sensor fusion has also been mentioned, as a family of techniques consisting in merging data coming from multiple, even heterogeneous sensors, to produce a single final pose estimate. The advantage of sensor fusion, over its increased complexity both in terms of mathematical formulation and implementation, is the possibility to overcome the issues of one sensor by leveraging the other ones. Over the years, many sensor fusion algorithms have been proposed, each one with its own strengths and weaknesses, and each one suitable for a specific kind of sensor or application. A good summary can be found in [23] together with a novel, synthetic approach to the problem.

In [23], the challenge of efficient position measurement fusion in a horizontal plane (x, y) is addressed for autonomous systems equipped with multiple visual sensors. The proposed methodology diverges from existing approaches by prioritizing reduced computational complexity and minimal sensor-specific information requirements. This design philosophy facilitates both rapid deployment on physical platforms and seamless integration of heterogeneous visual sensors within the system architecture, as validated through experimental evaluation. The fundamental research objective is to develop a computationally efficient sensor fusion framework that maintains high accuracy while reducing implementation complexity and sensor-specific dependencies. This stands in contrast to conventional approaches that often require extensive computational resources and detailed sensor characterization. Describing this work is instrumental since it has been carried out using a preliminary version of the DUA framework presented in Chapter 4, so the following discussion briefly summarizes the main results and then moves to the description of the implementation details.

The blending algorithm

Sensor fusion methodologies enhance measurement accuracy and reliability by combining data from multiple sources, achieving superior performance compared to individual sensor utilization. A fundamental challenge in sensor fusion implementation arises from the inherent diversity of sensor characteristics, with each device exhibiting distinct advantages and limitations that vary based on operational requirements and environmental conditions. Effective fusion systems must therefore incorporate mechanisms for dynamic sensor trustworthiness evaluation while maintaining operational capability during partial sensor failures.

Established fusion methodologies predominantly rely on precise characterization of data sources through covariance matricesand mathematical models. However, accurate determination of these parameters presents significant challenges in practical implementations, particularly for real-time estimation from data streams. The accuracy of these parameters directly impacts the performance of estimation algorithms and, consequently, the robustness of the complete localization system. This limitation serves as the primary motivation for the proposed fusion architecture: to achieve enhanced robustness with respect to sensor-specific parameters while maintaining architectural simplicity. The framework proposed in [23] reduces dependence on precise sensor characterization, thereby reducing implementation complexity while maintaining system reliability. This approach represents a departure from conventional methodologies that require extensive sensor modeling and parameter estimation. By reducing reliance on detailed sensor models, the system achieves improved robustness to parameter uncertainty while facilitating practical deployment in operational environments.

The aspects that concern this work, completely instrumental to the characterization of the implementation that achieved the experimental results, are limited to the structure of the algorithm. The algorithm, termed *blending*, can be summarized as follows: first, new samples are gathered from every sensor and their values are subtracted the previous sample to obtain an incremental measurement. Secondly, the incremental measurements are filtered using an appropriate method, to further reduce noise and reject outliers. Finally, the blending stage happens, where all incremental samples are combined to obtain a final increment to add to the previous one. The result of this last addition is the final position estimate, which is then used to control the robot.

Formally, let $n \in \mathbb{N}$ sensors be available, each one producing data at the same rate $f_s \in \mathbb{R}$, $f_s > 0$. The incremental, filtered (x, y) sample at time $k \in \mathbb{Z}_{\geq 0}$ corresponding to each sensor is then denoted by $\Delta_{f,i_k} \in \mathbb{R}^2$, $i = 1, \ldots, n$. Each sensor also provides information about its functioning state, considered either on or off for simplicity and denoted by $s_{i,k} \in \{0, 1\}$, $i = 1, \ldots, n$. Let $\Delta_{f_k} = \begin{bmatrix} \Delta'_{f,1_k} & \ldots & \Delta'_{f,n_k} \end{bmatrix}'$, $s_k = \begin{bmatrix} s_{1,k} & \ldots & s_{n,k} \end{bmatrix}'$, and $\alpha_1, \ldots, \alpha_n \in \mathbb{R}_{\geq 0}$ such that $\sum_{i=1}^{n} \alpha_i = 1$. Let $b : \mathbb{R}^{n \times 2} \times \mathbb{R}^n \to \mathbb{R}^2$ be the *blending map* that produces the final increment sample. The blending stage of the algorithm is summarized by the following equations:

$$\bar{\alpha}_{i_k} = s_{i_k} \alpha_i \left(1 + \frac{\sum_{j=1}^{n} (1 - s_{j_k}) \alpha_j}{\sum_{j=1}^{n} s_{j_k} \alpha_j} \right), \qquad i = 1, \ldots, n, \tag{5.1a}$$

$$b(\Delta_{f_k}, s_k) = \sum_{i=1}^{n} \bar{\alpha}_{i_k} \Delta_{f,i_k}, \qquad k \in \mathbb{Z}_{\geq 0}. \tag{5.1b}$$

The formulation presented in (5.1) exhibits simplicity while incorporating several advantageous characteristics. The parameters $(\alpha_1, \ldots, \alpha_n)$ enable fine-tuned calibration to accommodate varying levels of sensor accuracy and reliability and the coefficients $(\bar{\alpha}_{1_k}, \ldots, \bar{\alpha}_{n_k})$, whose sum is equal to 1, remodulate the final value accounting for the state of each sensor. The responsiveness of the method to sensor operational states enables resilient operation during both transient and permanent sensor failures, as shown in the experimental validation discussed in [23]. From a computational perspective, (5.1) shows high efficiency as it reduces to a series of sums of products that can be executed with minimal computational overhead on standard microprocessor architectures.

The distributed implementation

The experimental platform developed in [23] has been designed to integrate only visual senors, for both simplicity and because they are among the most subject to issues due to accumulated error and drift. The chosen sensors are a ZED Mini camera (see Section 5.2.2) and an Intel RealSense D435i depth camera. The first, through an early version of the `zed_driver` package presented in Section 5.2.2, produced samples of its position estimate, while the latter was used to run the ORB-SLAM2 algorithm in the IRD mode described in Section 5.3.1.

The hardware architecture was again distributed, including a Nvidia Jetson Xavier NX SBC and an Up Squared x86 SBC; the former was responsible for driving the ZED camera, while the latter had the task

Up Squared	
CPU	Intel Pentium N4200 (4C/4T, up to 2.5 GHz)
RAM	8 GB LPDDR4
OS	Ubuntu Linux 20.04
Nvidia Jetson Xavier NX	
CPU	NVIDIA Carmel ARM v8.2 (6C/6T, up to 1.4 GHz)
RAM	8 GB LPDDR4x
GPU	NVIDIA Volta (384 CUDA Cores, 48 Tensor Cores)
OS	Ubuntu Linux 20.04
Holybro Pixhawk 4	
CPU	STM32F765 (ARM Cortex-M7, up to 216 MHz)
RAM	512 KB
OS	NuttX 10.0.0

Table 5.5: Main technical specifications of the SBCs used in the implementation of [23].

of interfacing with the D435i and running the ORB-SLAM2 system. The choice of a x86 SBC to run ORB-SLAM2, in contrast to what has been shown in Section 5.3.1, was motivated by the possibility to leverage the vector instruction sets of the x86 architecture to speed up the computations of the algorithm in a way that is closer to its original implementation. The two SBCs were interconnected by an Ethernet connection and all of the software modules were implemented as ROS 2 nodes. Finally, to emulate the situation of an autonomous robot, a microcontroller was added, in the form of a Pixhawk 4 flight controller running PX4 (see Section 5.2.1). This FMU acted only as a microcontroller, receiving fused position estimates and interpolating them with data coming from its internal IMU. A summary of the technical characteristics of the hardware modules is given in Table 5.5.

The software framework used to manage the experimental implementation in [23] was a first, embryonic version of what has subsequently become the DUA framework presented in Chapter 4. There was, in fact, a variety of software tools involved, ranging from the ROS 2 middleware to the implementation of the ORB-SLAM2 algorithm and the ZED SDK (see Section 5.2.2). All of these software required their own dependencies, as well as specific setup steps that were slightly different on each of the two boards. For this reason, Docker was introduced as a tool to manage the software dependencies and setup steps, as well as to ensure a consistent and reproducible behavior across all the boards and the development PCs. This allowed the developers to focus on the solution and implementation of the blending algorithm, streamlining the experimental validation presented in the [23]. Different Docker images were created for each of the different platforms, including development PCs, which integrated all the necessary dependencies and software together. The techniques applied to the creation of these Docker images have led to the development of the base units of `dua-foundation`, introduced in Section 4.2. Then, the rest of the framework followed.

6 The case of tokamak devices

The previous chapter motivated the importance of the proposed DUA framework and of the technologies on which it is based in the context of autonomous robots. It clarified the relevance of the proposed solution in that context and provided some results that suggest its potential, as well as some practical applications that benefit of the DUA structure. This chapter shifts the focus onto a different, yet related context: that of tokamak devices for nuclear fusion with magnetic plasma confinement.

From the point of view of a system architecture developer, these two scenarios are not so different: both involve sophisticated systems with many interacting components and both require a high degree of autonomy. However, autonomy is the first item that differentiates these contexts: an autonomous robot, as discussed in Chapters 2 and 5, is a single entity that may have to operate independently of human operators for long periods of time, while a tokamak device may even surpass a robot in terms of complexity but it is an experimental device; this implies that it is not expected to operate without human operators involved and that its operational times might be shorter than those of a robot. Nonetheless, the various subsystems of a tokamak device are usually expected to operate autonomously during an experiment, thus the technologies and design paradigms discussed previously are still relevant. Among those, as the following clarifies, the distributed paradigm remains among the most suitable since the control system of a tokamak may be made of a variety of components that need to communicate and coordinate with each other.

Tokamak device are another application scenario for the work described herein. This chapter presents the concepts of nuclear fusion with magnetic confinement, then introduces the Tokamak à Configuration Variable (TCV) device that has been the subject of this work. Subsequently, a first innovation is presented: an application of the distributed DDS middleware (see Section 3.1.1) to the TCV control architecture made possible by the versatility of TCV's distributed control system.

6.1 Magnetic confinement fusion on TCV

Nuclear fusion occurs when light atomic nuclei combine to form heavier elements. This process powers stellar bodies and represents a promising avenue for clean, sustainable energy generation. These reactions, such as the fusion of Deuterium and Tritium nuclei to produce an α-particle and a high-energy neutron, release substantial energy with the potential for efficient power generation.

The development of fusion energy has acquired heightened significance in the context of contemporary global challenges. As nations worldwide confront climate change and seek to reduce greenhouse gas emissions, fusion technology offers a carbon-neutral alternative to fossil fuels. Furthermore, fusion power may address the growing energy demands of emerging economies while promoting sustainable development, as it requires minimal fuel resources and produces no long-lived radioactive waste like fission does. The abundance of fusion fuel sources in nature also promises enhanced energy security and reduced geopolitical tensions associated with conventional energy resources. Moreover, nuclear power plants can be scaled to meet the demand of the territory in which they are located, without the need to identify suitable locations to harvest renewable but intrinsically intermitted sources of energy such as solar or wind.

To facilitate fusion reactions, the participating elements must be heated to temperatures sufficient for charged nuclei to overcome their repulsive Coulomb forces. At these elevated temperatures, the matter exists as fully ionized gas, termed *plasma* [40], which must be confined for durations adequate to achieve practical reaction probabilities.

Tokamaks [244] are experimental devices that have originated in the 1950s in the USSR; Figure 6.1 depicts the TCV machine, which has been the subject of this work. They employ toroidal vacuum chambers for plasma confinement using magnetic field configurations [14, 31, 77]. Dedicated coils encircling the chamber generate a *toroidal* magnetic field, while *poloidal* field coils produce an *axisymmetric* poloidal field. This poloidal field serves multiple functions: controlling plasma position and shape while inducing *plasma current*; the plasma has to, in fact, carry a current that is instrumental to keep the fusion reaction going, as well as to maintain a reference temperature. Over the decades, tokamaks have contributed to study the behavior of plasma among different types of reactions, as well as to develop the technologies necessary to improve and maintain magnetic confinement. More recent devices, such as the ITER and the JT-60SA machines, incorporate the generation of energy into their project goals.

The control infrastructure of a tokamak fusion device represents a sophisticated integration of diverse technological components operating across multiple temporal and spatial scales. The hardware architecture encompasses an extensive array of elements: high-power electrical supplies for magnetic field generation, Programmable Logic Controllers (PLCs) for real-time control and safety systems, analog control and diagnostic devices, and server-grade computing systems for data acquisition and processing. This hardware foundation must support both high-bandwidth real-time operations and complex experimental procedures while maintaining strict safety protocols.

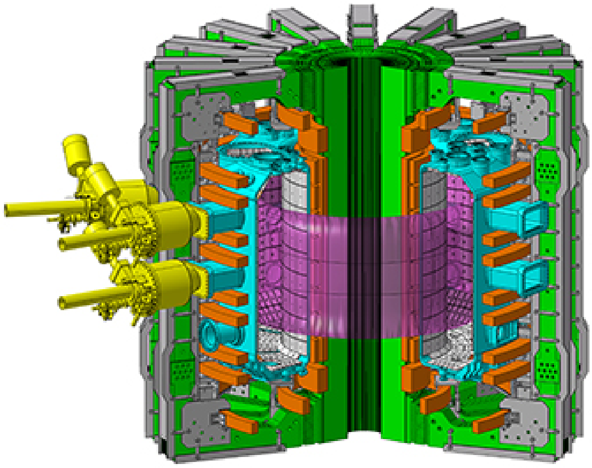

Figure 6.1: Schematic of the TCV device.

The operational complexity extends beyond hardware considerations to include intricate control schemes for plasma magnetic confinement, comprehensive diagnostic systems for plasma state monitoring, and experimental execution frameworks. These systems have to operate in concert to maintain plasma stability while facilitating scientific investigation. The human interface component adds another layer of complexity, requiring control rooms staffed by teams of skilled operators managing different aspects of machine operation. The supervision infrastructure must provide interfaces ranging from detailed module-level control to high-level Supervisory Control and Data Acquisition (SCADA) systems, enabling both fine-grained control and system-wide monitoring.

Such operational sophistication demands a robust and well-structured architectural foundation. The control system design must accommodate real-time requirements, ensure fault tolerance, and facilitate system maintenance and upgrades. A distributed systems paradigm offers particularly compelling advantages in this context, providing natural solutions for parallel operation, redundancy, and scalability. This approach enables effective management of the complexity of the system while maintaining the flexibility necessary for experimental physics operations.

There are various dimensional parameters that can be used to evaluate the size of a tokamak device; one of these is the *major radius* R_0, which is the distance from the center of the torus to the outer wall of the vessel. With a major radius $R_0 = 0.88\,\mathrm{m}$, TCV is classified as a medium-scale fusion research device. It is operated at the Swiss Plasma Center (SPC) of the École Polytechnique Fédérale de Lausanne (EPFL), Switzerland [123]. The device designation reflects its exceptional plasma shaping capabilities, enabled by an extensive array of independent Poloidal Field (PF) coils in conjunction with a distinctive rectangular vacuum vessel geometry, achieving toroidal magnetic fields up to 1.54 T.

The PF coil system, illustrated in Figure 6.2, comprises four distinct subsystems.

- The **OH coils**, consisting of a central solenoid (OH1) and an up-down symmetric arrangement of

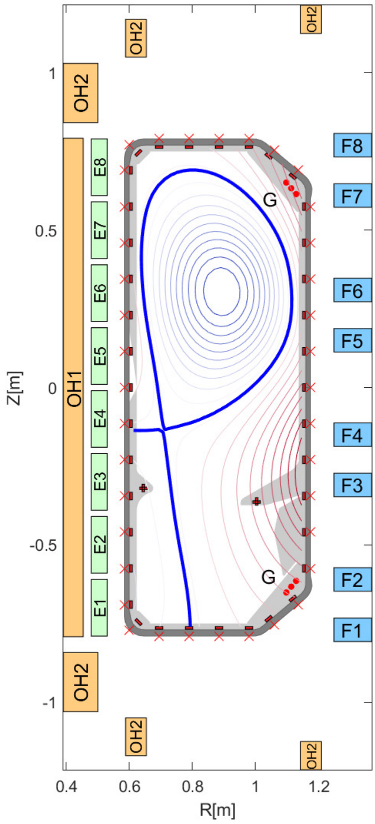

Figure 6.2: TCV poloidal cross-section with an example plasma configuration (lower single-null). The OH, E, F, and G coils are highlighted in different yellow, green, blue, and red, respectively. Removable baffles appear in the cross-section in gray protruding inward in the bottom part of the vessel.

6 series-connected coils (OH2), provide plasma current drive functionality.

- The **E coils**, comprising 8 independently controlled circuits positioned on the high-field side.
- The **F coils**, featuring 8 independent circuits located on the low-field side.
- The **G coils**, consisting of two up-down symmetric in-vessel coils in anti-series configuration for vertical stabilization of the plasma.

The device architecture accommodates removable *baffles* [208] that enhance plasma core-divertor separation. The configuration depicted in Figure 6.2 shows the Short-Inner-Long-Outer (SI-LO) baffles arrangement.

The exceptional capabilities of TCV, combined with its continuously enhanced control architecture, have established this device as a significant contributor to international fusion research [45, 67].

Through successive upgrades to both hardware and control systems spanning its three decades of operation, TCV has maintained its position at the forefront of experimental fusion science. Its contributions span multiple international research campaigns [110], including investigations of advanced plasma configurations and real-time control developments.

6.2 The Système de Contrôle Distribué

Real-time control is a fundamental requirement in modern tokamak operation. The Système de Contrôle Distribué (SCD) [11, 92, 121] represents a distributed real-time control architecture designed for the TCV tokamak. The system emerged from the necessity to handle a considerable number of actuators and diagnostics while maintaining precise control over plasma behavior. Its distributed nature, of particular interest in this work, stems from the need to minimize analog signal paths across the facility, employing real-time-capable networks to interconnect various subsystems.

The SCD interfaces with hundreds of diagnostic channels while controlling multiple actuator systems. These include independently controllable coil power supplies, steerable real-time Electron Cyclotron launchers with continuously variable power, multiple neutral beam injectors, and an array of gas injection valves. Such an extensive set of controllable elements demands a sophisticated control infrastructure capable of coordinating multiple operations simultaneously.

A distinctive characteristic of TCV operation lies in the fast plasma dynamics, which imposes stringent timing requirements on the control system. Despite its relatively small size compared to future reactor designs, TCV requires primary magnetic control loops executing within 100 µs. This temporal constraint represents a defining factor in the system architecture, influencing both hardware and software designs.

The SCD design philosophy emphasizes two seemingly contrasting requirements: operational flexibility and system reliability. The control infrastructure should facilitate rapid implementation and testing of novel control schemes while maintaining robust operation during regular experimental campaigns. This duality led to the adoption of MATLAB/Simulink as the primary development environment, enabling control algorithm prototyping through graphical programming and subsequent automated code generation.

The initial implementation of the SCD, now more than a decade old, required a complete recompilation of the control code before each plasma discharge when control parameters were modified. While this approach supported the exploratory phase of the SCD implementation, it proved increasingly difficult to maintain as the system expanded and the number of integrated modules and algorithms increased.

The transition to the MARTe2 framework (see Section 2.3.6) marked a significant improvement, introducing a modular, maintainable architecture with enhanced standardization capabilities. MARTe2, specifically designed for fusion applications, provides a comprehensive environment for real-time software applications. Its data-driven runtime workflow enables dynamic parameter and waveform loading during the initialization of a discharge, eliminating the need for frequent recompilations. This framework has also improved the integration between real-time computers and the TCV IT infrastructure, streamlining the deployment of new control algorithms from development to experimental validation.

The SCD hardware architecture comprises multiple computational nodes interconnected through dedicated networks. The main processing units consist of industrial PCs operating under real-time Linux, specifically customized with the `PREEMPT_RT` patch, CPU isolation, and specialized kernel parameters. These modifications enable deterministic operation while maintaining a conventional operating system environment, avoiding the complexity of custom-compiled kernels. A summary schematic of the system is given in Figure 6.3.

Input and output operations rely on high-density ADC/DAC modules. A dedicated reflective memory network (RFM) provides communication at 170 MB/s with sub-microsecond latency between computational nodes. This infrastructure is complemented by an EtherCAT real-time industrial network deployed to manage distributed low-count I/O subsystems, each handling up to 10 analog channels, with a total capacity of approximately 500 channels. The computational nodes, each performing specific functions in the control architecture, are organized as follows.

- **Node 01** operates the Far Infra Red interferometer (FIR) and related X ray diagnostics.
- **Node 02** and **Node 03** constitute the main control nodes of the system. Node 02 serves as the primary controller while Node 03 acts as its twin for debugging and backup purposes, *i.e.*, it runs the same code on the same hardware reading the same measurements but it is not connected to any actuator. These nodes manage all coils, gas valves, and auxiliary heating systems.
- **Nodes 05** and **Node 06** provide computational capabilities through 6-core Intel Core i7 processors. One unit incorporates a real-time network analyzer to monitor the performance of the reflective memory network.
- **Node 07** executes real-time MHD analysis, monitoring rotating tearing modes and locked modes.
- **Node 08** handles fast acquisition and real-time processing of Electron Cyclotron Emission data.
- **Node 09** manages real-time acquisition and processing of the Thomson scattering diagnostic, providing calibrated electron temperature and density profiles.
- **Node 10**, designated as MANTIS, operates a sophisticated 10-camera multispectral imaging system. It captures spectrally filtered tangential images of the plasma at frequencies between 200 Hz and 800 Hz, enabling real-time processing for applications such as radiation front position estimation along the divertor.

The primary control functionality, implemented on Node 02 with Node 03 as backup, represents a critical component of the system. This subsystem interfaces with a 192-channel ADC module, sending

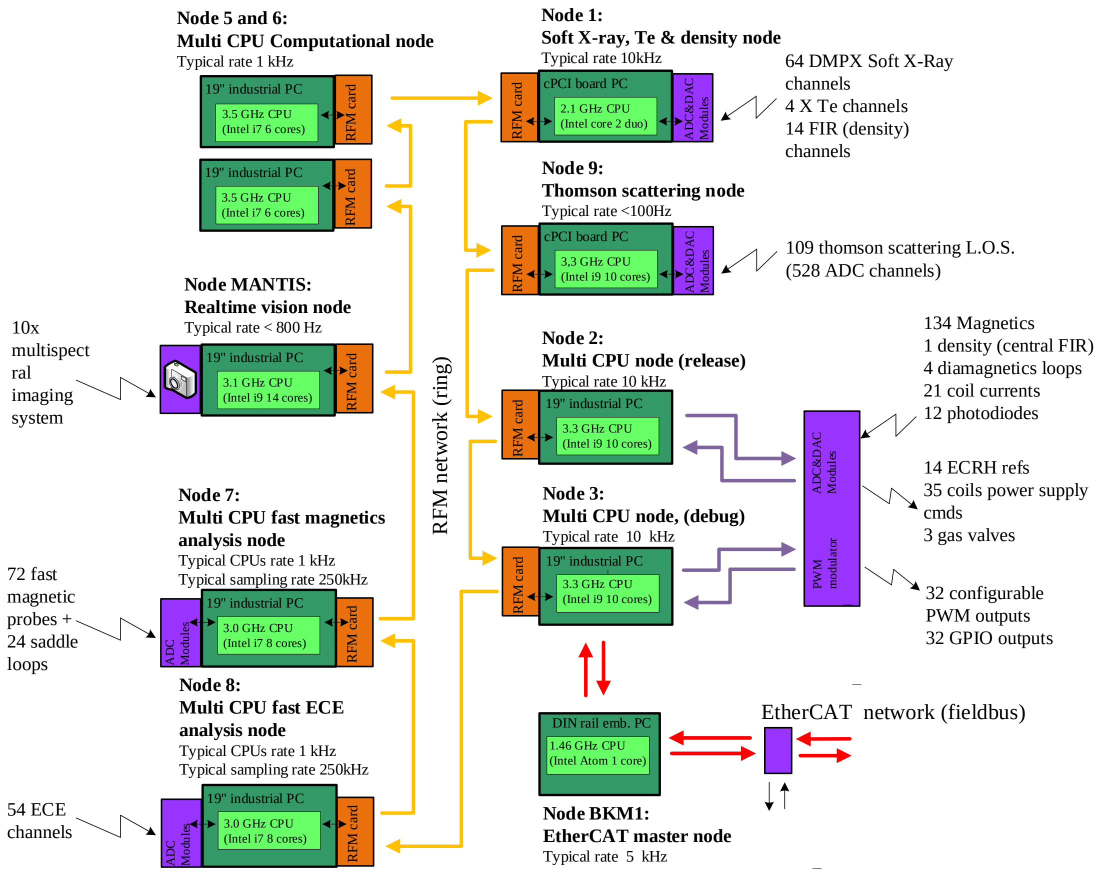

Figure 6.3: Schematic of the Système de Contrôle Distribué (SCD) architecture.

data streams through two high-speed fiber optic links to both the industrial PCs that implement the nodes. This redundant configuration ensures synchronized data delivery to both nodes, enabling parallel operation during plasma discharges.

A central server orchestrates the entire control system, managing a main shot state machine that coordinates all distributed computers. This server maintains an MDSplus database (see Section 2.3.6) containing control system data and real-time algorithm configurations, hosts the development environment for control algorithm implementation and provides a secure platform for authorized users to integrate new controllers into the TCV control framework.

The software development workflow employs an object-oriented simulation framework, designated as SCDDS (Simulink-based Control Design and Deployment Suite), which manages parameter and waveform access consistently between simulation and real-time environments. After new algorithms have been developed with the graphical programming tools offered by MATLAB/Simulink, the SCDDS framework handles the complexity of parameter management, enabling developers to focus on control logic implementation. This approach proves particularly valuable when validating new algorithms using historical plasma discharge data. Once validated in simulation, control algorithms undergo automated code generation, producing C code that replicates the behavior of the original Simulink design. The SCDDS framework ensures a streamlined integration process within the MARTe2 environment, in which the entirety of the real-time control algorithms run to ensure that the system satisfies the stringent timing requirements in a deterministic fashion.

6.3 Improving the SCD network with the DDS

The strict real-time requirements of a sophisticated control system such as the SCD, described in Section 6.2, ask for an implementation that is designed and configured for the task at hand in any of its parts. Operating system kernels are compiled with patches that enable fine-grained real-time task scheduling policies. The software is written with frameworks that guarantee real-time execution, such as MARTe2. The hardware is powerful enough to handle the computational load of the control algorithms, reserving to them all the system resources they require (*e.g.*, CPU, RAM, PCI bandwidth) in a deterministic fashion, cutting out every other task including system ones. When sampling times get to the microsecond precision declared in Section 6.2, every component of the system has to be carefully considered.

In this scenario, things are a bit complicated by the fact that SCD is *distributed* in the sense that has been described in Chapter 1. It is not a single system but an interconnection of multiple computational nodes each executing a predefined set of tasks encoded in specific software subsystem, also interfacing with dedicated hardware components when necessary. Each node cannot perform its tasks alone: for example, control nodes 02 and 03 cannot actuate the coil power supplies properly without the real-time data provided by the plasma equilibrium reconstruction, which runs on nodes 05 and 06, which in turn need the data from the plasma diagnostics running on other nodes. By this description, it is clear how the communication infrastructure among the nodes has to be carefully designed and implemented to guarantee the real-time performance of the whole system. It requires a technology that makes it fast, deterministic, and reliable.

Currently (see [92]), the SCD nodes are interconnected by a real-time fiber optic network that implements a *reflective memory* (RFM). In essence, reflective memories are shared memory systems that allow multiple machines to share a data area. The implementation of the RFM usually consists of a high-speed connection medium, in this case an optic fiber connection, and dedicated network cards to install on the machines forming the network. These cards contain the actual data area and their device drivers map that area within the main memory of the host machine. The RFM implementation is such that each local copy of the shared data area is kept coherent with the others, so that every write operation is propagated to all the other copies. The timings that these operations might take can be very tight but, most importantly, they are deterministic. This makes the RFM a suitable technology for this kind of application scenario.

The RFM technology has been commercially available since the 1980s and SCD adopts Abaco Systems network cards [2]. As stated in Section 6.2, these cards and their fiber optic interconnection allow SCD to maintain sub-microsecond latencies when transferring data among the nodes with a throughput up to 170 MB/s. This implementation has allowed SCD to perform well in the last decade but it is now showing some concerning issues, especially in terms of scalability and maintainability. First, as stated in [2], the Abaco cards are reaching their End Of Life (EOL) status with a limited production phase started in 2022. This, together with their high price (around 8.000€/card), is creating procurement concerns at SPC for such a critical component of the SCD. Secondly, the throughput of the RFM network is not enough to handle the increasing amount of data that the SCD nodes have to exchange. New diagnostics are being evaluated and added each year, with some of them capable of generating considerable amounts of data, such as the MANTIS camera system on Node 10. The problem becomes clear when one notes that the data area is shared among *all* the nodes, so they have to use that space to exchange data coming from every device and subsystem all at the same time. Despite some access policies that have been implemented to deal with this without increasing latencies (see, again, [92]), a saturation point has been reached and new diagnostics such as bolometry cannot currently be used in real-time control because their data cannot be shared among the nodes.

To enable further upgrades to the SCD increasing TCV's experimental capabilities, as well as to avoid ending up relying on an EOL component, a replacement solution for the RFM network has become necessary. Various criteria have been applied to its selection, including the cost-effectiveness of relying again on dedicated hardware components when common network hardware such as Ethernet has evolved so much in the meantime, guaranteeing bandwidths that were simply unthinkable at the time in which the reflective memory was invented. Ideally, a properly configured, reserved Ethernet network is enough if it is driven by a software protocol that is able to guarantee the same determinism and low latencies of the RFM. The Data Distribution Service, introduced in Section 3.1.1, is a candidate technology that can provide these features and has been selected as a replacement for the RFM network in the SCD.

This section describes the design of a software component for the SCD architecture that integrates the DDS middleware for communication purposes. Then, it presents some results of its application on SCD nodes, showing promising performance that motivated the decision to proceed with the integration of the DDS in the SCD. These results prove how the distributed technologies that constitute the base of the DUA framework presented in Chapter 4 can be applied successfully even to a different context such as the control system of a tokamak device.

6.3.1 Design of the DDSDataSource for the MARTe2 framework

As every hardware component of the SCD architecture, the reflective memory is integrated with it by the means that the MARTe2 framework provides. Recalling the overview of MARTe2 given in Section 2.3.6, the software components dedicated to the integration of specific hardware are called DataSources. The reflective memory makes no exception to this structure, since it is interfaced with MARTe2 by a DataSource called `RFM2g`, that can either read from or write to the shared data area. Thus, the end goal of the DDS integration process is the development of a new DataSource that replaces the RFM2g one, called DDSDataSource. In a similar way to the RFM2g, the DDSDataSource may act as either a publisher or a subscriber on a given topic. The following description of the design of the DDSDataSource is largely based on the characteristics of a DDS implementation, which have been illustrated in Section 3.1.1 and has been presented in [155]. The DDSDataSource class is a C++ class that inherits from the DataSourceI interface of MARTe2, which is the base class for all the DataSources in the framework. Thus, all the DDSDataSource features hereby described are implemented within the methods of this class, that the MARTe2 framework calls at specific times during the execution of the real-time loop.

Before the DDSDataSource features can be presented, it is instrumental to address the three main problems that steered the design process. Two of them are mainly related to the DDS implementation, while the third directly concerns the SCD use case. As the following discussion clarifies, the DDS middleware offers a vast set of configuration options and capabilities but the SCD use case requires a little subset of them which can ensure efficient and reliable operation. The chosen DDS implementation for this work is eProsima Fast DDS [72].

How to discover other entities?

One can recall (see Section 3.1.1) that the DDS standard specifies the Discovery Protocol to automate the discovery and configuration of all DDS entities in the network. Such entities include both DDS participants and the DataReaders and DataWriters that share the actual data being transmitted. By default, this protocol operates with multicast IP transmissions requiring no manual configuration.

In a distributed, large scenario such as a swarm of autonomous robots, being able to automate the discovery phase is a great advantage. However, in the SCD case it might not be necessary and even counterproductive. First, the SCD network is much smaller than that of a swarm of robots, with currently 10 nodes that are using the DDS over an isolated, dedicated 1 Gbps Ethernet network where all the hosts are known and preconfigured. Consequently, the number of topics and thus that of expected communication endpoints is also relatively small, especially compared to that of the complete software architecture of a sophisticated autonomous robot equipped with multiple sensors, actuators, and software modules.

For this reason, the Discovery Protocol feature of the DDS standard is not required in the implementation of the DDSDataSource. Instead, it is preferred to expose manual configuration options of the DDS network settings, to avoid the overhead of the discovery phase and let the SCD developers have full control over the network configuration. In the DDS implementation, the network settings are handled by the DomainParticipant object, which has to be unique for every application that uses the DDS. The DDSDataSource is a software component implemented as a C++ class, thus multiple instances of it may be present inside a single MARTe2 application. For this reason, the DataSource implementation automatically instantiates a single DomainParticipant for the entire MARTe2 application, in a way such that new DataSource instances retrieve a reference to the already existing DomainParticipant.

This feature is completely transparent to the developer using the DataSource, who only has to specify the network settings that are then applied process-wide through the `DomainParticipant`.

What kind of messages to exchange?

A great advantage of the DDS middleware over the RFM is that while the latter offers a single data area shared among all the nodes with a fixed size, the former allows the definition of multiple topics, each with its own dedicated data channel that can be configured to have a specific format. Nodes can subscribe and publish to the topics they require, even concurrently: this implies that the data that can flow over the network is much more than that shared by the RFM. This approach is more flexible and allows the nodes to use up to the whole bandwidth of the Ethernet network dedicated to DDS communications. As illustrated in Section 3.1.1, the middleware allows to define the data structure of the messages that are exchanged over the network. In the case of the SCD, considering all the different diagnostics and actuators, there are many different structured messages to define. Due to time constraints (a replacement for the RFM had to be quickly developed), such a sophistication is not required at this stage. Currently, the `DDSDataSource` is designed to exchange a single message type, which is a byte array that can contain any kind of signal. In MARTe2, outgoing signal data passes through memory exchange components, called `IOGAMs`, before it reaches the DataSource; there, multiple signal samples can be packed into a single byte array which then becomes a DataSource message. The same holds for data coming from the network: a byte array coming from the DDS layer can be unpacked by an `IOGAM`. The specific instructions to pack and unpack the signals are given in the `cfg` of the MARTe2 application.

How to exchange data with the SCD framework?

The last critical issue to address concerns the expected behavior of a data exchange involving the `DDSDataSource` within a MARTe2 application running on SCD. This is due to how the `RFM2g` DataSource to be replaced currently behaves. Data exchanged over the reflective memory network can be grouped in two distinct classes, differentiated by their purpose:

- data that is required for offline, slow reconstruction of the plasma equilibrium, sampled at lower rates and with the possibility of higher latencies;
- data that is required for *real-time fast control*, sampled at the highest rates allowed by the SCD.

The perspective that better highlights this problem is that of a `RFM2g` DataSource that reads data from the shared memory area. Since the reflective memory has an internal clock, and because MARTe2 threads have to execute a real-time loop at a fixed rate, the DataSource component can synchronize itself with the hardware and ensure that the thread rate is the same as the RFM clock. Ideally, since the RFM satisfies the timing requirements of magnetic control, this is feasible. In practice, it has been found that this is not always the case: there are situations in which the RFM network jitter occasionally becomes non-negligible. This may have critical and potentially destructive repercussions on the plasma equilibrium, as the control algorithms may not be able to actuate the coils in time to stabilize the plasma. The solution to this problem, currently implemented in the `RFM2g` DataSource, is to handle the two situations described above in two different ways, corresponding to two different operation modes of the `RFM2g` DataSource acting as a reader.

`Synchronized` The reader DataSource is synchronized with the RFM clock and reads data at the highest rate allowed by the network. This mode is used for data that is not involved in fast control, since the RFM jitter may cause the DataSource to miss deadlines. This also implies that, if new data is produced at each RFM clock cycle, the DataSource returns a new data sample at each iteration of the real-time thread loop.

`NotSynchronized` The reader DataSource is not synchronized with the RFM clock and reads data at a rate specified by some other component running in the real-time thread (*e.g.*, a `LinuxTimer` DataSource). This rate can be set equal to that of the RFM clock but the DataSource is not bound to it, allowing it to deterministically respect fast control deadlines. A direct consequence of this behavior is that, if new data is produced at each RFM clock cycle, the DataSource returns a new data sample only when the RFM clock cycle is completed, else it returns again the last valid sample. Clearly this makes the DataSource act as a sort of sample-and-hold device, but repeating a sample occasionally does not cause problems in the SCD control loop in practical scenarios and is preferred to missing deadlines.

To streamline the integration of the new `DDSDataSource` in the SCD, its behavior has to be exactly the same as described above.

6.3.2 Features of the DDSDataSource

As any DataSource, the `DDSDataSource` can handle data in two ways: it can either publish data to the network or subscribe to data coming from it. Thus, it can be configured to act either as a `Publisher` or as a `Subscriber`; sample MARTe2 configurations for each are shown in Listings 6.1 and 6.2 respectively. This first configuration option corresponds to the `Mode` parameter. The rest of the configuration options are described below and constitute the features of the `DDSDataSource`.

Each `DDSDataSource` instance has a specific `Name`, which is a character string set by the developer in the configuration file that uniquely identifies that DataSource in the network. This parameter is not required by the DDS implementation but by a functionality of the `DDSDataSource` itself, termed the *handshake protocol*. During the shot initialization phase, the SCD state machine initializes all the nodes and their software components and awaits for their readiness. The `DDSDataSource` instances, when started, provide a similar acknowledgement to the state machine so that the shot routine proceeds only when all communication channels and related DDS entities have been correctly initialized. To do so, the `DDSDataSources` synchronize with one another by transparently exchanging messages containing their names with a `TRANSIENT_LOCAL` durability policy (see Section 3.1.1). This is a feature that is not present in the `RFM2g` DataSource and that is entirely made possible by the DDS middleware. The `NamesToMatch` parameter is a string containing a semicolon-separated list of names of other `DDSDataSources` with which the `DDSDataSource` has to synchronize, awaiting for their presence message before completing its own initialization.

The `DDSDataSource` exchanges data over a specific network interface, which is specified by the `NetworkInterface` parameter: a string containing either the character representation of the NIC IP address or its system name. The `TopicName` parameter is a string that identifies the topic to which the `DDSDataSource` is associated. Transmissions belonging to that topic must observe a specific Quality of Service policy on all the nodes that interact with it; such a policy can be specified by setting the `ReliabilityKind`, `HistoryKind`, and `HistoryDepth` parameters. These parameters specify the reliability of the data transmission, the kind of buffering that the DDS middleware has to apply to the data, and the depth of the buffer, respectively (see, again, Section 3.1.1). Note that they have to match among configurations concerning DataSources operating on a same topic, else the corresponding DDS entities do not match and the communication is not established. The `MaxSamples` parameter specifies the maximum number of samples that the DDS middleware implementation can handle: in a real-time framework such as MARTe2, this setting is used to preallocate the memory that is used to store the samples, thus avoiding dynamic memory allocations during the execution of real-time loops.

The `ExecutionMode` parameter concerns the behavior of the DataSource with respect to the previous `RFM2g` implementation, as discussed in Section 6.3.1 where the effect of this setting on the `Subscriber` DataSource has been described. Regarding the implementation of this mechanism, the `Subscriber` DataSource always stores the last valid sample received from the DDS layer, returning either it or a new one depending on its `ExecutionMode` setting and the availability of new data. However, this setting also has a meaning for `Publisher` DataSources: in this case, the DDS implementation can spawn a dedicated thread to handle data writes to the network layer (*i.e.*, system calls that are part of the transport protocol OS API). Thus, the two possible behaviors for a `Publisher` are as follows.

`Synchronized` When writing data to the network layer, the DDS API blocks the calling thread until the data has been effectively passed to the transport layer. This is the preferred behavior in the MARTe2 framework, since it guarantees that the data is effectively transmitted to the network before the real-time loop proceeds to the next cycle.

`NotSynchronized` When writing data to the network layer, the DDS API does not block the calling thread and returns immediately. The data is just copied to a buffer that is then processed by a dedicated DDS implementation thread. This is the default behavior of the DDS implementation since it reduces application latencies, but it may not be suitable for fast real-time loops.

The last configuration parameters of a `DDSDataSource` concern real-time scheduling of internal threads within the MARTe2 application. For each `DomainParticipant`, the eProsima Fast DDS implementation spawns the following three threads.

- A thread handling asynchronous events.
- A thread handling the reception of data from the transport layer.
- A thread handling the transmission of data to the transport layer.

These threads are always necessary, since they handle both DDS control transmissions and actual data packets. This also clarifies why it is preferable to have a single `DomainParticipant` for every application on a single DDS domain, as mentioned in Section 3.1.1. In the MARTe2 framework, every thread must have a specific real-time scheduling policy. Usually, this task is handled by the MARTe2 scheduler in a way that is completely transparent to the developer, so there is no way to let the MARTe2 scheduler handle these three threads. In compliance to what is done in the core MARTe2 library, these threads are assigned a `SCHED_FIFO` scheduling policy with a priority level of `75`. The `SenderThreadCPUMask`, `EventsThreadCPUMask`, and `ReceiverThreadCPUMask` parameters are hexadecimal numbers that specify the CPU cores on which the threads are allowed to run. These parameters allow the developer to specify the CPU affinity of the DDS threads in a similar way to what is done for the MARTe2 threads.

Note that on every SCD node runs a single MARTe2 application, which can contain an arbitrary number of `DDSDataSources` that share the same `DomainParticipant`, thus the same three internal threads. This means that the price to pay to replace an expensive, EOL, obsolete hardware component with a software component that is more flexible and scalable is three CPU cores per node plus a few Ethernet cables.

The `Signals` section of the `DDSDataSource` configuration specifies the MARTe2 signals that the DataSource handles, in either the send or receive direction. To transmit a variable-length byte array, providing at the same time some basic diagnostic information, the following two signals are defined.

- `MsgCounter` is a signal of type `uint32` that contains the number of messages that have been exchanged by the DataSource since its initialization.
- `DataBuffer` is a signal of type `uint8` that contains the actual data to be exchanged over the network. It is a one-dimensional array whose dimension has to be set by the developer in accordance to the resulting size of the packed array of signals that is produced, or expected, by the `IOGAMs`.

```
+DDSPublisher = {
    Class = DDSDataSource
    Mode = Publisher
    Name = "Pub05"
    ExecutionMode = Synchronized
    NetworkInterface = "eno1"
    TopicName = "rfm_1"
    HistoryKind = KeepLast
    HistoryDepth = 10
    ReliabilityKind = Reliable
    MaxSamples = 1000
    SenderThreadCPUMask = 0x4
    EventsThreadCPUMask = 0x8
    ReceiverThreadCPUMask = 0x10
    NamesToMatch = "Sub06"
    Signals = {
        MsgCounter = {
            Type = uint32
            NumberOfDimensions = 1
            NumberOfElements = 1
        }
        DataBuffer = {
            Type = uint8
            NumberOfDimensions = 1
            NumberOfElements = 1024
        }
    }
}
```

Listing 6.1: Sample configuration settings of a `Publisher` `DDSDataSource`.

```
+DDSSubscriber = {
    Class = DDSDataSource
    Mode = Subscriber
    Name = "Sub06"
    ExecutionMode = NotSynchronized
    NetworkInterface = "eno1"
    TopicName = rfm_1
    HistoryKind = KeepLast
    HistoryDepth = 10
    ReliabilityKind = Reliable
    MaxSamples = 1000
    SenderThreadCPUMask = 0x4
    EventsThreadCPUMask = 0x8
    ReceiverThreadCPUMask = 0x10
    NamesToMatch = "Pub05"
    Signals = {
        MsgCounter = {
            Type = uint32
            NumberOfDimensions = 1
            NumberOfElements = 1
        }
        DataBuffer = {
            Type = uint8
            NumberOfDimensions = 1
            NumberOfElements = 1024
        }
    }
}
```

Listing 6.2: Sample configuration settings of a `Subscriber` `DDSDataSource`.

6.3.3 Benchmarking results

As illustrated in [92], the machines currently implementing SCD nodes form a heterogeneous set, spanning from recent servers to hardware that is even a decade old. This is due to the fact that the SCD has been developed over the years and, during each improvement, continuous operation of the TCV machine had to be ensured, deferring the upgrade of some of the most critical systems.

The eProsima Fast DDS implementation version used in this work is 2.14, which requires a Linux kernel 5 or later, and a GCC 11.4 toolchain supporting the `C++17` language standard. For this reason, at the time of writing, the `DDSDataSource` has not yet been tested during a real experiment on the TCV machine. For this to happen, the nodes have to be fully upgraded replacing the oldest machines, a process that is currently ongoing. However, the `DDSDataSource` has been tested in a laboratory environment, on the newer machines that are shortly going to replace the older ones. The results of these tests are promising and show that the DDS middleware can be successfully integrated into the SCD architecture.

Two tests have been performed, one for each of the two operation modes of the `DDSDataSource` described in Section 6.3.1. The first test has been performed on a `Publisher` `DDSDataSource` that

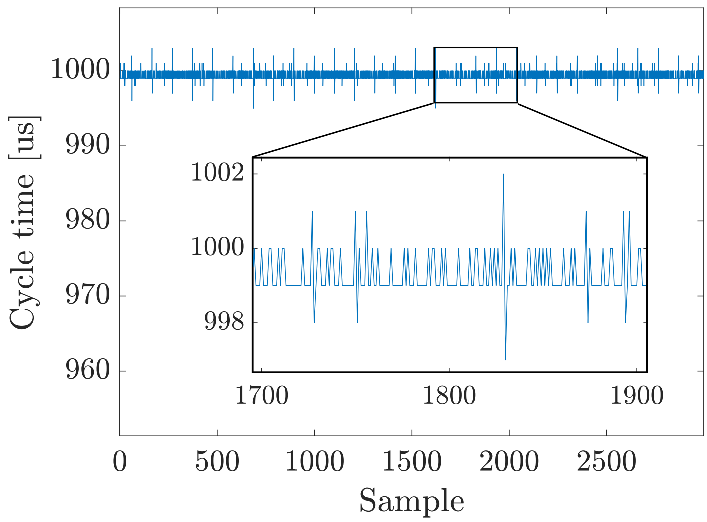

Figure 6.4: Steady-state cycle time of the real-time thread containing a `Publisher` `DDSDataSource` running in `Synchronized` mode.

sends data to a `Subscriber` `DDSDataSource` running in `Synchronized` mode, while the second one involved a `Subscriber` running in `NotSynchronized` mode. In both situations, data has been sent from one machine to another over a dedicated 1 Gbps Ethernet connection, with a payload of 4 kB corresponding to a page of main memory; this payload was always passed to proper `IOGAM`s, so the cycle times also involve significant memory copy operations. The `DDSDataSource` data exchange was the only task that the real-time threads had to perform, so that cycle time data gathered during the tests has been used to fully evaluate the latencies introduced by the DDS middleware. All the data to evaluate the benchmark has been gathered using MDSplus (see Section 2.3.6).

Publisher side

The `Publisher` ran in `Synchronized` mode during both tests because, even if this feature is also available to publishers, it is not desirable in this context to let an internal DDS thread buffer write operations to the transport layer. The steady-state cycle time of the real-time thread running the `Publisher` DataSource is shown in Figure 6.4. The thread runs at a rate enforced by a `LinuxTimer` DataSource, which is configured to run at 1 kHz. As Figure 6.4 shows, there is some overhead due to the DDS write operations towards the transport layer, but its maximum entity of 3 μs is negligible compared to the overall cycle time. Thus, the publisher side of the `DDSDataSource` is suitable for fast control applications.

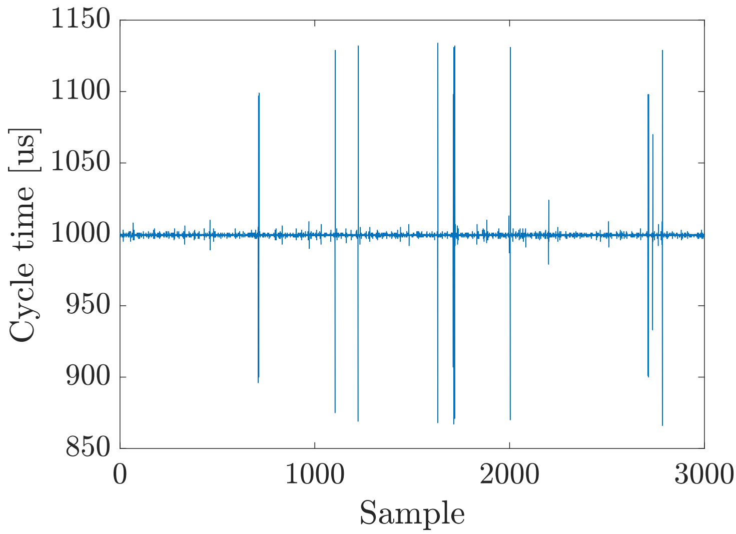

Figure 6.5: Steady-state cycle time of the real-time thread containing a `Subscriber` `DDSDataSource` running in `Synchronized` mode.

Subscriber side in Synchronized mode

The steady-state cycle time of a real-time MARTe2 thread running the subscriber `DDSDataSource` in `Synchronized` mode is shown in Figure 6.5. Recall that in this mode the DataSource is synchronized with the transport layer and blocks the running thread, waiting until a new data packet has been received. Thus, the expected cycle time of the real-time thread in ideal conditions is that of the publisher thread, equal to 1000 µs. Figure 6.5 shows that the actual cycle time averages to that value, but the jitter reaches considerable peaks of over 100 µs. This is due to the fact that each time the thread has to wait for a new data packet, it hands over control to the operating system: the thread is first removed from the run queue of its CPU, then it is put back in when a new data packet is received. This operation and all the context switches in between introduce non-deterministic latencies that, no matter how efficient and deterministic the kernel scheduler is, become noticeable when the timings have this order of magnitude. Thus, this behavior is expected and compliant with the `RFM2g` implementation but it is unavoidable and confirms that this execution mode suits data collection and offline diagnostics use cases.

Subscriber side in NotSynchronized mode

Figure 6.6 depicts the steady-state cycle time of a real-time MARTe2 thread running a subscriber `DDSDataSource` in `NotSynchronized` mode. Recall that, in this mode, the DataSource does not enforce a rate for the real-time thread loop, thus a `LinuxTimer` DataSource has been added to the tasks performed by the thread. The rate of the `LinuxTimer`, to test the capabilities of the implementation, has been pushed at 5 kHz: five times higher than that of the publisher, resulting in an expected cycle

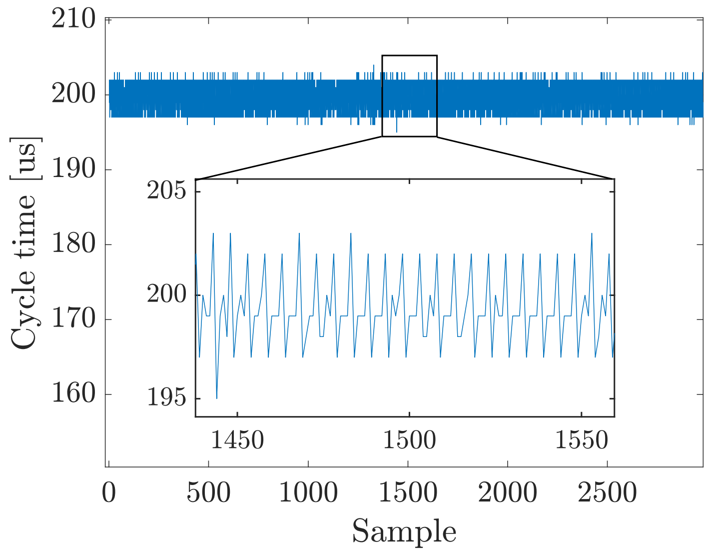

Figure 6.6: Steady-state cycle time of the real-time thread containing a `Subscriber` `DDSDataSource` running in `NotSynchronized` mode.

time of 200 µs. Note that, by configuring the real-time thread in this way and with this execution mode, the `DDSDataSource` copies and returns the same 4 kB sample five times for every publisher cycle. Figure 6.6 shows that the actual cycle time is, in practice, compatible with the expected value: the maximum jitter observed at steady-state is equal to 5 µs, which is negligible compared to the overall cycle time. Moreover, the jitter is so small and periodic that it is fully compatible with the maximum time resolution achievable by the `LinuxTimer`, which is providing the clock to the entire real-time thread. This final test confirmed that the `DDSDataSource` implementation is also suitable for fast feedback control applications when configured in this way and confirmed it as a replacement for the `RFM2g` DataSource in the SCD architecture.

7 Plasma shape control for tokamaks

In Chapter 6, the second application scenario of this work has been introduced. In particular, Section 6.1 illustrated the Tokamak à Configuration Variable device, and Section 6.2 provided a thorough description of the Système de Contrôle Distribué that controls TCV. That description clarified how the distributed control system of TCV allows multiple diagnostics, algorithms, estimators, and controller to coexist and interact within a single control architecture. Thanks to this design, the experimental capabilities of TCV have been extended significantly. One of the latest innovations in this context has been the introduction of a *shape controller*, *i.e.*, a subsystem that acts as a reference governor and PID controller for the coil power supplies in order to track a desired plasma shape. Its implementation currently works in coordination with other SCD subsystem, in particular those that generate feedback measurements concerning the equilibrium reconstruction; the measurement that they provide are used to evaluate the shape control law in real-time.

Plasma shapes, characterized by distinct cross-sectional geometries in the poloidal plane, exhibit varying properties regarding stability, performance, and interaction with vessel components. The plasma shape significantly influences the stability characteristics of global *magnetohydrodynamic* (MHD) modes and regulates particle transport phenomena. Furthermore, precise plasma shaping enables an optimal use of both the chamber and the plasma volumes while facilitating controlled management of power deposition from plasma-wall interactions in regions where energy escapes the confined plasma core.

The redundant array of poloidal field coils makes TCV a unique device in the tokamak landscape. The flexibility of the PF coil system allows the researchers to explore a wide range of plasma shapes, each of which exhibits different phenomena and is relevant for the advancement of both plasma and fusion research. This capability has been exploited extensively during the thirty years of TCV operation, leading to significant results in the field [8, 99, 106, 201].

However, regarding the SCD implementation, a real shape control subsystem of the SCD magnetic control has never been fully developed and made operational until 2024. Before, the shape control function has been achieved in feedforward mode, *i.e.*, by evaluating offline the coil current references to give to the power supplies during each shot. This procedure is certainly prone to errors, as it has no feedback and does not take into account real-time measurements of the plasma shape. However, it worked in most cases thanks to the high quality of the offline equilibrium reconstruction and the very brief duration of TCV shots. Nonetheless, to fully exploit the experimental capabilities of the device and to ensure an effective and reliable control of the plasma shape, a real-time shape controller is necessary.

This chapter presents the work that has been done on the implementation of the TCV shape controller while working on the SCD. First, the current state of the art in shape control of tokamak plants is reviewed, focusing subsequently on what has been done recently on TCV. Then, the limitations of the current implementation are discussed and a set of possible improvements is evaluated. Finally, an improved scheme is designed and evaluated in dedicated experiments.

7.1 State of the art in plasma shape control

The plasma-circuit interaction dynamics are fundamentally governed by nonlinear partial differential equations (PDEs). However, control system design typically employs reduced-order, linear time-invariant models based on ordinary differential equations (ODEs). This methodological gap is addressed through specialized plasma modeling software suites that generate simplified mathematical representations while preserving essential physical characteristics. These computational tools, termed *codes*, enable a systematic approach to model reduction, striking a balance between retaining the fundamental plasma physics and producing tractable formulations suitable for control synthesis.

To define a plasma shape and its position, two key elements are considered: the *centroid* and the *last closed flux surface* (LCFS). The centroid is the center of the plasma cross-section along the poloidal plane, and can be either *magnetic* or *geometric*; the type of centroid to consider depends on the specific application at hand. The magnetic centroid is usually involved when it is convenient to approximate the plasma as a filament carrying the plasma current; in that case, the magnetic centroid corresponds to this ideal filament revolving around the vessel. When dealing with magnetic control, which is also the case of this work, the magnetic centroid is usually considered.

Poloidal field coils, running along the torus, generate a poloidal field that induces a *poloidal flux* in the vessel. Along the plasma volume, there are points in which this flux has the same value. Examining the plasma from a poloidal cross-section, one can group this points by flux values, thus identifying *isoflux surfaces*. These run outward starting from the plasma centroid, initially as closed surfaces that enclose one another. The last closed one of these surfaces is identified as the LCFS.

Thus, controlling the plasma shape usually involves tracking reference setpoints for the centroid and the LCFS, expressed as a set of control points. The shape control capabilities directly impact operational parameters and plasma-material interactions, allowing for advanced scenarios that improve performance while preserving machine integrity. This type of control represents a critical tool for achieving optimal plasma conditions while maintaining operational safety margins. To achieve different plasma configurations in terms of shape, the machine must be equipped with both a vessel large enough to contain them and with a set of coils that can generate the necessary magnetic fields.

Plasma shape control techniques can be broadly classified into two families: *isoflux-based control* and *gap control*. In isoflux-based control techniques [9], a set of shape control points identifies the last closed flux surface, and the poloidal flux and magnetic field at these points are regulated to track the desired plasma shape. This approach is direct and yields a high tracking precision, requiring minimal modification to the equilibrium reconstruction code as it relies solely on the interpolation of flux and field maps at the control point locations. By minimizing discrepancies between measured and desired flux profiles, isoflux control ensures that the plasma maintains optimal stability and performance.

Conversely, gap control [243] focuses on maintaining fixed distances between the plasma surface boundary and the surrounding vacuum vessel or other wall components. These distances are maintained by acting on the magnetic fields generated by the coils. The control laws involved operate in feedback from the measurements of magnetic probes that evaluate the gaps at some *critical* points. Gap control proves particularly effective in ensuring that the plasma does not approach the vessel wall too closely, thereby reducing the risk of increased impurity influx, excessive cooling of the plasma edge, or even direct physical contact with the vessel.

Regulating plasma shape can be conceptualized as a Multi-Input Multi-Output (MIMO) control problem, which is commonly addressed using model-based methodologies that rely on linearized plasma response models, such as the CREATE-L [4, 241] model. A known application of this approach is the eXtreme Shape Controller (XSC) [7], originally developed and validated for the JET tokamak [5]. Over time, controllers with a similar structural design have been implemented in various other devices, including the FTU [6], EAST [9] and JT-60SA [53] tokamaks.

7.2 State of the art in plasma shape control at TCV

Despite TCV's redundant coil configuration and the broad range of plasma shapes it can support, the absence of a unified shape control system that fully integrates with existing shot design and preparation tools has remained a persistent limitation. Previous attempts at developing a shape controller for TCV have shed light on both the complexity of the task and the potential rewards brought by a successful solution. Early work reported in [15, 16] introduced an H_∞ controller aimed at managing key global shape parameters: triangularity, elongation, and vertical displacement. Although this approach represented a significant step forward, it relied heavily on a control system that is no longer in use at TCV, preventing its applicability in contemporary experiments. Another notable effort described in [12] proposed a novel shape control design but entirely replaced the pre-existing magnetic control architecture. This approach hindered seamless integration with the established framework and shot planning methodologies (see Section 6.2), ultimately reducing its practical value.

More recently, a pioneering AI-based plasma shape controller developed in collaboration with Google DeepMind [58] has shown remarkable capabilities during discharges. It has been developed to act on each phase of a shot, from the breakdown to the shape formation; it is currently being expanded to achieve more sophisticated plasma configurations. This innovative system highlighted the promise of advanced computational methods for achieving more agile and responsive control. However, its dependence on extensive computational resources (particularly for neural network tuning) has made it challenging to adapt and reconfigure between experiments.

The current approach is described in [162]. It implements an isoflux control strategy to regulate the plasma boundary configuration. This approach maintains the plasma shape by enforcing equivalent poloidal flux values between a set of control points positioned along the last closed flux surface and a reference flux value measured at a specified *boundary point*. The designation of this boundary point differs based on the plasma configuration: for *limiter* plasmas, it corresponds to the limiter contact point, while for *diverted* configurations, it is defined by the primary *X-point* location. The control methodology varies according to the configuration type. In limiter scenarios, the controller enforces a zero orthogonal field component at the contact point to ensure the boundary intersects with the reference position. Diverted configurations, conversely, require simultaneous regulation of both radial and vertical field components to zero at the X-point location, thereby maintaining the desired magnetic topology. Furthermore, the study of some specific divertor configurations requires the control of other quantities such as poloidal flux at non-boundary points, flux difference between active and inactive X-points, and the position of inactive X-points; these too can be controlled by the shape controller.

Once a desired shape has been selected, the first step of the shape controller design procedure consists in determining a linearized model of the TCV plant around the prescribed equilibrium. This is quite a crucial and challenging task, usually solved by the suite of equilibrium reconstruction codes developed at SPC. The result of this design stage also yields a numerical representation of the plasma shape in terms of its LCFS, encoded as a set of control points.

The controller architecture implements a decoupling strategy for the controlled channels at low frequencies through singular value decomposition (SVD) of the steady-state plant dynamics. This decomposition is performed on the weighted static gain matrix of the system. To ensure both numerical feasibility and prevention of excessive control action, the controller design considers only singular values exceeding a specified threshold. This reduction in the problem dimensionality serves the dual purpose of maintaining computational feasibility while limiting control effort.

The architecture incorporates a first-order low-pass filter for each channel to attenuate oscillatory behavior. This design choice addresses the resonance phenomena observed in the Bode plots of the dominant singular modes across various TCV operational scenarios. Control of the selected dominant modes is achieved through a diagonal multiple-input multiple-output (MIMO) PID controller. When a state-space representation of the hybrid controller is available, two approaches exist for gain tuning: an analytical method based on assumed ideal channel decoupling with specified cutoff frequency and phase margin requirements per channel, or optimization-based techniques for refined parameter selection. The PID tuning methodology enables complete automation of the controller design process, upon the availability of a linearized plasma response model. The end user only has to specify the weighting matrices along with fundamental control parameters, such as the desired crossover frequency and phase margin.

This degree of automation proves particularly valuable during experiments, allowing the operators to be effective without extensive control theory expertise and ensuring an easy integration within the existing SCD framework.

7.3 Limitations and possible improvements of the TCV shape controller

The current design of the TCV shape controller has been quite the challenge for the development team and its integration has turned out to be a delicate matter. Nonetheless, the result is a system that works: given a reference plasma shape and after some tuning, the TCV shape controller is able to control the plasma shape within the limits of the actuators and the diagnostics. Recalling the discussion of the redundant coil configuration of TCV in Section 6.1, which constitutes probably its most peculiar feature, the question arises whether the current shape controller exploits this redundancy to improve the performance of the coil current control or not. Up to this point, it does not.

The PID formulation of the shape controller, described in [162], does not account for the actuator redundancy offered by the TCV model, which may in turn be exploited to improve the performance of the shape controller itself. Performance indicators to be considered in this context are the overall power consumption of the coils, as well as the saturation limits of the power supplies. In practical experiments, these have turned out to be important aspects to consider when working with TCV.

The above mentioned question leads one to think whether something can be done to improve the present shape control scheme. This is the starting point for the novel work presented in this chapter. An immediate solution is to include the performance optimization goals in the design of the shape controller. This approach, although ideally feasible, leads to the definition of a control problem that is much different from the original one. Determining a solution to it, considering also all the intricacies and complications that are involved in the control of a plasma shape on TCV, is a challenging task. Another approach consists in keeping the current shape controller design, which works and has been validated through numerous experiments, essentially as it is. However, a new control problem can be formulated at this point, considering the current shape controller as part of the dynamic system and with the goal of optimizing the performance of the shape controller by altering its output before it is fed to the plant. This is the approach that has been followed in this work, relying on a specific subject of control theory that is receiving increasing interest in recent years: that of *dynamic control allocation*.

Dynamic control allocation techniques are used to distribute the control effort among a set of redundant actuators in order to optimize a given performance index, following an *allocation policy*. What distinguishes *dynamic* control allocation from similar techniques is that, in the former, the allocation policy is a control law itself that is determined through a control design process and evaluated in real-time. Subsystems that are typically involved in a dynamic control allocation scheme are denoted as *dynamic allocators* or simply as *allocators*. The peculiarity of dynamic allocators is that they are designed as *add-ons* to a preexisting control system, *i.e.*, their design does not require a complete redesign of the control system, which may yield significant complications when optimization objectives are included in the problem. Instead, given the models of the plant and the control system, their design can be carried out *a posteriori* and they can subsequently be integrated into the control scheme as additional subsystems. The art of dynamic control allocation, therefore, lies in the design of these additional components with the goal in mind of not only optimizing the control action in terms of some criteria of practical relevance but also of ensuring that the original control objectives are still met without any alteration of the desired output behavior.

Regarding TCV, a methodological study has been carried out in [229], in which an early implementation of a dynamic coil current allocator has been integrated in the MATLAB simulation code suite. Its performance has been validated by means of simulations on real shot data, which showed the feasibility of this approach. The rest of this chapter presents the work that has been carried out in this direction. First, a review of the current state of the art about dynamic control allocation is presented, together with a set of formulations that are of interest for the TCV case. Each of these has been considered, leading to some theoretical results that are illustrated in the following. Then, the design of a novel dynamic control allocator for TCV is presented, which is based on the latest advancements in the subject. Experimental results are discussed at the end of the chapter, showing the performance of the proposed novel allocator in a real scenario and constituting the first working implementation of its kind in the literature.

7.4 State of the art in dynamic control allocation

The evolution of technology over recent decades has led to the emergence of highly interconnected systems (see Chapters 1 and 2) characterized by increasingly complex mathematical models. A distinctive trait of many such systems is the presence of multiple actuation devices. These actuators, which may possess diverse capabilities and characteristics, may be kept as spare or operate in a *redundant* manner. In the latter case, the resulting controlled plants feature more actuators than sensors and measurement devices, leading to their classification as *over-actuated systems*. This configuration implies that an identical output behavior can be achieved through different input configurations. However, this flexibility is frequently perceived in practice as a liability, leading to control architectures that first reduce the model to a square plant through, *e.g.*, *squaring-down* techniques and then apply conventional control strategies. This approach not only foregoes the advantages of an intrinsically redundant plant structure but may also result in closed-loop systems with suboptimal performance or undesirable characteristics.

The ability to select control inputs that both align with prescribed closed-loop outputs and meet specific performance criteria has therefore become crucial. These performance objectives may encompass enhanced system behavior, reduced power consumption, or improved reliability with respect to practical limitations. Given these compelling benefits, the allocation of control in input-redundant plants remains an active and vital research domain, as detailed in [27, 104, 111]. The growing significance of this field is evident in its diverse applications, spanning automotive systems [48], aeronautics [219], and tokamak fusion reactors [56, 57, 100, 229].

To properly contextualize relevant contributions, it is essential to distinguish between *static* and *dynamic* control allocation techniques. Static allocators simply redistribute control signals across plant inputs according to fixed laws expressed as, *e.g.*, gain matrices. Conversely, dynamic allocation consists in the implementation of allocation units as dynamic compensators within existing control schemes. The aim of these units is to preserve the original controlled plant output while optimizing the input or even the plant state evolutions through input redundancy. Foundational work in dynamic control allocation for linear systems includes [43, 44, 218, 247].

The notion of a dynamic control allocator is formalized for the first time in [247], where it stems from the concepts of *weak* and *strong input redundancy*, which characterize two possible plant configurations with respect to the effect of the allocation action on the output behavior. According to [247], a *weakly input-redundant* plant is such that the allocation action becomes *invisible* in the output only at steady-state; conversely, a *strongly input-redundant* plant rejects the input allocation action within the state evolution, making it invisible in the output at all times. As shown in the following, this characterization has been superseded by the introduction of a dynamic subsystem of the input allocator that is entirely devoted to ensure output invisibility, thus proving that the output invisibility requirement does not completely depend on the state-space description and properties of the plant. The study conducted in [247] is also the base for some notable work regarding the design of control allocators for tokamak plants [56, 57, 100, 229].

The design of a full-information regulator for over-actuated linear systems is considered in [218], introducing a geometric approach. This method is motivated by the geometric characterization of a feedback regulator and leads to significant developments concerning the effect of input redundancy on the *output invisibility* of the allocation action. The geometric approach is extended in [43, 44] accounting for model uncertainties. Moreover, the full-information regulator problem is addressed again in [90] where the selected approach consists in the parametrization of the solutions to the regulator equations, defining a space over which optimization methods are evaluated.

The notion of output invisibility is further explored in [51, 89], with the goal of optimizing the steady-state plant input without altering the steady-state plant output. The authors consider control systems subject to a sinusoidal reference input, which is also available to the allocation device. The main novelty of these works lies in the introduction of an architecture for the allocation device based on the interconnection of two subsystems: a *steady-state optimizer* and an *annihilator*. The former is responsible for the optimization of the steady-state plant input, while the latter ensures that the allocation action is invisible in the steady-state plant output. Their design is not decoupled but the optimizer can be designed after the annihilator. This architecture is particularly suitable for input-redundant plants with a weak input redundancy property and constitutes the starting point for the work presented in this chapter.

Furthermore, [227] analyzes the impact of plant model uncertainties on the design of a dynamic control allocator, while [3] proposes a functional approximation methodology. Static control allocation is evaluated in [66] against linear quadratic control, examining data requirements and achievable approximations. Alternative approaches include reinforcement learning-based dynamic allocation [117] and static optimal allocation achieved with a model predictive control design [173]. Dynamic control allocation also addresses implementation challenges like actuator saturation, as shown in adaptive control contexts [235].

Some early results regarding control allocation for nonlinear systems are given in [38, 52]. In [52], the notion of weakly input-redundant plants is extended to nonlinear systems which admit a vector relative degree. This extension leads to the design of both static and dynamic input allocators for nonlinear systems that admit a special global normal form. Conversely, [38] proposes a Lyapunov-based approach to achieve optimal input allocation satisfying safety constraints, enforced by barrier functions.

7.5 Control requirements

A dynamic control allocator for the TCV shape controller is to be designed in a way that makes it easy to be integrated within the existing framework, not only in terms of its implementation but also of the modeling techniques on which it is based. For this work, the following formulations of a dynamic control allocation problem have been considered. Their evaluation has led to theoretical results that are illustrated in the following and allowed the author to identify the most suitable approach for the TCV case.

7.5.1 Preliminary definitions

Each dynamic system defined in the following is assumed to be *complete* unless differently specified. Consider a continuous-time, time-invariant dynamic system Σ described by the equations

$$\dot{\chi}(t) = f(\chi(t), r(t)) + g(\chi(t))v(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.1a}$$

$$\eta_p(t) = h_p(\chi(t), v(t)), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.1b}$$

$$\eta_o(t) = h_o(\chi(t)), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.1c}$$

where $\chi(t) \in \mathbb{R}^n$ is the state vector at time t, $r(t) \in \mathbb{R}^{n_r}$ is the *reference* input (intended as, in general, both references to track and disturbances to reject), $v(t) \in \mathbb{R}^m$ is the *auxiliary* control input, and $\eta_p(t) \in \mathbb{R}^{q_p}$, $\eta_o(t) \in \mathbb{R}^{q_o}$ are the output vectors. Let the functions $f : R^n \times \mathbb{R}^{n_r} \to \mathbb{R}^n$, $g_i : \mathbb{R}^n \to \mathbb{R}^n$, $i = 1,\dots,m$, $h_p : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^{q_p}$, and $h_o : \mathbb{R}^n \to \mathbb{R}^{q_o}$ be smooth. Σ is a *two-input two-output* (TITO) model that is convenient to deal with multiple kinds of real-world plants: the ones that are of interest here are those where Σ embeds a dynamic process and a given control system for it. These situations are further detailed in the following together with the peculiarities of each case that has been considered. η_p is an output that measures the overall performance of the system (namely, a *performance output*) and the output η_o follows a desired behavior (thus being termed a *regulated output*) usually by the action of some given component that is internal to Σ.

The reference input r is generated by the *exosystem* E described by the equations

$$\dot{w}(t) = \xi(w(t)), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.2a}$$

$$r(t) = \psi(w(t)), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.2b}$$

where $w(t) \in \mathbb{R}^{n_e}$ is the state vector, $w(0) = w_0$, and the functions $\xi : \mathbb{R}^{n_e} \to \mathbb{R}^{n_e}$ and $\psi : \mathbb{R}^{n_e} \to \mathbb{R}^{n_r}$ are smooth. The properties of E are characterized by the following "blanket assumptions", which implicitly hold from this point onward.

Assumption 7.1. *The origin $w = 0$ of $\mathbb{R}^{n_e}$ is a stable equilibrium point associated with ξ.* ◦

Assumption 7.2. *There exists a compact and invariant set $\mathscr{W} \subset \mathbb{R}^{n_e}$ such that $w_0 \in \mathscr{W}^\circ$ with $\mathscr{W}^\circ \subseteq \mathrm{int}(\mathscr{W})$, $\mathscr{W}^\circ$ neighborhood of the origin of $\mathbb{R}^{n_e}$. Moreover, any $w_0 \in \mathscr{W}^\circ$ is such that the flow $\Phi_t^\xi(w_0)$ is defined for all $t \in \mathbb{R}$ and, for any neighborhood $\mathscr{U}^\circ$ of w_0 and for any real number $\tau > 0$, there exists a time $t_1 > \tau$ such that $\Phi_{t_1}^\xi(w_0) \in \mathscr{U}^\circ$, and a time $t_2 < -\tau$ such that $\Phi_{t_2}^\xi(w_0) \in \mathscr{U}^\circ$.* ◦

In other words, Assumption 7.2 states that given $w_0 \in \mathscr{W}^\circ$, the trajectory $w(t)$ that originates in w_0 passes arbitrarily close to w_0 for $|t| \to +\infty$. This defines the point w_0 as *Poisson stable* (see [108]) and, since this holds for any $w_0 \in \mathscr{W}^\circ$, the system E is said to be Poisson stable. This property implies that no trajectory of (7.2) can decay to zero as time tends to infinity and that the reference input r is periodic of period $T \in \mathbb{R}_{>0}$. The interconnection of the system Σ with the exosystem E is depicted in Figure 7.1.

The model described by the equations (7.1)–(7.2) is further characterized by the following assumptions. The first one is instrumental to enable the formulation of the control allocation problems and the design of dynamic allocators.

Assumption 7.3. $m > q_o$. ◦

Assumption 7.3 states that the system Σ is *over-actuated* (namely, *input-redundant*) with respect to the auxiliary input v and the regulated output η_o.

Definition 7.4 (Degree of redundancy)**.** *The* degree of redundancy *of the system* (7.1) *with respect to*

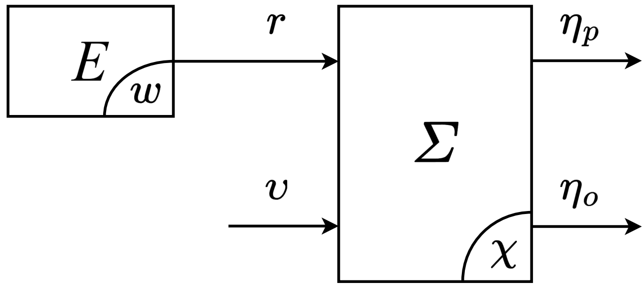

Figure 7.1: Block diagram of the system (7.1) interconnected to the exosystem (7.2).

the auxiliary input v *and the regulated output* η_o *is*

$$m_a = m - q_o. \tag{7.3}$$

∘

The next assumption characterizes Σ as a preexisting dynamic system whose auxiliary input can be exploited to improve the performance indicator.

Assumption 7.5. *The system* Σ *is input-to-state stable (ISS) with respect to the input* (r, v)*. Moreover, the equilibrium* $\chi = 0$ *of the system* $\dot{\chi} = f(\chi, 0)$ *is exponentially stable.* ∘

Finally, the following assumption clarifies the role of the exosystem E in the considered setting.

Assumption 7.6. *There exists a mapping* $\pi : \mathcal{W}^\circ \to \mathbb{R}^n$ *that, for any* $w \in \mathcal{W}^\circ$*, satisfies*

$$\frac{\partial \pi}{\partial w}\xi(w) = f(\pi(w), \psi(w)). \tag{7.4}$$

∘

It is worth to point out that Assumption 7.6 provides, for any $w_0 \in \mathcal{W}^\circ$, a *steady-state response* to the system (7.1)–(7.2) (see [108, Prop. 8.1.1]), denoted

$$\tilde{\chi}(t) = \chi(t, \pi(w_0), \psi(\Phi_t^\xi(w_0))), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.5}$$

The performance and regulated outputs also admit steady-state responses denoted $\tilde{\eta}_p$ and $\tilde{\eta}_o$, respectively.

Let $\tau_0 \in \mathbb{R}_{\geq 0}$. The performance criterion with respect to which the behavior of Σ may be optimized is expressed by the following *Lagrange functional* of the form

$$J(\upsilon) = \int_{\tau_0}^{\tau_0+T} \ell(\tilde{\eta}_p(\tau))\, d\tau, \tag{7.6}$$

where the function $\ell : \mathbb{R}^{q_p} \to \mathbb{R}_{\geq 0}$ is smooth, convex, and radially unbounded.

The two dynamic control allocation problems that are of interest for this work are formalized as follows. The allocation action is implemented by means of the auxiliary input υ of the system Σ and the nature of the following two problems originates from two different ways in which the effect of the auxiliary input on the regulated output can be evaluated.

Problem 7.7 (Dynamic allocation with steady-state output invisibility)**.** *Consider the system* (7.1) *and the exosystem* (7.2). *Suppose that Assumptions 7.3, 7.5 and 7.6 hold. Find, if any, an input allocation device (briefly, an* allocator*) having input ϖ and output ς such that, when interconnected to Σ and E according to*

$$\varpi = r, \qquad\qquad \varsigma = \upsilon, \tag{7.7a}$$

ensures that the following requirements hold.

(P) Steady-state periodicity: *the allocator admits a bounded steady-state output response, namely $\tilde{\varsigma}$, and it is periodic of period T, where T is given in Assumption 7.6.*

(I) Steady-state output invisibility: *the steady-state regulated output responses of the system* (7.1)*, both in presence and absence of the allocation action, namely $\tilde{\eta}_o^a$ and $\tilde{\eta}_o$, respectively, are such that*

$$\tilde{\eta}_o^a(t) = \tilde{\eta}_o(t)\, t \in \mathbb{R}_{\geq 0}. \tag{7.8}$$

(O) Steady-state optimality: *the interconnection of the allocator to* (7.1)–(7.2) *brings the cost functional* (7.6) *to its minimum, if any.*

∘

Problem 7.8 (Dynamic allocation with full output invisibility)**.** *Consider the system* (7.1) *and the exosystem* (7.2). *Suppose that Assumptions 7.3, 7.5 and 7.6 hold. Find, if any, an input allocation device (briefly, an* allocator*) having input ϖ and output ς such that, when interconnected to Σ and E according to*

$$\varpi = r, \qquad\qquad \varsigma = \upsilon, \tag{7.9a}$$

ensures that the following requirements hold.

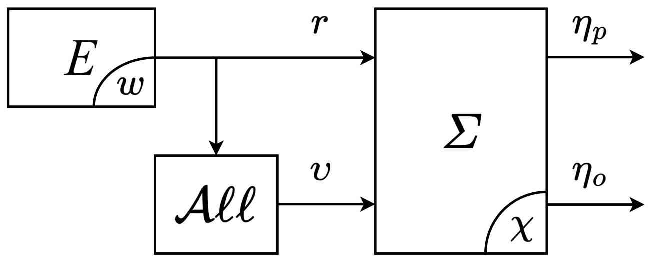

Figure 7.2: Block diagram of the system (7.1) interconnected to the exosystem (7.2) and to the input allocation device, enclosed in the $\mathscr{A}\ell\ell$ block.

(P) Steady-state periodicity: *the allocator admits a bounded steady-state output response, namely* $\tilde{\zeta}$*, and it is periodic of period* T*, where* T *is given in Assumption 7.6.*

(I) Full output invisibility: *the regulated output responses of the system given by* (7.1)*, both in presence and absence of the allocation action, namely* η_o^a *and* η_o*, respectively, are such that*

$$\eta_o^a(t) = \eta_o(t) \tag{7.10}$$

for all $t \in \mathbb{R}_{\geq 0}$.

(O) Steady-state optimality: *the interconnection of the allocator to* (7.1)–(7.2) *brings the cost functional* (7.6) *to its minimum, if any.*

∘

It is finally worth to point out that, in both problems, the optimization process is allowed to exhibit a transient behavior: what distinguishes the two situations is whether the effect of this transient behavior on the regulated output can be visible or not. The desired configuration is depicted in Figure 7.2.

7.6 Design of dynamic control allocators

The results that follow in the rest of this chapter provide solutions to Problems 7.7 and 7.8 based on different configurations of (7.1)–(7.2). The case of Problem 7.7 is examined considering both a linear and a nonlinear model for the system (7.1). On the other hand, Problem 7.8 is studied from the only point of view of linear systems but in multiple scenarios: when the model of Σ is fully known, when it is affected by parametric uncertainties, and when it includes sampled data devices.

7.6.1 Dynamic allocation with steady-state output invisibility

The case identified by Problem 7.7 where the systems (7.1)–(7.2) are linear has been studied in the literature and multiple solutions have been proposed, as reported in Section 7.4. However, none of these solutions retain their applicability when the model exhibits less convenient features, which is one of the reasons why the dynamic control allocation problem for nonlinear systems is still an open issue. This section presents the work that has been done in this direction, starting from the following motivating idea. Since, at its core, the dynamic control allocation problem is a constrained dynamic optimization problem, it is beneficial to reformulate it as such. This allows one to use approaches and tools belonging to the theory of optimal control.

This approach has been tested initially on a linear feedback system of the form of Σ, interconnected to a linear exosystem E [91]. Consider a continuous-time, linear time-invariant dynamic plant Θ of the form

$$\dot{x}(t) = A_p x(t) + B_p u(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.11a}$$

$$y(t) = C_p x(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.11b}$$

where $x(t) \in \mathbb{R}^{n_p}$ is the plant state vector at time t, $u(t) \in \mathbb{R}^m$ is the control input, $y(t) \in \mathbb{R}^{q_o}$ is the plant output vector, and $A_p \in \mathbb{R}^{n_p \times n_p}$, $B_p \in \mathbb{R}^{n_p \times m}$, and $C_p \in \mathbb{R}^{q_o \times n_p}$ are constant matrices. In the considered setting, system (7.11) is subject to a continuous-time, linear time-invariant feedback controller Γ given by

$$\dot{\varphi}(t) = A_c \varphi(t) + B_c y(t) + P_c r(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.12a}$$

$$\rho(t) = C_c \varphi(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.12b}$$

where $\varphi(t) \in \mathbb{R}^{n_c}$ is the controller state vector at time t and $\rho(t) \in \mathbb{R}^m$ is the controller output vector, and $A_c \in \mathbb{R}^{n_c \times n_c}$, $B_c \in \mathbb{R}^{n_c \times q_o}$, $P_c \in \mathbb{R}^{n_c \times n_r}$, and $C_c \in \mathbb{R}^{m \times n_c}$ are constant matrices. The interconnection of the plant Θ and the control system Γ forms the feedback loop Σ according to $u = \rho + v$, with the dimension of the full state vector being $n = n_p + n_c$; to keep the notation as concise as possible, the plant output y is considered as the regulated output of the system Σ, *i.e.*, $\eta_o = y$, unless specified otherwise. The following assumption characterizes the properties of the feedback interconnection.

Assumption 7.9. *The pairs* (A_p, B_p), (A_c, B_c) *are reachable, and the pairs* (C_p, A_p), (C_c, A_c) *are detectable. Moreover, the closed-loop feedback interconnection of* (7.11) *and* (7.12) *does not induce pole-zero cancellations and it is asymptotically stable.* ∘

Let s be a variable taking values in $\mathbb{C}$, and $W_{yu}(s)$ be the transfer matrix of system (7.11) from u to y, computed from the state-space description of Θ. Let $W_{yv}(s)$ be the transfer matrix of (7.1) from v to y; moreover, let $(A_\Sigma, B_\Sigma, C_\Sigma, 0)$ be a realization of $W_{yv}(s)$. The Smith-McMillan degrees of $W_{yu}(s)$ and $W_{yv}(s)$ coincide with those of their respective state-space description thanks to Assumption 7.9.

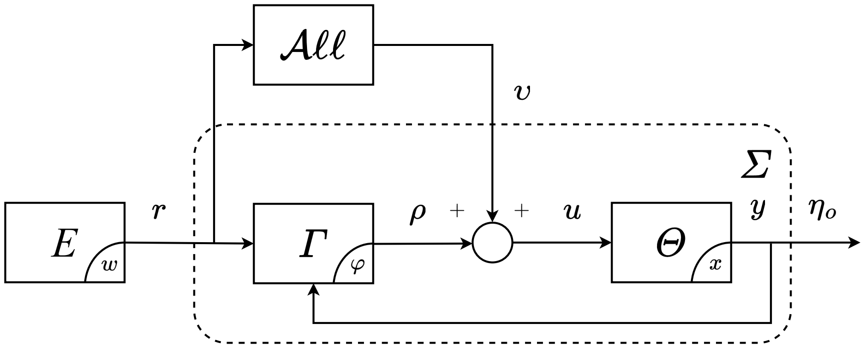

Figure 7.3: Block diagram of the feedback control system (7.11)–(7.12) interconnected to the exosystem (7.13); the input allocation device is enclosed in the $\mathscr{A}\ell\ell$ block.

The reference input in this setting is defined as the state of the exosystem E, which is a continuous-time linear time-invariant system of the form

$$\dot{w}(t) = Sw(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.13a}$$

$$r(t) = w(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.13b}$$

where $w(t) \in \mathbb{R}^{n_e}$ is the state vector at time t and $S \in \mathbb{R}^{n_e \times n_e}$ is a block-diagonal matrix defined as

$$S = \operatorname{diag}(S_0, \dots, S_h), \tag{7.14a}$$

$$S_0 = 0, S_i = \begin{bmatrix} 0 & \omega_i \\ -\omega_i & 0 \end{bmatrix}, \qquad i = 1, \dots, h, \tag{7.14b}$$

with $w(0) = w_0 \in \mathbb{R}^{n_e}$. The angular frequencies $\{\omega_1, \dots, \omega_h\}$ in (7.14) are positive integer multiples of $\frac{2\pi}{T}$, *i.e.*, such that $\omega_i = k_i \frac{2\pi}{T}$, $k_i \in \mathbb{N}$, $i = 1, \dots, h$.

The feedback controller (7.12), together with exosystem (7.14), ensure that Assumptions 7.5 and 7.6 hold. The complete block diagram of the closed loop is shown in Figure 7.3.

Both the transfer matrices $W_{yu}(s)$ and $W_{yv}(s)$ admit left coprime polynomial factorizations (see [39, §7.8]) of the form

$$W_{yu}(s) = D_p^{-1}(s) N_p(s), \tag{7.15a}$$

$$W_{yv}(s) = D_\Sigma^{-1}(s) N_\Sigma(s), \tag{7.15b}$$

where $D_p(s)$, $D_\Sigma(s)$, $N_p(s)$ and $N_\Sigma(s)$ are polynomial matrices. The following assumption is introduced to simplify the discussion.

Assumption 7.10. *The left coprime factorization* (7.15) *is such that the following holds:*

$$\ker(N_\Sigma(0)) = \ker(N_p(0)), \tag{7.16a}$$

$$\ker(N_\Sigma(j\omega_i)) = \ker(N_p(j\omega_i)), \qquad i = 1,\dots,h. \tag{7.16b}$$

∘

Although the factorization (7.15) is not unique, it can be shown that Assumption 7.10 holds for one of them if and only if it holds for each one of them. The performance output in this case is defined as the plant input, *i.e.*, $\eta_p = u$. The effort demanded to the actuators is thus evaluated by the instantaneous cost function $\ell(\tilde{\eta}_p(t)) = \|\tilde{u}(t)\|_2^2$. Setting $\tau_0 = 0$ in eq. (7.6) yields the following formulation of the cost functional J:

$$J(\tilde{u}) = \frac{1}{2}\int_0^T \|\tilde{u}(\tau)\|_2^2\, d\tau. \tag{7.17}$$

Requirements (P), (I), and (O) of Problem 7.7 reveal the connection between *dynamic control allocation* on one hand and *constrained optimal control* on the other: the (P) statement requires the asymptotic behavior to be T-periodic, the (O) statement introduces the cost to optimize, and finally the (I) statement describes the constraint to be satisfied to ensure that the optimal steady-state solution does not affect the original steady-state output performance induced by the controller (7.12). To substantiate this evidence, the definition of a few more items is required. Let $(A_\varphi, B_\varphi, C_\varphi, 0)$ be a minimal state-space realization of the feedback loop Σ having w as input and ρ as output. Let $\Pi \in \mathbb{R}^{n\times n_e}$ be the solution of the Sylvester equation

$$\Pi S = A_\varphi \Pi + B_\varphi, \tag{7.18a}$$

and define $\bar{M} \in \mathbb{R}^{m\times n_e}$ as

$$\bar{M} = C_\varphi \Pi, \tag{7.18b}$$

which is termed the *pseudo-steady-state output map*. Let $\tilde{\rho}$ denote the steady-state controller output. The following equality holds (see Appendix C, as well as [17, 215] for a proof):

$$\tilde{\rho}(t) = \bar{M} w(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.19}$$

The finite-horizon optimal control problem corresponding to Problem 7.7 is formulated as follows.

Problem 7.11 (Optimal input allocation for linear dynamics)**.** *Consider the continuous-time, linear time-invariant feedback control system described by* (7.11), (7.12), *and the exosystem* (7.13). *Suppose that Assumptions 7.3, 7.5, 7.6, 7.9 and 7.10 hold, and define* $\bar{M}$ *as in* (7.18). *Determine, if it exists, a function* $v^\star \in \mathscr{L}_2([0,T])$ *that*

(i) minimizes the cost functional

$$J(v) = \frac{1}{2}\int_0^T \left\|\bar{M} w(\tau) + v(\tau)\right\|_2^2\, d\tau; \tag{7.20}$$

(ii) satisfies the integral constraint

$$\lim_{t_0 \to -\infty} \int_{t_0}^{t} C_\Sigma e^{A_\Sigma (t-\tau)} B_\Sigma \hat{v}(\tau)\, d\tau = 0, \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.21}$$

where $\hat{v} : \mathbb{R} \to \mathbb{R}^m$ *denotes the periodic extension of* v, *namely* $\hat{v}(t) = v(\mathrm{mod}(t, T))$, $t \in \mathbb{R}_{\geq 0}$. ∘

It is worth pointing out that statement (ii) of Problem 7.11 constitutes a somewhat novel class of constraints for optimal control problems. In particular, since the left-hand side of (7.21) represents the steady-state output response of the closed loop identified by $(A_\Sigma, B_\Sigma, C_\Sigma, 0)$ with $v = v$, statement (I) of Problem 7.7 is equivalent to the constraint (7.21) of Problem 7.11.

Remark 7.12. *The relation between Problem 7.7 and Problem 7.11 establishes the basis for a deeper understanding of both. Having determined the solution* $v^\star$ *of Problem 7.11, it is possible to satisfy the requirements of Problem 7.7 simply by injecting* $v(t) = v^\star(\mathrm{mod}(t, T))$ *into the closed loop; the linearity and asymptotic stability of* Σ *imply that the resulting closed loop converges to a steady-state satisfying the requirements of Problem 7.7. Note that whereas an allocated closed loop using a generic allocator solving Problem 7.7 generally exhibits transients in both* v *(which is computed asymptotically) and in the overall system response; the proposed approach directly generates the steady-state evolution of* v*, although the remaining signals in the overall system response still exhibit transients before converging to their allocated evolution.* ▲

It is possible to derive a closed-form solution to Problem 7.11, as shown in the following. However, some preliminary technical results are required. For clarity, introduce the notation

$$\zeta(t) = \bar{M} w(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.22}$$

By standard results about Fourier series expansion of functions in $\mathscr{L}_2([0, T])$, it is possible to write

$$\zeta(t) = \zeta_0 + \sum_{k=1}^{+\infty} (a_k \sin(k\omega t) + b_k \cos(k\omega t)), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.23a}$$

$$v^\star(t) = v_0^\star + \sum_{k=1}^{+\infty} (\alpha_k \sin(k\omega t) + \beta_k \cos(k\omega t)), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.23b}$$

where $\zeta_0, v_0^\star, a_k, b_k, \alpha_k, \beta_k \in \mathbb{R}^m$. Moreover, by the properties of (7.2) and (7.13), only a finite number of coefficients a_k, b_k are non-zero in (7.23a); the same property holds for (7.23b) as shown in the following. The following lemma, considering a single frequency component (briefly, an *harmonic*) of (7.23b), characterizes the relation imposed by the constraint (7.21) on its coefficients.

Lemma 7.13. *Let* $\bar{\omega} = 2\pi \bar{k}/T$, *with* $\bar{k} \in \mathbb{Z}_{\geq 0}$, *and*

$$v(t) = \alpha \sin(\bar{\omega} t) + \beta \cos(\bar{\omega} t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.24}$$

where $\alpha, \beta \in \mathbb{R}^m$*; let* $\vartheta = \begin{bmatrix} \alpha' & \beta' \end{bmatrix}'$ *and*

$$\Psi(\bar{\omega}) = \begin{bmatrix} \mathrm{Im}(N_\Sigma(j\bar{\omega})) & \mathrm{Re}(N_\Sigma(j\bar{\omega})) \\ -\mathrm{Re}(N_\Sigma(j\bar{\omega})) & \mathrm{Im}(N_\Sigma(j\bar{\omega})) \end{bmatrix}. \tag{7.25}$$

Then, (7.21) *holds for* v *in* (7.24) *if and only if either one of the following conditions holds:*

$$\Psi(\bar{\omega})\vartheta = 0, \tag{7.26a}$$

$$\vartheta = \Psi^\perp(\bar{\omega})z, \qquad z \in \mathbb{R}^\nu, \tag{7.26b}$$

where $\nu = \dim(\ker(\Psi(\bar{\omega})))$. ∘

Proof. Rewrite

$$v(t) = \alpha\sin(\bar{\omega}t) + \beta\cos(\bar{\omega}t) = \frac{1}{2}\left(e^{j\bar{\omega}t}(\beta - j\alpha) + e^{-j\bar{\omega}t}(\beta + j\alpha)\right). \tag{7.27}$$

By linearity and Assumption 7.9, the steady-state response of Σ to v in the left hand side of (7.21) is given by

$$\tilde{y}(t) = \frac{1}{2}e^{j\bar{\omega}t}(\beta - j\alpha)W_{yv}(j\bar{\omega}) + \frac{1}{2}e^{-j\bar{\omega}t}(\beta + j\alpha)W_{yv}(-j\bar{\omega}). \tag{7.28}$$

Considering that the two addends in the last expression are each the complex conjugate of the other, it follows that $\tilde{y}$ is zero if and only if

$$(\beta - j\alpha)W_{yv}(j\bar{\omega}) = 0, \tag{7.29}$$

as a matter of fact, considering (7.15) and that Assumption 7.9 implies that $D_\Sigma(j\bar{\omega})$ is invertible, (7.29) is equivalent to

$$(\beta - j\alpha)N_\Sigma(j\bar{\omega}) = 0. \tag{7.30}$$

Imposing that both the imaginary part and the real part of $(\beta - j\alpha)N_\Sigma(j\bar{\omega})$ must be zero, (7.26a) follows. It is worth to point out that the case $\bar{k} = 0$, *i.e.*, $\bar{\omega} = 0$, is also included by (7.26a) since in this case $\alpha = 0$, $\beta = v_0^\star$, $\mathrm{Im}(N_\Sigma(0)) = 0$, $\mathrm{Re}(N_\Sigma(0)) = N_\Sigma(0)$, thus (7.26a) is an identity with the condition $N_\Sigma(0)v_0^\star = 0$. □

The property stated by Lemma 7.13 parametrizes the functions $\hat{v}^\star$ satisfying (7.21), thus allowing one to translate the problem of optimizing (7.20) under the constraint (7.21) into an unconstrained optimization problem on a properly-defined, smaller domain. The next lemma shows how the coefficients of ζ and $v^\star$ are related when the cost (7.20) is also minimized.

Lemma 7.14. *Let* $\bar{\omega} = 2\pi\bar{k}/T$*, with* $\bar{k} \in \mathbb{Z}_{\geq 0}$*, and*

$$\zeta(t) = a\sin(\bar{\omega}t) + b\cos(\bar{\omega}t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.31a}$$

$$v(t) = \alpha\sin(\bar{\omega}t) + \beta\cos(\bar{\omega}t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.31b}$$

where $a, b, \alpha, \beta \in \mathbb{R}^m$, $\theta = \begin{bmatrix} a' & b' \end{bmatrix}'$, $\vartheta = \begin{bmatrix} \alpha' & \beta' \end{bmatrix}'$.

Then, v minimizes (7.20) *under the constraint* (7.21) *if and only if $\vartheta = \vartheta^\star(\bar\omega)$, with*

$$\vartheta^\star(\bar\omega) = -\Psi^\perp(\bar\omega)(\Psi^\perp(\bar\omega))^\dagger\theta, \tag{7.32}$$

and $\vartheta^\star(\bar\omega) = \begin{bmatrix}\alpha^\star(\bar\omega)' & \beta^\star(\bar\omega)'\end{bmatrix}'$. ∘

Proof. The computation of the cost (7.20) with (7.31) reduces to

$$\begin{aligned}\frac{1}{2}\int_0^T \|\zeta(\tau)+v(\tau)\|_2^2\,d\tau \\ &= \frac{1}{2}\int_0^T \left\|(a+\alpha)\sin(\bar\omega\tau)+(b+\beta)\cos(\bar\omega\tau)\right\|_2^2\,d\tau \\ &= \frac{1}{4}\left\|\begin{bmatrix}(a+\alpha)\\(b+\beta)\end{bmatrix}\right\|_2^2,\end{aligned} \tag{7.33}$$

where the integral over a period of the mixed term $\sin(\bar\omega\tau)\cos(\bar\omega\tau)$ is zero. From Lemma 7.13, the vector ϑ must be of the form (7.26b) to satisfy the constraint (7.21), namely $\vartheta = \Psi^\perp(\bar\omega)z$ with $z \in \mathbb{R}^\nu$. Then, the coefficients $\vartheta^\star(\bar\omega) = \begin{bmatrix}\alpha^\star(\bar\omega)' & \beta^\star(\bar\omega)'\end{bmatrix}'$ are obtained by finding $z^\star \in \mathbb{R}^\nu$ such that

$$z^\star = \arg\min_{z\in\mathbb{R}^\nu} \bar J(z), \tag{7.34}$$

where

$$\bar J(z) = \frac{1}{2}\left\|\theta + \Psi^\perp(\bar\omega)z\right\|_2^2. \tag{7.35}$$

Note that (7.34) is an unconstrained, finite-dimensional optimization problem completely equivalent to Problem 7.11 in the setting defined by Lemma 7.14. Thus, the solution may be computed by solving the equation

$$\frac{\partial \bar J(z)}{\partial z} = \left(\theta + \Psi^\perp(\bar\omega)z\right)'\Psi^\perp(\bar\omega) = 0, \tag{7.36}$$

yielding

$$\left(\Psi^\perp(\bar\omega)\right)'\theta + \left(\Psi^\perp(\bar\omega)\right)'\Psi^\perp(\bar\omega)z = 0, \tag{7.37}$$

$$z^\star = -\left(\Psi^\perp(\bar\omega)\right)^\dagger\theta, \tag{7.38}$$

and then (7.32) is given by (7.26b). Finally, this solution yields the unique global minimum for (7.34) and consequently the solution for Problem 7.11 by the strict convexity of $\bar J$:

$$\frac{\partial^2 \bar J(z)}{\partial z^2} = (\Psi^\perp(\bar\omega))'\Psi^\perp(\bar\omega) > 0 \tag{7.39}$$

holds by construction, since $\Psi^\perp(\bar\omega)$ is full column rank. □

Remark 7.15. *Consider* (7.31a) *and recall* (7.22), *namely the fact that $\zeta(t) = \bar M w(t)$ with $\bar M$ given by*

(7.18). *The two expressions are reconciled noting that if*

$$w(t) = a_w \sin(\bar{\omega} t) + b_w \cos(\bar{\omega} t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.40}$$

with $a_w, b_w \in \mathbb{R}^{n_e}$, *then* $\theta = \begin{bmatrix} a' & b' \end{bmatrix}' = \bar{M} \begin{bmatrix} a'_w & b'_w \end{bmatrix}'$ *holds.* ▲

A straightforward corollary of the previous result follows from (7.32). It tells, in essence, that if ζ contains a single harmonic with zero amplitude, then the optimal v with the same single harmonic has zero amplitude as well.

Corollary 7.16. *If* v, ζ *are given by* (7.31) *(with* v *minimizing* (7.20) *under constraint* (7.21) *and* $a = b = 0$ *in* (7.31a)*) then* $\alpha = \beta = 0$ *in* (7.31b). ∘

The following result exploits the previous lemmas and corollary to describe the optimal solution in the general case where ζ and v are given by (7.23).

Lemma 7.17. *Let* (7.22), (7.23) *hold. Assume that* $v^\star$ *is the minimizer of* (7.20) *under the constraint* (7.21) *and the excitation* ζ. *Then,* $v^\star$ *contains all and only the frequency components present, with nonzero coefficients, in* ζ, i.e.

$$\zeta(t) = \zeta_0 + \sum_{k=1}^{h} a_k \sin(\omega_k t) + b_k \cos(\omega_k t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.41a}$$

$$v^\star(t) = v_0^\star + \sum_{k=1}^{h} \alpha_k \sin(\omega_k t) + \beta_k \cos(\omega_k t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.41b}$$

and the coefficients $\theta_k = \begin{bmatrix} a'_k & b'_k \end{bmatrix}'$, $\vartheta_k = \begin{bmatrix} \alpha'_k & \beta'_k \end{bmatrix}'$ *are related by* (7.32) *to* $\bar{\omega} = k\omega$. ∘

Proof. The proof simply consists of showing that the minimization of the cost in the case of several frequency components reduces to the separate minimization of each frequency component so that the previous results immediately apply. To avoid cumbersome notation, let

$$A_k = a_k + \alpha_k, \qquad B_k = b_k + \beta_k, \qquad k = 1, 2, \ldots, \tag{7.42}$$

which allow the instantaneous cost in (7.20) to be written as

$$\begin{aligned} &\frac{1}{2}\|\zeta(t) + v(t)\|_2^2 = \frac{1}{2}\left\|\gamma_0 + \delta(t) + \epsilon(t)\right\|_2^2 \\ &= \frac{1}{2}(\gamma'_0\gamma_0 + \delta(t)'\delta(t) + \epsilon(t)'\epsilon(t)) + \gamma'_0\delta(t) + \delta(t)'\epsilon(t) + \gamma'_0\epsilon(t), \qquad t \in \mathbb{R}_{\geq 0}, \end{aligned} \tag{7.43}$$

and the vector γ_0 and the functions δ and ϵ given by

$$\gamma_0 = \zeta_0 + v_0^\star, \tag{7.44a}$$

$$\delta(t) = \sum_{k=1}^{+\infty} A_k \sin(k\omega t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.44b}$$

$$\epsilon(t) = \sum_{k=1}^{+\infty} B_k \cos(k\omega t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.44c}$$

The functional to be minimized is

$$\begin{aligned}
&\int_0^T \|\gamma_0\|_2^2 + \|\delta(t)\|_2^2 + \|\epsilon(t)\|_2^2 \, dt \\
&\quad = T\,\|\gamma_0\|_2^2 + \sum_{k=1}^{+\infty} \|A_k\|_2^2 \int_0^T \sin^2(k\omega t)\, dt + \sum_{k=1}^{+\infty} \|B_k\|_2^2 \int_0^T \cos^2(k\omega t)\, dt \\
&\quad = T(\zeta_0 + v_0^\star)'(\zeta_0 + v_0^\star) + \frac{\pi}{\omega} \sum_{k=1}^{+\infty} ((a_k + \alpha_k)'(a_k + \alpha_k) + (b_k + \beta_k)'(b_k + \beta_k)),
\end{aligned} \tag{7.45}$$

which is obtained by considering the expansion of (7.43), noting that mixed terms contain products involving sine and cosine functions that, when integrated over a period, yield zero, and recalling (7.42). It is clear that the overall cost is simply the sum of the costs of the single frequency components; these costs are minimized by the coefficient determined in the previous Lemmas 7.13 and 7.14. □

The following result is thus obtained collecting the ones provided by the previous Lemmas 7.13 and 7.14.

Theorem 7.18. *Consider the feedback control system* (7.11)–(7.13) *and suppose that Assumptions 7.3, 7.5, 7.6, 7.9 and 7.10 hold. Then, there exists a unique solution $v^\star$ to Problem 7.11, of the form*

$$v^\star(t) = v_0^\star + \sum_{i=1}^{h} \alpha_i^\star \sin(\omega_i t) + \beta_i^\star \cos(\omega_i t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.46}$$

with

$$v_0^\star = -N_\Sigma^\perp(0)(N_\Sigma^\perp(0))^\dagger \zeta_0, \tag{7.47a}$$

$$\begin{bmatrix} \alpha_i^\star \\ \beta_i^\star \end{bmatrix} = -\Psi^\perp(\omega_i)(\Psi^\perp(\omega_i))^\dagger \begin{bmatrix} a_i \\ b_i \end{bmatrix}, \qquad i = 1, \dots, h. \tag{7.47b}$$

∘

Example 7.19. *Consider the following numerical model for the plant* (7.11)*:*

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -2 & 0 & 0 \\ 0 & 0 & 0 & 3 \\ 0 & -4 & 1 & 0 \end{bmatrix}, \qquad B = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & 1 \\ -1 & -1 & 0 \\ 0 & -1 & 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix}. \tag{7.48}$$

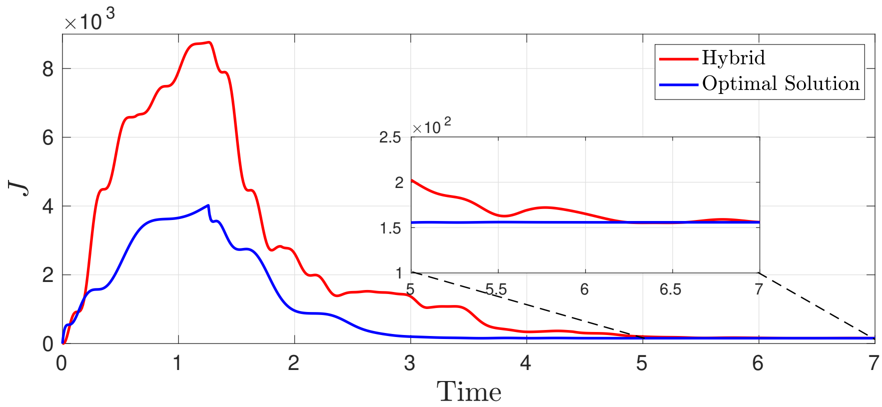

Figure 7.4: Cost in the presence of input allocation by means of a hybrid allocator (red), and of the periodic extension of the optimal solution to Problem 7.11 (blue). The last value is the relevant one.

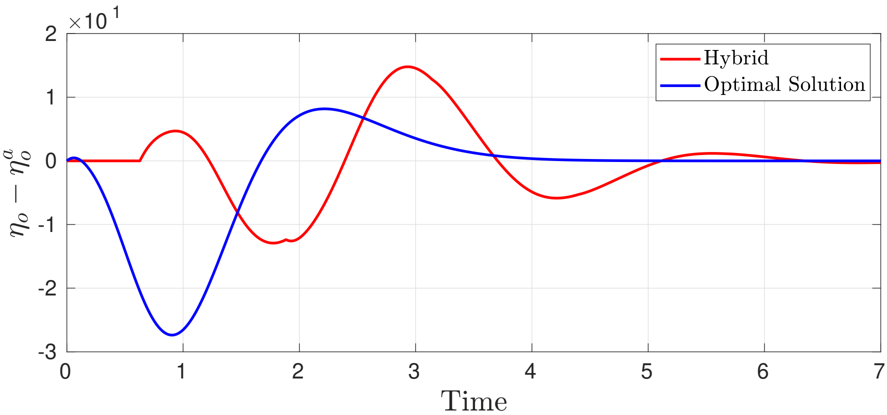

Figure 7.5: Difference $\eta_o - \eta_o^a$ between the regulated outputs of the closed loop in absence and presence of the allocator, respectively, for the hybrid allocator (red) and for the periodic extension of the optimal solution to Problem 7.11 (blue).

The matrices describing the stabilizing controller (7.12) *have been obtained by an LQG design with unit weights, restricting the design to the first two inputs of the plant* (7.48) *only. This choice has been made to better highlight the optimization action of the allocation device, which is instead allowed to modulate the control action among all the three inputs of* (7.48). *The exosystem is defined as in* (7.13) *for the set of angular frequencies given by* $\{\omega_1 = 5,\ \omega_2 = 10,\ \omega_3 = 15\}$, *whereas its initial condition has been chosen equal to* $w_0 = \begin{bmatrix} 2 & 0 & 4 & 0 & 6 & 0 \end{bmatrix}'$. *Finally, it is assumed that the initial conditions for both the plant and the controller are equal to zero, and all simulations are carried out by setting the final time* $t_f = 7$ s.

The purpose of the proposed simulations is to compare the performance of a state-of-the-art hybrid allocator, designed as in [89], against the one that is implemented by applying the results presented above. The hybrid allocator is based on a gradient descent algorithm with the gradient decay rate γ *selected as* $\gamma = 2$. *The implementation of the proposed allocator, based on the previous results, is based on the approach described in Remark 7.12, namely applying a periodic extension of the optimal solution*

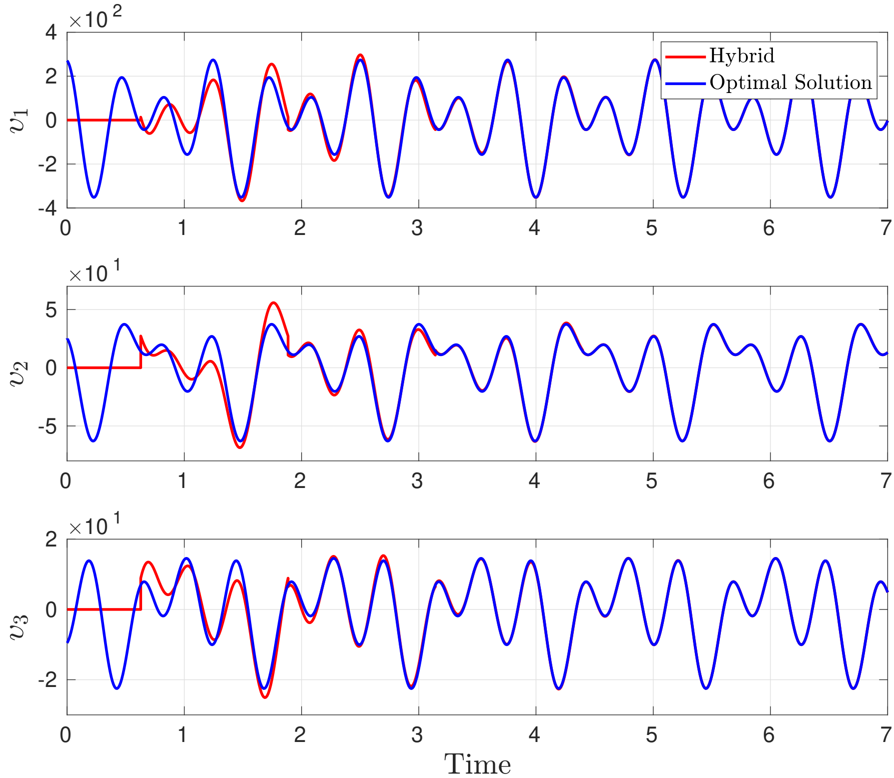

Figure 7.6: Outputs of a hybrid allocator (red) versus the components of the periodic extension of the optimal solution to Problem 7.11 (blue).

$v^\star$ to Problem 7.11. Figure 7.4 reports the time histories of the costs for the considered cases. After a transient motion, which is not relevant at all given that the cost has to be evaluated at steady-state, both solutions converge to the same value of the steady-state cost, which is the relevant one and is equal to $J(v^\star) = 155.836$. Even if, again, the transients in this plot are not relevant, it is interesting to note that the proposed solution converges much faster to the steady-state value, and also with a less intense transient, with respect to the state of the art. To evaluate the achieved output invisibility, consider the difference $\eta_o - \eta_o^a$ between the regulated outputs of the closed loop in absence and presence of the allocator, respectively. The time histories of these differences for the proposed approach and the state of the art are depicted in Figure 7.5: the convergence to zero of both signals proves that both solutions, including the proposed one, meet the steady-state output invisibility requirement. The outputs of the state-of-the-art hybrid allocator are shown in Figure 7.6 against the components of $v^\star(t)$, showing that both signals become coincident after a transient phase.

This study provides a first step towards the application of optimal control theory results within dynamic control allocation settings. The proposed approach allows the designer to build a control allocation device that acts, essentially, as a periodic function generator whose model is entirely determined in terms of its frequency components by Theorem 7.18. Conversely, the previous state-of-the-art solution is described by more complex hybrid dynamics. ■

The nonlinear case is now addressed. Consider the dynamic system (7.1), subject to the reference input produced by the exosystem (7.2). Suppose that Assumptions 7.3, 7.5 and 7.6 hold and consider the steady-state cost functional (7.6), which evaluates the steady-state performance output of the system (7.1) subject to the allocation action of the auxiliary input v. In a similar way to the previous linear case, let $\ell(\tilde{\eta}_p(t)) = \|\tilde{\eta}_p(t)\|_2^2$ and $\tau_0 = 0$ in (7.6), yielding

$$J(v) = \frac{1}{2}\int_0^T \|\tilde{\eta}_p(\tau)\|_2^2 \, d\tau. \tag{7.49}$$

In this scenario, as in many others reported in the literature (see Section 7.4), the most cumbersome requirement set by Problem 7.7 is the output invisibility statement (I), which, in the previous linear case of Problem 7.11, was still included as a problem constraint. Conversely, the approach hereby proposed for the original formulation of Problem 7.7 involving (7.1)–(7.2) stems from a different reasoning: Problem 7.7 is reformulated as an optimal control problem that approximates the original one by penalizing the steady-state output invisibility requirement in the cost functional [156]. Then, a numerical algorithm is developed to determine candidate solutions to the approximated problem.

To substantiate this evidence few preliminary definitions are in order. Let $\chi_N(t), \chi_A(t) \in \mathbb{R}^n$ denote the state vector of (7.1) at time t in the absence or presence of the allocator, respectively, that is

$$\dot{\chi}_N(t) = f(\chi_N(t), r(t)), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.50a}$$

$$\dot{\chi}_A(t) = f(\chi_A(t), r(t)) + g(\chi_A(t))v(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.50b}$$

and let $\tilde{\chi}_N$, $\tilde{\chi}_A$ denote the corresponding steady-state responses, provided that the latter exists. Define the *output invisibility function* $h_I(\chi_N, \chi_A) : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ in the following way:

$$h_I(\chi_N(t), \chi_A(t)) = \frac{1}{2}\|h_o(\chi_N(t)) - h_o(\chi_A(t))\|_2^2, \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.51}$$

A relaxed formulation of Problem 7.7 is the following constrained optimal control problem.

Problem 7.20 (Optimal input allocation for nonlinear dynamics)**.** *Consider the system* (7.1) *and suppose that Assumptions 7.3, 7.5 and 7.6 hold. Given $w_0 \in \mathscr{W}^\circ$, let $w(0) = w_0$. Let $h_I(\chi_N(t), \chi_A(t))$ be defined as in* (7.51). *Let $v : \mathbb{R} \to \mathbb{R}^m$ and define the cost functionals*

$$J_p(v) = \frac{1}{2}\int_0^T \|h_p(\tilde{\chi}_A(t), v(t))\|_2^2 \, dt, \tag{7.52a}$$

$$J_I(v) = \int_0^T h_I(\tilde{\chi}_N(t), \tilde{\chi}_A(t)) \, dt. \tag{7.52b}$$

Determine, if any, a function $v^\star \in \mathscr{L}_2([0, T])$, periodic of period T, such that the cost functional

$$J(v) = J_p(v) + \alpha J_I(v), \tag{7.53}$$

with a fixed $\alpha \in \mathbb{R}_{>0}$, is brought to its minimum value, satisfying the differential constraints (7.50a)–(7.50b) *with $v = v$.* ∘

Remark 7.21. *The output invisibility requirement,* i.e., *statement (I) of Problem 7.7, represents the main challenge of dynamic input allocation problems in general. In Problem 7.20, by* (7.52b) *and* (7.53)*, this requirement is included as part of the minimization objective, implying that the solutions to Problem 7.20 can only approximate the behavior of those to Problem 7.7. It is reasonable to expect that the quality of the approximation depends on the specific solutions to Problem 7.20 that can be determined, using numerical methods, as* $\alpha \to +\infty$. ▲

Remark 7.22. *Although a generic allocator that solves Problem 7.7 may generally exhibit transients in its output* ς*, the solution to Problem 7.20 consists of a steady-state behavior repeated over the time interval* $[0, T]$*, similar to the previous linear case [91]. However, there may still be transients in the overall system response of* (7.1) *and in the remaining signals as they converge to their steady-state, allocated behavior.* ▲

Let $\lambda_N(t), \lambda_A(t) \in \mathbb{R}^n$ be the costate variables, associated to the nominal (*i.e.*, non-allocated) and allocated state variables, respectively. In the following, time dependencies are omitted to ease the notation. Recalling (7.50)–(7.53), define the Hamiltonian function

$$\begin{aligned} &H(\chi_N(t), \chi_A(t), \lambda_N(t), \lambda_A(t), v(t)) \\ &\qquad = \frac{1}{2}\left\| h_p(\chi_A(t), v(t)) \right\|_2^2 + \alpha h_I(\chi_N(t), \chi_A(t)) + \lambda_N(t)' f(\chi_N(t), r(t)) \\ &\qquad\qquad + \lambda_A(t)'(f(\chi_A(t), r(t)) + g(\chi_A(t)) v(t)), \quad t \in \mathbb{R}_{\geq 0}. \end{aligned} \tag{7.54}$$

Consider (omitting, for the sake of simplicity, some dependencies)

$$H_v = h_p(\chi_A, v)' \frac{\partial h_p}{\partial v} + \lambda_A' g(\chi_A), \tag{7.55}$$

and note that the costate dynamics are given by

$$\begin{aligned} \dot{\lambda}_N(t) = &-\alpha \left.\frac{\partial h_I(\chi_N, \chi_A)'}{\partial \chi_N}\right|_{\chi_N = \chi_N(t), \chi_A = \chi_A(t)} \\ &- \left.\frac{\partial f(\chi_N, r)'}{\partial \chi_N}\right|_{\chi_N = \chi_N(t), r = r(t)} \lambda_N(t), \qquad t \in \mathbb{R}_{\geq 0}, \end{aligned} \tag{7.56a}$$

$$\begin{aligned} \dot{\lambda}_A(t) = &-\frac{\partial h_p(\chi_A(t), v(t))'}{\partial \chi_A} h_p(\chi_A(t), v(t)) \\ &- \alpha \left.\frac{\partial h_I(\chi_N, \chi_A)'}{\partial \chi_A}\right|_{\chi_N = \chi_N(t), \chi_A = \chi_A(t)} - \left.\frac{\partial f(\chi_A, r)'}{\partial \chi_A}\right|_{\chi_A = \chi_A(t), r = r(t)} \lambda_A(t) \\ &- \sum_{i=1}^{m} v_i(t) \left.\frac{\partial g_i(\chi_A)'}{\partial \chi_A}\right|_{\chi_A = \chi_A(t)} \lambda_A(t), \qquad t \in \mathbb{R}_{\geq 0}, \end{aligned} \tag{7.56b}$$

where v_i and g_i denote the i-th entry and column of v and g, respectively. Given the function u with values $u(t) \in \mathbb{R}^m$, the periodic extension of u is denoted by $\hat{u} : \mathbb{R} \to \mathbb{R}^m$, *i.e.*, $\hat{u}(t) = u(\mathrm{mod}(t, T))$. Let $\beta, \gamma \in \mathbb{R}_{>0}$, and χ_N be the solution of (7.50a). Fix a final time $t_f = aT$ with $a \in \mathbb{N}$ large enough to include a "reasonable" number of periods, an arbitrary v_0 with values in $v_0(t) \in \mathbb{R}^m$, fix $(\lambda_N(t_f), \lambda_A(t_f)) = (0, 0)$,

Algorithm 1 Steady-state steepest descent algorithm [156].

Input: γ, T, t_f, $[0, t_f]$, v_0, χ_N, $(\lambda_N(t_f), \lambda_A(t_f))$, $\chi_A(0)$
Output: $\tilde{v}$
1: $\tilde{v}_0 \leftarrow v_0$
2: **for** $k = 1, \ldots$ **do**
3: $\quad \chi_{A,k} \leftarrow$ the solution of (7.50b) with $v = \tilde{v}_{k-1}$, $\chi_{A0} = \chi_A(0)$
4: $\quad (\lambda_{N,k}, \lambda_{A,k}) \leftarrow$ backward integration of (7.56) with
$\qquad (\chi_A, v) = (\chi_{A,k}, \tilde{v}_{k-1})$, $(\lambda_N(t_f), \lambda_A(t_f))$ given
5: $\quad$ **if** $H_v^\delta(\chi_{A,k}, \lambda_{A,k}, \tilde{v}_{k-1}) = 0$ **then return** $\tilde{v} \leftarrow \tilde{v}_{k-1}$
6: $\quad$ **else**
7: $\qquad u_k \leftarrow \tilde{v}_{k-1} - \gamma H_v(\chi_{A,k}, \lambda_{A,k}, \tilde{v}_{k-1})$
8: $\qquad \tilde{v}_k \leftarrow \hat{\tilde{u}}_k$ $\quad \triangleright$ Periodic extension of the steady-state.
9: $\quad$ **end if**
10: **end for**

and define

$$H_v^\delta(\chi_A, \lambda_A, v) = \left(\int_{t_f - T}^{t_f} H_v(\chi_A(t), \lambda_A(t), v(t)) H_v(\chi_A(t), \lambda_A(t), v(t))' \, dt \right)^{1/2}. \tag{7.57}$$

The proposed method is stated in Algorithm 1, and its properties are characterized by the following theorem.

Theorem 7.23. *Consider system* (7.1)*, fix a large enough* $\alpha > 0$*, and suppose that Assumptions 7.3, 7.5 and 7.6 hold. Given* $v_0 : \mathbb{R} \to \mathbb{R}^m$*, and* $t_f = aT$*,* $a \in \mathbb{N}$*, there exists a sequence of updates* $\{\Delta v_k\}_{k=1}^h$*,* $\Delta v_k \in \mathbb{R}^m$*,* $h \in \mathbb{N}$*, such that Algorithm 1 terminates in h steps yielding the sequence* $\{\tilde{v}_k\}_{k=1}^h$ *that converges to an extremal solution of Problem 7.20.* ∘

Proof. Given the standing assumptions and hypotheses, the existence of the sequences $\{\Delta v_k\}_{k=1}^h$ and $\{\tilde{v}_k\}_{k=1}^h$, as well as their convergence properties, are guaranteed by the Steepest Descent Algorithm (see [115, Sec. 6.2] and references therein), properly adapted as in Algorithm 1 to face Problem 7.20. □

Remark 7.24. *In practical scenarios, a numerical stopping criterion evaluating the norm of the gradient at each step is put in place, trading asymptotic convergence for implementation simplicity. One possible example is, for a fixed* $\beta \in \mathbb{R}_{>0}$*, evaluating*

$$H_v^\delta(\chi_{A,k}, \lambda_{A,k}, \tilde{v}_{k-1}) \le \beta, \qquad k \in \mathbb{N}, \tag{7.58}$$

instead of

$$H_v^\delta(\chi_{A,k}, \lambda_{A,k}, \tilde{v}_{k-1}) = 0, \qquad k \in \mathbb{N}. \tag{7.59}$$

Regarding the update step γ*, it is selected in principle as a constant value. Consequently, if the magnitude of the update step is large the algorithm may not converge. Furthermore, the feasibility of Algorithm 1 may also depend on the value of the parameter* α*. Recalling* (7.56b)*, a large value made the backward* *integration numerically unfeasible in the simulations that have been performed to test this method.* ▲

Remark 7.25. *Note that Algorithm 1 is purely methodological: in practical scenarios, numerical integrations over the considered time interval* $[0, t_f]$ *are necessary, adding multiple layers of configurable parameters to the algorithm. First, the choice of the numerical algorithm,* e.g., Runge-Kutta *methods, is crucial to ensure the convergence of the backward integration of* (7.56)*. Moreover, since discretizations are involved, a key aspect to consider is also the interpolation method to obtain all the signals from computed samples. Finally, another parameter of interest concerning the numerical implementation of Algorithm 1 is the sampling interval with respect to which the time interval* $[0, t_f]$ *is discretized. The magnitude of the sampling interval should, in general, be small to enable a feasible approximation of the continuous-time dynamics involved in Problem 7.7 and has direct repercussions on the computational complexity of the integration steps.* ▲

The choice of the update step in Algorithm 1 is a crucial design task. Many heuristic techniques have emerged in the literature to improve the convergence of this method, in particular thanks to its extensive diffusion in finite-dimensional optimization fields such as, *e.g.*, machine learning for the training of neural networks [96].

The ADAM scheme yields the update rule given in Algorithm 2. With an abuse of notation, the variables s and r store at each iteration, respectively, H_v and $H_v \odot H_v$ of the current and of the previous iterations. Note also that the operation $(\cdot)^{1/2}$ on $\hat{r}$, as well as the operation $(\cdot)^{-1}$ on $(\hat{r}^{1/2} + \delta)$, reported on line 14 of Algorithm 2, are both performed component-wise. The values ρ_1 for s, and ρ_2 for r, constitute a measure of the importance of the past history, which is then used for the proper evaluation of the update step. In particular, an appropriate choice of these parameters allows one to discard all the information related to previous iterations, thus improving the update step by relying only on the most recent information. It is worth mentioning that this scheme adjusts the update step dynamically during the iterations, differently from the scheme provided by Algorithm 1. The observations made in Remark 7.25 continue to hold also for Algorithm 2.

Example 7.26. *Consider the following continuous-time exosystem of the form* (7.2)*:*

$$\dot{w}(t) = \begin{bmatrix} 0 & 1 & 0 \\ -\omega^2 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} w(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.60a}$$

$$r(t) = \begin{bmatrix} w_1(t) + w_3(t) \\ w_2(t) \end{bmatrix}, \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.60b}$$

with $\omega = 2\pi f$, $f = 0.2\,\mathrm{Hz}$, *and* $w_0 = \begin{bmatrix} 0 & \omega & 2 \end{bmatrix}'$*. Hence, consider the following continuous-time system*

Algorithm 2 Steady-state ADAM algorithm [156].

Input: γ, T, t_f, $[0, t_f]$, v_0, χ_N, $(\lambda_N(t_f), \lambda_A(t_f))$, $\chi_A(0)$, ρ_1, ρ_2, δ
Output: $\tilde{v}$

$\tilde{v}_0 \leftarrow v_0$
$s_0 \leftarrow 0$
$r_0 \leftarrow 0$
for $k = 1, \ldots$ **do**
    $\chi_{A,k} \leftarrow$ the solution of (7.50b) with $v = \tilde{v}_{k-1}$, $\chi_{A0} = \chi_A(0)$
    $(\lambda_{N,k}, \lambda_{A,k}) \leftarrow$ backward integration of (7.56) with
        $(\chi_A, v) = (\chi_{A,k}, \tilde{v}_{k-1})$, $(\lambda_N(t_f), \lambda_A(t_f))$ given
    if $H_v^\delta(\chi_{A,k}, \lambda_{A,k}, \tilde{v}_{k-1}) = 0$ **then return** $\tilde{v} \leftarrow \tilde{v}_{k-1}$
    else
        $H_v \leftarrow H_v(\chi_{A,k}, \lambda_{A,k}, \tilde{v}_{k-1})$
        $s_k \leftarrow \rho_1 s_{k-1} + (1-\rho_1) H_v$
        $r_k \leftarrow \rho_2 r_{k-1} + (1-\rho_2) H_v \odot H_v$
        $\hat{s} \leftarrow s/(1-\rho_{1,k})$
        $\hat{r} \leftarrow r/(1-\rho_{2,k})$
        $\Delta u_k \leftarrow -\gamma\, \hat{s} \odot ((\hat{r})^{1/2} + \delta)^{-1}$
        $u_k \leftarrow \tilde{v}_{k-1} + \Delta u_k$
        $\tilde{v}_k \leftarrow \hat{\bar{u}}_k$ ▷ Periodic extension of the steady-state.
    end if
end for

of the form (7.1)*:*

$$\dot{\chi}(t) = \begin{bmatrix} 0 \\ \chi_1(t) + \chi_2(t) \\ \chi_1(t) - \chi_2(t) \end{bmatrix} + \begin{bmatrix} e^{\chi_2(t)} & 1 \\ e^{\chi_2(t)} & 1 \\ 0 & 0 \end{bmatrix} (h_c(\chi(t), r(t)) + v(t)), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.61a}$$

$$\eta_p(t) = h_c(\chi(t), r(t)) + v(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.61b}$$

$$\eta_o(t) = (1 + \chi_3(t))^3, \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.61c}$$

where $h_c : \mathbb{R}^3 \times \mathbb{R}^3 \to \mathbb{R}^2$ is a given feedback control law, and $v : \mathbb{R} \to \mathbb{R}^2$ is the auxiliary input, to be determined as an extremal solution to Problem 7.20.

The control objective in this example is that of tracking, with the regulated output η_o, the reference input $r_1(t) = w_1(t) + w_3(t) = \sin(\omega t) + 2$, $t \in \mathbb{R}_{\geq 0}$. To this end, the control law h_c has been designed to first apply a linearizing feedback *to the system* (7.61) *and then to stabilize the resulting linearized dynamics. Subsequently, the tracking objective is achieved by injecting the reference input into the feedback control law. A detailed description of the synthesis procedure is provided in [108, Chap. 4]. To highlight the optimization action operated by the auxiliary input v, in a similar way to the previous linear case, the control law h_c has been designed to operate only on the first input, setting the second input to zero (as described in [108, Chap. 4], this is possible because an inspection of* (7.61) *reveals that the system has relative degree $r = 3$ from the first component of the input, affecting the system via the input map $g_1(\chi) = \begin{bmatrix} e^{\chi_2} & e^{\chi_2} & 0 \end{bmatrix}'$, to the output η_o).*

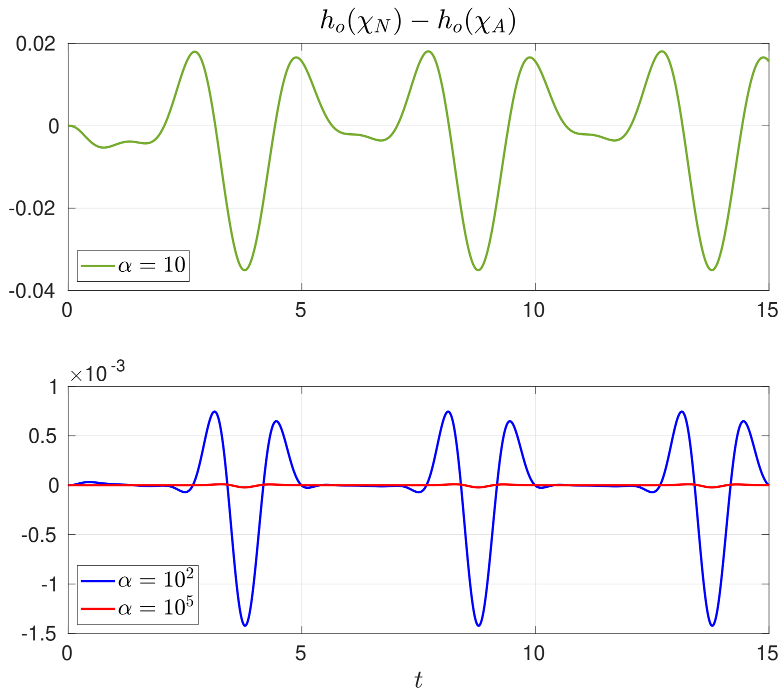

Figure 7.7: Difference between the regulated output with and without input allocation, for different values of the invisibility weight α, during the final run of Algorithm 1.

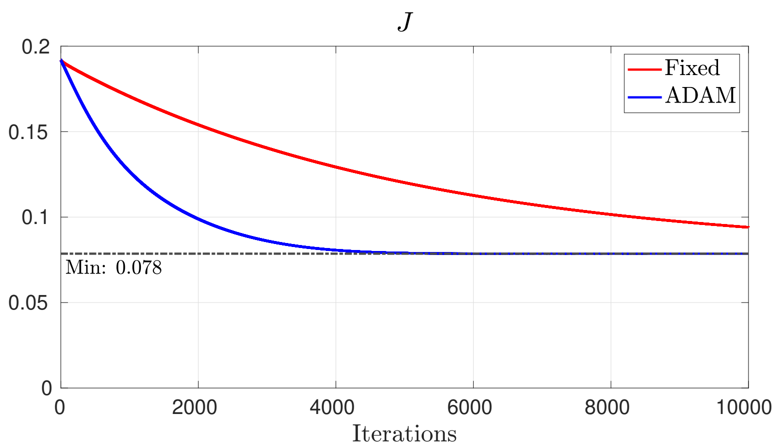

Figure 7.8: Cost during all the iterations performed with Algorithm 1 (red) versus those with Algorithm 2 (blue).

Algorithm 1 has been tested with parameters $(\gamma, t_f) = (10^{-4}, 15)$, $\upsilon_0 = 0$, *and the initial condition of* (7.61) *given by* $\chi(0) = \begin{bmatrix} 0.1872 & -0.0767 & 0.2599 \end{bmatrix}'$. *All algorithms have been implemented numerically in MATLAB following the procedure described in Remark 7.25,* i.e., *by discretizing the time interval* $[0, t_f]$ *with a sampling interval of* 10^{-2}, *performing fixed-step numerical integrations of the state and costate dynamics, and operating with sampled signals.*

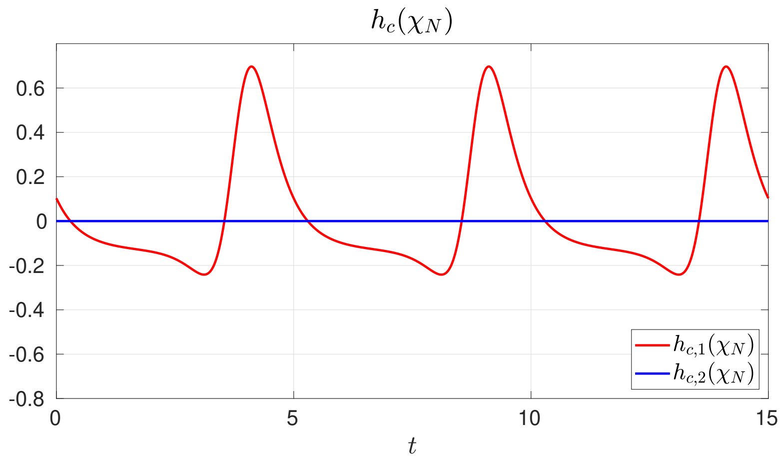

Figure 7.9: Nominal feedback-linearizing system input.

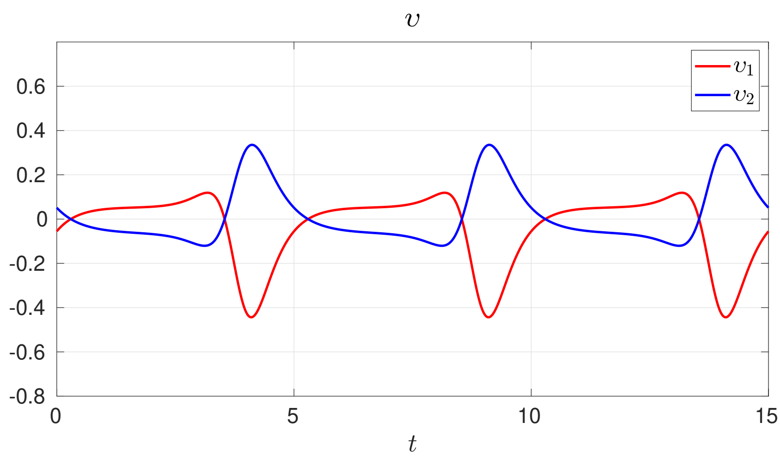

Figure 7.10: Auxiliary input determined by Algorithm 2.

The stopping criterion has been disregarded intentionally, setting a maximum number of iterations equal to 10^4*. The goal of this test is twofold: first, check the performance of the algorithm, and secondly, tune the parameter* α*. Figure 7.7 shows the steady-state tracking errors achieved with different values of* α*: the problem remains computationally feasible and it is possible to set even a large invisibility weight,* i.e., $\alpha = 10^5$*, achieving a small output error.*

A comparison has been made, retaining all the parameters, between the rates of convergence of Algorithm 1 and of Algorithm 2, selecting for the latter the parameters $\rho_1 = 0.9$*,* $\rho_2 = 0.999$ *and* $\delta = 10^{-8}$*. Figure 7.8 shows how Algorithm 2 is faster than Algorithm 1 in this example, converging in about half the maximum number of iterations allowed and well before the convergence of Algorithm 1. Note that it has not been possible to make Algorithm 1 achieve the same speed of convergence of Algorithm 2: increasing the constant update step* γ *leads to a divergence of the algorithm.*

Figure 7.9 shows the nominal system input given by the feedback linearization and stabilization process

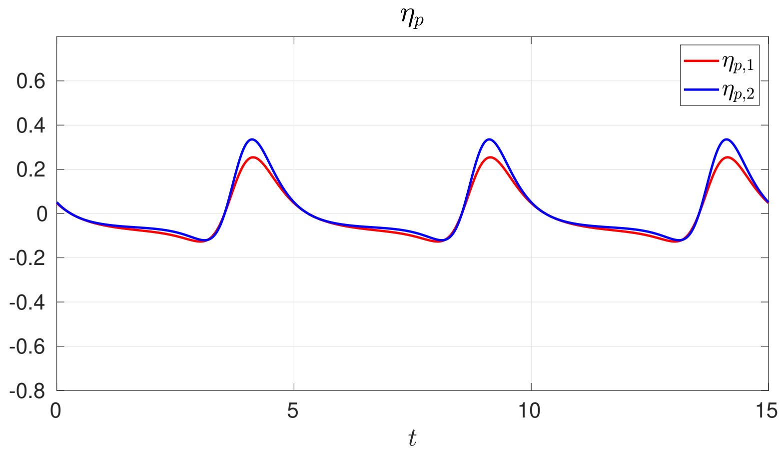

Figure 7.11: Performance output subject to the input allocation determined by Algorithm 2, to be compared with the time histories in Figure 7.9.

and Figure 7.10 depicts the auxiliary input determined by Algorithm 2. Finally, Figure 7.11 illustrates the performance output subject to the allocator action, to be compared with the time histories in Figure 7.9. It is worth pointing out that, even if the feedback-linearizing law uses only one input, the auxiliary one uses both. Thus, in accordance with the definition of the performance output in this example, η_p *exhibits a reduced amplitude leading to a smaller norm, all of which while only a small alteration in the regulated output* η_o *is observed.* ■

7.6.2 Dynamic allocation with full output invisibility

The discussion of Problem 7.7 in Section 7.6.1 has highlighted a common aspect of all dynamic control allocation problems: the invisibility requirement, formulated either as a problem statement or as an optimization constraint, is the hardest one to manage. The steady-state formulation of Problem 7.7 allows the designer to exploit the steady-state behavior of the system, once it is determined, to design the dynamics of the allocator. This is not possible when dealing with Problem 7.8, which requires to account for the full dynamics of (7.1)–(7.2).

The intuition at the heart of the design of dynamic allocators achieving full output invisibility is illustrated in [51]. In essence, given items (I) and (O) of Problem 7.8, the allocator dynamics have to be split into multiple subsystems, each of which deals with one of the two requirements. While [51] deals with a problem similar to Problem 7.8 but in which a steady-state cost *function* is considered, its results provide a way to develop an approach suitable for Problem 7.8. An allocator solving Problem 7.8 is composed by, at least, the interconnection of the following two subsystems:

- an *annihilator*, responsible for the full output invisibility requirement and acting as a filter that masks the effect of the allocation action on the regulated output;
- an *optimizer*, responsible for the optimality requirement and acting as a dynamic controller that

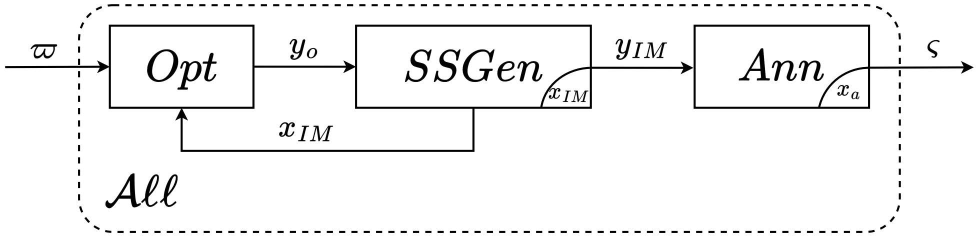

Figure 7.12: Block diagram of a dynamic allocator achieving full output invisibility as requested by Problem 7.8: the optimizer, steady-state generator, and annihilator subsystems are represented by the blocks Opt, $SSGen$, and Ann, respectively.

computes the allocation action to bring the cost functional (7.6) to its minimal value;

- a *steady-state generator*, which contains an internal model of the exosystem (7.2) and is in charge of producing functions of time within a prescribed set.

The interconnection of these three basic components is depicted in Figure 7.12. Note that, when the considered steady-state is constant, a possible implementation of the optimizer is based on the gradient descent method [114] and the steady-state generator is naturally embedded within its integrator, thus reducing the architecture to only two distinct components instead of three.

As highlighted in [51, Remark 4], this construction allows the designer to overcome cumbersome limitations encountered in earlier works that attempt to achieve full output invisibility but fail to do so without relying on quite non-trivial properties of Σ; an example of this for linear systems is found in [247] with the *strong input redundancy* property. Another important aspect of the allocator proposed in [51] is that the design of its three components is carried out in sequence, as summarized in the following steps.

1. The annihilator is designed first, exploiting the dynamic model of the system Σ and its properties. The idea is to design a system whose input-output behavior is *orthogonal* to that of the system Σ, with respect to the regulated output η_o. The results in [51] provide two different ways to design such a component: a polynomial method to compute orthogonal transfer matrices, and a geometric approach based on invariant subspaces.
2. The optimizer is designed next, using a static description of the steady-state behavior of the annihilator that is now available as, *e.g.*, a static gain (see again [17, 215]). The design of this component is aimed at minimizing the steady-state cost in (7.6), so it mainly depends on the formulation of the cost itself.
3. The steady-state generator is designed last, integrating an internal model of the exosystem E.

This work focuses on the design of an allocator solving Problem 7.8 in the case of a linear model of the form of (7.1)–(7.2) and in two different situations: when the model is fully known and when it is affected by parametric uncertainties. In particular, the design of the annihilator is carried out applying a polynomial approach that is derived from the one discussed in [51] but that is then adapted to the uncertain case. Since the two methods share many similarities, especially in the nominal case, an overview of the original method presented in [51] is omitted; the interested reader is referred to the original work for more insights on the problem formulation thereby considered, as well as on the geometric annihilator design approach.

Consider now a continuous-time, linear time-invariant model given by

$$\dot{\chi}(t) = A(\mu)\chi(t) + B(\mu)v(t) + P(\mu)w(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.62a}$$

$$\eta_o(t) = C_1(\mu)\chi(t) + D_{11}(\mu)v(t) + D_{12}(\mu)w(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.62b}$$

$$\eta_p(t) = C_2(\mu)\chi(t) + D_{21}(\mu)v(t) + D_{22}(\mu)w(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.62c}$$

where $\chi(t) \in \mathbb{R}^n$ is the state vector at time t, $v(t) \in \mathbb{R}^m$ is the auxiliary control input, $w(t) \in \mathbb{R}^{n_e}$ is the reference input, and $\eta_o(t) \in \mathbb{R}^{q_o}$ and $\eta_p(t) \in \mathbb{R}^{q_p}$ are the regulated and the performance outputs, respectively. The reference input is again taken as the full state of the exosystem (7.2), *i.e.*, $r = w$ and $n_r = n_e$. The exosystem E is described by the continuous-time, linear time-invariant model

$$\dot{w}(t) = Sw(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.63}$$

where $w(t) \in \mathbb{R}^{n_e}$ is the state vector at time t, with initial condition $w(0) = w_0$. The interconnection is depicted in Figure 7.1.

It is assumed that the matrices in (7.62) depend continuously on an unmeasured vector μ belonging to a set $\mathscr{M}$ that contains the origin: $\mu = 0$ identifies the nominal case. Although in similar cases studied in the literature (see, *e.g.*, [44]), the set $\mathscr{M}$ is assumed to be fully described and compact, these properties are not necessary for the results that follow. To simplify the notation in the following, the dependence of the matrices in (7.62) on μ is omitted unless strictly necessary. The properties of the uncertain model are characterized by the following assumption.

Assumption 7.27. (7.62) *is fully known for* $\mu = 0$ *and the matrix* A *in* (7.62a) *is Hurwitz for any* $\mu \in \mathscr{M}$*. The matrix* $S \in \mathbb{R}^{n_e \times n_e}$ *in* (7.63) *is such that* $\operatorname{spec}(S) \subset \mathbb{C}_0$*; moreover, its eigenvalues are all distinct and possess angular frequencies that are positive integer multiples of a given angular frequency* $\omega_b \in \mathbb{R}_{>0}$. ∘

As an immediate consequence of Assumption 7.27 and of the "blanket" Assumptions 7.1 and 7.2, the exosystem (7.63) is Poisson stable (see Section 7.6.1) and its evolution is periodic of period T defined by the least common multiple of the individual periods.

For $s \in \mathbb{C}$,

$$W_{\eta_o v}(s) = C_1(sI - A)^{-1}B + D_{11}, \tag{7.64a}$$

$$W_{\eta_p v}(s) = C_2(sI - A)^{-1}B + D_{21}, \tag{7.64b}$$

denote the transfer matrices of (7.62) from v to η_o and from v to η_p, respectively. Recalling Assumptions 7.3 and 7.5, it is necessary to further characterize the over-actuation property in the uncertain case introduced by (7.62) to ensure that input allocation remains possible.

Assumption 7.28. *For any $\mu \in \mathscr{M}$, the system* (7.62a), (7.62b) *is such that $m > q_o$,* $\mathrm{rank}(B) = m$, *and*

$$\mathrm{rank}(W_{\eta_o v}(s)) = q_o, \qquad s \in \mathbb{C} \setminus (\mathscr{Z}(W_{\eta_o v}(s)) \cup \mathscr{P}(W_{\eta_o v}(s))). \tag{7.65}$$

∘

Finally, recall the definition of the cost functional (7.6) as well as the properties of the instantaneous cost function ℓ. To provide a concise description of the control objective in this scenario, let

$$\eta_0(t, t_0, \chi_0, w, v) \tag{7.66}$$

be the output response for output η_o at time $t \geq t_0$ of the plant Σ initialized at $\chi(t_0) = \chi_0$ and driven by the inputs w and v. The control problem of interest in this context is formulated as follows.

Problem 7.29 (Dynamic allocation with robust output invisibility)**.** *Consider the systems* (7.62), (7.63) *and the steady-state cost functional* (7.6). *Suppose that Assumptions 7.5, 7.27 and 7.28 hold. Find, if possible, a* dynamic control allocator *with input ϖ and output ς, to be interconnected to Σ and E according to*

$$\varpi = w, \tag{7.67a}$$

$$\varsigma = v, \tag{7.67b}$$

such that the following requirements hold.

(i) Fixed-time robust output invisibility*: there exists a finite time $t_0 \geq 0$ with the property that, for any $\mu \in \mathscr{M}$, the regulated output η_o satisfies*

$$\eta_0(t, t_0, \chi_0, w, 0) = \eta_0(t, t_0, \chi_0, w, v), \qquad t \geq t_0,\ (\chi_0, w_0) \in \mathbb{R}^n \times \mathscr{W}^\circ. \tag{7.68}$$

(ii) Steady-state periodicity*: for any $w_0 \in \mathscr{W}^\circ$, the internal state of the control allocator is bounded for any time and its unique steady-state response is periodic of period T.*

(iii) Optimality*: the Lagrange cost functional* (7.6) *is minimized with respect to the class of inputs in a* prescribed *finite-dimensional function space.*

∘

The construction of a solution to Problem 7.29 for any $\mu \in \mathscr{M}$ begins by identifying the data whose knowledge is necessary for the design of the allocator. This knowledge is obtained in the uncertain case by means of algorithms that are described in the following and are based on input-output measurements. Let the spectrum of the matrix S in (7.63) be

$$\operatorname{spec}(S) = \{0, j\omega_1, -j\omega_1, \ldots, j\omega_{\bar{n}_e}, -j\omega_{\bar{n}_e}\}, \qquad \bar{n}_e = \frac{n_e - 1}{2}. \tag{7.69}$$

Recall that $m_a = m - q_o$ (see Definition 7.4). The design of the allocator requires the following information.

Definition 7.30 (Data for the dynamic control allocator (7.78))**.** *The data necessary for the design of the dynamic control allocator* (7.78) *is divided into three items.*

K1 *The* steady-state map $\bar{M}_{\eta_p,w}$ *from w to η_p,* i.e., *a matrix such that, denoting by $\tilde{\eta}_p^w$ the part of the steady-state response in η_p due to w,*

$$\tilde{\eta}_p^w(t) = \bar{M}_{\eta_p,w}\, w(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.70}$$

K2 *The* moments *of the transfer matrix $W_{\eta_p v}(s)$ of* (7.62) *at $s \in \{0, j\omega_1, , \ldots, j\omega_{\bar{n}_e}\} \subset \operatorname{spec}(S)$, namely*

$$W_{\eta_p v}(0), W_{\eta_p v}(j\omega_1), \ldots, W_{\eta_p v}(j\omega_{\bar{n}_e}). \tag{7.71}$$

K3 *An* annihilator *for the transfer matrix $W_{\eta_o v}(s)$ of* (7.62)*, namely $W_a(s) \in \mathbb{C}^{m \times m_a}$, with poles with negative real part, such that*

$$W_{\eta_o v}(s) W_a(s) = 0, \qquad s \in \mathbb{C} \setminus (\mathscr{P}(W_{\eta_o v}(s)) \cup \mathscr{P}(W_a(s))). \tag{7.72}$$

∘

If the data **K1**, **K2**, and **K3** are available, the proposed dynamic allocator can be constructed as follows. Let (A_a, B_a, C_a) be a minimal realization of $W_a(s)$ in (7.72). Define

$$\Omega = \operatorname{diag}\left\{0, \begin{bmatrix} 0 & \omega_1 \\ -\omega_1 & 0 \end{bmatrix}, \ldots, \begin{bmatrix} 0 & \omega_{\bar{n}_e} \\ -\omega_{\bar{n}_e} & 0 \end{bmatrix}\right\}, \tag{7.73a}$$

$$\Lambda = \left[\, 1 \,\vdots\, 1 \;\; 0 \,\vdots\, \cdots \,\vdots\, 1 \;\; 0 \,\right], \tag{7.73b}$$

yielding the matrices (A_{IM}, C_{IM}):

$$A_{IM} = \Omega \otimes I_{m_a}, \tag{7.74a}$$

$$C_{IM} = \Lambda \otimes I_{m_a}. \tag{7.74b}$$

Furthermore, using the data **K2**, define

$$W_{\eta_p \varpi}(j\omega) = W_{\eta_p v}(j\omega) W_a(j\omega), \qquad \omega \in \{0, \omega_1, \ldots, \omega_{\bar{n}_e}\}, \tag{7.75}$$

and build the matrix $\bar{N}_\mu$ as

$$\bar{N}_\mu = \left[\, W_{\eta_p \varpi}(0) \,\middle|\, \mathrm{Re}(W_{\eta_p\varpi}(j\omega_1)) \;\; \mathrm{Im}(W_{\eta_p\varpi}(j\omega_1)) \,\middle|\, \cdots \,\middle|\, \mathrm{Re}(W_{\eta_p\varpi}(j\omega_{\bar{n}_e})) \;\; \mathrm{Im}(W_{\eta_p\varpi}(j\omega_{\bar{n}_e})) \,\right]. \tag{7.76}$$

Define the functions $\xi(t)$ and $L(w(t), x_{IM}(t))$ as

$$\xi(\sigma) = \bar{M}_{\eta_p,w} e^{S\sigma} w(\sigma) + \bar{N}_\mu e^{A_{IM}\sigma} x_{IM}(\sigma), \qquad \sigma \in \mathbb{R}_{\geq 0}, \tag{7.77a}$$

$$L(w, x_{IM}) = \left(\int_0^T \nabla \ell(\xi(\sigma))' \bar{N}_\mu e^{A_{IM}\sigma} d\sigma \right)'. \tag{7.77b}$$

The proposed dynamic allocator is given by

$$\dot{x}_{IM}(t) = A_{IM} x_{IM}(t) - \gamma L(w(t), x_{IM}(t)), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.78a}$$

$$\dot{x}_a(t) = A_a x_a(t) + B_a C_{IM} x_{IM}(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.78b}$$

$$\varsigma(t) = C_a x_a(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.78c}$$

Remark 7.31. *The equations* (7.77)–(7.78) *allow the designer to directly implement the proposed allocation scheme (more insights on how to do this in software routines as well as on devices such as,* e.g.*, digital microcontrollers, are given in the following). Nonetheless, a deeper understanding of* (7.78) *is necessary to clarify its construction with respect to the architecture introduced in [51], anticipating the following results. The optimizer unit is given by* (7.77) *and it embeds, in essence, a gradient descent scheme as shown in the following. The state vector $x_{IM}(t)$ is the state of the steady-state generator at time t, whose dynamics are defined by* (7.73)–(7.74)*. The state vector $x_a(t)$ is the state, at time t, of the annihilator given by $W_a(s)$ and the matrices (A_a, B_a, C_a) are a minimal realization of it.* ▲

Remark 7.32. *The definition of A_{IM} given in* (7.74a)*, together with its eigenvalues selected by* (7.73a)*, define the* prescribed *function space mentioned in statement (iii) of Problem 7.29: it consists of the functions of time generated as combinations of the modes of* (7.63), i.e., *any function $v(t)$ such that there exist constants c_0, c_i, ϕ_i, $i = 1, \ldots, \bar{n}_e$, with the property that $v(t) = c_0 + \sum_{i=1}^{\bar{n}_e} c_i \sin(\omega_i t + \phi_i)$.* ▲

Remark 7.33. *In [89, Problem 1], a formulation similar to Problem 7.29 is provided, although concerning the only case of $\mu = 0$. The results shown therein lead to the development of a dynamic control allocator that is implemented as a hybrid dynamic system, which has also been compared against the solution of the optimal control approach presented in Section 7.6.1. With respect to that work, the solution proposed in* (7.78) *has two main advantages. First, it is a continuous-time system with no hybrid dynamics, which also simplifies its implementation. Secondly, the solution presented in [89] is based on a gradient descent approach, as is the one hereby proposed (see* (7.78a)*), and both these solutions depend on a gradient descent step paramter $\gamma \in \mathbb{R}_{>0}$. What differentiates the proposed approach from the state of the art is that, while in [89] the step parameter γ has to be tuned carefully to ensure convergence due to the hybrid dynamics, here it can be set* arbitrarily large, *thus leading to an arbitrarily fast allocation action.* ▲

The equations (7.78) allow the designer to directly implement the proposed control allocator. The properties of this solution are clarified in the following. In the case of $\mu = 0$, the gathering of the data **K1**, **K2**, and **K3** is straightforward.

Remark 7.34 (**K1**, **K2**, **K3** for $\mu = 0$)**.** *If $\mu = 0$,* (7.62) *is fully known by Assumption 7.27 and the data **K1**, **K2**, and **K3** is computed as follows.*

K1 *Solve the following:*

$$\Pi_\Sigma S = A\Pi_\Sigma + P, \tag{7.79a}$$

$$\bar{M}_{\eta_p,w} = C_2\Pi_\Sigma + D_{22}. \tag{7.79b}$$

K2 $\{W_{\eta_p v}(0), W_{\eta_p v}(j\omega_1), \ldots, W_{\eta_p v}(j\omega_{\bar{n}_e})\}$ *are given by*

$$W_{\eta_p v}(s), \qquad s \in \{0, j\omega_1, \ldots, j\omega_{\bar{n}_e}\}. \tag{7.80}$$

K3 *Find $N_a(s)$ whose columns are a minimal polynomial basis for the null space of $W_{\eta_o v}(s)$. Choose a Hurwitz monic polynomial $d_a(s)$ of degree larger than each element of $N_a(s)$. Define*

$$W_a(s) = d_a^{-1}(s)N_a(s), \tag{7.81}$$

and choose $(A_a, B_a, C_a, 0)$ as any minimal realization of $W_a(s)$. ▲

The following statement characterizes the properties of (7.78) in the case of $\mu = 0$.

Lemma 7.35. *Consider the system* (7.62) *with $\mu = 0$. Then, the dynamic control allocator* (7.78) *interconnected to* (7.62) *as in* (7.67) *solves Problem 7.29 for any $\gamma \in \mathbb{R}_{\geq 0}$ with $t_0 = 0$.* ◦

Proof. Item (i) of Problem 7.29 with $\mu = 0$ follows immediately by knowledge of **K3** computed as in Remark 7.34; this guarantees full output invisibility of the allocator unit as requested. Therefore, the optimality requirement remains to be addressed. To this end, it must be shown that the behavior of the proposed allocator (7.78) leads to the minimization of the cost functional (7.6). This is achieved by showing that (7.78a), (7.77b) define a gradient descent algorithm for the cost functional (7.6). Define the *moving-horizon cost* over a period as

$$J_t = \int_t^{t+T} \ell(\tilde{\eta}_p(\sigma))\, d\sigma, \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.82}$$

and consider (7.78a) with an input $u_{IM}(t)$ to be determined, *i.e.*,

$$\dot{x}_{IM}(t) = A_{IM}x_{IM}(t) + u_{IM}(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.83}$$

To evaluate the effect of a change in the cost J due to a change in x_{IM}, consider the following selections of u_{IM}.

$$u_{IM}^{\delta}(\sigma) = \begin{cases} \delta, & \text{if } \sigma \in [t, t+\Delta],\ \Delta > 0,\ t \in \mathbb{R}_{\geq 0}, \\ 0, & \text{otherwise.} \end{cases} \tag{7.84a}$$

$$u_{IM}^{\circ}(\sigma) = 0, \qquad \forall \sigma \geq t,\ t \in \mathbb{R}_{\geq 0}. \tag{7.84b}$$

Denote the respective evolutions of $\tilde{\eta}_p$, x_{IM}, and J_t by the superscripts δ or $\circ$, *e.g.*, $\tilde{\eta}_p^{\delta}$ and $\tilde{\eta}_p^{\circ}$. Integrating (7.83), (7.84) from the common initial condition $x_{IM}^{\circ}(t)$ yields

$$x_{IM}^{\circ}(t+\Delta) = e^{A_{IM}\Delta} x_{IM}^{\circ}(t), \tag{7.85a}$$

$$\begin{aligned} x_{IM}^{\delta}(t+\Delta) &= e^{A_{IM}\Delta} x_{IM}^{\circ}(t) + \int_0^{\Delta} e^{A_{IM}\tau} u_{IM}(t+\Delta-\tau)\, d\tau \\ &= x_{IM}^{\circ}(t+\Delta) + \int_0^{\Delta} e^{A_{IM}\tau}\, d\tau \delta = x_{IM}^{\circ}(t+\Delta) + \Delta \bar{B}\delta. \end{aligned} \tag{7.85b}$$

The matrix $\bar{B}$ is such that

$$\|\bar{B}\| \leq 1, \qquad \lim_{\Delta \to 0^+} \bar{B} = I, \tag{7.86}$$

which follow from the fact that $\|e^{A_{IM}\tau}\| = 1$ (since $(e^{A_{IM}\tau})' e^{A_{IM}\tau} = I$) and from integration of the series expansion of $e^{A_{IM}\tau}$, respectively. Since u_{IM}° and u_{IM}^{δ} are zero after $t+\Delta$, it holds that

$$x_{IM}^{\delta}(\tau) - x_{IM}^{\circ}(\tau) = e^{A_{IM}(\tau-(t+\Delta))} \Delta \bar{B}\delta, \qquad \tau \geq t+\Delta, \tag{7.87}$$

and the corresponding steady-state responses of η_p are

$$\tilde{\eta}_p^{\circ}(\tau) = \bar{M}_{\eta_p,w} w(\tau) + \bar{N}_{\mu} x_{IM}^{\circ}(\tau), \tag{7.88a}$$

$$\tilde{\eta}_p^{\delta}(\tau) = \bar{M}_{\eta_p,w} w(\tau) + \bar{N}_{\mu} x_{IM}^{\delta}(\tau), \tag{7.88b}$$

$$\hat{\tilde{\eta}}_p(\tau) = \tilde{\eta}_p^{\delta}(\tau) - \tilde{\eta}_p^{\circ}(\tau) = \bar{N}_{\mu} e^{A_{IM}(\tau-(t+\Delta))} \Delta \bar{B}\delta. \tag{7.88c}$$

If Δ is sufficiently small, the difference between the two cost integrand functions along the two solutions satisfy

$$\begin{aligned} &\ell(\tilde{\eta}_p^{\delta}(\tau)) - \ell(\tilde{\eta}_p^{\circ}(\tau)) \\ &\qquad = \nabla \ell|_{\tilde{\eta}_p^{\circ}(\tau)} \hat{\tilde{\eta}}_p(\tau) + o(\|\hat{\tilde{\eta}}_p(\tau)\|) \\ &\qquad\qquad = \nabla \ell|_{\tilde{\eta}_p^{\circ}(\tau)} \bar{N}_{\mu} e^{A_{IM}(\tau-(t+\Delta))} \Delta \bar{B}\delta + o(\|\Delta\|), \end{aligned} \tag{7.89}$$

where the equality $o(\|\hat{\tilde{\eta}}_p(\tau)\|) = o(\|\Delta\|)$ follows from (7.88c), (7.86), implying in turn that

$$\|\bar{N}_{\mu} e^{A_{IM}(\tau-(t+\Delta))} \Delta \bar{B}\delta\| \leq \|\bar{N}_{\mu}\| \cdot \|\Delta\| \cdot \|\delta\|, \tag{7.90}$$

provided that $||\delta||$ is bounded by a quantity independent of Δ, which is shown in the following.

Periodicity of $\tilde{\eta}_p^\circ$ yields

$$J_t^\circ = J_{t+\Delta}^\circ, \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.91}$$

hence, for small Δ:

$$\begin{aligned}
J_{t+\Delta}^\delta - J_t^\circ = J_{t+\Delta}^\delta - J_{t+\Delta}^\circ \\
&= \int_{t+\Delta}^{t+\Delta+T} \left(\ell(\tilde{\eta}_p^\delta(\tau)) - \ell(\tilde{\eta}_p^\circ(\tau)) \right) d\tau \\
&= \int_{t+\Delta}^{t+\Delta+T} \left(\nabla \ell|_{\tilde{\eta}_p^\circ(\tau)} \hat{\tilde{\eta}}_p(\tau) + o(\|\Delta\|) \right) d\tau \\
&= \int_{t+\Delta}^{t+\Delta+T} \nabla \ell|_{\tilde{\eta}_p^\circ(\tau)} \hat{\tilde{\eta}}_p(\tau)\, d\tau + o(\|\Delta\|).
\end{aligned} \tag{7.92}$$

Hence, recalling that $\lim_{\Delta \to 0^+} \frac{1}{\Delta} o(\|\Delta\|) = 0$ by definition of $o(\cdot)$, and (7.88c), it follows that

$$\begin{aligned}
\dot{J}_t = \lim_{\Delta \to 0^+} \frac{J_{t+\Delta}^\delta - J_t^\circ}{\Delta} \\
&= \lim_{\Delta \to 0^+} \frac{1}{\Delta} \int_{t+\Delta}^{t+\Delta+T} \nabla \ell|_{\tilde{\eta}_p^\circ(\tau)} \hat{\tilde{\eta}}_p(\tau)\, d\tau \\
&= \int_t^{t+T} \nabla \ell|_{\tilde{\eta}_p^\circ(\tau)} \bar{N}_\mu e^{A_{IM}(\tau - t)}\, d\tau \delta \\
&= F(t)\delta, \qquad t \in \mathbb{R}_{\geq 0}.
\end{aligned} \tag{7.93}$$

Selecting $\delta = -\gamma F(t)$ for any $\gamma \in \mathbb{R}_{\geq 0}$ yields

$$\dot{J}_t = -\gamma \|F(t)\|^2, \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.94}$$

Note that $F(t)$ only depends on the nominal $\tilde{\eta}_p^\circ$ trajectory and is independent of Δ, as required after (7.90). The change of the variable $\sigma = \tau - t$ in the definition of $F(t)$ shows that

$$F(t)' = L(w, x_{IM})|_{w=w(t), x_{IM}=x_{IM}(t)}, \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.95}$$

with the latter defined in (7.77). The latter equation (7.94), in turn, together with boundedness of $\bar{M}_{\eta_p,w} w(t)$ for any $t \in \mathbb{R}_{\geq 0}$ as in (7.70), implies that x_{IM} in (7.78a) is bounded and that, at steady-state, it is described by the differential equation $\dot{\tilde{x}}_{IM}(t) = A_{IM}\tilde{x}_{IM}(t)$, the solution of which is, by construction, periodic of period T. Finally, the annihilator is asymptotically stable since the matrix A_a in (7.78b) is Hurwitz; hence, the condition on $\dot{J}_t$ implies that items (ii) and (iii) of Problem 7.29 hold, concluding the proof. □

Remark 7.36. *In practice, the dynamic control allocation scheme* (7.78) *can be implemented by approximating the integral in* (7.77b) *with a finite sum. Given a sufficiently small sampling time* $\tau_s \in \mathbb{R}_{>0}$, $\tau_s = \frac{T}{K}$, $K \in \mathbb{N}$, *define* $\bar{S} = e^{S\tau_s}$ *and* $\bar{A}_{IM} = e^{A_{IM}\tau_s}$. *Then, the gradient of the cost functional is approxi-*

mated as

$$\xi_k = \bar{M}_{\eta_p,w}\bar{S}^k w(k\tau_s) + \bar{N}_\mu \bar{A}_{IM}^k x_{IM}(k\tau_s), \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.96a}$$

$$L(w, x_{IM})' \approx \sum_{k=1}^{K} \nabla \ell(\xi_k)' \cdot \bar{N}_\mu \bar{A}_{IM}^k, \tag{7.96b}$$

which is a nonlinear function of the measurements of $x_{IM}(t)$ *and* $w(t)$*. The core difficulties in the implementation of* (7.78) *arise in the nonlinear terms of* (7.96) *and fall within the wide spectrum of problems concerning function approximation with digital devices. In this specific case, assuming that the quantities* $w(t)$ *and* $x_{IM}(t)$ *remain constant during sampling intervals, a digital device with a sampling time equal to* τ_s *in* (7.96) *can be used to implement the control allocator* (7.78). ▲

The following result reveals that the structure of the control allocator (7.78) is simplified significantly when the cost functional (7.6) is quadratic: the correction term (7.77a)–(7.77b) becomes a *static linear feedback* from the state vector in (7.78a) and a *feedforward* from the state of the exosystem (7.63).

Proposition 7.37. *Consider the control allocator* (7.78) *and suppose that* $\ell(\tilde{\eta}_p(t)) = \frac{1}{2}\|\tilde{\eta}_p(t)\|_R^2$, *with* $R = R' > 0$. *Then,* $L(w(t), x_{IM}(t))$ *in* (7.77a)–(7.77b) *becomes*

$$L(w, x_{IM}) = H_1 x_{IM}(t) + H_2 w(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.97}$$

with

$$H_1 = \int_0^T e^{A_{IM}'\sigma} \bar{N}_\mu' R \bar{N}_\mu e^{A_{IM}\sigma}\, d\sigma, \tag{7.98a}$$

$$H_2 = \int_0^T e^{A_{IM}'\sigma} \bar{N}_\mu' R \bar{M}_{\eta_p,w} e^{S\sigma}\, d\sigma. \tag{7.98b}$$

Hence, for any $\gamma \in \mathbb{R}_{>0}$, (7.78a) *becomes*

$$\dot{x}_{IM}(t) = (A_{IM} - \gamma H_1) x_{IM}(t) - \gamma H_2 w(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.99}$$

Moreover, if the pair $(A_{IM}, R^{\frac{1}{2}}\bar{N}_\mu)$ *is observable, then the matrix* $(A_{IM} - \gamma H_1)$ *is Hurwitz for any* $\gamma \in \mathbb{R}_{>0}$.

∘

Proof. In this case, the gradient of the cost function ℓ in (7.6) is linear:

$$\nabla \ell(\tilde{\eta}_p(t)) = R\tilde{\eta}_p(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.100}$$

Applying standard properties of integrals, (7.77b) reduces to

$$\begin{aligned}
&L(w, x_{IM})|_{w=w(t), x_{IM}=x_{IM}(t)} \\
&\qquad = \int_0^T e^{A'_{IM}\sigma} \bar{N}'_\mu R \bar{N}_\mu e^{A_{IM}\sigma}\, d\sigma\, x_{IM} + \int_0^T e^{A'_{IM}\sigma} \bar{N}_\mu R \bar{M}_{\eta_p, w} e^{S\sigma}\, d\sigma\, w(t) \\
&\qquad = H_1 x_{IM}(t) + H_2 w(t), \quad t \in \mathbb{R}_{\geq 0}. \qquad (7.101)
\end{aligned}$$

The last claim of Proposition 7.37 follows from two facts: first, H_1 is the observability Gramian matrix of the pair $(A_{IM}, R^{\frac{1}{2}} \bar{N}_\mu)$, and secondly, the matrix A_{IM} is such that $\mathrm{spec}(A_{IM}) \subset \mathbb{C}_0$. □

Remark 7.38. *In practice, the matrices H_1 and H_2 in* (7.98) *can be computed offline. Define*

$$A^1_{12} = \bar{N}'_\mu R \bar{N}_\mu, \qquad (7.102a)$$
$$A^1_{22} = A_{IM}, \qquad (7.102b)$$
$$A^2_{12} = \bar{N}'_\mu R \bar{M}_{\eta_p, w}, \qquad (7.102c)$$
$$A^2_{22} = S, \qquad (7.102d)$$

and compute the matrix exponentials

$$H_i = \begin{bmatrix} I & 0 \end{bmatrix} \exp\left(\begin{bmatrix} -A'_{IM} & A^i_{12} \\ 0 & A^i_{22} \end{bmatrix} T \right) \begin{bmatrix} 0 \\ I \end{bmatrix}, \qquad i \in \{1, 2\}. \qquad (7.103)$$

Furthermore, denoting by $x^\star_{IM}(t)$ the optimal steady-state evolution of $x_{IM}(t)$, the fact that the gradient $L(w(t), x^\star_{IM}(t))$ is equal to zero for any $t \in \mathbb{R}_{\geq 0}$ yields

$$x^\star_{IM}(t) = -H_1^{-1} H_2 w(t), \qquad t \in \mathbb{R}_{\geq 0}. \qquad (7.104)$$

The first consequence of (7.104) *is that the correct initialization for the steady-state generator is $x_{IM}(0) = -H_1^{-1} H_2 w(0)$. Secondly, one may* bypass *the steady-state generator entirely by redefining the control allocator* (7.78) *as*

$$\dot{x}_a(t) = A_a x_a(t) - B_a C_{IM} H_1^{-1} H_2 w(t), \qquad t \in \mathbb{R}_{\geq 0}, \qquad (7.105a)$$
$$y_a(t) = C_a x_a(t), \qquad t \in \mathbb{R}_{\geq 0}. \qquad (7.105b)$$

▲

Remark 7.39. *The observability property in the last sentence of Proposition 7.37 is always satisfied if the columns of the numerator of the annihilator $W_a(s)$, namely $N_a(s)$, are chosen as a minimal basis, as it can be inferred from [112, Thm. 6.5.10].* ▲

Dealing with the general case of $\mu \in \mathcal{M}$ requires data-driven algorithms for the estimation of the data in Definition 7.30. To this end, the steady-state behavior of the uncertain system (7.62)–(7.63) must first be estimated. The following results and algorithms provide a solution to this problem based on the concept of *moment matching* for a linear system; the basic definitions and results required in the following are given in Appendix C. Starting from **K1**, the first item to be estimated is the steady-state map $\bar{M}_{\eta_p,w}$. Usually, this implies disconnecting the exosystem (7.62) from the system (7.62); in the current context this procedure is not permitted, since one of the main goals of the allocation approach is to not alter or reconfigure in any way the preexisting control system. Thus, a data-driven algorithm for the online estimation of $\bar{M}_{\eta_p,w}$ with $v = 0$ is proposed. Since it is based on sampled measurements of $\tilde{\eta}_p(t)$ and $w(t)$, the first step consists in defining the characteristics of the sampling procedure. Let $\tau_s \in \mathbb{R}_{>0}$ be the sampling time, characterized by the following statements.

Definition 7.40 (Non-pathological sampling time)**.** *The sampling time* $\tau_s > 0$ *is* non-pathological *if, for any* $\lambda_1, \lambda_2 \in \text{spec}(S) \cup \text{spec}(A)$*, there is no* $h \in \mathbb{Z}$ *such that*

$$\tau_s(\lambda_1 - \lambda_2) = h2\pi j. \tag{7.106}$$

∘

Assumption 7.41. τ_s *is non-pathological.* ∘

Let $S_d = e^{S\tau_s}$, $A_d = e^{A\tau_s}$. Assumption 7.41 implies that all distinct eigenvalues of S and A are mapped to distinct eigenvalues of S_d and A_d; moreover all structural properties (*e.g.*, observability and reachability) of (7.62)–(7.63) are preserved in their sampled data equivalents. More details about this topic are found in [41, Sec. 3.2].

Remark 7.42. *Algorithm 3 holds onto Assumptions 7.27 and 7.41. Moreover, here it is defined for data sampled at times* $k = 0, 1, \ldots, K-1$ *but it can be applied (with obvious changes) to any sequence of* K *data samples,* i.e.*, for* $k = \bar{k}, \ldots, \bar{k}+K-1$ *for an arbitrary* $\bar{k} \in \mathbb{Z}$. ▲

Proposition 7.43. *Let Assumptions 7.27 and 7.41 hold. Fix* $w_0 \in \mathcal{W}^\circ$*. Then, the following statements hold.*

i. Algorithm 3 terminates yielding a matrix $\bar{M}_{\eta_p,w}$.

ii. $\bar{M}_{\eta_p,w}$ *satisfies* $\tilde{\eta}_p(t) = \bar{M}_{\eta_p,w} w(t)$ *for all* $t \in \mathbb{R}_{\geq 0}$.

iii. If w_0 *excites all natural modes of* (7.63)*, then* $\bar{M}_{\eta_p,w}$ *is equal to the unique solution of* (7.79b). ∘

Note that statement (ii) of Proposition 7.43 highlights that the matrix $\bar{M}_{\eta_p,w}$ computed by Algorithm 3 always ensures a correct prediction of the steady-state evolution *for the trajectory used in the computation*, even if such matrix does not coincide with the unique solution of (7.79b). Further conditions, concerning the quantity of information that the data provides, are required to ensure the coincidence of the two matrices; this is specified in statement (ii) of Proposition 7.43. Since the algorithm is intended to be used for data-driven, online estimation, the trajectory used in the computation (arising from the fixed w_0) is exactly the one of interest for correctly controlling the plant, hence statement (ii) is the one actually useful in practice. The issue at hand is the same as the one in classic adaptive

Algorithm 3 Data-driven pseudo-steady-state output map estimation.

Input: τ_s, $K \in \{a \in \mathbb{N} : a \geq n+1+q_p n_e\}$, $\{w_k\}_{k=0}^{K-1}$, $\{\eta_{p,k}\}_{k=0}^{K+n-1}$.
Output: $\bar{M}_{\eta_p,w}$.

1: Define $T(\eta_p, w)$ as:

$$T(\eta_p, w) = \begin{bmatrix} \eta_p(n\tau_s) & \dots & \eta_p(0) & -w(0)' \otimes I_{q_p} \\ \eta_p((n+1)\tau_s) & \dots & \eta_p(\tau_s) & -w(\tau_s)' \otimes I_{q_p} \\ \vdots & \dots & \vdots & \vdots \\ \eta_p((n+K-1)\tau_s) & \dots & \eta_p((K-1)\tau_s) & -w((K-1)\tau_s)' \otimes I_{q_p} \end{bmatrix}. \tag{7.107}$$

2: Find the smallest $\nu \in \mathbb{N}$, $1 \leq \nu \leq n$, for which there exists $v \in \mathbb{R}^{n+1+q_p n_e}$ satisfying:

$$T(\eta_p, w)v = 0, \tag{7.108}$$

with v such that

$$v' = \begin{bmatrix} 0_{1\times(n-\nu)} & 1 & \hat{a}_{\nu-1} & \hat{a}_{\nu-2} & \cdots & \hat{a}_0 & \hat{M}' \end{bmatrix}. \tag{7.109}$$

3: Define $\tilde{M} \in \mathbb{R}^{q_p \times n_e}$ such that $\mathrm{vec}(\tilde{M}) = \hat{M}$.

4: Let $\hat{a}_\nu = 1$ and compute $\bar{M}_{\eta_p,w}$ as:

$$\bar{M}_{\eta_p,w} = \tilde{M}\left(\sum_{h=0}^{\nu} \hat{a}_h S_d^h\right)^{-1}. \tag{7.110}$$

---

regulation, where the control objective is achieved even without fully identifying the parameters of the plant. Statement (iii) is given to complete the analysis and provide a full understanding of the issues at play.

Proof. To prove statement (i) of Proposition 7.43, notice that all steps in Algorithm 3 can be performed, provided that (7.108) is solvable and the matrix inverse in (7.110) exists; both facts are shown in the following. In the following, given a signal $x(t)$, $x^\circ(t)$ denotes its transient response, provided that the latter exists. For all $k \in \mathbb{N}$, the following holds:

$$\eta_p(k\tau_s) = \eta_p^\circ(k\tau_s) + \tilde{\eta}_p(k\tau_s) = C_2\chi^\circ(k\tau_s) + \bar{M}_{\eta_p,w}w(k\tau_s). \tag{7.111}$$

Recall [112, App. A.8] that a matrix satisfies the equation defined by its *minimal polynomial* (*i.e.*, the smallest degree polynomial which enjoys this property), and that the minimal polynomial has the same roots, with lower or equal multiplicity, of the *characteristic polynomial.* Define $\hat{a}_\nu = 1$ and let the minimal (monic) polynomial of A_d be:

$$\rho_{A_d}(z) = \sum_{h=0}^{\nu} \hat{a}_h z^h, \qquad z \in \mathbb{C}. \tag{7.112}$$

Since $\rho_{A_d}(A_d) = 0$ by definition and

$$w((k+h)\tau_s) = S_d^h w(k\tau_s), \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.113a}$$

$$\chi^\circ((k+h)\tau_s) = A_d^h \chi^\circ(k\tau_s), \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.113b}$$

it is possible to define a linear combination of the measurements (7.111) that is independent of $\chi_0^\circ = \chi^\circ(0)$ as

$$\begin{aligned}\sum_{h=0}^{\nu} \hat{a}_h \eta_p((k+h)\tau_s) &= \sum_{h=0}^{\nu} \hat{a}_h C_2 \chi^\circ((k+h)\tau_s) + \sum_{h=0}^{\nu} \hat{a}_h \bar{M}_{\eta_p,w} w((k+h)\tau_s) \\ &= C_2 \rho_{A_d}(A_d)\chi^\circ(k\tau_s) + \bar{M}_{\eta_p,w}\rho_{A_d}(S_d) w(k\tau_s) \\ &= \bar{M}_{\eta_p,w}\rho_{A_d}(S_d) w(k\tau_s), \quad k \in \mathbb{Z}_{\geq 0}.\end{aligned} \tag{7.114}$$

Defining

$$H(k\tau_s) = \begin{bmatrix} \eta_p((k+\nu)\tau_s) & \eta_p((k+\nu-1)\tau_s) & \cdots & \eta_p((k+1)\tau_s) & \eta_p(k\tau_s) \end{bmatrix}, \tag{7.115a}$$

$$\hat{a} = \begin{bmatrix} \hat{a}_\nu & \hat{a}_{\nu-1} & \cdots & \hat{a}_1 & \hat{a}_0 \end{bmatrix}, \tag{7.115b}$$

$$\tilde{M} = \bar{M}_{\eta_p,w}\rho_{A_d}(S_d), \tag{7.115c}$$

yields

$$H(k\tau_s)\hat{a} = \tilde{M}\chi(k\tau_s), \qquad k \in \mathbb{Z}_{\geq 0}. \tag{7.116}$$

In turn, (7.116) can be vectorized [248, Sec. 2.5] introducing $\hat{M} = \mathrm{vec}(\tilde{M})$, yielding:

$$H(k\tau_s)\hat{a} = (w(k\tau_s)' \otimes I_{q_p})\hat{M}, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.117}$$

and collecting (7.117) for $k = 0, 1, \ldots, K-1$ yields (7.108).

Since the degree of the minimal polynomial of a matrix never exceeds the degree of its characteristic polynomial, the above reasoning proves that a vector $v \in \mathbb{R}^{n+1+q_p n_e}$ as required at line 2 of Algorithm 3 can be found.

As for the invertibility of the matrix at line 4, note that this matrix is exactly $\rho_{A_d}(S_d)$. Since S is diagonalizable, there exists an invertible matrix T_I such that $S = T_I \Lambda_S T_I^{-1}$, with Λ_S diagonal. Since τ_s is non-pathological, S_d is diagonalizable by using the same T_I as S, yielding $S_d = T_I \Lambda_{S_d} T_I^{-1}$ with Λ_{S_d}

diagonal. By rewriting

$$\rho_{A_d}(S_d) = \rho_{A_d}(T_I \Lambda_{S_d} T_I^{-1}) = T_I \rho_{A_d}(\Lambda_{S_d}) T_I^{-1}, \tag{7.118}$$

and considering that $\rho_{A_d}(\Lambda_{S_d})$ is diagonal, it is clear that $\rho_{A_d}(S_d)$ is not invertible if and only if one of the elements on its diagonal is zero; this implies that S_d and A_d have at least one eigenvalue in common, which is impossible since by hypothesis τ_s is non-pathological and $\mathrm{spec}(S) \cap \mathrm{spec}(A) = \emptyset$ due to Assumption 7.27.

To prove statement (ii), note that $\tilde{\eta}_p$ is a q_p−dimensional vector function whose scalar components are linear combinations of the n_e linearly independent functions generated by the exosystem; hence, for any fixed $w_0 \in \mathscr{W}^\circ$, the ensuing $\tilde{\eta}_p$ is uniquely associated to a set of $q_p n_e$ scalars. Consequently, if Algorithm 3 is able to determine a matrix $\bar{M}$ (possibly different from the *true* $\bar{M}_{\eta_p,w}$ solving (7.79b)) which ensures that $\bar{M}w$ interpolates $\tilde{\eta}_p$ at $q_p n_e$ (or more) points in time, then for the fixed w_0 the same $\bar{M}$ interpolates $\tilde{\eta}_p$ at any time, since it matched all the $q_p n_e$ degrees of freedom in q_p. Since Algorithm 3 is based on satisfying (7.116) at $K \geq n + 1 + q_p n_e > q_p n_e$ sample points, the claim in statement (ii) is proved once it is clear that (7.116) expresses a matching condition for the relation $\tilde{\eta}_p(t) = \bar{M}_{\eta_p,w} w(t)$ at time $t = k\tau_s$. To cancel out the transient terms in (7.111), a linear combination of responses has been performed in (7.114), hence (7.116) describes the steady-state response due to the initial state $\bar{w}_0 = \rho_{A_d}(w_d) w_0$ (see (7.114)). Note that $\bar{w}_0$ excites in S_d all and only the natural modes excited by w_0 in S_d, which implies that the coefficients of such modes are exactly estimated.

To prove statement (iii), denote by T_w the block depending on w in (7.107). Note that T_w is full column rank if and only if

$$\mathrm{rank}\left(\begin{bmatrix} w(0) & w(\tau_s) & \cdots & w((K-1)\tau_s) \end{bmatrix}\right) = n_e. \tag{7.119}$$

Since $K > n_e$ and each element $w(\tau_s)$ can be expressed as $w(\tau_s) = S_d^k w_0$, Cayley-Hamilton's theorem implies that (7.119) is equivalent to:

$$\mathrm{rank}\left(\begin{bmatrix} w_0 & S_d w_0 & \cdots & S_d^{n_e-1} w_0 \end{bmatrix}\right) = n_e. \tag{7.120}$$

The condition in (7.120) has the form of a reachability matrix test for the pair (S_d, w_0), which is equivalent to w_0 exciting all natural modes of S_d if S_d has all distinct eigenvalues. By Assumption 7.41, S_d has all distinct eigenvalues since, by Assumption 7.27, S has all distinct eigenvalues; this implies, in turn, that S and S_d have the same eigenvectors, so that w_0 excites all natural modes of S_d if and only if it excites all natural modes of S. Hence, T_w is full column rank by the condition in statement (iii).

Let T_{η_p} denote the block depending on η_p in (7.107), and by $T_{\eta_p}^h$ the submatrix containing its rightmost h columns. Since the columns of T_w are a linearly independent set, the algorithm starts to consider matrices $\begin{bmatrix} T_{\eta_p}^h & T_w \end{bmatrix}$ increasing h until for $h = \nu$, it happens that $\begin{bmatrix} T_{\eta_p}^{\nu-1} & T_w \end{bmatrix}$ is still full column rank, whereas $\begin{bmatrix} T_{\eta_p}^{\nu} & T_w \end{bmatrix}$ has a non-trivial kernel, which has necessarily dimension one since $\begin{bmatrix} T_{\eta_p}^{\nu} & T_w \end{bmatrix}$ has exactly one column more than $\begin{bmatrix} T_{\eta_p}^{\nu-1} & T_w \end{bmatrix}$. Thus, there is a unique solution v to (7.108) with the structure in (7.109) and minimal value of ν. Since a solution (7.110) is found in statement (i) on the basis of (7.109) and since the latter is shown to be uniquely defined, under the stated hypotheses the unique solution found by Algorithm 3 must result in the same $\bar{M}$ arising from (7.79b). □

Remark 7.44. *Although Algorithm 3 is provided in this context with the specific purpose of estimating $\bar{M}_{\eta_p,w}$ to compute the dynamic allocator* (7.78)*, it is a general-purpose algorithm for the estimation of the pseudo-steady-state output map of a linear system such as,* e.g.*, system* (C.6) *in Appendix C. Thus, its interest falls within the broader context of online data-driven moment estimation for linear systems, as do the statements of Proposition 7.43 and its proof requiring only an adapted notation.* ▲

Algorithm 3 provides a procedure to compute in finite time the information summarized in the item **K1**, for any $\mu \in \mathcal{M}$. Conversely, computing **K2** entails estimating the moments $W_{\eta_p v}(0), W_{\eta_p v}(j\omega_i)$, $\omega_i \in \mathrm{spec}(S)$, $i = 1, ..., \bar{n}_e$. In this case, however, the measurements of $\eta_p(t)$ are not influenced only by $v(t)$ but also by $w(t)$ (since the exosystem cannot be disconnected at any given time). Thanks to the computation of **K1**, $\bar{M}_{\eta_p,w}$ is now available. The next statement follows trivially from the previous results.

Lemma 7.45. *If **K1** is available, then Algorithm 4 computes an estimate of **K2**.* ∘

Algorithm 4 Data-driven computation of **K2**.

Input: $\bar{M}_{\eta_p,w}$, Ω, Λ, measurements of $\eta_p(t)$.
Output: $W_{\eta_p v}(0), W_{\eta_p v}(j\omega_i)$, $\omega_i \in \mathrm{spec}(S)$, $i = 1, ..., \bar{n}_e$.

1: Define

$$\eta_p^\delta(t) = \eta_p(t) - \bar{M}_{\eta_p,w}\, w(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.121}$$

2: Define the *virtual dynamics*

$$\dot{\zeta}(t) = \Omega\zeta(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.122a}$$
$$\psi(t) = \Lambda\zeta(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.122b}$$

with $\zeta(0) = \Lambda'$.

3: **for** $i = 1, \ldots, m$ **do**

4: Apply $v(t) = e_i\psi(t)$, $t \in \mathbb{R}_{\geq 0}$, and measure $\eta_p^\delta(t)$.

5: Apply Algorithm 3 to the measurements of $(\zeta(t), \eta_p^\delta(t))$, yielding

$$\bar{M}_i = \begin{bmatrix} \bar{M}_{i,0} & \bar{M}_{i,1} & \ldots & \bar{M}_{i,2\bar{n}_e} \end{bmatrix}, \tag{7.123}$$

with $\bar{M}_{i,h} \in \mathbb{R}^{q_p}$, $h = 0, \ldots, 2\bar{n}_e$.

6: The i-th column of $W_{\eta_p v}(0)$ is $\bar{M}_{i,0}$.

7: The i-th column of $W_{\eta_p v}(j\omega_h)$ is $\bar{M}_{i,2h-1} + j\bar{M}_{i,h}$, $h = 1, \ldots, \bar{n}_e$.

8: **end for**

Finally, the key step to determine **K3** is to find a polynomial annihilator $N_a(s)$ such that its columns are a minimal polynomial base of the null space of $W_{\eta_o v}(s)$. It is then transformed into a rational annihilator $W_a(s)$ by adding a Hurwitz denominator to make it strictly proper. Since the numerators in the elements of $W_{\eta_o v}(s)$ have degree at most n, then, according to [250, Prop. 1], $N_a(s)$ can be chosen with elements of degree not larger than nq_o exactly by selecting the columns of $N_a(s)$ as a minimal polynomial base of the null space of $W_{\eta_o v}(s)$ [82, Main Thm.].

Define

$$n_c = nq_o + 1, \tag{7.124a}$$

$$n_r = \min\{v \in \mathbb{N} : 2vq_o \geq mn_c\}. \tag{7.124b}$$

For a set of frequencies $\{\bar{\omega}_1, ..., \bar{\omega}_{n_r}\}$, define the block matrix H with the (k,h)-th block of size $2q_o \times m$ as

$$H_{[k,h]} = \begin{bmatrix} \mathrm{Re}\left((j\bar{\omega}_k)^{h-1} W_{\eta_o v}(j\bar{\omega}_k)\right) \\ \mathrm{Im}\left((j\bar{\omega}_k)^{h-1} W_{\eta_o v}(j\bar{\omega}_k)\right) \end{bmatrix}, \qquad k = 1, \ldots, n_r, \ h = 1, \ldots, n_c, \tag{7.125}$$

where $\mathrm{rank}(W_{\eta_o v}(j\bar{\omega}_k)) = q_o$ provided that $s = j\bar{\omega}_k$ is not a pole or a zero of the transfer matrix $W_{\eta_o v}(s)$.

The computation of a polynomial annihilator for $W_{\eta_o v}(s)$ can be translated into the computation of the null space of the real matrix H. The key issue is that, while every polynomial vector in a minimal basis of the null space of $W_{\eta_o v}(s)$ is associated to a single real vector in the kernel of H, the opposite transformation maps a basis of $\ker(H)$ to a linearly dependent set of polynomial vectors in the null space of $W_{\eta_o v}(s)$. This problem is avoided by computing a *minimal coefficient basis* for the null space of H, whose columns, when translated back into their polynomial form, yield a *minimal polynomial basis* for the null space of $W_{\eta_o v}(s)$ with dimension $\rho = m - q_o$. The following Algorithm 5 is adapted from [250], where several possible optimized computations are also proposed. A minimal *polynomial* basis matrix of order $h-1$ is computed from a minimal *coefficient* basis matrix $N \in \mathbb{R}^{mh\times\rho}$ as

$$\begin{bmatrix} I_m & sI_m & \cdots & s^{h-1}I_m \end{bmatrix} N. \tag{7.126}$$

Proposition 7.46. *Recall* (7.124)*. Consider the transfer matrix* $W_{\eta_o v}(s) \in \mathbb{C}^{q_o\times m}$ *and an arbitrary collection of frequencies* $\{\bar{\omega}_1, \ldots, \bar{\omega}_n\}$ *such that* $\mathrm{rank}(W_{\eta_o v}(j\bar{\omega}_i)) = q_o$*,* $i = 1, \ldots, n$*. Let the matrix* $H \in \mathbb{R}^{q_o n_r \times mn_c}$ *be defined as in* (7.125)*. Let* $N \in \mathbb{R}^{mh\times(m-q_o)}$ *be a* minimal coefficient basis *matrix computed by Algorithm 5 and partitioned as*

$$N = \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_{h-1} \end{bmatrix}, \qquad N_i \in \mathbb{R}^{m\times(m-q_o)}, \ i = 0, \ldots, h-1. \tag{7.127}$$

Then, the matrix function

$$N_a(s) = N_0 + sN_1 + \ldots + s^{h-1}N_{h-1} \tag{7.128}$$

Algorithm 5 Computation of a minimal coefficients basis.

Input: $H = \begin{bmatrix} H_{[:,1]} & H_{[:,2]} & \ldots \end{bmatrix}$.
Output: A minimal coefficient basis N for H.
1: $\bar{\Sigma} \leftarrow H_{[:,1]}$
2: $k_1 \leftarrow \dim(\ker(\bar{\Sigma}))$
3: $\bar{p} \leftarrow \emptyset$
4: $h \leftarrow 1$
5: **if** $k_1 = 0$ **then**
6: $\quad X_2 \leftarrow 0_{1\times(2m)}$
7: $\quad h \leftarrow 2$
8: $\quad$**goto** 15
9: **else if** $0 < k_1 < \rho$ **then**
10: $\quad X_1 \leftarrow 0_{1\times m}$
11: $\quad$**goto** 22
12: **else if** $k_1 = \rho$ **then**
13: $\quad$**goto** 29
14: **end if**
15: $\bar{\Sigma} \leftarrow \begin{bmatrix} H'_{[:,1:h]} \\ X'_h \end{bmatrix}$
16: $k_h \leftarrow \dim(\ker(\bar{\Sigma}))$
17: **if** $k_h = \rho$ **then**
18: $\quad$**goto** 29
19: **else**
20: $\quad$**goto** 22
21: **end if**
22: $\bar{Q}, \bar{R} \leftarrow$ QR decomposition of $\bar{\Sigma}'$.
$\qquad\qquad\qquad\qquad\qquad\qquad \triangleright$ $\bar{\Sigma}' = \bar{Q}\bar{R}$ with $\bar{Q}$ orthogonal and $\bar{R}$ upper-triangular.
23: $\bar{L} \leftarrow$ The last $(p - \mathrm{rank}(\bar{R}))$ rows of $\bar{Q}'$.
24: $\bar{p} \leftarrow \bar{p} \cup \bar{p}_h$, where $\bar{p}_h$ is the set of pivot indices in a reduced row echelon form of $\bar{L}$.
25: $\bar{X} \leftarrow$ The rows of I_m with indices in $\bar{p}$.
26: $X_{h+1} \leftarrow \mathrm{diag}(X_h, \bar{X})$
27: $h \leftarrow h + 1$
28: **goto** 15
29: $N \in \mathbb{R}^{mh\times\rho} \leftarrow$ Any basis matrix of $\ker(\bar{\Sigma})$.

is a polynomial annihilator of $W_{\eta_o v}(s)$ whose columns are a minimal polynomial basis for the null space of $W_{\eta_o v}(s)$. ◦

Proof. Since each element in the polynomial matrix $\det(sI - A)W_{\eta_o v}(s)$ has degree not larger than n and $\mathrm{rank}(\det(sI - A)W_{\eta_o v}(s)) = \eta_o$ by Assumption 7.27, it follows from [250, Prop. 1] that a polynomial annihilator $N_a(s)$ with entries having maximum degree not larger than nq_o exists. Then, the equation $W_{\eta_o v}(s)N_a(s) = 0$ has a solution of the form

$$N_a(s) = N_0 + sN_1 + \ldots + s^{\bar{n}}N_{\bar{n}}, \qquad \bar{n} < n_c. \tag{7.129}$$

Let

$$N_{\bar{n}} = \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_{\bar{n}} \end{bmatrix} \tag{7.130}$$

so that

$$N_a(s) = \begin{bmatrix} I_m & sI_m & \cdots & s^{\bar{n}} I_m \end{bmatrix} N_{\bar{n}}. \tag{7.131}$$

Thus, the annihilator equation to be solved becomes

$$W_{\eta_o v}(s) N_a(s) = W_{\eta_o v}(s) \begin{bmatrix} I_m & sI_m & \dots & s^{\bar{n}} I_m \end{bmatrix} N_{\bar{n}} = 0. \tag{7.132}$$

By evaluating (7.132) at the frequencies $\{\bar{\omega}_1, \dots, \bar{\omega}_n\}$ and imposing that both real and imaginary parts of the left hand side of (7.132) are equal to zero, it follows that $N_{\bar{n}}$ lies in the null space of the matrix $H_{[:,1:\bar{n}]}$ whose blocks are defined in (7.125). Finally, arguments similar to those in [250], Algorithm 5 computes the minimal $\bar{n} = h$ and a suitable $N_{\bar{n}} = N$, yielding the desired result. □

Remark 7.47. *The requirement of selecting frequencies* $\{\bar{\omega}_i\}_{i=1}^n$ *with a full-rank property, namely such that* $\operatorname{rank}(W_{\eta_o v}(j\bar{\omega}_i)) = q_o$, $i = 1, \dots, n$, *originates from the necessity to rule out pathological cases that may arise in control systems such as those modeled by* (7.62). *More precisely, suppose that* (7.62) *embeds a feedback control loop (recall the linear case of Section 7.6.1), which has been designed by incorporating an internal model of the exosystem, as is customary in output regulation problems. In that case, choosing frequencies that belong to the internal model yields null rows in* H, *making the polynomial annihilator's computation unfeasible. It is worth observing that, in the uncertain setting, a random selection of such frequencies* generally *ensures the required full-rank property.* ▲

Remark 7.48. *Algorithm 5 can be applied to find a minimal coefficient basis for a polynomial annihilator of the polynomial matrix* $A(s) = s^{\bar{n}} A_{\bar{n}} + \cdots + sA_1 + A_0$ *where* $A_i \in \mathbb{R}^{q_o \times m}$ *and* $\operatorname{rank}(A(s)) = q_o$. *In such a case, instead of forming matrix* H *with blocks described by* (7.125), *it is possible to use a block Toeplitz matrix* T *whose* $(k,h)-th$ *block is* $T_{[k,h]} = A_{k-h}$ *if* $0 \le k - h \le \bar{n}$, *and* $T_{[k,h]} = 0$ *otherwise. This applies to,* e.g., *the case of* $\mu = 0$, *in which the polynomial matrix* $N_{\eta_o v}(s) = \det(sI - A) W_{\eta_o v}(s)$ *is known.* ▲

Recalling the definition of matrix H in (7.125), it appears evident that Proposition 7.46 is stated with the ideal premise of a perfect knowledge of the transfer matrix $W_{\eta_o v}(s)$ evaluated at the frequencies $\{\bar{\omega}_1, \dots, \bar{\omega}_n\}$; such a knowledge is not available for a generic $\mu \neq 0$. Therefore, to extend the previous results to the case of a generic $\mu \in \mathcal{M}$, it is necessary to perform a data-driven estimation of the moments $W_{\eta_o v}(j\bar{\omega}_i)$, $i = 1, \dots, n$. The proposed procedure is illustrated in Algorithm 6.

The following result summarizes all the previous claims, and proves that the proposed control allocator is determined for any $\mu \in \mathcal{M}$.

Theorem 7.49. *Consider the system* (7.62)–(7.63) *with* $\mu \in \mathcal{M}$. *Suppose that Assumptions 7.5, 7.27, 7.28 and 7.41 hold. Then, the dynamic control allocator* (7.78) *interconnected to* (7.62)–(7.63) *as in* (7.67) *solves Problem 7.29 for any* $\gamma \in \mathbb{R}_{>0}$. ◦

Algorithm 6 Computation of H in (7.125).

Input: $\{\bar{\omega}_i\}_{i=1}^{n}$, τ_s, $K \in \{a \in \mathbb{N} : a > K > n + n_e + 1\}$.
Output: H.

Define the *virtual dynamics*

$$
\begin{aligned}
\bar{\Omega} &= \operatorname{diag}\left\{\begin{bmatrix} 0 & \bar{\omega}_1 \\ -\bar{\omega}_1 & 0 \end{bmatrix}, \ldots, \begin{bmatrix} 0 & \bar{\omega}_n \\ -\bar{\omega}_n & 0 \end{bmatrix}\right\}, && && (7.133a) \\
\bar{\Lambda} &= \left[\, 1 \;\; 0 \,\middle|\, \cdots \,\middle|\, 1 \;\; 0 \,\right], && && (7.133b) \\
\dot{\zeta}(t) &= \bar{\Omega}\zeta(t), && t \in \mathbb{R}_{\geq 0}, && (7.133c) \\
\psi(t) &= \bar{\Lambda}\zeta(t), && t \in \mathbb{R}_{\geq 0}, && (7.133d)
\end{aligned}
$$

with $\zeta(0) = \bar{\Lambda}'$.

$v(t) \leftarrow 0$, $t \in \mathbb{R}_{\geq 0}$

Collect samples $\mathscr{W}_d = \{w(k\tau_s)\}_{k=0}^{n+K-1}$, $\mathscr{H}_d = \{\eta_o(k\tau_s)\}_{k=0}^{n+K-1}$.

Apply Algorithm 3 to $(\mathscr{W}_d, \mathscr{H}_d)$ to obtain $\bar{M}_{\eta_o,w}$.

$\eta_o^{\delta}(t) \leftarrow \eta_o(t) - \bar{M}_{\eta_o,w}\, w(t)$

for $i = 1, \ldots, m$ **do**

  $v(t) \leftarrow e_i \psi(t)$, $t \in \mathbb{R}_{\geq 0}$

  ▷ Connect (7.133) to the i-th input channel.

  Collect samples $\mathscr{Z}_d = \{\zeta(k\tau_s)\}_{k=0}^{n+K-1}$, $\bar{\mathscr{H}}_d = \{\eta_o^{\delta}(k\tau_s)\}_{k=0}^{n+K-1}$.

  Apply Algorithm 3 to $(\mathscr{Z}_d, \bar{\mathscr{H}}_d)$ and obtain $\bar{M}_{\eta_o,\zeta}^{i}$ partitioned as

$$
\bar{M}_{\eta_o,\zeta}^{i} = \begin{bmatrix} \operatorname{Re}\left(W_{\eta_o v}^{i}(j\bar{\omega}_1)\right)' \\ \operatorname{Im}\left(W_{\eta_o v}^{i}(j\bar{\omega}_1)\right)' \\ \vdots \\ \operatorname{Re}\left(W_{\eta_o v}^{i}(j\bar{\omega}_n)\right)' \\ \operatorname{Im}\left(W_{\eta_o v}^{i}(j\bar{\omega}_n)\right)' \end{bmatrix}'. \tag{7.134}
$$

end for

Rearrange the columns of $\{\bar{M}_{\eta_o,\zeta}^{i}\}_{i=1}^{m}$ to obtain H as in (7.125).

Proof. The validity of Theorem 7.49 follows from the combination of all the previous results. Indeed, the full design procedure for any $\mu \in \mathscr{M}$ can now be illustrated, providing a constructive proof. Consider the data **K1**, **K2**, **K3** in Definition 7.30. If $\mu = 0$, *i.e.*, (7.62) is fully known, then Remark 7.34 provides a way to compute all the items. Conversely, if $\mu \neq 0$, it is necessary to perform an *online estimation* of all the data items. The procedure is summarized as follows.

1. **K1** is determined using Algorithm 3, whose validity is proven by Proposition 7.43.

2. **K2** is given by Algorithm 4, whose correctness is guaranteed by Lemma 7.45.

3. **K3**, *i.e.*, the polynomial annihilator of (7.62) with respect to the regulated output, is determined by first computing the matrix H in (7.125) using Algorithm 6, then using it in Algorithm 5 to obtain a minimal polynomial basis for the null space of $W_{\eta_o v}(s)$, as stated in Proposition 7.46.

Once the data **K1**, **K2**, and **K3** have been estimated, it is possible to build the control allocator (7.78), which solves Problem 7.29 for any $\gamma \in \mathbb{R}_{>0}$ as stated in Lemma 7.35. □

Example 7.50. *The following numerical example is offered to show in practice the implications of the results presented above. The dynamics* (7.62) *are now interpreted as the output feedback interconnection of a plant and a controller, whose state vectors are denoted by* $x(t) \in \mathbb{R}^{n_p}$ *and* $\varphi(t)\mathbb{R}^{n_c}$*, respectively. This situation is similar to the one addressed by the linear case of Section 7.6.1. Thus, the matrices in* (7.62) *are decomposed into*

$$A(\mu) = \begin{bmatrix} A_p(\mu) & B_p(\mu)C_c \\ B_cC_p(\mu) & A_c \end{bmatrix}, \qquad B(\mu) = \begin{bmatrix} B_p(\mu) \\ 0 \end{bmatrix}, \qquad P(\mu) = \begin{bmatrix} 0 \\ B_cQ \end{bmatrix}. \tag{7.135}$$

The nominal values for the plant are

$$A_p(0) = \begin{bmatrix} -1 & 1 & -2 \\ 0 & -1 & 1 \\ 1 & -1 & -1 \end{bmatrix}, \qquad B_p(0) = \begin{bmatrix} 1 & 1 \\ -1 & 0 \\ 2 & 1 \end{bmatrix}, \qquad C_p(0) = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}, \tag{7.136}$$

and $Q = \begin{bmatrix} -1 & 0 \end{bmatrix}$*. The exosystem is defined by* (7.63) *with* $n_e = 2$ *and* $\omega_1 = 1$.

By relying on a classic architecture, the design of the controller comprises an internal model unit Θ_1*, which contains a single copy (namely, as the number of regulated outputs) of the modes of the exosystem, and a dynamic stabilizer* Θ_2*, which contains an observer for the cascade of the plant and* Θ_1 *and a stabilizing feedback from the observer dynamics. In particular, a state-space realization of* Θ_1 *is provided by the matrices* $A_{\Theta_1} = S$*,* $C_{\Theta_1} = \begin{bmatrix} 1 & 0 \end{bmatrix}$ *and* $B_{\Theta_1} = I_2$*, while the interconnection between* Θ_1 *and the system* (7.136) *is achieved by considering a constant* squaring-down *matrix* $\Xi = \begin{bmatrix} 1 & 2 \end{bmatrix}'$*. The state-space description of the compensator* Θ_2 *is*

$$\dot{x}_{\Theta_2}(t) = (A_{\Theta_2} + B_{\Theta_2}K_{\Theta_2} - L_{\Theta_2}C_{\Theta_2})x_{\Theta_2}(t) + L_{\Theta_2}\eta_o(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.137}$$

together with the interconnection given by

$$u_{\Theta_1}(t) = y_{\Theta_2}(t) = K_{\Theta_2}x_{\Theta_2}(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.138}$$

where

$$A_{\Theta_2} = \begin{bmatrix} A_p(0) & B_p(0)\Xi C_{\Theta_1} \\ 0 & A_{\Theta_1} \end{bmatrix}, \qquad B_{\Theta_2} = \begin{bmatrix} 0 \\ I \end{bmatrix}, \qquad C_{\Theta_2} = \begin{bmatrix} C_p(0) & 0 \end{bmatrix}. \tag{7.139}$$

The gain matrices

$$L_{\Theta_2} = \begin{bmatrix} 10.85 & -3.51 & 12.25 & 1.40 & -0.17 \end{bmatrix}', \tag{7.140a}$$

$$K_{\Theta_2} = \begin{bmatrix} -8.09 & -4.41 & -3.40 & -8.07 & -0.92 \\ -0.41 & -0.89 & -0.44 & -0.92 & -1.41 \end{bmatrix}, \tag{7.140b}$$

have been obtained with an LQR approach. Finally, the regulated and performance outputs are defined as in (7.62b) *and* (7.62c)*, respectively, with*

$$C_1(\mu) = \begin{bmatrix} C_p(\mu) & 0 \end{bmatrix}, \qquad C_2(\mu) = \begin{bmatrix} 0 & C_c \end{bmatrix}, \qquad C_c = \begin{bmatrix} C_{\Theta_1} & 0 \end{bmatrix}, \tag{7.141}$$

and $D_{21} = 0$, $D_{12} = Q$, $D_{21} = I$, $D_{22} = 0$.

Denote by $y_c(t) \in \mathbb{R}^2$ *the output vector of the full regulator and by* $u(t) \in \mathbb{R}^2$ *the input vector of the plant under control. The interconnection of the plant, the regulator, and the allocator to be designed is given by*

$$u(t) = y_c(t) + \varsigma(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.142}$$

A polynomial annihilator of the nominal transfer matrix $W_{\eta_o v}(s)$ *is*

$$N_a(s) = \begin{bmatrix} -s^2 - s - 3 \\ -3s - 3 \end{bmatrix}, \qquad s \in \mathbb{C}, \tag{7.143}$$

hence, its state-space description is given by

$$A_a = \begin{bmatrix} -3 & -3 & -1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \qquad B_a = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad C_a = \begin{bmatrix} -1 & -1 & -3 \\ 0 & -3 & -3 \end{bmatrix}. \tag{7.144}$$

The optimization unit is designed as in (7.78) *with* $R = I$ *for multiple values of* γ.

The top graph of Figure 7.13 shows the time histories of $y_c(t)$, i.e., *the non-allocated control input, whereas the middle and bottom graphs display the time histories of* $u(t) = y_c(t) + \varsigma(t)$, i.e., *the optimally-allocated control input, for different values of* γ. *In the nominal case, the time histories of the output* $\eta_o(t)$ *induced by the allocated plant input is identical to that induced by the regulator output alone, hence they are not shown. The steady-state allocated input is the same in the middle and bottom graphs of Figure 7.13 but a larger* γ *yields a faster convergence to the optimal, allocated input.*

It is assumed that the matrices defined in (7.136) *for the nominal case are perturbed according to* $A(\mu) = A(0) + \Delta A$, $B(\mu) = B(0) + \Delta B$, *and* $C(\mu) = C(0) + \Delta C$, *with the norms of the* unstructured *perturbation matrices bounded by* 0.15. *The perturbation is such that the uncertain plant in closed loop with the nominal controller remains asymptotically stable. Figures 7.14 and 7.15 depict the continuous-time signals and their sampled data (denoted by the circles) employed to implement Algorithms 3 and 5. By*

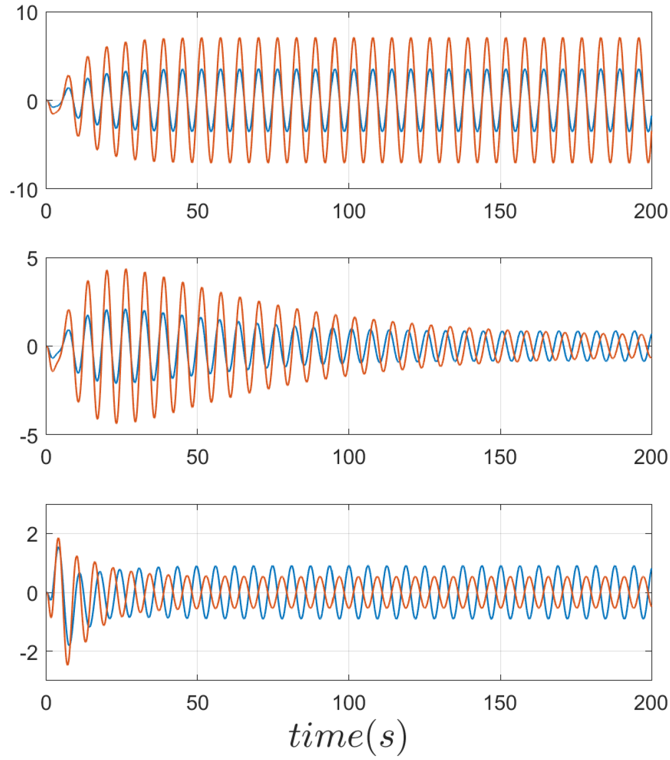

Figure 7.13: *(Nominal case)* Top graph: time histories of $y_c(t)$, *i.e.*, the non-allocated control input. Middle and bottom graphs: time histories of $u(t) = y_c(t) + \varsigma(t)$, *i.e.*, the optimally-allocated control input, with $\gamma = 0.01$ and $\gamma = 0.1$. The steady-state values in the middle and bottom graphs are the same, but a larger γ yields a faster convergence.

inspecting the top graph of Figure 7.14, it is worth stressing the difference between the continuous-time measurements of η_o *and the samples* $\eta_o^\delta(k\tau_s)$ *used in line 8 of Algorithm 6, in which the effect of the exosystem alone has been compensated. In the absence of measurement noise, these algorithms permit the exact computation of the matrices* $\bar{M}_{\eta_p,w}$ *and* $W_a(s)$ *on the basis of which the control allocator* (7.78) *is implemented, thus obtaining arbitrarily fast and optimal allocation even in the uncertain case.*

Consider again the nominal plant (7.136)*, driven by the same exosystem defined in the nominal case, and suppose that the controller has been designed according to* (7.139)–(7.140)*. Suppose further that the cost functional is defined as in* (7.6) *with* $\lambda(\cdot) = \mathrm{dz}_\sigma(\cdot)^2$ *where* dz_σ *denotes the (component-wise)* deadzone *function with levels prescribed by the vector* $\sigma \in \mathbb{R}^{q_p}$*. The purpose of the following numerical simulation is to compare alternative implementations of the definition of the dynamics in charge of generating the steady-state behavior of the allocator unit,* i.e.*, of the matrix* A_{IM} *in* (7.78a)*. The latter is first defined with the property that* $\mathrm{spec}(A_{IM}) = \mathrm{spec}(S)$*, in line with the constructions of the LQ setting above, and the achieved optimal behavior is compared with the control allocation induced by the selection* $\mathrm{spec}(A_{IM}) = \bigcup_{\kappa=1}^{4} \mathrm{spec}(\kappa S)$*, namely containing the modes of the exosystem together with a certain number of its higher-order harmonics. In the former case the output of the allocator is denoted by* $\varsigma(t)$*, whereas in the latter it is denoted by* $\varsigma^h(t)$*. The deadzone levels for the two components of the*

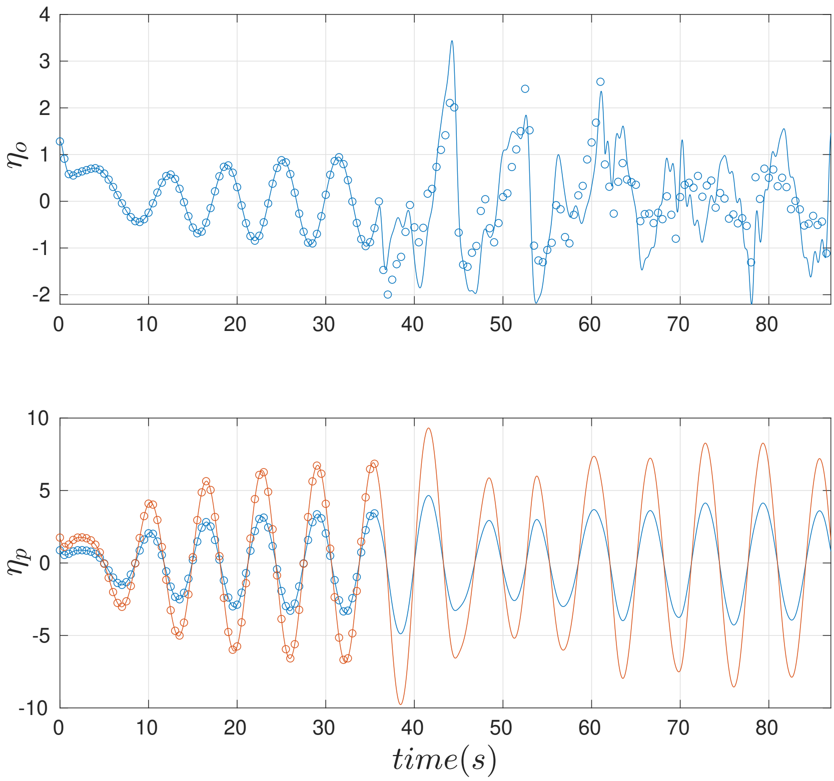

Figure 7.14: *(Uncertain case)* Regulated (top graph) and performance (bottom graph) outputs of the uncertain plant during the two-phase identification process. The markers on each plot identify the sampling times and the values used by the data-driven procedures. In the last part of the upper plot, the markers show the values of $\eta_o^\delta(k\tau_s)$ (see line 8 of Algorithm 6).

control input are $\sigma = \begin{bmatrix} 0.9 & 0.35 \end{bmatrix}'$. *Figure 7.16 depicts the behavior of the allocated control inputs at steady-state, together with the deadzone representation (dashed gray box). The presence of the higher-order harmonics permits to achieve a zero value of the cost functional, since at steady-state the inputs remain for all time within the dead-zone levels. Figure 7.17 shows the steady-state time histories of the non-allocated control input (top graph), with the time histories of control actions allocated via an allocator with* $\mathrm{spec}(A_{IM}) = \mathrm{spec}(S)$ *and in the presence of higher-order harmonics (middle and bottom graphs, respectively).* ■

7.6.3 Sampled-data dynamic allocation with full output invisibility

The previous results show how the design of dynamic allocators achieving full output invisibility for linear systems can be carried out in a systematic way, by splitting the allocator into three subsystems that are designed in sequence. Moreover, it has also been shown how the data required to design

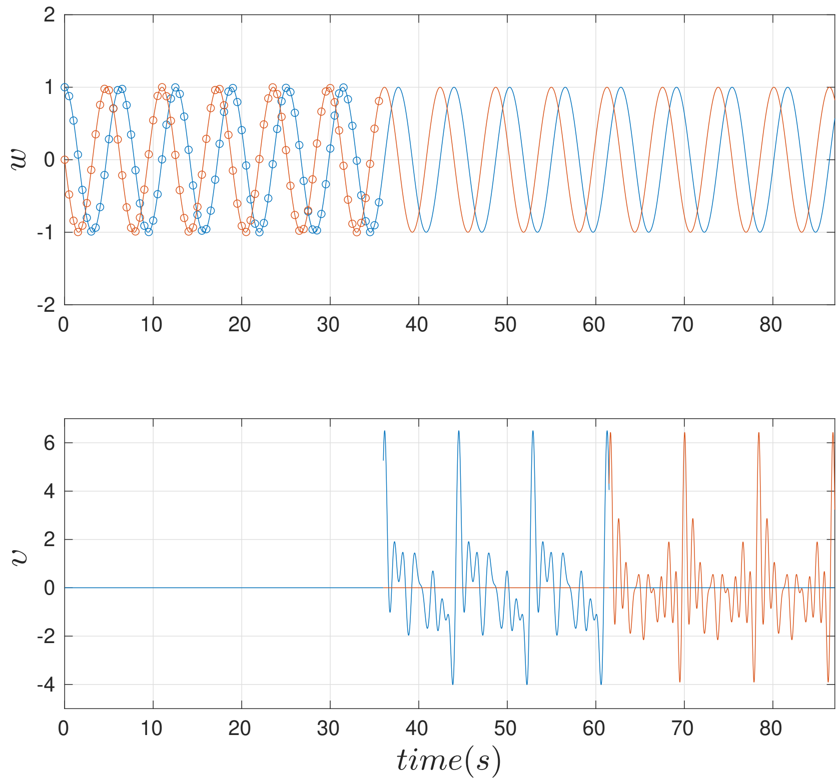

Figure 7.15: *(Uncertain case)* Time histories of the reference input (top graph) and of the controlled input (bottom graph) during the two-phase identification process. The markers identify the sampling times.

these subsystems can be estimated online from sampled measurements of the inputs and outputs of a preexisting control system. In practice, it is reasonable to expect that system (7.1) includes digital devices, *e.g.*, microcontrollers, as part of the control loop to implement the given control laws. Moreover, it is also reasonable to expect that the allocator itself, together with its estimation algorithms (if required), is to be implemented on a digital device connected to the control system. In this regard, some considerations have been made in Remark 7.36 but the behavior of an input allocation device of the form shown in Figure 7.12 implemented on a digital device remains an unexplored topic; the majority of the available literature on the subject is focused on the continuous-time design of dynamic allocators (see Section 7.4). The first question to address is whether one has to expect a different behavior when sampled-data devices are involved.

To investigate this, consider the continuous-time, linear time-invariant plant Θ given by

$$\dot{x}(t) = A_p x(t) + B_p u(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.145a}$$

$$y(t) = C_p x(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.145b}$$

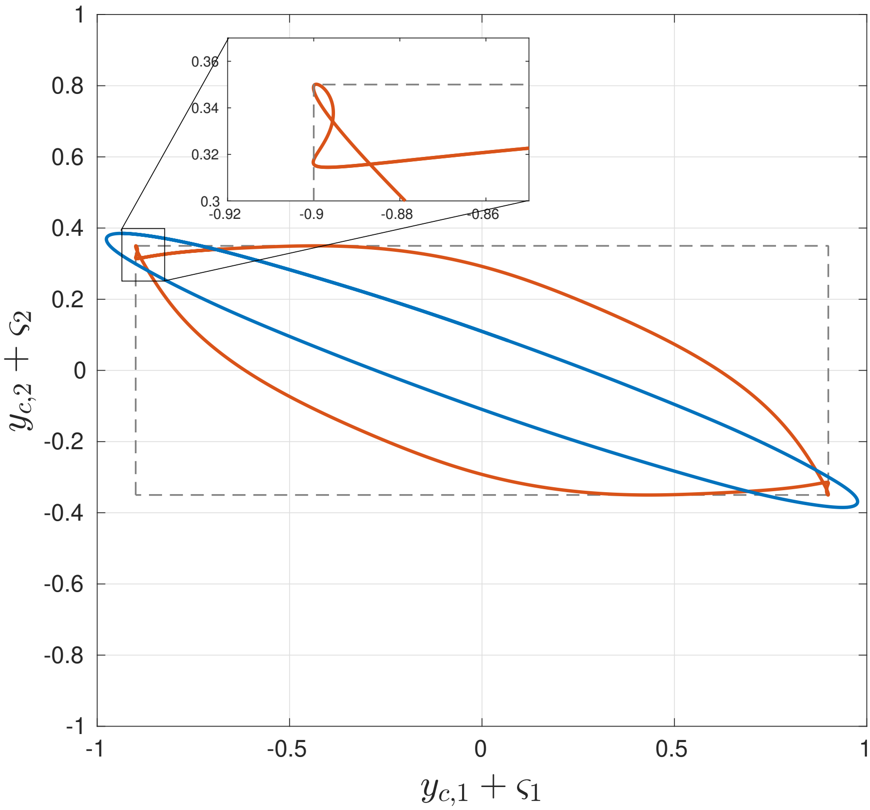

Figure 7.16: *(Deadzone)* Behavior of the allocated control inputs, together with the deadzone representation (dashed gray box). The control actions allocated via an allocator with $\mathrm{spec}(A_{IM}) = \bigcup_{\kappa=1}^{4} \mathrm{spec}(\kappa S)$ (red graph) achieve further amplitude reduction compared to an allocator with $\mathrm{spec}(A_{IM}) = \mathrm{spec}(S)$ (blue graph) (see Figure 7.17).

where $x(t) \in \mathbb{R}^{n_p}$ is the plant state vector, $u(t) \in \mathbb{R}^m$ is the plant input vector, and $y(t) \in \mathbb{R}^{q_o}$ is the plant output vector at time t, and $A_p \in \mathbb{R}^{n_p \times n_p}$, $B_p \in \mathbb{R}^{n_p \times m}$, and $C_p \in \mathbb{R}^{q_o \times n_p}$ are constant matrices. Let $\tau_s \in \mathbb{R}{\geq 0}$ be the sampling time. The plant (7.145) is part of a feedback control scheme comprising a sampled-data controller Γ described by the following discrete-time, linear time-invariant state-space model:

$$\varphi_{k+1} = A_c \varphi_k + B_c y_k + P_c r_k, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.146a}$$

$$\rho_k = C_c \varphi_k + D_c y_k + Q_c r_k, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.146b}$$

where $\varphi_k \in \mathbb{R}^{n_c}$ is the controller state vector, and $\rho_k \in \mathbb{R}^m$ is the controller output vector at time k. The matrices $A_c \in \mathbb{R}^{n_c \times n_c}$, $B_c \in \mathbb{R}^{n_c \times q_o}$, $P_c \in \mathbb{R}^{n_c \times n_r}$, $C_c \in \mathbb{R}^{m \times n_c}$, $D_c \in \mathbb{R}^{m \times q_o}$, and $Q_c \in \mathbb{R}^{m \times n_r}$ are constant. $r(t) \in \mathbb{R}^{n_r}$ is the reference input at time t, generated by an exosystem of the form (7.2). Let

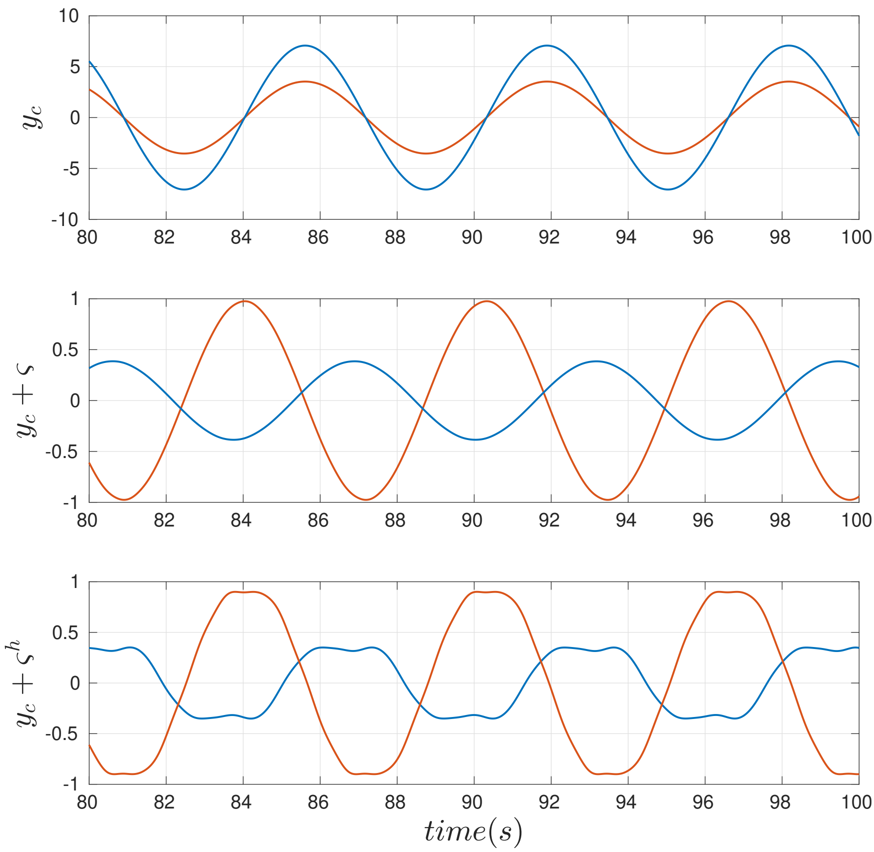

Figure 7.17: *(Deadzone)* Steady-state time histories of the non-allocated control input (top graph). Time histories of control actions allocated via an allocator with $\text{spec}(A_{IM}) = \text{spec}(S)$ (middle graph) and an allocator with $\text{spec}(A_{IM}) = \bigcup_{\kappa=1}^{4} \text{spec}(\kappa S)$, (bottom graph). Higher harmonics, enlarging the finite-dimensional function space used for optimization, make possible a further cost reduction (see Figure 7.16).

$A_{p,d} \in \mathbb{R}^{n_p \times n_p}$, $B_{p,d} \in \mathbb{R}^{n_p \times m}$, and $C_p \in \mathbb{R}^{q_o \times n_p}$ be constant matrices given by

$$A_{p,d} = e^{A_p \tau_s}, \tag{7.147a}$$

$$B_{p,d} = \int_0^{\tau_s} e^{A_p \sigma} B_p \, d\sigma, \tag{7.147b}$$

$$C_{p,d} = C_p. \tag{7.147c}$$

The matrices in (7.147) yield the following *discretized* state-space description of the plant Θ, equivalent to (7.145) at the sampling instants $t = k\tau_s$, $k \in \mathbb{Z}_{\geq 0}$:

$$x_{k+1} = A_{p,d}x_k + B_{p,d}u_k, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.148a}$$

$$y_k = C_{p,d}x_k, \qquad k \in \mathbb{Z}_{\geq 0}. \tag{7.148b}$$

Let $\{0,\omega_1,\ldots,\omega_h\}$ be angular frequencies such that there exist $T \in \mathbb{R}_{>0}$ and $\{k_i\}_{i=1}^{h} \subset \mathbb{N}$ such that

$$\omega_i = k_i \frac{2\pi}{T}, \qquad i = 1,\ldots,h. \tag{7.149}$$

The characteristics of the control system resulting from the feedback interconnection of (7.145) and (7.146), as well as the properties of the exosystem considered in this scenario, are stated by the following assumptions.

Assumption 7.51. *The pair (A_p, B_p) is controllable, and the pair (C_p, A_p) is detectable. The design of the discrete-time controller* (7.146) *is based on the discrete-time equivalent description* (7.148) *of the plant. Moreover, the feedback interconnection of* (7.145) *and* (7.146) *given by $u = \rho$ by means of a* zero-order hold *device on the plant input, and a* sample-and-hold *device on the plant output, each with sampling time τ_s, is well-posed and asymptotically stable.* ◦

Assumption 7.52. *The reference input r is generated by the exosystem E given by*

$$\dot{w}(t) = Sw(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.150a}$$

$$r(t) = C_e w(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.150b}$$

where $S \in \mathbb{R}^{n_e \times n_e}$ is such that

$$S_0 = 0, \tag{7.151a}$$

$$S_i = \begin{bmatrix} 0 & \omega_i \\ -\omega_i & 0 \end{bmatrix}, \qquad i = 1,\ldots,h, \tag{7.151b}$$

$$S = \operatorname{diag}(S_0, S_1, S_2, \ldots, S_h). \tag{7.151c}$$

The matrix $C_e \in \mathbb{R}^{n_r \times n_e}$ is constant and the pair (C_e, S) is observable. Moreover, sampled measurements of the reference input, namely r_k at time k, are available by means of a sample-and-hold *device with the same sampling time τ_s as in Assumption 7.51.* ◦

It is worth pointing out that Assumption 7.52 implies that the reference input r is periodic of period T.

Assumption 7.53. *There exists $K \in \mathbb{N}$ such that $T = K\tau_s$.* ◦

Although Assumption 7.53 is instrumental to simplify the following analysis, it can be relaxed in practice as shown later on.

Remark 7.54. *In a similar way the continuous-time, linear case presented in Section 7.6.1, the feedback* *control system given by* (7.145)–(7.148) *and* (7.150)–(7.151) *can be modeled as in* (7.1)*:*

$$u(t) = \rho(k\tau_s) + v(k\tau_s), \qquad t \in [k\tau_s, (k+1)\tau_s), k \in \mathbb{Z}_{\geq 0}, \tag{7.152a}$$

$$\chi(t) = \begin{bmatrix} x(t)' & \varphi(k\tau_s)' \end{bmatrix}', \qquad t \in [k\tau_s, (k+1)\tau_s), k \in \mathbb{Z}_{\geq 0}, \tag{7.152b}$$

$$\eta_p(t) = u(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.152c}$$

$$\eta_o(t) = y(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.152d}$$

where $\chi(t) \in \mathbb{R}^{n_p + n_c}$. ▲

Let $R \in \mathbb{R}^{m \times m}$ be a constant matrix such that $R = R'$, $R \succ 0$. The steady-state plant input is denoted by $\tilde{u}$. Consider the following cost functional:

$$J = \int_0^T \|\tilde{u}(t)\|_R^2 \, dt. \tag{7.153}$$

The dynamic input allocation problem for the feedback control system hereby defined is formulated as follows.

Problem 7.55 (Dynamic allocation with sampled output invisibility)**.** *Consider the sampled-data feedback control system given by* (7.145)–(7.148) *interconnected as in* (7.152)*, and* (7.150)–(7.151)*. Let Assumptions 7.3, 7.51, 7.52 and 7.53 hold. Find, if any, a* discrete-time input allocation device *(namely, an* allocator*) such that, when interconnected to the feedback loop according to*

$$\varpi = r, \qquad \varsigma = v, \tag{7.154}$$

yielding the plant input $u = \rho + \varsigma$*, ensures that the following requirements hold.*

(I) Sampled output invisibility: *the regulated outputs both in presence and absence of the allocator are such that*

$$\eta_o(k\tau_s) = \eta_o^a(k\tau_s), \qquad k \in \mathbb{Z}_{\geq 0}. \tag{7.155}$$

(O) Steady-state optimality: *the cost functional* (7.153) *is brought to its minimum value.*

∘

The full control scheme is depicted in Figure 7.18.

Remark 7.56. *The statement of Problem 7.55 is indeed a sampled-data variant of Problem 7.8. Although the two problems share the steady-state optimality requirement, what differentiates them is the full output invisibility one. From a continuous-time perspective,* i.e.*, the time domain of the plant* (7.145)*, Problem 7.8 requires the effect of the allocator to be invisible in the plant output at all times. Conversely, Problem 7.55 requires the effect of the allocator to be invisible only at the sampling instants. Conceptually,*

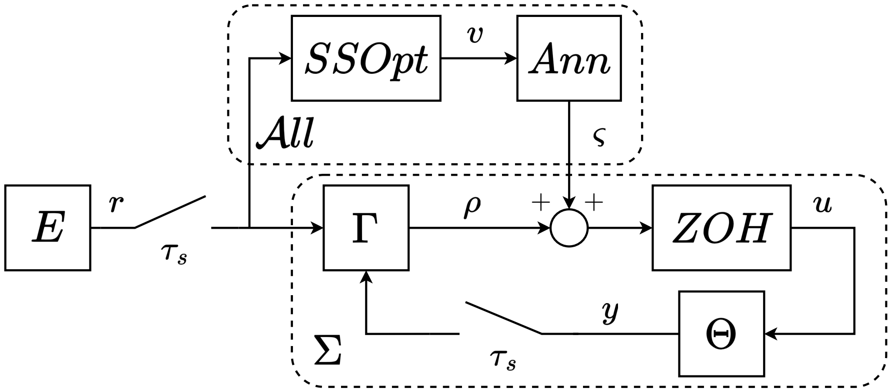

Figure 7.18: Sampled-data feedback control system given by (7.145)–(7.148) interconnected as in (7.152), and (7.150)–(7.151); the discrete-time allocator is enclosed in the $\mathscr{A}\ell\ell$ block.

this difference is motivated by the fact that the control system is implemented using digital devices that are now modeled with discrete-time dynamics; this fact reasonably leads to the formulation of the full output invisibility requirement given in Problem 7.55. ▲

This scenario, completely new in the literature to the best of the author's knowledge, poses two questions which are the subject of the following investigation: how the structure of the allocator introduced in Section 7.6.2 (see Figure 7.12) has to be adapted to a sampled-data design, and how the allocator action affects the plant output between the sampling instants [88].

Example 7.57. *The necessity of a discrete-time allocator design approach to solve Problem 7.55 is highlighted by the following numerical example. Consider the continuous-time, linear time-invariant plant of the form* (7.145) *given by*

$$A_p = \begin{bmatrix} -8 & 2 & -9 \\ 9 & -12 & 8 \\ 5 & -7 & -3 \end{bmatrix}, \qquad B_p = \begin{bmatrix} 4 & 7 \\ -10 & -6 \\ 10 & -7 \end{bmatrix}, \qquad C_p = \begin{bmatrix} -7 & -4 & 1 \end{bmatrix}. \tag{7.156}$$

An inspection of (7.156) *reveals that the plant is asymptotically stable since* $\operatorname{spec}(A_p) \subset \mathbb{C}_{<0}$, *that the pair* (A_p, B_p) *is controllable, and that the pair* (C_p, A_p) *is observable. Let*

$$v(t) = 5\sin(10t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.157}$$

where $v(t) \in \mathbb{R}$ *since* (7.156) *has degree of redundancy* $m_a = 1$. *Then, design a* polynomial annihilator *for* (7.156) *following the procedure described in Proposition 7.46. Fix the sampling time* $\tau_s = 0.1\,\mathrm{s}$. *Let* u_a *be the input and* ς *be the output of the resulting annihilator; interconnect the plant* (7.156) *and the*

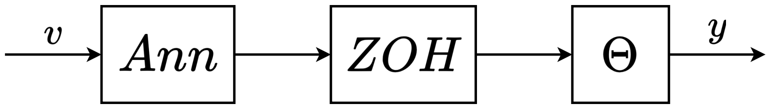

Figure 7.19: Interconnection of the continuous-time annihilator (represented by the Ann block) with the continuous-time, linear time-invariant plant (7.156) by means of a zero-order hold device.

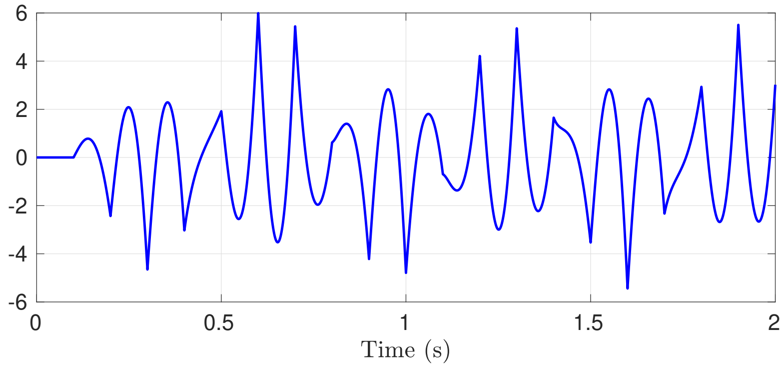

Figure 7.20: Output of the continuous-time, linear time-invariant plant (7.156) in presence of its polynomial annihilator subject to the sinusoidal input (7.157).

annihilator such that

$$u_a(t) = v(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.158}$$

$$u(t) = \varsigma(k\tau_s), \qquad t \in [k\tau_s, (k+1)\tau_s),\ k \in \mathbb{Z}_{\geq 0}, \tag{7.159}$$

i.e., *the annihilator output is fed to the plant* (7.156) *by means of a zero-order hold device with sampling time* τ_s. *The situation hereby described is depicted in Figure 7.19. The output of* (7.156) *in presence of its annihilator subject to the sinusoidal input* (7.157) *is shown in Figure 7.20. The interconnection* (7.158) *of the plant* (7.156) *with its annihilator by means of a zero-order hold device represents the straightforward implementation of the continuous-time annihilator designed as in Proposition 7.46 on a digital device. As shown in Figure 7.20, not only is the plant output not equal to zero between the sampling instants but it is also not equal to zero at the sampling instants.* ■

Example 7.57 suggests that the entire annihilator design approach should be revised in a discrete-time setting when system (7.1) involves sampled-data devices. Let z be a variable taking values in $\mathbb{C}$, and $W^d_{yu}(z) \in \mathbb{C}^{q_o \times m}$ be the transfer function of the discretized plant (7.148). The approach outlined in [51] and further characterized in Proposition 7.46 is extended to the discrete-time case as follows.

Proposition 7.58. *Let* $N_a(z) \in \mathbb{C}^{m \times m_a}$ *be a polynomial matrix whose columns form a minimal polynomial basis for the null space of* $W^d_{yu}(z)$*, and* $d_a(z)$ *be a monic Schur polynomial with degree larger than* *that of each element of* $N_a(z)$. *Define*

$$W_a(z) = N_a(z) d_a^{-1}(z). \tag{7.160}$$

Then, it holds that

$$W_{yu}^d(z) W_a(z) = 0, \qquad z \in \mathbb{C} \setminus (\mathscr{P}(W_{yu}^d(z)) \cup \mathscr{P}(W_a(z))). \tag{7.161}$$

Furthermore, given any minimal realization $(A_a, B_a, C_a, 0)$ *of* $W_a(z)$, *the system*

$$\vartheta_{k+1} = A_a \vartheta_k + B_a v_k, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.162}$$

$$\varsigma_k = C_a \vartheta_k, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.163}$$

where $\vartheta_k \in \mathbb{R}^{n_a}$ *is the state vector at time* k, *and* $v_k \in \mathbb{R}^{m_a}$, *is a* discrete-time annihilator *for plant* (7.148).

◦

Remark 7.59. *Suppose that* $W_{yu}^d(z)$ *admits a left coprime polynomial factorization [39, §7.8] such that*

$$W_{yu}^d(z) = D_{p,d}(z)^{-1} N_{p,d}(z), \tag{7.164}$$

where $D_{p,d}(z)$, $N_{p,d}(z)$ *are polynomial matrices. Then, the polynomial matrix* $N_a(z)$ *in Proposition 7.58 may be selected among the solutions of*

$$N_{p,d}(z) N_a(z) = 0. \tag{7.165}$$

▲

Remark 7.60. *The discrete-time system* (7.162) *is such that* (7.161) *holds. Hence, the discrete-time annihilator* (7.162) *satisfies the sampled output invisibility requirement in Problem 7.55. However, the effect of the allocator on the plant output between the sampling instants is not characterized.* ▲

The proof of Proposition 7.58 follows a reasoning similar to that of Proposition 7.46, hence it is omitted. To describe the proposed steady-state optimizer, some preliminary definitions are in order. Recall Assumption 7.53 and consider the following cost functional, obtained as a sampled-data variant of (7.153):

$$J_d = \tau_s \sum_{k=0}^{K-1} \tilde{u}_k' R \tilde{u}_k. \tag{7.166}$$

S_d is the sampled exogenous dynamics given by

$$S_d = e^{\tau_s S}. \tag{7.167}$$

Let $n_{IM} = m_a n_e$ and

$$A_{IM} = I_{m_a} \otimes S_d, \tag{7.168}$$

with $A_{IM} \in \mathbb{R}^{n_{IM} \times n_{IM}}$. Furthermore, let $(A_\varphi, B_\varphi, C_\varphi, 0)$ be a minimal realization of the feedback loop having ρ as output and note that $B_\varphi = \begin{bmatrix} B_r & B_v \end{bmatrix}$ since the feedback loop has two exogenous inputs, namely r and v. Select $C_{IM} \in \mathbb{R}^{m_a \times n_{IM}}$ such that the pair (C_{IM}, A_{IM}) is observable and solve the following Sylvester equations for $\Pi_{\bar{M}} \in \mathbb{R}^{(n_p+n_c)\times n_e}$, $\Pi_{\bar{N}} \in \mathbb{R}^{n_a \times n_{IM}}$:

$$\Pi_{\bar{M}} S_d = A_\varphi \Pi_{\bar{M}} + B_r C_e, \tag{7.169a}$$

$$\Pi_{\bar{N}} A_{IM} = A_a \Pi_{\bar{N}} + B_v C_{IM}, \tag{7.169b}$$

yielding (see, again, Appendix C and references therein)

$$\bar{M} = C_\varphi \Pi_{\bar{M}}, \tag{7.170a}$$

$$\bar{N} = C_a \Pi_{\bar{N}}, \tag{7.170b}$$

with $\bar{M} \in \mathbb{R}^{m \times n_e}$, $\bar{N} \in \mathbb{R}^{m \times n_{IM}}$.

To simplify the notation, let

$$Z = S_d^{n_e} O(C_e, S_d)^\dagger. \tag{7.171}$$

Finally, define in addition

$$\bar{R} = I_K \otimes R, \qquad \bar{R} \in \mathbb{R}^{mK \times mK}, \tag{7.172a}$$

$$L = \begin{bmatrix} L_1 & L_2 \end{bmatrix} = \begin{bmatrix} \bar{M} S_d Z & \bar{N} \\ \bar{M} S_d^2 Z & \bar{N} A_{IM} \\ \vdots & \vdots \\ \bar{M} S_d^K Z & \bar{N} A_{IM}^{K-1} \end{bmatrix}, \qquad L \in \mathbb{R}^{mK \times (n_e n_r + n_{IM})}, \tag{7.172b}$$

and

$$F_\zeta = -(L_2' \bar{R} L_2)^{-1} L_2' \bar{R} L_1, \qquad F_\zeta \in \mathbb{R}^{n_{IM} \times n_e n_r}, \tag{7.173a}$$

$$F_{IM} = -(L_2' \bar{R} L_2)^{-1} L_2' \bar{R} L_2 A_{IM}, \qquad F_{IM} \in \mathbb{R}^{n_{IM} \times n_{IM}}, \tag{7.173b}$$

$$A_\zeta = \begin{bmatrix} 0 & I_{n_r} & 0 & 0 & \cdots & 0 \\ 0 & 0 & I_{n_r} & 0 & \cdots & 0 \\ & & \cdots & & & \\ 0 & 0 & 0 & \cdots & 0 & I_{n_r} \\ 0 & & \cdots & & & 0 \end{bmatrix}, \qquad A_\zeta \in \mathbb{R}^{n_e n_r \times n_e n_r}, \tag{7.173c}$$

$$B_\zeta = \begin{bmatrix} 0 & \cdots & 0 & I_{n_r} \end{bmatrix}', \qquad B_\zeta \in \mathbb{R}^{n_e n_r \times n_r}. \tag{7.173d}$$

The proposed steady-state optimizer is given by

$$\psi_{k+1} = (A_{IM} + F_{IM})\psi_k + F_\zeta \zeta_k, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.174a}$$

$$\zeta_{k+1} = A_\zeta \zeta_k + B_\zeta r_k, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.174b}$$

$$v_k = C_{IM}\psi_k, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.174c}$$

where $\psi_k \in \mathbb{R}^{n_{IM}}$ is the state vector of the steady-state optimizer at time k, and $\zeta_k \in \mathbb{R}^{n_e n_r}$ is a buffer of samples of the reference input r. The properties of the discrete-time allocator formed by the interconnection of (7.162) and (7.174) are summarized in the following result.

Theorem 7.61. *Consider the sampled-data feedback control system given by* (7.145)–(7.148) *interconnected as in* (7.152), *and* (7.150)–(7.151). *Let Assumptions 7.3, 7.51, 7.52 and 7.53 hold. Then, the allocator given by* (7.162), (7.174) *and interconnected to the feedback loop such that*

$$\varpi = r, \qquad \varsigma = v, \tag{7.175}$$

solves Problem 7.55. ◦

Proof. The discrete-time annihilator (7.162) ensures that the sampled output invisibility requirement holds by construction, as stated in Proposition 7.58. The motivation at the heart of the structure of (7.174) is that, at each time k, ψ must be steered towards an optimal trajectory $\psi^\star$ along which the cost functional (7.153) is minimized; the formulation of Problem 7.55 guarantees that such a trajectory exists.

Denote the steady-state controller output by $\tilde{\rho}$ and let

$$\pi_{k,i} = \tilde{\rho}_{k+i} + \bar{N}\psi_{k+i}, \qquad i = 1, \dots, K,\ k \in \mathbb{Z}_{\geq 0}, \tag{7.176a}$$

$$\pi_k = \begin{bmatrix} \pi'_{k,1} & \pi'_{k,2} & \cdots & \pi'_{k,K} \end{bmatrix}', \qquad k \in \mathbb{Z}_{\geq 0}. \tag{7.176b}$$

Equations (7.166), (7.176) yield

$$J_d = \tau_s \pi'_k \bar{R} \pi_k, \qquad k \in \mathbb{Z}_{\geq 0}. \tag{7.177}$$

Recall (7.170) and note that, by embedding a copy of the modes of the exosystem (namely, embedding a *steady-state generator*), the steady-state optimizer can compute π_k at each time k by forward-propagating the current value of the state vector ψ over an interval of K steps according to

$$\psi_{h+1} = A_{IM}\psi_h + \delta, \qquad h = k,\ \delta \in \mathbb{R}^{n_{IM}}, \tag{7.178a}$$

$$\psi_{h+1} = A_{IM}\psi_h, \qquad h = k+1, \dots, k+K-1, \tag{7.178b}$$

where it has been assumed that the correction term δ at time h has been designed to steer the state ψ towards its optimal steady-state trajectory $\psi^\star$. To compute samples of $\tilde{\rho}$ in (7.176), a measurement of the state of the exosystem w_k is required; since the optimizer operates in feedforward from the reference input r and Assumption 7.52 holds, a buffer ζ of the form

$$\zeta_k = \begin{bmatrix} r'_{k-n_e} & r'_{k-n_e+1} & \cdots & r'_{k-1} \end{bmatrix}' \tag{7.179}$$

is enough to compute

$$w_k = Z\zeta_k = ZO(C_e, S_d)\, w_{k-n_e}, \qquad k \in \mathbb{Z}_{\geq 0}. \tag{7.180}$$

This justifies the structure of $L1$ in (7.172b), yielding

$$\pi_k = L\begin{bmatrix} \zeta'_k & \psi'_k A'_{IM} + \delta' \end{bmatrix}', \qquad k \in \mathbb{Z}_{\geq 0}. \tag{7.181}$$

Given π_k in (7.181), the value of the correction term δ that brings the cost functional (7.153) to its minimum value is determined by imposing that the gradient of the cost functional (7.166) with respect to δ is equal to zero. Recalling (7.177), this implies

$$2\tau_s(L'_2\bar{R}L_2\delta + L'_2\bar{R}L_1\zeta_k + L'_2\bar{R}L_2 A_{IM}\psi_k) = 0, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.182}$$

yielding what is found in (7.174), that is

$$\delta = -(L'_2\bar{R}L_2)^{-1}(L'_2\bar{R}L_1\zeta_k + L'_2\bar{R}L_2 A_{IM}\psi_k), \qquad k \in \mathbb{Z}_{\geq 0}. \tag{7.183}$$

□

Remark 7.62. *As expressed by* (7.174), δ *is computed and applied at every time* k *instead of every* K *steps as assumed in* (7.178). *However, this does not invalidate the correctness of the procedure because a single update is necessary to drive* ψ *to the optimal trajectory* $\psi^\star$, *namely*

$$F_\zeta\zeta_k + F_{IM}\psi_k = 0, \qquad k > n_e + 1, \tag{7.184}$$

and (7.174a) *generates the free response of the discretized exosystem. Moreover, if the closed-loop system goes to steady-state in* N_c *time steps, and the allocator also does in* N_{all} *time steps, then the allocated control system goes to steady-state in* $\max\{N_c, N_{all}\}$ *time steps.* ▲

Proposition 7.63. *Let*

$$\tilde{u}_k = \tilde{\rho}_k + \bar{N}\psi_k, \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.185a}$$

$$\psi^\star_k = \arg\min_{\psi} J_d. \tag{7.185b}$$

If the pair $(\bar{N}, A_{IM})$ *is observable, then the proposed allocator is such that*

$$\psi_k = \psi^\star{}_{\bmod (k,K)}, \qquad k \geq n_e + 1. \tag{7.186}$$

∘

Recall that discrete-time systems are termed *deadbeat*, namely such that their transient response goes to zero in a finite number of steps, if and only if all their poles are in zero.

Corollary 7.64. *Suppose that the closed-loop given by* (7.145)–(7.148) *interconnected as in* (7.152) *and the annihilator* (7.162) *are deadbeat, then the allocated plant input converges to its optimal trajectory in a finite number of steps.* ∘

Remark 7.65. $\psi^\star$ *is uniquely defined since, in this setting, the cost functional* (7.166) *is strictly convex with respect to* $\tilde{u}$. ▲

Remark 7.66. *Some observations about the space of admissible control laws are in order here. In [89], it is observed that these belong to a prescribed set (namely, the modes of the exosystem), which however remains an infinite-dimensional space. Instead, the admissible allocator actions in the present context, hence the optimal control input with respect to* (7.153)*, lie in a finite-dimensional space because in this setting the controls are sampled,* i.e., *piecewise-constant periodic signals.* ▲

Remark 7.67. *As anticipated above, Assumption 7.53 may be relaxed in practice provided a few observations are discussed. To begin with, the intrinsic mismatch between the continuous time period* T *and any positive integer multiple of the sampling time* τ_s *prevents the possibility of covering exactly one period of the reference input with an integer number of discrete-time samples. This in turn implies that the correction term* δ *in* (7.178) *exhibits a persistent oscillation for which* ψ *cannot converge to* $\psi^\star$*. As a further consequence, also the cost functional* (7.153) *does not settle to a constant value, oscillating instead around the minimal steady-state cost. Finally, note that* K *can still be chosen according to*

$$K = \begin{cases} \left\lfloor \frac{T}{\tau_s} \right\rfloor, & \text{if } \frac{T}{\tau_s} \le \frac{1}{2}, \\ \left\lceil \frac{T}{\tau_s} \right\rceil, & \text{otherwise,} \end{cases} \tag{7.187}$$

in order to reduce the amplitude of such oscillations, as illustrated in the following. ▲

Remark 7.68. *In* (7.174)*, the update law proposed for* ψ *effectively steers it towards the optimal trajectory* $\psi^\star$*, as the proof of Theorem 7.61 shows, without any assumption about the initial states of the optimizer unit. In practical applications, however, it might be desirable to start updating* ψ *only after the buffer* ζ *has been filled with meaningful samples of the reference input* r, i.e.

$$\psi_{k+1} = \begin{cases} A_{IM}\psi_k + F_\zeta \zeta_k + F_{IM}\psi_k, & \text{if } k \ge n_e, \\ A_{IM}\psi_k, & \text{otherwise,} \end{cases} \tag{7.188}$$

to prevent potentially unwanted transients of the optimizer from being added to the control input. ▲

Remark 7.69. *In* (7.174)*, after* ζ *has been filled with* n_e *samples of* r*, only one state update is necessary for* ψ *to reach* $\psi^\star$*. In practical applications, one might prefer a softer allocation action trading more time steps necessary to reach the optimal trajectory for a less intense allocator output. A possible alternative* *implementation is*

$$\psi_{k+1} = A_{IM}\psi_k + \alpha(F_\zeta \zeta_k + F_{IM}\psi_k), \qquad k \in \mathbb{Z}_{\geq 0}, \tag{7.189}$$

where $\alpha \in (0,1)$. ▲

Remark 7.70. *In* (7.179)*, the buffer* ζ *is designed to store* n_e *samples of the reference input, namely as many as the order of the exosystem. In practice,* $\bar{k} < n_e$ *may be enough, where* $\bar{k} \in \mathbb{N}$ *is the smallest integer such that* $O_{\bar{k}}(C_e, S_d)$ *has full column rank.* ▲

Example 7.71. *Consider the following state-space representation for system* (7.145)*:*

$$A_p = \begin{bmatrix} -3 & 2 & -9 \\ 9 & -7 & 8 \\ 5 & -7 & 2 \end{bmatrix}, \qquad B_p = \begin{bmatrix} 4 & 7 \\ -10 & -6 \\ 10 & -7 \end{bmatrix}, \qquad C_p = \begin{bmatrix} -7 & -4 & 1 \end{bmatrix}. \tag{7.190}$$

In this framework, the objective of the preexisting feedback control system consists in tracking the sinusoidal reference input $r(t)$ *with the plant output* $y(t)$*. The reference input has* $h = 2$ *frequency components and is generated by an exosystem of the form* (7.150)*, which satisfies Assumption 7.52 with*

$$T = \frac{2\pi}{\omega_1} = 1\,\mathrm{s}, \qquad \omega_2 = \frac{\omega_1}{2}, \qquad C_e = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 \end{bmatrix}, \qquad w(0) = \begin{bmatrix} 1 & 0 & 5 & 0 & 2 \end{bmatrix}'. \tag{7.191}$$

An inspection of (7.190) *reveals that it satisfies Assumption 7.3 with* $m_a = 1$ *and that the pair* (A_p, B_p) *is controllable, as is the pair* $(A_p, B_{p_{[:,1]}})$ *that is relative to the first input component alone. Thus, the regulator is designed to act only on the first input, setting the second to zero to better highlight the allocation action. The sampling time is* $\tau_s = 0.1\,\mathrm{s}$*. The regulator solves the error-feedback regulation problem applying the Internal Model Principle [83], ensuring that the closed-loop system is asymptotically stable and that the steady-state tracking error is equal to zero at the sampling instants. The regulator is of the form given in* (7.146) *and, together with the plant* (7.190)*, satisfies Assumption 7.51. The discrete-time annihilator* (7.162) *is designed as stated in Proposition 7.58. The parameters of the steady-state optimizer* (7.174) *are*

$$R = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}, \qquad C_{IM} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}, \qquad K = \frac{T}{\tau_s} = 10, \tag{7.192}$$

which satisfy Assumption 7.53.

The performance of the allocated scheme is compared with the reference *one yielded by the control scheme without the allocator. Figure 7.21 shows that the proposed allocator solves Problem 7.55, since it brings the steady-state cost to a value that is lower than the one reached with the reference scheme. The initial transients in the allocated time history are almost entirely due to the regulator and not relevant to the analysis, since what matters is the end value, nonetheless it is interesting to note how the time history of the allocated cost maintains lower values than those of the reference one even during transients. Figure 7.22 shows how the allocator applies the optimal correction to its internal state in just one time*

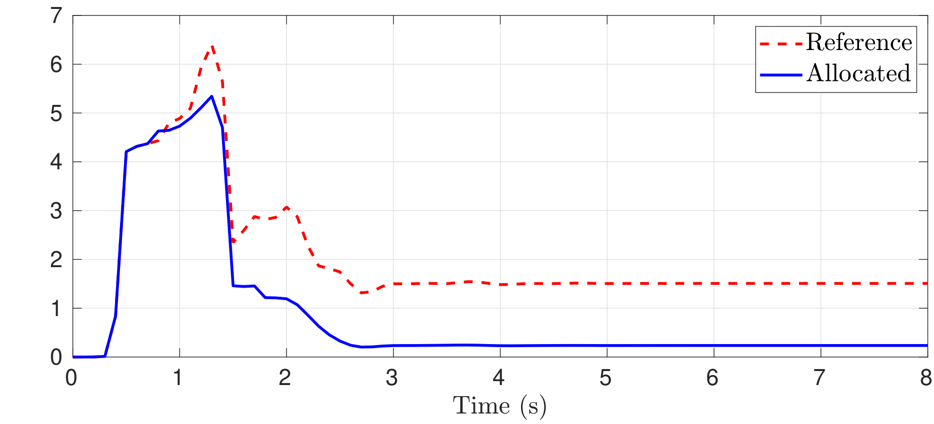

Figure 7.21: Continuous-time steady-state cost with allocation (continuous) versus steady-state cost without allocation (dashed); the final value is the relevant one.

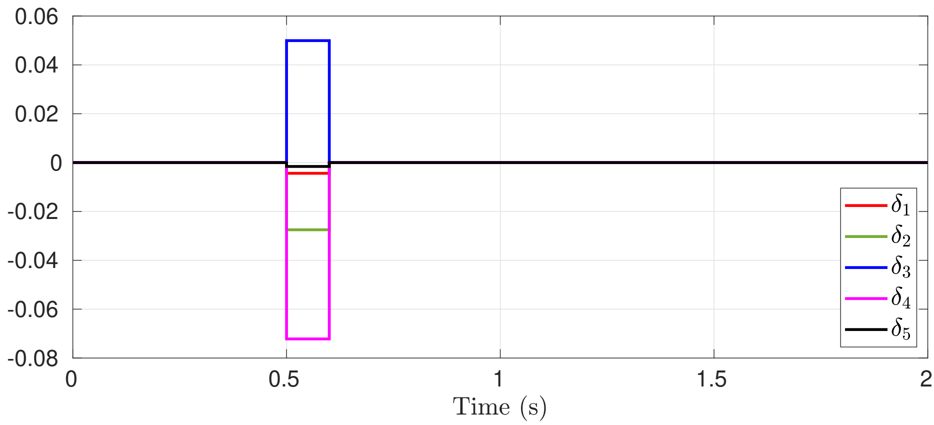

Figure 7.22: Steady-state optimizer correction term δ.

step, as anticipated in the proof of Theorem 7.61, and how then δ remains equal to zero. Figure 7.23 shows the difference between the continuous-time tracking errors of the allocated and non-allocated schemes, respectively. The time histories of the tracking error differences show that they exhibit different tracking errors at all times except the sampling ones. This agrees with the sampled output invisibility requirement of Problem 7.55, and proves that the discrete-time allocator induces a behavior which is invisible to the discrete-time control system but is visible onto the continuous-time plant output. In this example, the ripple trend depends on both the reference signal's amplitude and the sampling time being relatively long, but also on the way in which the control inputs are weighted by R. Figure 7.25 shows what the optimizer does if implemented as explained in Remark 7.69. In that case, more than one time step is required to converge but intermediate corrections are less intense in terms of their amplitude. Fig. 7.26 shows a similar simulation performed on a control scheme where T is such that $K \notin \mathbb{N}$, i.e. *relaxing Assumption 7.53 as anticipated, but still picked according to* (7.187). *Both the steady-state cost and the correction term δ never converge but start oscillating, as stated in Remark 7.67. The other*

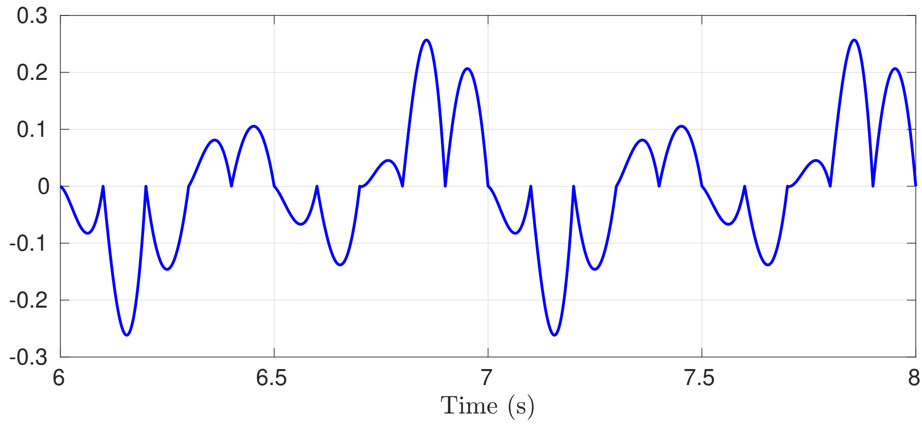

Figure 7.23: Difference between the continuous-time tracking errors with and without control allocation, respectively.

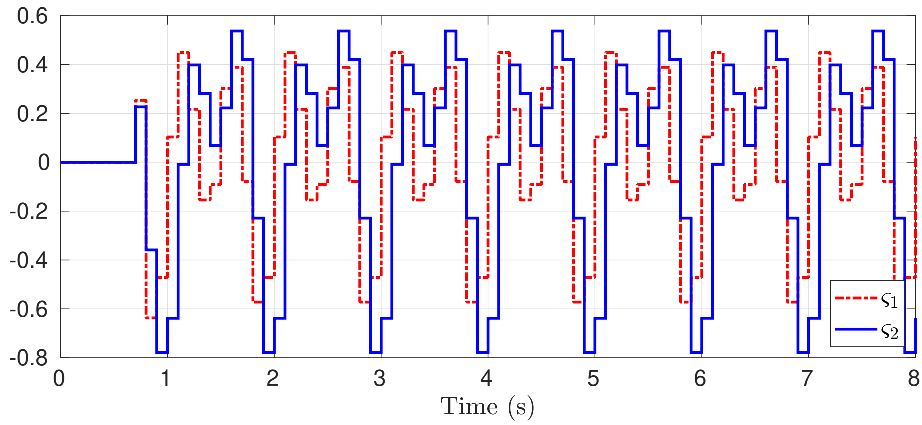

Figure 7.24: Discrete-time allocator output ς.

considerations made above are still valid in this case. ■

7.7 Design of a dynamic current allocator for the TCV shape controller

The design of a dynamic coil current allocator for the TCV shape controller requires an inspection of the preexisting control scheme, to clarify which among the novel formulations presented in Section 7.6 is the most appropriate. Subsequently, the selected solution is adapted to the specific case of the TCV shape controller, accounting for any differences arising with respect to the previously analyzed scenarios.

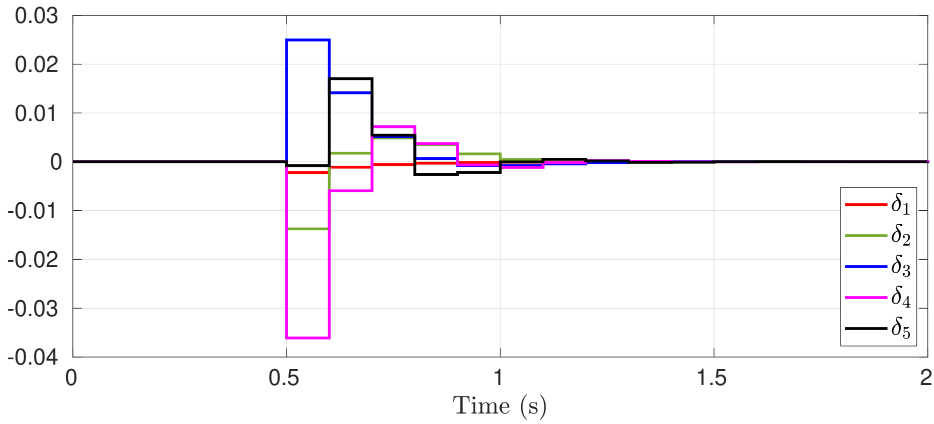

Figure 7.25: Optimizer correction term δ smoothened by $\alpha = 0.5$ (see Remark 7.69).

7.7.1 Problem statement

Consider the simplified control scheme given in Figure 7.27, which summarizes the interconnection of the main control subsystems with the TCV plant. Before going further, it is instrumental to anticipate that every dynamic model considered in the following is continuous-time, linear time-invariant; this is possible thanks to the powerful real-time equilibrium reconstruction codes that are part of the SCD control suite, which allow control engineers to work on a linearized model of the plant around the prescribed shape equilibrium for every shot (see Sections 6.1, 6.2 and 7.2).

The central block P_h represents the dynamics of the plasma and coils, described by the following equations:

$$\dot{x}_{ph}(t) = A_{ph}x_{ph}(t) + B_{ph}u_h(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.193a}$$

$$y(t) = C_{ph}x_{ph}(t) + D_{ph}u_h(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.193b}$$

where $x_{ph}(t) \in \mathbb{R}^{275}$ is the state vector at time t, containing the currents in the active circuits $I_a(t) \in \mathbb{R}^{19}$ and in the filaments used to represent the vessel, namely $I_u(t) \in \mathbb{R}^{256}$; $u_h(t) \in \mathbb{R}^{19}$ is the plant input, containing the voltages $V_a(t) \in \mathbb{R}^{19}$ to be applied to the active circuits; $y(t) = \begin{bmatrix} y_h(t)' & y_{sh}(t)' & I_{EF}(t)' \end{bmatrix}'$ is the plant output, containing the controlled variables $y_h(t) \in \mathbb{R}^{24}$, namely the plasma current and its radial and vertical positions as well as the currents of the PF coils, the shape output $y_{sh}(t) \in \mathbb{R}^{q_{sh}}$ whose dimension depends on the design of the shape controller for the desired equilibrium, and the measurements $I_{EF}(t) \in \mathbb{R}^{16}$ of the currents of the EF coils.

The inner control loop comprises also the feedback controller C_h, informally termed *hybrid* due to its composition of both analog and digital devices [123]. The role of C_h is to compute the active voltages V_a which are to be applied to the PF coils to track the reference input r_h, taking care of the vertical and radial stabilization of the plasma and controlling the current flow in the PF coils. Its state-space

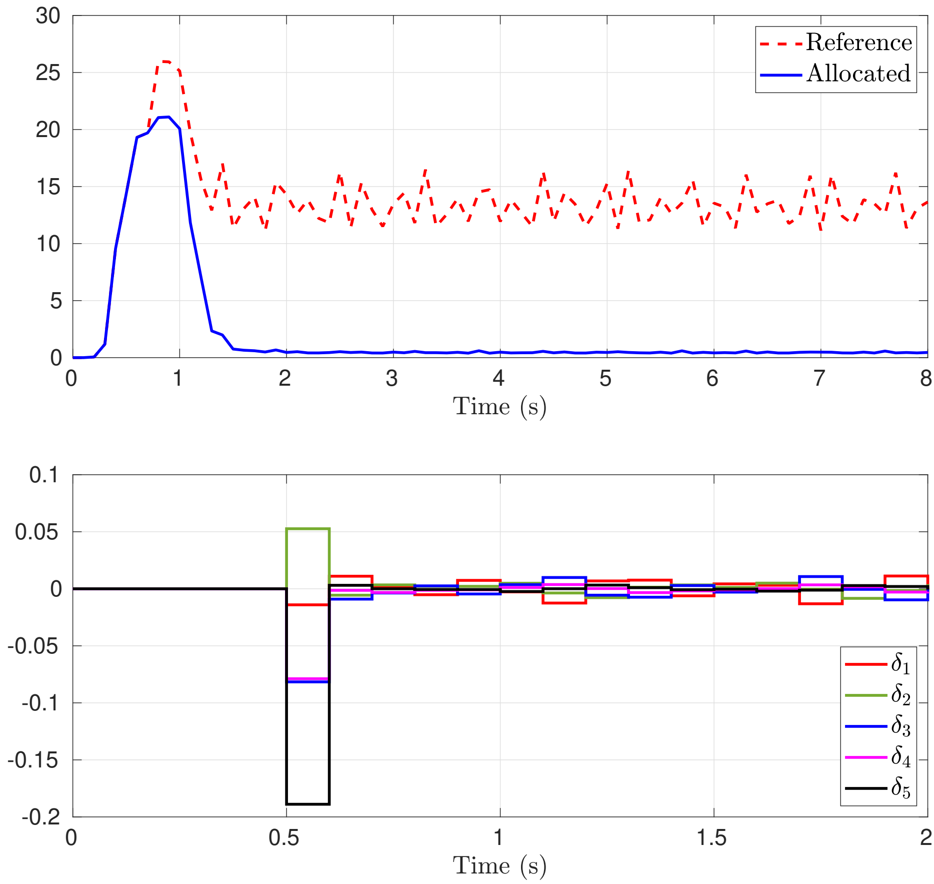

Figure 7.26: Continuous-time steady-state cost (top) and optimizer correction term δ (bottom) in a case where $K = \frac{T}{\tau_s} \notin \mathbb{N}$ (see Remark 7.67); for the cost (top), the final value is the relevant one.

representation is given by

$$\dot{x}_{ch}(t) = A_{ch}x_{ch}(t) + B_{ch}e_h(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.194a}$$

$$u_h(t) = C_{ch}x_{ch}(t) + D_{ch}e_h(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.194b}$$

where $x_{ch}(t) \in \mathbb{R}^{n_{ch}}$ is the state vector of the hybrid controller at time t, $e_h(t) \in \mathbb{R}^{24}$ is the hybrid error, $e_h = r_h - y_h$, and $u_h(t) \in \mathbb{R}^{19}$ is the hybrid controller output.

The shape controller is embedded in the C_{sh} block and its state-space representation is

$$\dot{x}_{csh}(t) = A_{csh}x_{csh}(t) + B_{csh}e_{sh}(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.195a}$$

$$u_{sh}(t) = C_{csh}x_{csh}(t) + D_{csh}e_{sh}(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.195b}$$

where $x_{csh}(t) \in \mathbb{R}^{n_{csh}}$ is the state vector of the shape controller at time t, $e_{sh}(t) \in \mathbb{R}^{q_{sh}}$ is the shape error, $e_{sh} = r_{sh} - y_{sh}$, and $u_{sh}(t) \in \mathbb{R}^{24}$ is the shape controller output, namely the variations to be applied to

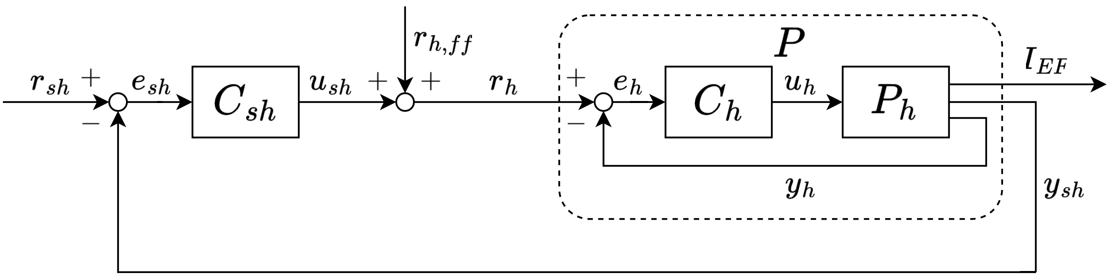

Figure 7.27: Simplified diagram of the TCV control scheme: the P_h block represents the actual plant, interconnected to the *hybrid* feedback controller C_h and to the shape controller, represented by the C_{sh} block.

modify the prescribed feedforward references $r_{h,ff}$ for the hybrid controller, which are precomputed for each shot.

Assumption 7.72. *The pairs* (A_{ph}, B_{ph}), (A_{ch}, B_{ch}) *are controllable, and the pairs* (C_{ph}, A_{ph}), (C_{ch}, A_{ch}) *are detectable. Moreover, the closed-loop system given by* (7.193)–(7.195) *interconnected as in*

$$e_{sh} = r_{sh} - y_{sh}, \qquad r_h = r_{h,ff} + u_{sh}, \tag{7.196}$$

is well-posed and aymptotically stable. ○

Assumption 7.73. $q_{sh} < 24$. ○

In practice, the validity of Assumption 7.73 (that is analogous to Assumption 7.3) depends on the design of the shape controller, *i.e.*, on the number q_{sh} of shape outputs that are considered to track the desired shape. This is a tunable parameter of the TCV shape controller [162], because it depends on the number of control points that define the desired shape. Thus, if the control objective for a given experiment is suitable, the designer can reduce the number of regulated outputs and exploit TCV's redundant array of coils to improve the performance of the control system.

Assumption 7.74. *The shape reference signal* r_{sh} *is such that*

$$r_{sh}(t) = \bar{r}_{sh}, \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.197}$$

with $\bar{r}_{sh} \in \mathbb{R}^{q_{sh}}$. ○

It is worth to point out that even if Assumption 7.74 holds, which in other words implies that a desired shape to track is encoded as a set of q_{sh} constant scalar references, the resulting reference for the hybrid controller C_h, namely r_h, may not be a constant signal; this is due to the precomputed hybrid reference $r_{h,ff}$ that may include ramps. However, for the sake of simplicity, the design of the allocator can be carried out discarding the presence of the feedforward term $r_{h,ff}$. Thus, in accordance with the analysis carried out in Section 7.6, Assumption 7.74 implies that the steady-state behavior of the closed loop is constant. Hence, the steady-state cost functional (7.6) reduces to the *steady-state cost function*

$$J = \ell(\tilde{I}_{EF}), \tag{7.198}$$

where $\tilde{I}_{EF}$ is the steady-state value of the EF coil currents, and the function $\ell : \mathbb{R}^{16} \to \mathbb{R}$ is at least locally convex. A specific choice of the performance criteria with respect to which the steady-state behavior of the EF coil currents must be optimized yields a specific form of the function ℓ. The dynamic EF coil current allocation problem for the TCV tokamak can be formulated as follows.

Problem 7.75 (Dynamic EF coil current allocation for TCV shape control)**.** *Consider the TCV feedback control system* (7.193)–(7.194) *with the shape controller* (7.195) *and suppose that Assumptions 7.72, 7.73 and 7.74 hold. Find, if any, an input allocation device (briefly, an* allocator*) having input ϖ and output ς such that, when interconnected according to*

$$\varpi = u_{sh} + r_{h,ff}, \qquad r_h = u_{sh} + r_{h,ff} + \varsigma, \tag{7.199}$$

ensures that the following requirements hold.

(I) Full shape output invisibility: *the shape output both in presence and absence of the allocator is such that*

$$y_{sh}(t) = y^a_{sh}(t), \qquad t \in \mathbb{R}_{\geq 0}. \tag{7.200}$$

(O) Steady-state optimality*: the cost function* (7.198) *is minimized.*

∘

7.7.2 Design of the dynamic EF current allocator

The statement of Problem 7.75 shows many similarities with that of Problem 7.8. Thus, recalling the literature review offered in Section 7.4, a suitable solution to it is found starting from those presented in Section 7.6.2. First, the full shape output invisibility requirement stated in statement (I) justifies an approach similar to that introduced in [51], *i.e.*, the design of an allocator comprising three subsystems each responsible for one of the requirements, such as the dynamic annihilator that deals with the shape output preservation. However, the main difference with respect to all the cases analyzed in Section 7.6.2 and in [51] is stated in Assumption 7.74, *i.e.*, the intended steady-state behavior is constant. This implies that the *steady-state generator* component of the architecture introduced in [51] and further developed in this work is not necessary and that the interconnection of the allocator with the preexisting shape control system can be carried out as in (7.199). The proposed architecture is depicted in Figure 7.28, whereas the design procedure of both the *dynamic annihilator* and *steady-state optimizer* units is described in the following. After the selection of the most suitable architecture, the next design decision that has been taken concerned the nature of the dynamic systems to be

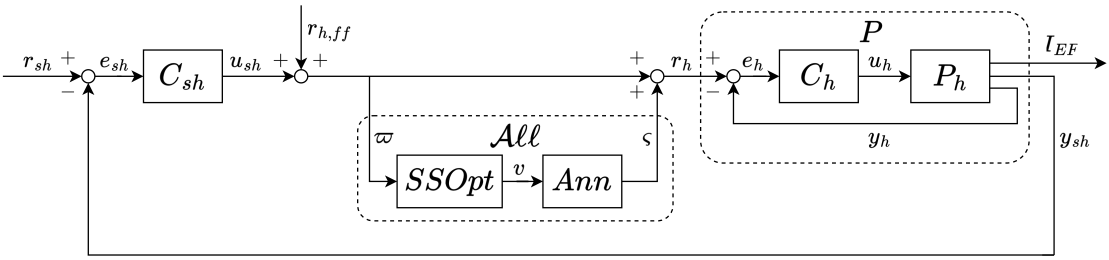

Figure 7.28: Full diagram of the TCV shape control scheme: the P_h block represents the actual plant, interconnected to the hybrid feedback controller C_h and to the shape controller C_{sh}; the proposed allocator is enclosed in the $\mathscr{A}\ell\ell$ block.

implemented, having to choose among either a continuous-time or a discrete-time design. Even if the TCV control system is now almost entirely composed of sampled-data, digital devices (see Sections 6.2 and 7.6.2), the proposed design has been carried out in the continuous time domain. This choice has been made for two technical reasons related to the SCD implementation: first, the design of all the control systems integrated in the SCD is performed in continuous-time, and their discretization is automatically performed by the build system that generates and compiles the real-time control code. Secondly, the sampling rates of the SCD are very high, typically in the order of 10^4 Hz, thus making negligible any error induced by the discretization in the majority of contexts. The validity of the proposed design follows from the results cited in Section 7.4 and from those illustrated in Section 7.6.2.

Design of the dynamic annihilator

Let $m_a = 24 - q_{sh}$ be the degree of redundancy of the closed loop system (7.193)–(7.194) with respect to the shape output y_{sh}. Let $W_{y_{sh}r_h}(s)$ be the transfer matrix from the hybrid reference input r_h to the shape output y_{sh}, $W_{y_{sh}r_h}(s) \in \mathbb{C}^{q_{sh}\times 24}$. Recalling Section 7.6.2, the transfer matrix of the dynamic annihilator is designed as $W_a(s) \in \mathbb{C}^{24\times m_a}$ such that

$$W_a(s) = N_a(s) d^{-1}(s), \tag{7.201}$$

where $d^{-1}(s)$ is a Hurwitz polynomial of degree larger than that of any element of $N_a(s)$, and $N_a(s) \in \mathbb{C}^{24\times m_a}$ is a polynomial annihilator for $W_{y_{sh}r_h}(s)$, *i.e.*, it is such that

$$W_{y_{sh}r_h}(s) N_a(s) = 0, \qquad s \in \mathbb{C} \setminus (\mathscr{P}(W_{y_{sh}r_h}(s))). \tag{7.202}$$

Then, the state-space representation of the annihilator is obtained by a minimal realization $(A_a, B_a, C_a, 0)$ of $W_a(s)$, yielding

$$\dot{x}_a(t) = A_a x_a(t) + B_a v(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.203a}$$

$$\varsigma(t) = C_a x_a(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.203b}$$

where $x_a(t)$ is the state vector of the annihilator at time t, $v(t) \in \mathbb{R}^{m_a}$ is the input of the annihilator, and $\varsigma(t) \in \mathbb{R}^{24}$ is the output of the annihilator.

Design of the steady-state optimizer

Recall (7.199) and the steady-state cost function (7.198). Let $P^\star_{I_{EF}} \in \mathbb{R}^{16\times 24}$ be the static gain of the TCV plant from the input r_h to the output I_{EF}, and $P^\star_a \in \mathbb{R}^{24\times m_a}$ be the static gain of the dynamic annihilator (7.203). It holds that

$$\tilde{I}_{EF} = P^\star_{I_{EF}}(\tilde{\varpi} + \tilde{\varsigma}) = P^\star_{I_{EF}}(\tilde{\varpi} + P^\star_a \tilde{v}). \tag{7.204}$$

Thus, let $x_o(t) \in \mathbb{R}^{m_a}$ for any $t \in \mathbb{R}_{\geq 0}$ and consider the cost function

$$\tilde{J}(x_o, \varpi) = \ell(P^\star_{I_{EF}}(\varpi + P^\star_a x_o)), \tag{7.205}$$

with ℓ as in (7.198). Let $\gamma \in \mathbb{R}_{>0}$ and $K \in \mathbb{R}^{m_a\times m_a}$ such that $K > 0$. According to [51, 247], applying the gradient descent method [114] yields the following dynamics for the steady-state optimizer:

$$\dot{x}_o(t) = -\gamma K \left.\frac{\partial \tilde{J}(x_o, \varpi)}{\partial x_o}\right|'_{x_o = x_o(t), \varpi = \varpi(t)}, \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.206a}$$

$$v(t) = x_o(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.206b}$$

where γ is the update step of the gradient descent method, which can be tuned to control the convergence speed of the optimizer, and K further tunes the convergence speed of the single optimization directions in x_o.

Offline design and validation procedure

Further comments regarding the actual implementation of the design procedure for the proposed allocator are in order here. A linearized model is first obtained, given the shape equilibrium and shape controller design corresponding to the experiment to be performed, using the code suites developed at SPC, namely the MATLAB EQuilibrium Toolbox (MEQ) [199] and the LIUQE code [170]. The model of the closed loop comprising the TCV plant (7.193) and the hybrid controller (7.194) linearized around the shape equilibrium possesses, in general, a high number of states (typically greater than 250). Then, a model reduction is performed applying a balancing transformation [120] and a principal component analysis [169], to ensure that the design procedure of the dynamic annihilator remains numerically feasible; the desired order for the reduced model is a design parameter that can be tuned according to the resulting input-output characteristics of the reduced model, which are compared against those of the original linearized model. This procedure allows the designer to reduce the order of the model down to, in practice, no more than 25 states. The polynomial annihilator (7.202) is determined by first computing a left coprime factorization of the transfer matrix $W_{y_{sh}r_h}(s)$, which is obtained from the reduced model, and then applying the numerical algorithm described in [227] to compute the polynomial matrix $N_a(s)$; this procedure, although numerically and computationally intensive, yields a solution to the problem of determining the polynomial annihilator that can be implemented in a straightforward fashion within the SCD code suite. The remaining parameters of the steady-state optimizer (7.206) are then tuned, and the entire allocator design is then validated, by means of simulations of the shot performed using the MEQ and FBT codes [47, 105, 140, 164].

7.8 Experimental results of dynamic annihilation on the TCV tokamak

This section presents the results of the first experiments conducted on the TCV tokamak to test a dynamic EF coil current allocator integrating a dynamic annihilator to achieve full invisibility in the shape output. The purpose of the allocation device is to improve the performance of the shape controller in terms of specific criteria concerning the EF coil currents. The considered shape controller has been presented in Section 7.2 and [162], and its limitations with respect to such an improvement have been highlighted in Section 7.3. The dynamic coil current allocator with dynamic annihilator for the TCV shape control system has been designed in Section 7.7.2, based on the results provided in Sections 7.4 and 7.6.2.

The selected plasma shape is a *diverted single null* configuration, informally termed a *standard shot*, yielding a specific shape controller design that is the same in both experiments; this decision has been made to test the new allocation scheme in a well-known and stable plasma configuration, where both the plant and the control systems exhibit little to no technical issues. Two experiments, *i.e.*, two plasma discharges have been performed: #81457 and #81460, corresponding to two different coil current optimization criteria illustrated in the following, each yielding a different allocator design as described in Section 7.7.2.

Each result of these discharges is compared against two other experiments. First, a comparison is made against the configuration with the shape controller alone achieving the same plasma shape, namely discharge #79742. This comparison is made to ensure that the allocation scheme, in practice, does not disrupt the operation of the shape controller. Secondly, each discharge is compared against an experiment in which a coil current allocator is also present, designed to meet the same optimization criteria but embedding a *static* annihilator as described in [229]. The purpose of this comparison is to highlight in practice the differences between a state-of-the-art dynamic allocator (see again Section 7.4) and the new scheme developed in this work. To this end, it is worth to point out that this work also constitutes the first real implementation of a dynamic allocation device integrating the dynamic annihilator since its very first proposal in [51].

Recall (7.200) and let

$$\delta y_{sh}(t) = y^a_{sh}(t) - y_{sh}(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{7.207}$$

be the *shape variation* induced by the allocator. For each experiment, two sets of data are analyzed and discussed. First, the coil currents are presented in two forms: their steady-state value is shown in bar plots to verify that the optimization objective has been met, and their time histories are plotted to show the behavior induced by the allocator during the entire discharge. Secondly, the shapes are presented in two forms as well: the time histories of the components of δy_{sh} as in (7.207) are presented to highlight the performance of the dynamic annihilator with respect to the other schemes, and the steady-state shapes are plotted against the reference one to show the overall performance of the shape control scheme in each case. Each experiment lasts 1 s from the breakdown to the shutdown initiation, and the activation instant of the shape control system (including the allocation devices, if any) is marked in each time history plot, happening at 0.6 s. There is one aspect that is worth anticipating here concerning the time histories of the shape errors, to avoid any confusion: at the beginning of each discharge, there are noticeable transient motions that usually fade beyond 0.75 s. Such transients are caused by the OH coil current crossing the zero value while ramping up: due to a technical problem with the OH power supplies of TCV, when the ramping OH current crosses the zero value it stays flat for a few milliseconds, then it resumes its ramping. This undesired behavior induces unmodeled effects in the plasma that are reflected in the shape error time histories, which exhibit non-deterministic transient motions. This issue is known by the TCV operators and is currently not addressed.

7.8.1 Steady-state current norm minimization

This experiment corresponds to the discharge #81457. The EF coil current optimization objective is given by the cost function

$$J = \frac{1}{2} \left\| \tilde{I}_{EF} \right\|_2^2, \tag{7.208}$$

i.e., the intention is to alter the shape controller output to achieve the same reference plasma shape minimizing the norm of the EF coil currents that are necessary to track it at steady-state. The cost function (7.208) is strictly convex and radially unbounded, hence a unique solution exists and the steady-state optimizer (7.206) converges to it with speed determined by the choice of the parameters. The main purpose of this experiment is to verify that the entire implementation of the novel dynamic allocation scheme works without issues, before testing it with more sophisticated optimization criteria, because there is not a significant optimization margin in this case. The FBT and MEQ codes (see Section 7.7.2), when used to compute the feedforward reference for the standard plasma shape, already apply the EF coil current norm minimization as a regularization criterion. Hence, the dynamic allocator does not have much room to improve the performance of the shape controller in this case. The steady-state optimizer parameters for this experiment are set to

$$\gamma = 10, \qquad K = I_{m_a}. \tag{7.209}$$

This experiment is compared against pulse #81032, in which a state-of-the-art allocator with static annihilator has been used.

Coil currents

Figure 7.29 shows the steady-state values of the EF coil currents for both pulses #81457 and #81032, respectively, against the reference #79742. An inspection reveals that both allocators perform in a similar way, remodulating all the currents to achieve the desired optimization objective. Figure 7.30 shows the time histories of the EF coil currents for the same pulses. It is interesting to note the similarity in the current traces of the two allocated pulses #81457 and #81032, which is a consequence of the similarity in the optimization criteria and of the limited optimization margin in this case. This proves that, from the point of view of the EF coil currents, the proposed dynamic allocator works as expected, maintaining a meaningful similarity with the state-of-the-art solution in the considered scenario.

Shape variations

Figure 7.31 shows the time histories of the components of δy_{sh} for pulses #81457 and #81032, respectively. Both pulses exhibit a similar behavior, including unwanted and non-deterministic transients induced by the OH zero crossing discussed above. The fact that the shape variations remain bounded in both cases suggest that both allocation scheme work without disrupting the desired plasma shape and the operation of the shape controller. The dynamic allocator with static annihilator in pulse #81032 exhibits a steadier performance with less oscillatory motions but a higher average shape error with respect to that achieved by the dynamic annihilator in pulse #81457, in particular in the time intervals $[0.6\,\mathrm{s}, 0.7\,\mathrm{s}]$ and $[0.85\,\mathrm{s}, 0.95\,\mathrm{s}]$. However, due to the limited optimization margin and the unwanted transients, this result may only be interpreted again as a confirmation that the proposed dynamic allocator works as expected, a fact further confirmed by Figure 7.32 which depicts the steady-state shapes achieved in presence of both allocation schemes against the one obtained by the shape controller alone and the reference: their negligible differences yield that, in practice, all three schemes are equivalent in terms of the steady-state shape output.

7.8.2 Saturation and deadzone avoidance

This experiment corresponds to the pulse #81460 and it is motivated by practical necessities that arise when dealing with the actual TCV EF coils, with which a dynamic allocator is conceptually helpful. The EF coil power supplies are subject to deadzone effects, *i.e.*, when the requested current values are below 1 kA in absolute value, the power supplies may not respond and produce a current that is not only different from the requested one but also unpredictable and uncontrollable. This issue is known but can currently be addressed only by ensuring that the current requests do not drop below the deadzone threshold, identified by the TCV team practically at $\pm 600\,\mathrm{A}$. This requirement, as stated in Section 7.3, is not observed by the shape controller, which may request currents below the deadzone threshold. Let $\bar{a}_i, \underline{a}_i, \bar{\delta}_i, \underline{\delta}_i \in \mathbb{R}_{\geq 0}$, then let $\bar{b}_i, \underline{b}_i, \bar{c}_i, \underline{c}_i \in \mathbb{R}$, and consider

$$\xi_i(\tilde{I}_{EF,i}) = \begin{cases} \frac{1}{2}\bar{a}_i(\tilde{I}_{EF,i} - \bar{b}_i)^2 & \text{if } \tilde{I}_{EF,i} > \bar{b}_i, \\ \frac{1}{2}\bar{\delta}_i(\tilde{I}_{EF,i} - \bar{c}_i)^2 & \text{if } \tilde{I}_{EF,i} \in [0, \bar{c}_i], \\ \frac{1}{2}\underline{\delta}_i(\tilde{I}_{EF,i} - \underline{c}_i)^2 & \text{if } \tilde{I}_{EF,i} \in [\underline{c}_i, 0], \\ \frac{1}{2}\underline{a}_i(\tilde{I}_{EF,i} - \underline{b}_i)^2 & \text{if } \tilde{I}_{EF,i} < \underline{b}_i, \\ 0 & \text{otherwise,} \end{cases} \qquad i = 1, \ldots, 16. \tag{7.210}$$

The EF coil current optimization objective is given by the cost function

$$J(\tilde{I}_{EF}) = \sum_{i=1}^{16} \xi_i(\tilde{I}_{EF,i}), \tag{7.211}$$

where, essentially, $(\underline{c}_i, \bar{c}_i)$ are the deadzone thresholds for the i-th EF coil, and $(\underline{b}_i, \bar{b}_i)$ are saturation limits for the i-th EF coil, respectively, for each $i = 1, \ldots, 16$; the coefficients $(\underline{a}_i, \bar{a}_i)$ and $(\underline{\delta}_i, \bar{\delta}_i)$ allow the designer to weigh differently each optimization objective on each coil, with $i = 1, \ldots, 16$. The cost function (7.211) is only locally convex but admits local minima for each component, hence the gradient descent-based steady-state optimizer (7.206) moves each component towards a local minimum that depends on the initial condition, a behavior that is totally acceptable in practice. The saturation boundaries are set to ± 3 kA to further test the capabilities of the proposed allocation scheme. The remaining dynamic allocator parameters are set to

$$\gamma = 22, \qquad K = I_{m_a}, \qquad (\underline{a}_i, \bar{a}_i, \underline{\delta}_i, \bar{\delta}_i) = (1, 1, 10^4, 10^4), \qquad i = 1, \ldots, 16. \tag{7.212}$$

This experiment is compared against pulse #79868, in which a state-of-the-art allocator with static annihilator has been used to meet the same optimization criteria with the same design parameters.

Coil currents

The steady-state values of the EF coil currents are shown in Figure 7.33 for pulses #81460 and #79868, respectively, against the reference #79742. The most important thing to note is that, despite the similarities in the design as well as in the optimization objective, the novel proposed dynamic allocator performs better than the state-of-the-art solution in this case: the EF coil currents are remodulated in a way such that they avoid both the deadzone and the saturation boundaries, while the state-of-the-art allocator fails to ensure so in particular for the coils $E4$, $E8$, and $F7$ whose currents remain within the deadzone threshold. Furthermore, Figure 7.34 shows the time histories of the EF coil currents for the same pulses, providing another important result: the proposed dynamic allocator with dynamic annihilator induces significant variations in the EF coil currents with respect not only to the reference discharge but also to the state-of-the-art allocator test, requesting steeper current variations that the power supplies successfully achieve. The steady-state values of the EF coil currents for pulse #81460 are, thus, different from those of the reference pulse #79742, a fact that would have important consequences on the plasma shape if not for the presence of the dynamic annihilator, as discussed below.

Shape variations

The time histories of the components of δy_{sh} for the pulse #81460 are shown in Figure 7.35, compared against the results of pulse #79868. After the initial unmodeled transient motion due to the OH coil zero crossing fades, both the allocators achieve a similar, steady performance in terms of shape variation, with the proposed novel dynamic allocator resulting in a lower shape variation towards the end of the experiment. The validity of this approach is confirmed by the steady-state shapes depicted in Figure 7.36 for the pulses #81460 and #79868, respectively, against the reference shape and the shape obtained by the shape controller alone in pulse #79742. Again, the shapes are practically the same, confirming that the proposed dynamic allocator works as expected even if, in this case, the steady-state currents are much different as highlighted above. The possibility to achieve the same prescribed plasma shape with such different EF coil currents without disrupting the operation of the shape controller is clearly the effect of the dynamic annihilator, confirming the performance improvements expected since the theoretical results in Section 7.6.2.

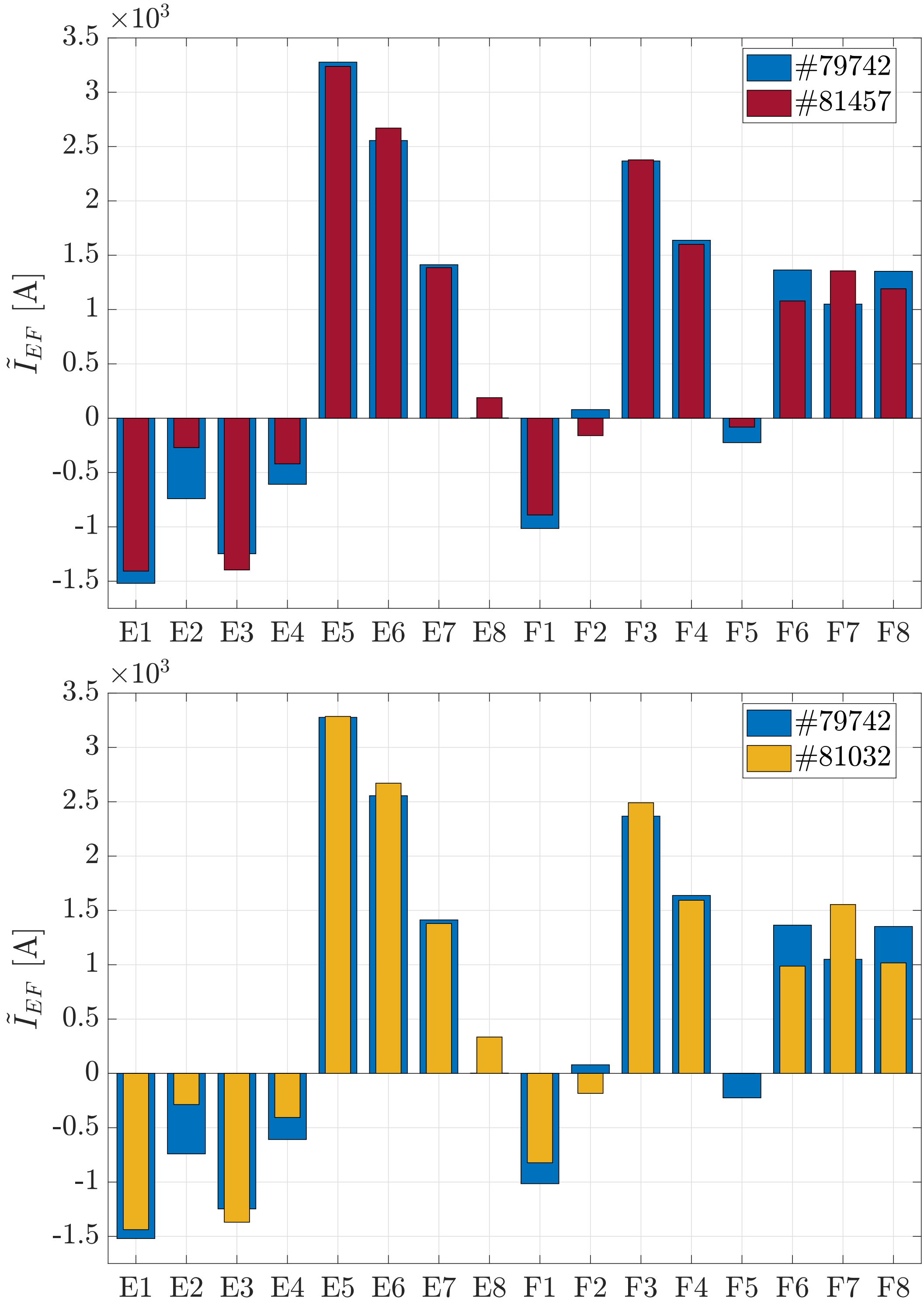

Figure 7.29: Steady-state values of the EF coil currents for pulses #81457 (top) and #81032 (bottom) against the reference pulse #79742.

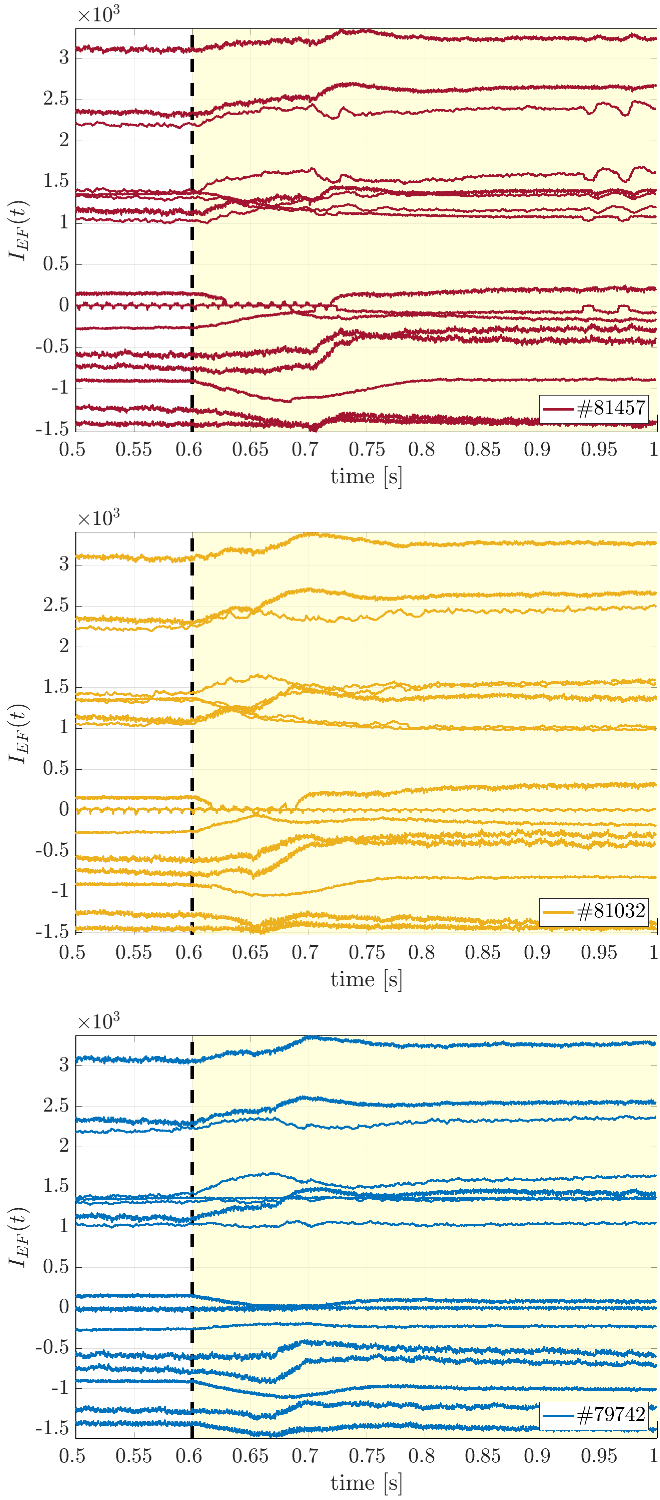

Figure 7.30: Time histories of the EF coil currents for pulses #81457 (top), #81032 (middle), and #79742 (bottom).

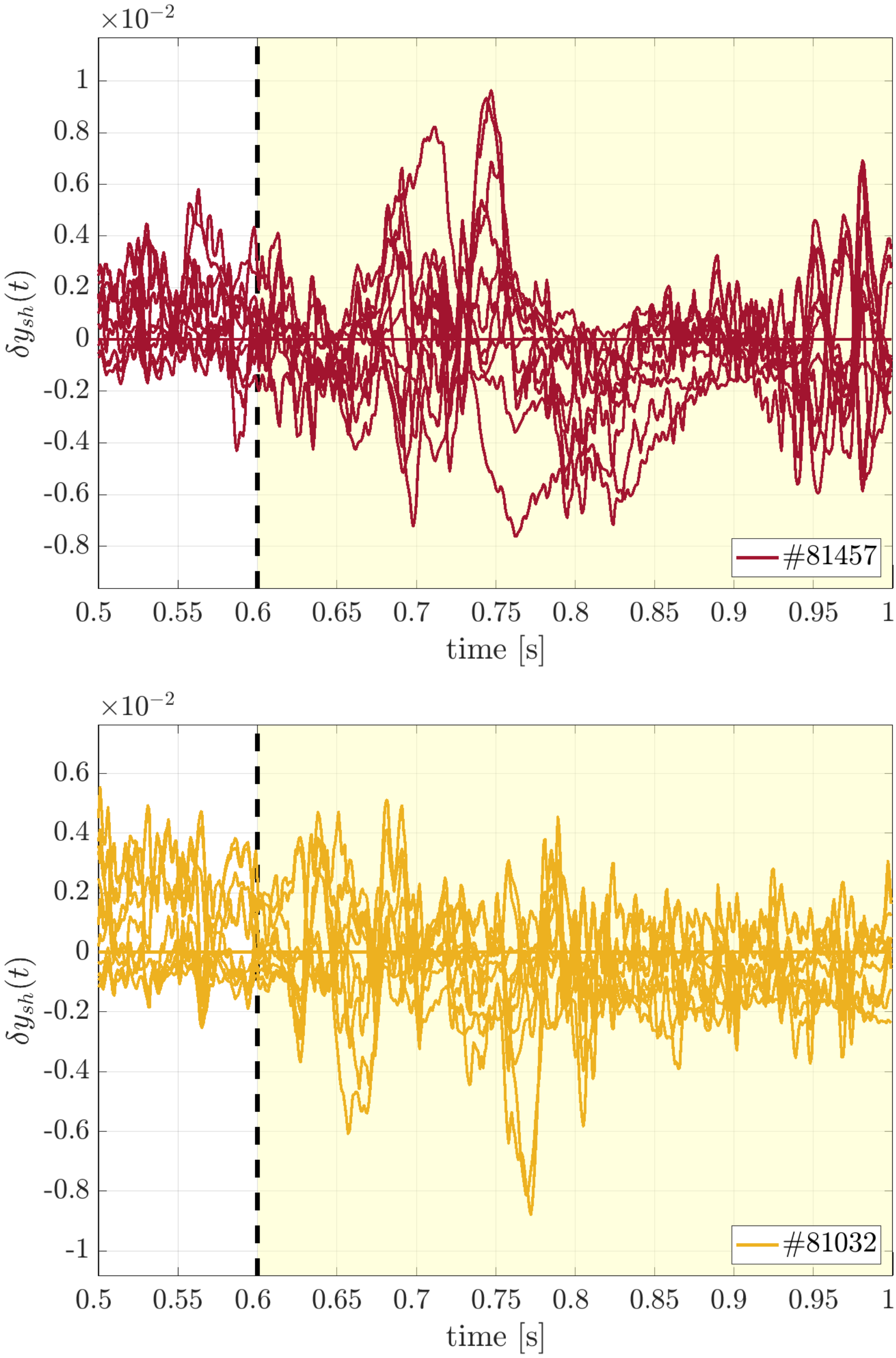

Figure 7.31: Time histories of the components of δy_{sh} for pulses #81457 with the dynamic annihilator (top) and #81032 with the static annihilator (bottom).

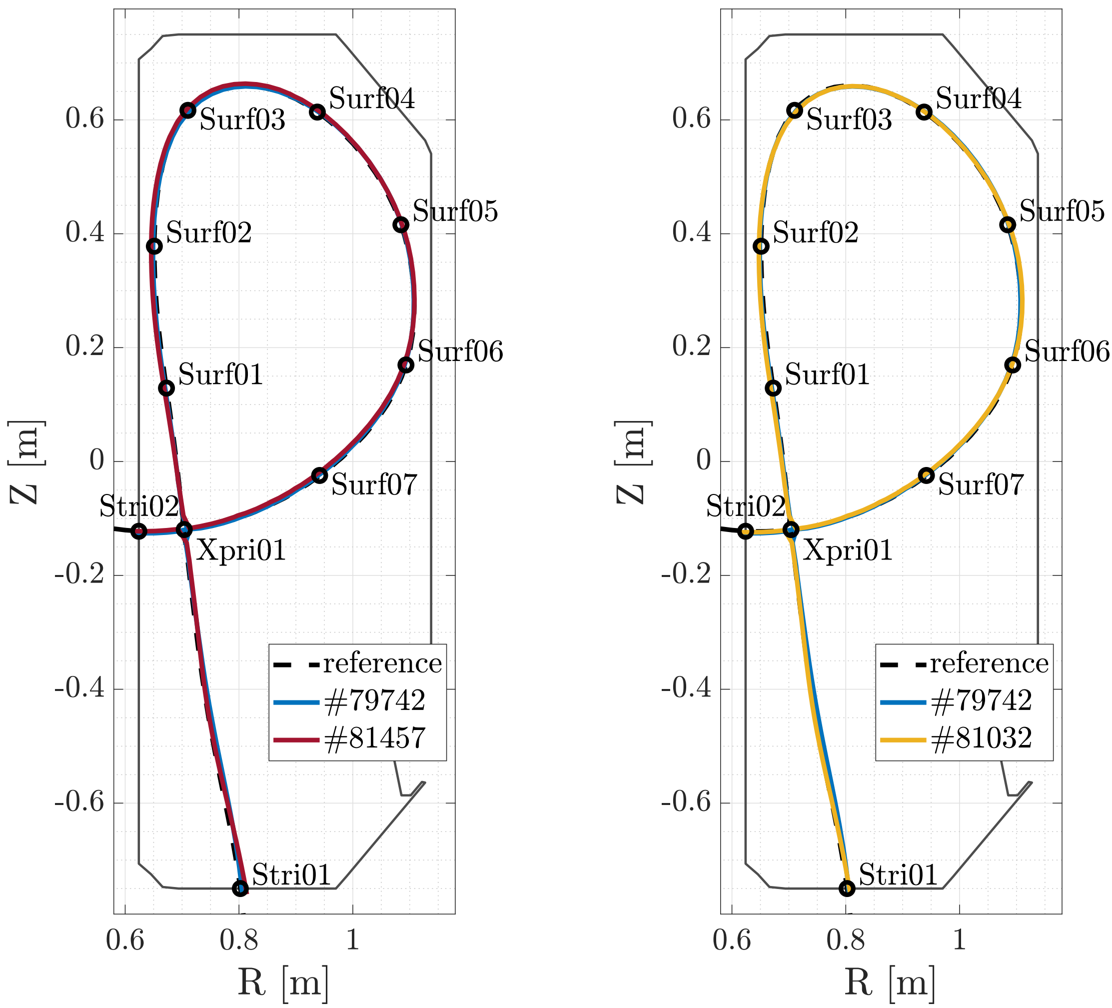

Figure 7.32: Steady-state shapes achieved in pulses #81457 with the dynamic annihilator (left) and #81032 with the static annihilator (right) against the shape obtained by the shape controller alone in pulse #79742 and the reference shape.

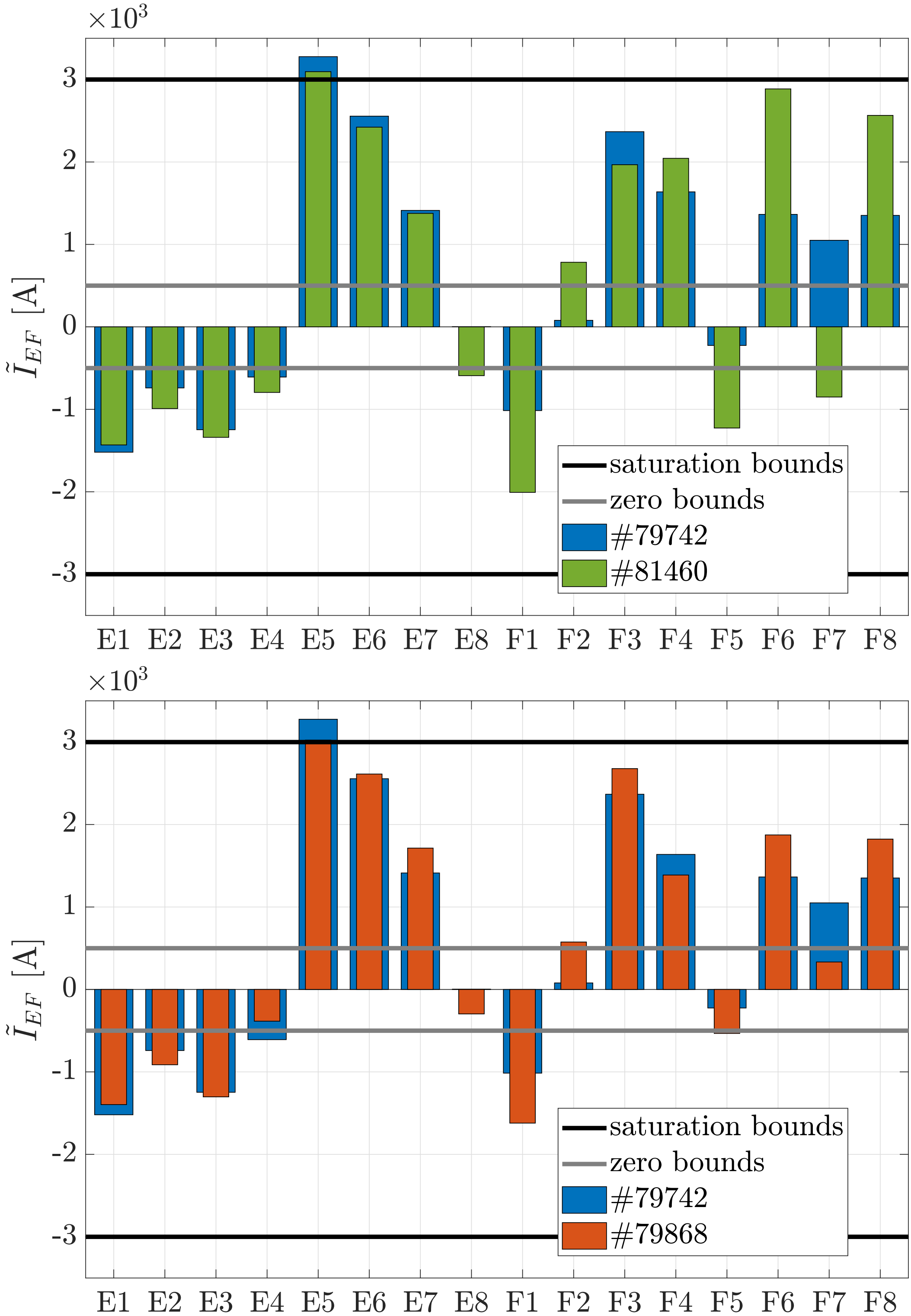

Figure 7.33: Steady-state values of the EF coil currents for pulses #81460 (top) and #79868 (bottom) against the reference pulse #79742.

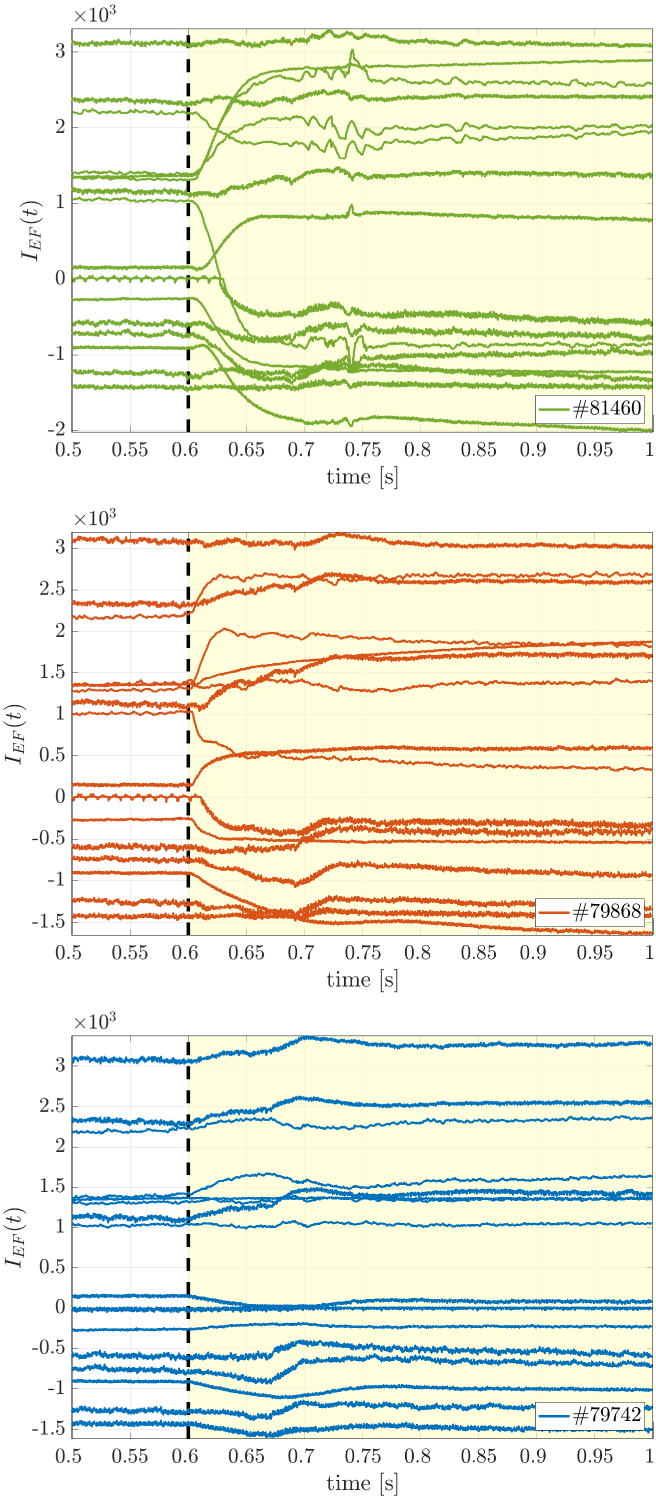

Figure 7.34: Time histories of the EF coil currents for pulses #81460 (top), #79868 (middle), and #79742 (bottom).

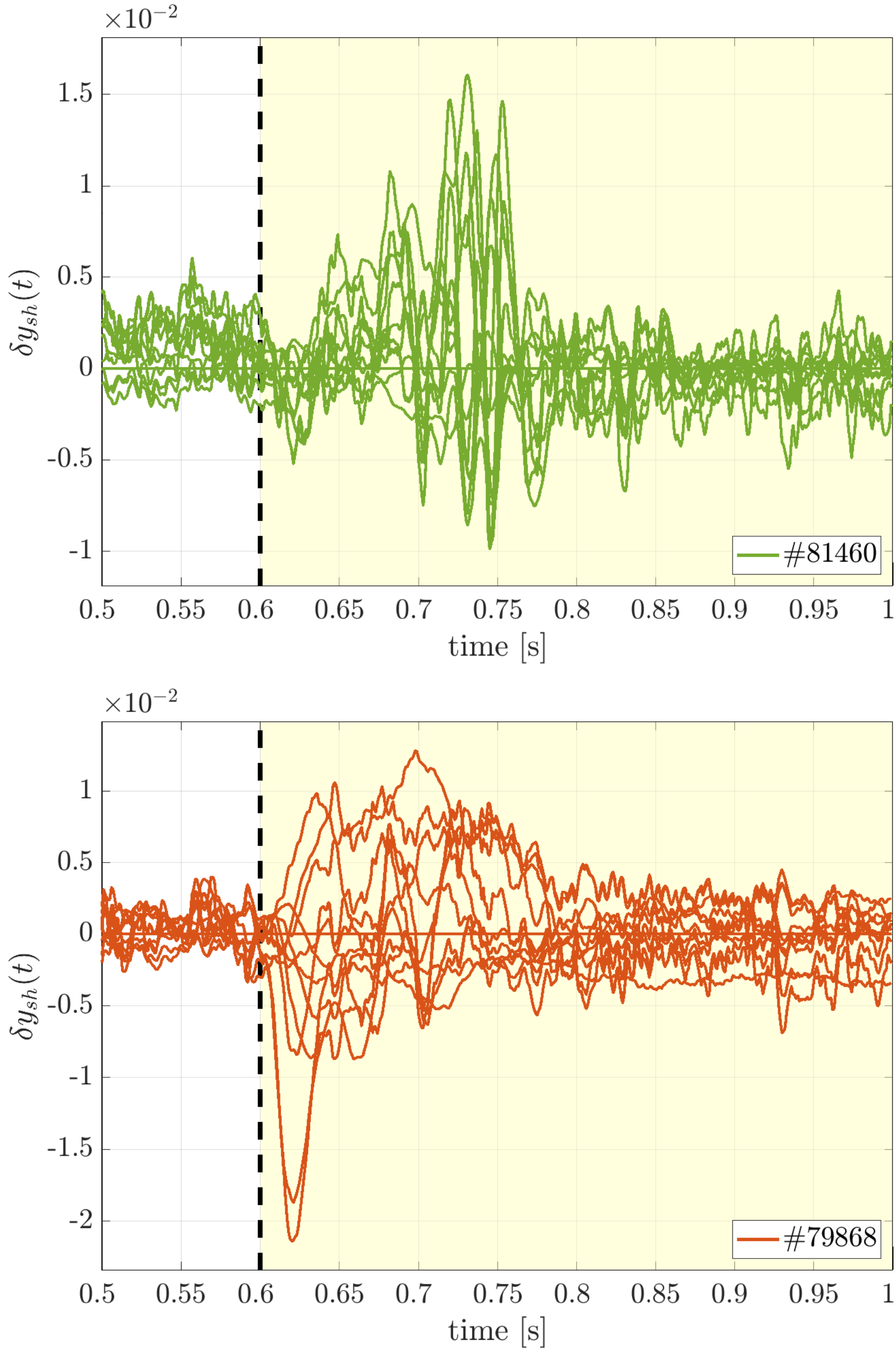

Figure 7.35: Time histories of the components of δy_{sh} for pulses #81460 with the dynamic annihilator (top) and #79868 with the static annihilator (bottom).

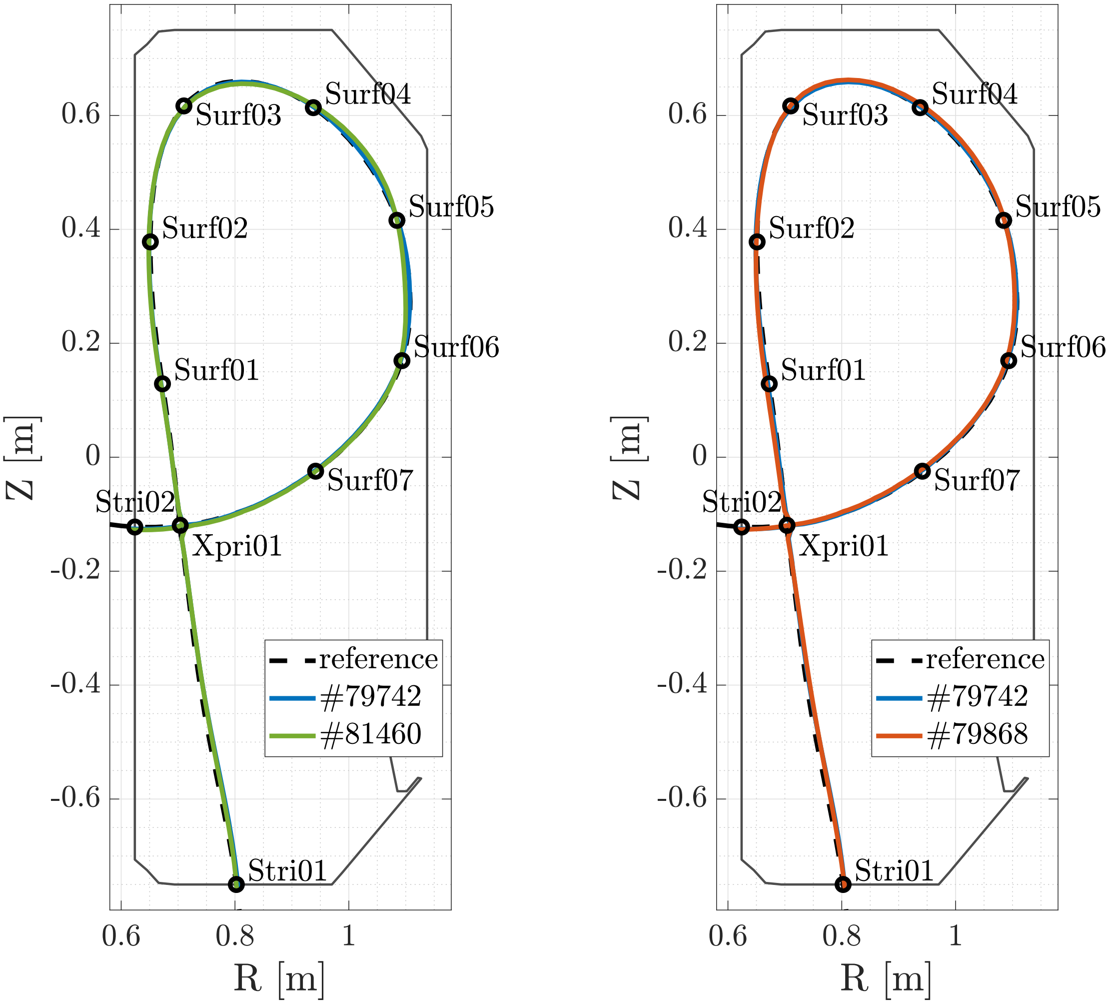

Figure 7.36: Steady-state shapes achieved in pulses #81460 with the dynamic annihilator (left) and #79868 with the static annihilator (right) against the shape obtained by the shape controller alone in pulse #79742 and the reference shape.

8 Conclusions

To provide a closing remark on the work hereby presented, it is worth to summarize the main contributions and results obtained, also to highlight possible improvements to consider and future directions to explore. This work addressed the context of distributed autonomous systems with the purpose of designing an architecture supporting research activities and workflows involving them. The importance of such an architecture, as well as its basic requirements, have been illustrated in Chapter 2. In Section 2.3, it has been recognized how one of the major drawbacks of existing solutions is the intent to provide a complete implementation from the ground up, instead of relying on existing tools and technologies that help to solve at least a part of the issues at hand. The relevant technologies have been introduced in Chapter 3, highlighting for each the main features that make them suitable for the design of the architecture. The Distributed Unified Architecture has been presented in Chapter 4 as a framework born from the integration of existing technologies with specific, novel solutions. The discussion addressed each of the basic requirements with a DUA feature, showing how the proposed architecture provides a common base environment for the development of software modules for autonomous systems, on both PCs and dedicated hardware.

The proposed solution has been tested on a variety of hardware platforms, whose interoperability has also been shown in the use cases considered in Chapter 5. A possible improvement in this regard is certainly the development of new base units (see Section 4.2): since the proposed approach worked on the set of hardware platforms discussed so far, it is worth expanding this set including new solutions and refining the existing ones; a first step in this regard is, *e.g.*, the development of a `jetson6` base unit supporting the most recent JetPack 6 SDK for the Nvidia Jetson platforms. Another possible improvement concerns the reliance on Docker (see Sections 3.2.1 and 4.2) even on SBCs and similar platforms. While the container-based approach of `dua_foundation` clearly helps to abstract the hardware layer in a repeatable and reliable fashion, it adds a degree of complexity that is preferable to avoid in embedded systems. To this end, a restructuring of the `dua_foundation` module is considered, continuing to provide the development base units in the form of Docker images but developing the ones targeted to embedded systems as classic system images using, *e.g.*, crosstool-NG

[171] and Buildroot [118] to build the toolchains and the root filesystems, respectively. Another aspect to consider is that, as discussed in Section 4.4, the Development (D) requirement is still not completely satisfied by the proposed architecture. Two areas for improvement are identified in this context: the addition and configuration of cross-compilation toolchains and debuggers in the development base units, and the integration of support for Continuous-Integration/Continuous-Deployment (CI/CD) pipelines in the `dua_template`, to support the most modern automated development workflows.

The many benefits of such an architecture in the autonomous robotics field have been discussed in Chapter 5, with some results highlighted in Section 5.3. To this end, the improvement of the development tools that DUA offers to ROS 2 and robotic software developers is certainly a path to consider. Among the many possible outcomes of such an effort, some notable ones include the development of a version of the `params_manager` module supporting Python packages, and a mathematical library for the management of the state of rigid bodies in 3D space offering APIs to interface directly with ROS 2 communication primitives. More in general, getting the community more involved in the DUA project is certainly beneficial, for both the project itself and the ROS 2 ecosystem: some of the features that `dua_utils` modules currently offer may be integrated into the ROS 2 API, and the development of new modules may be driven by the broader needs of the community. In this regard, other directions to explore include the enhancement of the real-time capabilities of the ROS 2 executors (see Section 3.1.3) and the improvement of the ROS 2 data recording and playback tools to support structured and organized archival technologies, such as MDSplus (see Section 2.3.6).

The other application scenario considered in Chapters 6 and 7, that is, that of magnetic confinement nuclear fusion, also provides some notable results. In Chapter 6, the application of the DDS middleware to the distributed control system of the TCV tokamak clarified the importance of these technologies even in critical research environments as that of the SCD distributed control system. These solutions also solved critical constraints affecting the expandability of the research infrastructure, paving the way for the integration of new real-time diagnostic systems and control algorithms. The benchmarking results shown in Section 6.3.3 confirmed that the intergation of the DDSDataSource in the SCD architecture, to replace the previous reflective memory network, is an update to be performed. Furthermore, the results concerning the dynamic allocation of EF coil currents in Chapter 7 also deserve further investigation. The methodological solutions developed in Section 7.6 cover a wide variety of scenarios but also uncover further questions to be researched. First, the study presented in Section 7.6.1 testifies that the dynamic input allocation problem for nonlinear systems is meaningful and may also be solved by framing it in the context of optimal control; however, further developments are necessary to understand to what extent this approach is conclusive. The dynamic annihilator presented in Section 7.6.2 and introduced for the first time in [51] broke the tight coupling involving full output invisibility and state-space properties since [247] but also shed light of more questions; more research is required to investigate how input-output characterization of the invisibility property may be reflected on state-space properties of the plant. Furthermore, the sampled-data case has been addressed for the first time in the literature as reported in Section 7.6.3 and many questions remain open, such as the proper definition of the inter-sample output ripple due to the sampled input allocation action. The implementation of a sampled-data dynamic allocator may also be performed on the TCV control system, extending the developments of Section 7.7 and providing, hopefully, a first practical proof of its effectiveness. Finally, the experimental results provided in Section 7.8, obtained with the dynamic EF coil current allocator designed in Section 7.7.2, constitute the first practical example of the effectiveness of a dynamic control allocator designed for full output invisibility, translating the theoretical results of Sections 7.4 and 7.6.2 into a real-world application involving a sophisticated plant, such as the TCV tokamak, whose technical constraints make it benefit from the proposed solution. Another way to provide more experimental results corroborating the methodological approaches mentioned above is to test the proposed dynamic allocators on a wider variety of plasma shapes and optimization objectives.

A The dua-template update workflow

```
# Workflow to sync local repository with latest version of DUA template.
#
# Roberto Masocco <r.masocco@dotxautomation.com>
#
# June 13, 2024

# Copyright 2024 dotX Automation s.r.l.
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

name: Sync DUA template

on:
  # Run every day at 00:01
  schedule:
    - cron: "1 0 * * *"

  # Run when base branch is updated
  push:
    branches:
      - master

  # Run when manually triggered
  workflow_dispatch:

env:
  BASE_BRANCH: master
  HEAD_BRANCH: chore/sync-dua-template
  GIT_AUTHOR_NAME: ${{ github.repository_owner }}
```

```
  GIT_AUTHOR_EMAIL: ${{ github.repository_owner }}@users.noreply.github.com
  REPO_TEMPLATE: dotX-Automation/dua-template
  THIS_REPO: ${{ github.repository }}

jobs:
  sync-dua-template:
    # Do not run on the template repository itself
    if: github.repository != 'dotX-Automation/dua-template'

    name: Sync DUA template
    runs-on: ubuntu-latest
    continue-on-error: false

    steps:
      # Configure Git to reduce warnings
      - name: Configure Git to reduce warnings
        run: git config --global init.defaultBranch master

      # Clone the template repository (we need the full history of this)
      - name: Check out template repository
        uses: actions/checkout@v4
        with:
          repository: ${{ env.REPO_TEMPLATE }}
          ref: 'master'
          token: ${{ github.token }}
          path: ${{ env.REPO_TEMPLATE }}
          fetch-depth: 0

      # Clone the target repository
      - name: Check out ${{ github.repository }}
        uses: actions/checkout@v4
        with:
          repository: ${{ github.repository }}
          ref: ${{ env.BASE_BRANCH }}
          token: ${{ github.token }}
          path: ${{ github.repository }}

      # Checkout a branch
      - name: Create branch in ${{ env.THIS_REPO }}
        run: |
          git -C "${THIS_REPO}" fetch origin "${HEAD_BRANCH}" || true
          git -C "${THIS_REPO}" branch -a
          git -C "${THIS_REPO}" checkout -B "${HEAD_BRANCH}" \
            "remotes/origin/${HEAD_BRANCH}" || \
          git -C "${THIS_REPO}" checkout -b "${HEAD_BRANCH}"

      # Parse template contents and synchronize changes
      - name: Parse template contents and synchronize changes
        run: |
          _files="$(find ${REPO_TEMPLATE} \
            ! -path "*/.git/*" \
            ! -path "*/.github/workflows/*" \
            ! -path "*/.vscode/*" \
            ! -name ".gitignore" \
            ! -name "LICENSE" \
            ! -name "README.md" \
            -type f \
            -print)"
          _deleted_files="$(git -C ${REPO_TEMPLATE} log \
            --diff-filter=D \
            --name-only \
```

```
          --pretty=format: | grep -v '^$')"
        for _deleted_file in ${_deleted_files}; do
          _file_path="${THIS_REPO}/${_deleted_file#${REPO_TEMPLATE}/}"
          if [[ -f "${_file_path}" ]]; then
            echo "INFO: Deleting ${_file_path}"
            rm -f "${_file_path}"
          fi
        done
        for _file in ${_files}; do
          _src="${_file}"
          _dst="${THIS_REPO}/${_file#${REPO_TEMPLATE}/}"
          _dst="${_dst%/*}/"
          mkdir -p "${_dst}"
          echo "INFO: Copying '${_src}' to '${_dst}'"
          cp "${_src}" "${_dst}"
        done
        git -C "${THIS_REPO}" diff

    # Commit changes, push, and create a pull request
    - name: Push changes and create a pull request
      env:
        GH_TOKEN: ${{ github.token }}
      run: |
        if [[ -n $(git -C "${THIS_REPO}" status --porcelain) ]]; then
          git -C ${THIS_REPO} config user.name "${GIT_AUTHOR_NAME}"
          git -C ${THIS_REPO} config user.email "${GIT_AUTHOR_EMAIL}"
          git -C ${THIS_REPO} add .
          git -C ${THIS_REPO} commit -m \
          "chore: sync to dua-template@${{ github.sha }}"
          if [[ -n $(git -C ${THIS_REPO} branch -r --list \
          "origin/${HEAD_BRANCH}") ]]; then
            git -C ${THIS_REPO} push
            echo "INFO: Pushed changes to branch ${HEAD_BRANCH}"
          else
            git -C ${THIS_REPO} push -u origin "${HEAD_BRANCH}"
            pushd ${THIS_REPO}
            gh pr create \
              --base "${BASE_BRANCH}" \
              --head "${HEAD_BRANCH}" \
              --title "Sync to dua-template@${{ github.sha }}" \
              --body "Sync to dua-template@${{ github.sha }}."
            popd
            echo "INFO: Created new pull request from \
            ${HEAD_BRANCH} to ${BASE_BRANCH}"
          fi
        else
          echo "INFO: No changes to commit"
        fi
```

Listing A.1: `dua-template` update GitHub workflow as of November 22, 2024 [145].

B Unit configurations

```
# flight_stack START #
# Install PX4 general dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
  astyle \
  libxml2-dev \
  libgstreamer-plugins-base1.0-dev \
  rsync && \
  rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*

# Install PX4 Python general dependencies
RUN . /opt/dua-venv/bin/activate && yes | pip3 install -U \
  future \
  jinja2>=2.8 \
  jsonschema \
  kconfiglib \
  lxml \
  psutil \
  pygments \
  pymavlink \
  pyulog

# Install NuttX toolchain dependencies
# Get these from: PX4-Autopilot/Tools/setup/ubuntu.sh
# and merge with your base unit's contents
RUN apt-get update && apt-get install -y --no-install-recommends \
  automake \
  binutils-dev \
  bison \
  flex \
  g++-multilib \
  gcc-multilib \
  gdb-multiarch \
  genromfs \
  gettext \
  gperf \
  libelf-dev \
  libexpat-dev \
  libgmp-dev \
  libisl-dev \
```

```
  libmpc-dev \
  libmpfr-dev \
  libncurses5 \
  libncurses5-dev \
  libncursesw5-dev \
  libtool \
  texinfo \
  u-boot-tools \
  util-linux \
  vim-common && \
  rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*/apt/lists/*

# Download and install NuttX toolchain (pray that this remains online)
WORKDIR /opt
RUN wget -O /tmp/gcc-arm-none-eabi-9-2020-q2-update-linux.tar.bz2 \
  https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu-rm/9-2020q2/\
  gcc-arm-none-eabi-9-2020-q2-update-x86_64-linux.tar.bz2 && \
  tar -jxf /tmp/gcc-arm-none-eabi-9-2020-q2-update-linux.tar.bz2 -C /opt/ && \
  rm /tmp/gcc-arm-none-eabi-9-2020-q2-update-linux.tar.bz2 && \
  chgrp -R internal gcc-arm-none-eabi-9-2020-q2-update && \
  chmod -R g+rw gcc-arm-none-eabi-9-2020-q2-update
ENV PATH=/opt/gcc-arm-none-eabi-9-2020-q2-update/bin:${PATH}
WORKDIR /root

# Build and install Foonathan memory allocator
WORKDIR /opt
RUN git clone --single-branch --branch 'v1.3.0' \
  https://github.com/eProsima/foonathan_memory_vendor.git && \
  cd foonathan_memory_vendor && \
  mkdir build && \
  cd build && \
  cmake .. && \
  cmake --build . --target install && \
  cd ../.. && \
  chgrp -R internal foonathan_memory_vendor && \
  chmod -R g+rw foonathan_memory_vendor
WORKDIR /root

# Build and install Fast-DDS-Gen (required by microRTPS Bridge)
WORKDIR /opt
RUN git clone --recursive --single-branch --branch 'v2.4.0' \
  https://github.com/eProsima/Fast-DDS-Gen.git && \
  cd Fast-DDS-Gen && \
  ./gradlew assemble && \
  ln -s /opt/Fast-DDS-Gen/scripts/fastddsgen scripts/fastrtpsgen && \
  cd .. && \
  chgrp -R internal Fast-DDS-Gen && \
  chmod -R g+rw Fast-DDS-Gen
WORKDIR /root
ENV PATH=/opt/Fast-DDS-Gen/scripts:${PATH}

# Build and install Fast-CDR
WORKDIR /opt
RUN git clone --single-branch --branch 'v1.1.1' \
  https://github.com/eProsima/Fast-CDR.git && \
  cd Fast-CDR && \
  mkdir build && \
  cd build && \
  cmake .. && \
  make -j$(nproc --all) && \
  make install -j$(nproc --all) && \
```

```
  cd ../.. && \
  chgrp -R internal Fast-CDR && \
  chmod -R g+rw Fast-CDR
WORKDIR /root

# Build and install Fast-DDS
WORKDIR /opt
RUN git clone --single-branch --branch 'v2.10.1' \
  https://github.com/eProsima/Fast-DDS.git && \
  cd Fast-DDS && \
  mkdir build && \
  cd build && \
  cmake -DTHIRDPARTY=ON .. && \
  make -j$(nproc --all) && \
  make install -j$(nproc --all) && \
  cd ../.. && \
  chgrp -R internal Fast-DDS && \
  chmod -R g+rw Fast-DDS
WORKDIR /root
# flight_stack END #
```

Listing B.1: Unit configuration of the `x86-cudev` target of the `flight_stack` DUA unit [150] as of November 26, 2024.

C Moments of a linear system

The contents of this appendix are based on [17, 215] and references therein. Some more insights are also found in [36, 37], concerning in particular off-line and on-line estimation methods from input-output measurement data. The interested reader is referenced to those works for a detailed discussion of the following topics.

Consider the system

$$\dot{x}(t) = Ax(t) + Bu(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{C.1a}$$

$$y(t) = Cx(t) + Du(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{C.1b}$$

where $x(t) \in \mathbb{R}^n$ is the state vector, $u(t) \in \mathbb{R}^m$ is the input vector, and $y(t) \in \mathbb{R}^q$ is the output vector. $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times m}$, $C \in \mathbb{R}^{q\times n}$, $D \in \mathbb{R}^{q\times m}$ are constant matrices. The transfer matrix of (C.1) is

$$W(s) = C(sI - A)^{-1}B + D, \qquad s \in \mathbb{C}. \tag{C.2}$$

Definition C.1 (Moment)**.** *Let $s^\star \in \mathbb{C}$ be such that $s^\star \notin \operatorname{spec}(A)$ for the system* (C.1)*. The* moment of (C.1) at $s^\star$ *is $W(s^\star)$.* ◦

The *moment of $W(s)$ (or (A,B,C,D)) at $s^\star$* has the following dynamic meaning: under the assumption that $s^\star \notin \operatorname{spec}(A)$, if (C.1) is subject to the input $U_0 e^{s^\star t}$ with $U_0 \in \mathbb{C}^m$, then its output will contain, among others, one term of the form $Y_0 e^{s^\star t}$ with $Y_0 = W(s^\star)U_0$.

Moments can be computed by using only real matrix computations, via the solution of the linear equations

$$\Pi S = A\Pi + BL, \tag{C.3a}$$

$$M = C\Pi + DL, \tag{C.3b}$$

in the unknowns Π, M, provided that

$$S = \begin{bmatrix} \alpha I_m & \omega I_m \\ -\omega I_m & \alpha I_m \end{bmatrix}, \qquad L = \begin{bmatrix} I_m & 0_{m\times m} \end{bmatrix}, \qquad \text{if } s^\star \notin \mathbb{R}, \tag{C.4a}$$

$$S = \alpha I_m, \qquad L = I_m, \qquad \text{if } s^\star \in \mathbb{R}, \tag{C.4b}$$

where $\alpha = \mathrm{Re}(s^\star)$, $\omega = \mathrm{Im}(s^\star)$. Under such conditions, it holds that M satisfies

$$M = \begin{bmatrix} \mathrm{Re}(W(s^\star)) & \mathrm{Im}(W(s^\star)) \end{bmatrix}, \qquad \text{if } s^\star \notin \mathbb{R}, \tag{C.5a}$$

$$M = W(s^\star), \qquad \text{if } s^\star \in \mathbb{R}, \tag{C.5b}$$

i.e., knowing M is equivalent to knowing $W(s^\star)$. Since $s^\star \notin \mathrm{spec}(A)$ implies $\mathrm{spec}(S) \cap \mathrm{spec}(A) = \emptyset$, by standard results [248, Lemma 2.7] the Sylvester equation (C.3a) is solvable and has a unique solution.

Remark C.2. *While the relation between* (C.3) *and* (C.5) *has been discussed focusing for simplicity on the computation of one moment of* $W(s)$ *at a single* $s^\star \in \mathbb{C}$, *similar relations hold as well for the simultaneous computation of the moments at several complex values, choosing* $S \in \mathbb{R}^{n_e\times n_e}$ *as a block diagonal matrix* $S = \mathrm{diag}(S_1, S_2, \ldots)$ *and* $L \in \mathbb{R}^{m\times n_e}$ *as a block row matrix* $L = \mathrm{row}(L_1, L_2, \ldots)$, *each having one block for each different* $s_1^\star$, $s_2^\star, \ldots$, *and each pair of blocks* (S_i, L_i) *having the structure of the pair* (S, L) *in* (C.4). ▲

More in general, considering the dynamics

$$\dot{w}(t) = Sw(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{C.6a}$$

$$\dot{x}(t) = Ax(t) + Pw(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{C.6b}$$

$$y(t) = Cx(t) + Qw(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{C.6c}$$

with $S \in \mathbb{R}^{n_e\times n_e}$ and under the assumption that $\mathrm{spec}(S) \cap \mathrm{spec}(A) = \emptyset$, the matrices Π, M solving

$$\Pi S = A\Pi + P, \tag{C.7a}$$

$$M = C\Pi + Q, \tag{C.7b}$$

have the following dynamic interpretation. If the initial condition for (C.6) satisfies $x_0 = \Pi w_0$, then

$$x(t) = \Pi w(t), \qquad y(t) = Mw(t), \qquad \forall t \geq 0, \tag{C.8}$$

motivating the names of *pseudo-steady-state maps* (respectively, in the state and in the output) for Π and M.

Under the stronger assumption that

$$\mathrm{spec}(A) \subset \mathbb{C}_{<0}, \qquad \mathrm{spec}(S) \subset \mathbb{C}_{\geq 0}, \tag{C.9}$$

which ensures the existence of a steady-state response for (C.6b), (C.6c) under the input generated by (C.6a), matrices Π and M will be called *steady-state maps* (respectively, in the state and in the output). Under (C.9), the steady-state response of (C.6) satisfies the relations

$$\tilde{x}(t) = \Pi w(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{C.10a}$$

$$\tilde{y}(t) = M w(t), \qquad t \in \mathbb{R}_{\geq 0}, \tag{C.10b}$$

and, for generic initial states $x_0 \neq \Pi w_0$, of (C.6), the corresponding transient response is given by

$$\hat{x}_0 = x_0 - \Pi w_0, \tag{C.11a}$$

$$x^\circ(t) = e^{At} \hat{x}_0, \qquad t \in \mathbb{R}_{\geq 0}, \tag{C.11b}$$

$$y^\circ(t) = C e^{At} \hat{x}_0, \qquad t \in \mathbb{R}_{\geq 0}. \tag{C.11c}$$

References

[1] AAEON Technology. *UP Bridge the Gap - Edge Computing Devices.* URL: https://up-board.org/ (visited on 11/21/2024).

[2] Abaco Systems. *PCIE-5565RC Interface Card | Abaco Systems.* URL: https://www.abaco.com/products/pcie-5565rc-interface-card (visited on 12/03/2024).

[3] Akbari, S., Galeani, S., and Sassano, M. "Towards Spline-based Dynamic Input Allocation". In: *Proc. of the 7th International Conference on Control, Automation and Diagnosis (ICCAD).* IEEE, May 2023, pp. 1–6.

[4] Albanese, R. and Villone, F. "The linearized CREATE-L plasma response model for the control of current, position and shape in tokamaks". In: *Nuclear Fusion* 38.5 (May 1998), p. 723.

[5] Albanese, R. et al. "Plasma modeling for position and current control in FTU". In: *Fusion Engineering and Design* 66-68 (2003), pp. 681–689.

[6] Albanese, R. et al. "Plasma response models for current, shape and position control in JET". In: *Fusion Engineering and Design* 66-68 (2003), pp. 715–718.

[7] Albanese, R. et al. "Design, implementation and test of the XSC extreme shape controller in JET". In: *Fusion Engineering and Design* 74.1 (Nov. 2005), pp. 627–632. ISSN: 0920-3796.

[8] Ambrosino, R. et al. "Optimization of experimental snowflake configurations on TCV". In: *Nuclear Fusion* 54.12 (Nov. 2014), p. 123008.

[9] Ambrosino, R. et al. "Model-based MIMO isoflux plasma shape control at the EAST tokamak: experimental results". In: *2020 IEEE Conference on Control Technology and Applications (CCTA)* (Aug. 2020), pp. 770–775.

[10] Anaconda Inc. *Conda.* URL: https://docs.conda.io/projects/conda/en/stable/ (visited on 11/06/2024).

[11] Anand, H., Galperti, C., Coda, S., et al. "Distributed digital real-time control system for the TCV tokamak and its applications". In: *Nuclear Fusion* 57.5 (2017).

[12] Anand, H. et al. “A novel plasma position and shape controller for advanced configuration development on the TCV tokamak”. In: *Nuclear Fusion* 57.12 (Sept. 2017), p. 126026.

[13] Apache Software Foundation. *Apache NuttX*. URL: https://nuttx.apache.org/ (visited on 11/26/2024).

[14] Ariola, M. and Pironti, A. *Magnetic control of tokamak plasmas*. Vol. 187. Springer, 2008.

[15] Ariola, M. et al. “A Modern Plasma Controller Tested on the TCV Tokamak”. In: *Fusion Technology* 36.2 (Sept. 1999), pp. 126–138.

[16] Ariola, M. et al. “Design and experimental testing of a robust multivariable controller on a tokamak”. In: *IEEE Transactions on Control Systems Technology* 10.5 (Sept. 2002), pp. 646–653.

[17] Astolfi, A. “Model Reduction by Moment Matching for Linear and Nonlinear Systems”. In: *IEEE Transactions on Automatic Control* 55.10 (Oct. 2010), pp. 2321–2336.

[18] Baldoni, G. et al. “Facilitating distributed data-flow programming with Eclipse Zenoh: the ERDOS case”. In: *Proc. of the 1st Workshop on Serverless mobile networking for 6G Communications*. Association for Computing Machinery, July 2021, pp. 13–18.

[19] Barbalace, A. et al. “Performance Comparison of EPICS IOC and MARTe in a Hard Real-Time Control Application”. In: *IEEE Transactions on Nuclear Science* 58.6 (Dec. 2011), pp. 3162–3166.

[20] Bédard, C. et al. “Message flow analysis with complex causal links for distributed ROS 2 systems”. In: *Robotics and Autonomous Systems* 161 (Mar. 2023), p. 104361.

[21] Bellard, F. *QEMU*. URL: https://www.qemu.org/ (visited on 11/20/2024).

[22] Bianchi, L. et al. “A novel distributed architecture for unmanned aircraft systems based on Robot Operating System 2”. In: *IET Cyber-Systems and Robotics* 5.1 (2023), e12083.

[23] Bianchi, L. et al. “Efficient visual sensor fusion for autonomous agents”. In: *Proc. of the 7th International Conference on Control, Automation and Diagnosis (ICCAD)*. IEEE, May 2023, pp. 01–06.

[24] Biggs, G., Perron, J., and Loretz, S. *Actions*. URL: https://design.ros2.org/articles/actions.html (visited on 11/16/2024).

[25] Bird, C. et al. “The promises and perils of mining git”. In: *2009 6th IEEE International Working Conference on Mining Software Repositories*. May 2009, pp. 1–10.

[26] Bocchino, R. L. et al. “F Prime: An Open-Source Framework for Small-Scale Flight Software Systems”. In: *Proc. of the 2018 Small Satellite Conference*. Aug. 2018.

[27] Bodson, M. “Evaluation of optimization methods for control allocation”. In: *Journal of Guidance, Control, and Dynamics* 25.4 (2002), pp. 703–711.

[28] Boettiger, C. “An introduction to Docker for reproducible research”. In: *SIGOPS Operating Systems Review* 49.1 (Jan. 2015), pp. 71–79.

[29] Boncagni, L. et al. “MARTe at FTU: The new feedback control”. In: *Fusion Engineering and Design* 87.12 (2012), pp. 1917–1920.

[30] Bonci, A. et al. “Robot Operating System 2 (ROS2)-Based Frameworks for Increasing Robot Autonomy: A Survey”. In: *Applied Sciences* 13.23 (Jan. 2023), p. 12796.

[31] Braams, C. M. and Stott, P. E. *Nuclear Fusion: Half a Century of Magnetic Confinement Fusion Research*. CRC Press, June 2002.

[32] Brand, D. and Zafiropulo, P. “On Communicating Finite-State Machines”. In: *Journal of the ACM* 30.2 (1983), pp. 323–342.

[33] Calacci, L., Carnevale, D., and Abruzzese, D. “BlockNet: a new flexible control architecture”. In: *2020 28th Mediterranean Conference on Control and Automation (MED)*. Sept. 2020, pp. 1039–1044.

[34] Canonical. *ubuntu - Official Image | Docker Hub*. URL: https://hub.docker.com/_/ubuntu (visited on 11/21/2024).

[35] Canonical. *Ubuntu Linux*. URL: https://ubuntu.com/ (visited on 11/12/2024).

[36] Carnevale, D., Galeani, S., and Sassano, M. “Data Driven Moment Computation, with Application to Output Tracking with External Models”. In: *2021 29th Mediterranean Conference on Control and Automation (MED)*. June 2021, pp. 610–615.

[37] Carnevale, D. et al. “External Models for Output Regulation based on Moment Estimation from Input-Output Data”. In: *IFAC-PapersOnLine* 50.1 (July 2017), pp. 7777–7782.

[38] Castro, R. De. “Safe and High-Performance Control Allocation”. In: *IEEE Transactions on Automatic Control* 67.6 (June 2022), pp. 3120–3127.

[39] Chen, C.-T. *Linear Systems Theory and Design*. 3rd ed. Oxford University Press, 2009.

[40] Chen, F. F. *Introduction to plasma physics*. Springer Science & Business Media, 2012.

[41] Chen, T. and Francis, B. A. *Optimal Sampled-Data Control Systems*. London: Springer-Verlag, 1995.

[42] Chollet, F. and ONEIROS. *Keras*. URL: https://keras.io/ (visited on 11/21/2024).

[43] Cocetti, M., Serrani, A., and Zaccarian, L. “Dynamic input allocation for uncertain linear over-actuated systems”. In: *Proc. of the 2016 American Control Conference (ACC)*. 2016, pp. 2906–2911.

[44] Cocetti, M., Serrani, A., and Zaccarian, L. “Linear output regulation with dynamic optimization for uncertain linear over-actuated systems”. In: *Automatica* 97 (Nov. 2018), pp. 214–225.

[45] Coda, S. et al. “Physics research on the TCV tokamak facility: from conventional to alternative scenarios and beyond”. In: *Nuclear Fusion* 59.11 (2019), p. 112023.

[46] Coleman, D. T. et al. “Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study”. In: *Journal of Software Engineering for Robotics* 5.1 (May 2014), pp. 3–16.

[47] Coosemans, R. et al. “Application of the Fenix flight simulator to TCV”. In: *50th EPS Conference on Controlled Fusion and Plasma Physics*. 2024.

[48] Cordiner, S. et al. “Dynamic input allocation of torque references for a parallel HEV”. In: *Proc. of the 49th IEEE Conference on Decision and Control (CDC)*. IEEE, 2010, pp. 4920–4925.

[49] Corsaro, A. et al. "Quality of service in publish/subscribe middleware". In: *Global Data Management* 19.20 (2006), pp. 1–22.

[50] Corsaro, A. et al. "Zenoh: Unifying Communication, Storage and Computation from the Cloud to the Microcontroller". In: *2023 26th Euromicro Conference on Digital System Design (DSD)*. Sept. 2023, pp. 422–428.

[51] Cristofaro, A. and Galeani, S. "Output invisible control allocation with steady-state input optimization for weakly redundant plants". In: *Proc. of the 53rd IEEE Conference on Decision and Control (CDC)*. IEEE, 2014, pp. 4246–4253.

[52] Cristofaro, A., Galeani, S., and Serrani, A. "Output invisible control allocation with asymptotic optimization for nonlinear systems in normal form". In: *Proc. of the 56th IEEE Conference on Decision and Control (CDC)*. IEEE, 2017, pp. 2563–2568.

[53] Cruz, N. et al. "Control-oriented tools for the design and validation of the JT-60SA magnetic control system". In: *Control Engineering Practice* 63 (June 2017), pp. 81–90.

[54] Culjak, I. et al. "A brief introduction to OpenCV". In: *2012 Proceedings of the 35th International Convention MIPRO*. May 2012, pp. 1725–1730.

[55] Dalesio, L. R., Kozubal, A. J., and Kraimer, M. R. "EPICS architecture". English. In: *Proc. of the International Conference on Accelerator and Large Experimental Physics Control Systems*. Los Alamos National Lab., NM (United States), 1991.

[56] De Tommasi, G. et al. "Trading output performance for input allocation: application to the JET tokamak shape controller". In: *Proc. of the 48th IEEE Conference on Decision and Control (CDC)*. Dec. 2009.

[57] De Tommasi, G. et al. "Nonlinear dynamic allocator for optimal input/output performance trade-off: application to the JET tokamak shape controller". In: *Automatica* 47.5 (2011), pp. 981–987.

[58] Degrave, J. et al. "Magnetic control of tokamak plasmas through deep reinforcement learning". In: *Nature* 602.7897 (Feb. 2022), pp. 414–419.

[59] Docker Inc. *Docker*. URL: https://www.docker.com/ (visited on 11/14/2024).

[60] Docker Inc. *Docker Compose*. URL: https://github.com/docker/compose (visited on 11/14/2024).

[61] Docker Inc. *Docker Desktop*. URL: https://www.docker.com/products/docker-desktop/ (visited on 11/14/2024).

[62] Docker Inc. *Docker Hub Container Image Library*. URL: https://hub.docker.com (visited on 11/14/2024).

[63] Dronecode Foundation. *The Dronecode Foundation*. URL: https://dronecode.org/ (visited on 11/25/2024).

[64] Dua, R., Raja, A R., and Kakadia, D. "Virtualization vs Containerization to Support PaaS". In: *2014 IEEE International Conference on Cloud Engineering*. Mar. 2014, pp. 610–614.

[65] Dua, R. et al. “Performance analysis of Union and CoW File Systems with Docker”. In: *2016 International Conference on Computing, Analytics and Security Trends (CAST)*. Dec. 2016, pp. 550–555.

[66] Duan, M. and Okwudire, C. E. “Connections between control allocation and linear quadratic control for weakly redundant systems”. In: *Automatica* 101 (2019), pp. 96–102.

[67] Duval, B. P. et al. “Experimental research on the TCV tokamak”. In: *Nuclear Fusion* 64.11 (Oct. 2024). Publisher: IOP Publishing, p. 112023.

[68] Eby, P. et al. *Setuptools*. URL: https://setuptools.pypa.io/en/latest/userguide/ (visited on 11/13/2024).

[69] Eclipse Foundation. *Cyclone DDS*. URL: https://cyclonedds.io/ (visited on 11/20/2024).

[70] EPICS Controls. *EPICS - Experimental Physics and Industrial Control System*. URL: https://epics-controls.org/ (visited on 11/23/2024).

[71] eProsima. *1.1. What is DDS? — Fast DDS 3.1.0 documentation*. URL: https://fast-dds.docs.eprosima.com/en/latest/fastdds/getting_started/definitions.html (visited on 11/13/2024).

[72] eProsima. *Fast DDS*. URL: https://fast-dds.docs.eprosima.com/en/latest/ (visited on 11/20/2024).

[73] eProsima. *Fast-DDS-Gen*. URL: https://github.com/eProsima/Fast-DDS-Gen (visited on 11/26/2024).

[74] eProsima. *Micro XRCE-DDS*. URL: https://micro-xrce-dds.docs.eprosima.com/en/latest/ (visited on 11/13/2024).

[75] eProsima. *micro-ROS*. URL: https://micro.ros.org/ (visited on 11/13/2024).

[76] eProsima. *Vulcanexus, the All-in-One ROS 2 tool set*. URL: https://vulcanexus.org/ (visited on 11/06/2024).

[77] Esposito, B. et al. “Runaway electron generation and control”. In: *Plasma Physics and Controlled Fusion* 59.1 (Nov. 2016), p. 014044.

[78] Espressif Systems. *ESP32 Wi-Fi & Bluetooth SoC*. URL: https://www.espressif.com/en/products/socs/esp32 (visited on 11/13/2024).

[79] Essers, M. S. and Vaneker, T. H. J. “Evaluating a Data Distribution Service System for Dynamic Manufacturing Environments: A Case Study”. In: *Procedia Technology*. 2nd International Conference on System-Integrated Intelligence: Challenges for Product and Production Engineering 15 (Jan. 2014), pp. 621–630.

[80] Ferron, N. et al. “Implementation and Review of the Axisymmetric Equilibrium System of RFX-Mod2 within the MARTe2 Framework”. In: *Electronics* 11.17 (2022), p. 2751.

[81] Fischer, T. et al. “A RoboStack Tutorial: Using the Robot Operating System Alongside the Conda and Jupyter Data Science Ecosystems”. In: *IEEE Robotics & Automation Magazine* 29.2 (June 2022), pp. 65–74.

[82] Forney Jr., G. D. “Minimal bases of rational vector spaces, with applications to multivariable linear systems”. In: *SIAM Journal on Control* 13.3 (1975), pp. 493–520.

[83] Francis, B. A. and Wonham, W. M. “The internal model principle for linear multivariable regulators”. In: *Applied mathematics and optimization* 2.2 (1975), pp. 170–194.

[84] Fredian, T., Stillerman, J., and Manduchi, G. “MDSplus extensions for long pulse experiments”. In: *Fusion Engineering and Design* 83.2 (Apr. 2008), pp. 317–320.

[85] Fredian, T. et al. “MDSplus yesterday, today and tomorrow”. In: *Fusion Engineering and Design* 127 (2018), pp. 106–110.

[86] Free Software Foundation. *GCC, the GNU Compiler Collection*. URL: https://gcc.gnu.org/ (visited on 11/20/2024).

[87] Fusion for Energy. *MARTe2*. URL: https://vcis.f4e.europa.eu/marte2-docs/master/html/ (visited on 11/16/2024).

[88] Galeani, S., Masocco, R., and Sassano, M. “Dynamic control allocation for sampled-data closed-loop systems”. In: *Proc. of the IEEE 61st Conference on Decision and Control (CDC)*. IEEE, Dec. 2022, pp. 4206–4211.

[89] Galeani, S. and Pettinari, S. “On dynamic input allocation for fat plants subject to multi-sinusoidal exogenous inputs”. In: *Proc. of the 53rd IEEE Conference on Decision and Control (CDC)*. IEEE, 2014, pp. 2396–2403.

[90] Galeani, S. et al. “On input allocation-based regulation for linear over-actuated systems”. In: *Automatica* 52 (Feb. 2015), pp. 346–354.

[91] Galeani, S. et al. “An optimal control perspective on the dynamic allocation problem for linear systems*”. In: *IFAC-PapersOnLine*. 22nd IFAC World Congress 56.2 (Jan. 2023), pp. 7492–7497.

[92] Galperti, C. et al. “Overview of the TCV digital real-time plasma control system and its applications”. In: *Fusion Engineering and Design* 208 (Nov. 2024), p. 114640.

[93] Garrido-Jurado, S. et al. “Generation of fiducial marker dictionaries using Mixed Integer Linear Programming”. In: *Pattern Recognition* 51 (Mar. 2016), pp. 481–491.

[94] Gerkey, B. *Why ROS 2?* June 2014. URL: https://design.ros2.org/articles/why_ros2.html (visited on 11/13/2024).

[95] GitHub, Inc. *GitHub*. URL: https://github.com (visited on 11/21/2024).

[96] Goodfellow, I., Bengio, Y., and Courville, A. *Deep learning*. MIT Press, 2016.

[97] Google Brain Team. *TensorFlow*. URL: https://www.tensorflow.org/ (visited on 11/21/2024).

[98] Görner, M. et al. “MoveIt! Task Constructor for Task-Level Motion Planning”. In: *2019 International Conference on Robotics and Automation (ICRA)*. May 2019, pp. 190–196.

[99] Gorno, S. et al. “Power exhaust and core-divertor compatibility of the baffled snowflake divertor in TCV”. In: *Plasma Physics and Controlled Fusion* 65.3 (Jan. 2023), p. 035004.

[100] Grazia, L. E. di et al. “Current Limit Avoidance Algorithms for DEMO Operation”. In: *Journal of Optimization Theory and Applications* 198.3 (Sept. 2023), pp. 958–987. ISSN: 1573-2878.

[101] Greising, S. and Reichert, A. “Zenoh Whitepaper”. FLECS Technologies GmbH, Jan. 2023.

[102] Guibas, L. J. and Sedgewick, R. “A dichromatic framework for balanced trees”. In: *19th Annual Symposium on Foundations of Computer Science (SFCS 1978)*. Oct. 1978, pp. 8–21.

[103] Guruji, A. K., Agarwal, H., and Parsediya, D. K. “Time-efficient A* Algorithm for Robot Path Planning”. In: *Procedia Technology* 23 (Jan. 2016), pp. 144–149.

[104] Härkegård, O. and Glad, S. T. “Resolving actuator redundancy—optimal control vs. control allocation”. In: *Automatica* 41.1 (2005). Publisher: Elsevier, pp. 137–144.

[105] Hofmann, F. “FBT - a free-boundary tokamak equilibrium code for highly elongated and shaped plasmas”. In: *Computer Physics Communications* 48.2 (Feb. 1988), pp. 207–221.

[106] Hofmann, F. et al. “Creation and control of variably shaped plasmas in TCV”. In: *Plasma Physics and Controlled Fusion* 36.12B (Dec. 1994), B277.

[107] Iovino, M. et al. “A survey of Behavior Trees in robotics and AI”. In: *Robotics and Autonomous Systems* 154 (2022), p. 104096.

[108] Isidori, A. *Nonlinear Control Systems, Third Edition*. Communications and Control Engineering. Springer, 1995.

[109] Jacob, B. and Guennebaud, G. *Eigen*. URL: https://eigen.tuxfamily.org/index.php?title=Main_Page (visited on 11/20/2024).

[110] Joffrin, E. et al. “Overview of the EUROfusion Tokamak Exploitation programme in support of ITER and DEMO”. In: *Nuclear Fusion* 64.11 (Aug. 2024), p. 112019.

[111] Johansen, T. A. and Fossen, T. I. “Control allocation—A survey”. In: *Automatica* 49.5 (2013). Publisher: Elsevier, pp. 1087–1103.

[112] Kailath, T. *Linear Systems*. Prentice Hall, 1980.

[113] Kalman, R. E., Falb, P. L., and Arbib, M. A. *Topics in Mathematical Systems Theory*. McGraw-Hill, 1969.

[114] Khalil, H. K. *Nonlinear systems*. Prentice Hall, 2002.

[115] Kirk, D. E. *Optimal control theory: an introduction*. Prentice Hall, 1970.

[116] kitware. *CMake*. URL: https://cmake.org/ (visited on 11/13/2024).

[117] Kolaric, P., Lopez, V. G., and Lewis, F. L. “Optimal dynamic Control Allocation with guaranteed constraints and online Reinforcement Learning”. In: *Automatica* 122 (Dec. 2020), p. 109265.

[118] Korsgaard, P. *Buildroot*. URL: https://buildroot.org/ (visited on 12/24/2024).

[119] Kronauer, T. et al. “Latency Analysis of ROS2 Multi-Node Systems”. In: *2021 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*. Sept. 2021, pp. 1–7.

[120] Laub, A. et al. “Computation of system balancing transformations and other applications of simultaneous diagonalization algorithms”. In: *IEEE Transactions on Automatic Control* 32.2 (Feb. 1987), pp. 115–122.

[121] Le, H. B. et al. “Distributed digital real-time control system for TCV tokamak”. In: *Fusion Engineering and Design* 89.3 (Mar. 2014), pp. 155–164.

[122] Linux Foundation. *ONNX*. URL: https://onnx.ai/ (visited on 11/21/2024).

[123] Lister, J. B. et al. “The control of tokamak configuration variable plasmas”. In: *Fusion Technology* (1997).

[124] Loeliger, J. and McCullough, M. *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development*. "O’Reilly Media, Inc.", Aug. 2012.

[125] Macenski, S. et al. “The Marathon 2: A Navigation System”. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Oct. 2020, pp. 2718–2725.

[126] Macenski, S. et al. “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66 (May 2022), eabm6074.

[127] Macenski, S. et al. “From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2”. In: *Robotics and Autonomous Systems* 168 (Oct. 2023), p. 104493.

[128] Macenski, S. et al. “Impact of ROS 2 Node Composition in Robotic Systems”. In: *IEEE Robotics and Automation Letters* 8.7 (July 2023), pp. 3996–4003.

[129] Macenski, S. et al. “Regulated pure pursuit for robot path tracking”. In: *Autonomous Robots* 47.6 (Aug. 2023), pp. 685–694.

[130] Manduchi, G, Taliercio, C, and Luchetta, A. “The Java interface of MDSplus: towards a unified approach for local and remote data access”. In: *Fusion Engineering and Design* 48.1 (Aug. 2000), pp. 163–170.

[131] Manduchi, G. “Commonalities and differences between MDSplus and HDF5 data systems”. In: *Fusion Engineering and Design* 85.3 (July 2010), pp. 583–590.

[132] Manduchi, G. “New EPICS channel archiver based on MDSplus data system”. In: *IEEE Transactions on Nuclear Science* 58.6 (Oct. 2011), pp. 3158–3161.

[133] Manduchi, G., De Marchi, E., and Mandelli, A. “A new LabVIEW interface for MDSplus”. In: *Fusion Engineering and Design* 88.6 (Oct. 2013), pp. 1196–1199.

[134] Manduchi, G., Fredian, T. W., and Stillerman, J. A. “MDSplus evolution continues”. In: *Fusion Engineering and Design* 87.12 (Dec. 2012), pp. 2095–2099.

[135] Manduchi, G. et al. “Upgrade of the RFX-mod real time control system”. In: *Fusion Engineering and Design* 87.12 (2012), pp. 1907–1911.

[136] Manduchi, G. et al. “The New Feedback Control System of RFX-mod Based on the MARTe Real-Time Framework”. In: *IEEE Transactions on Nuclear Science* 61.3 (June 2014), pp. 1216–1221.

[137] Manduchi, G. et al. “Integration of Simulink, MARTe and MDSplus for rapid development of real-time applications”. In: *Fusion Engineering and Design* 96-97 (Oct. 2015), pp. 645–648.

[138] Manduchi, G. et al. “Fast development of real-time applications using MDSplus and MARTe frameworks”. In: *Fusion Engineering and Design* 112 (Nov. 2016), pp. 942–945.

[139] Manduchi, G. et al. “A portable control and data acquisition solution using EPICS, MARTe and MDSplus”. In: *Fusion Engineering and Design* 127 (Feb. 2018), pp. 50–53.

[140] Marchioni, S. et al. “Vertical instability growth rate studies with rigid and deformable plasma models and proximity controller development in the TCV tokamak”. In: *APS Division of Plasma Physics Meeting Abstracts*. Vol. 2023. 2023, NP11.085.

[141] Maruyama, Y., Kato, S., and Azumi, T. “Exploring the performance of ROS2”. In: *2016 International Conference on Embedded Software (EMSOFT)*. Oct. 2016, pp. 1–10.

[142] Masocco, R. et al. *transitions_ros*. 2022. URL: https://github.com/dotX-Automation/transitions_ros (visited on 11/25/2024).

[143] Masocco, R. *dotxautomation/dua-foundation - Docker Hub*. URL: https://hub.docker.com/r/dotxautomation/dua-foundation (visited on 11/20/2024).

[144] Masocco, R. *dua-foundation*. URL: https://github.com/dotX-Automation/dua-foundation (visited on 11/16/2024).

[145] Masocco, R. *dua-template*. URL: https://github.com/dotX-Automation/dua-template (visited on 11/16/2024).

[146] Masocco, R. *dua-utils*. URL: https://github.com/dotX-Automation/dua-utils (visited on 11/16/2024).

[147] Masocco, R. *dua_app_management*. URL: https://github.com/dotX-Automation/dua_app_management (visited on 11/16/2024).

[148] Masocco, R. *dua_node*. URL: https://github.com/dotX-Automation/dua_node (visited on 11/16/2024).

[149] Masocco, R. *dua_qos*. URL: https://github.com/dotX-Automation/dua_qos (visited on 11/16/2024).

[150] Masocco, R. *flight_stack*. URL: https://github.com/IntelligentSystemsLabUTV/flight_stack (visited on 11/25/2024).

[151] Masocco, R. *simple_actionclient*. URL: https://github.com/dotX-Automation/simple_actionclient (visited on 11/16/2024).

[152] Masocco, R. *simple_serviceclient*. URL: https://github.com/dotX-Automation/simple_serviceclient (visited on 11/16/2024).

[153] Masocco, R. *zed_drivers*. URL: https://github.com/IntelligentSystemsLabUTV/zed_drivers (visited on 11/25/2024).

[154] Masocco, R. and Bianchi, L. *params_manager*. URL: https://github.com/dotX-Automation/params_manager (visited on 11/16/2024).

[155] Masocco, R. et al. “A distributed real-time diagnostic and control network for the TCV tokamak based on the Data Distribution Service”. In: *33rd Symposium on Fusion Technology (SOFT 2024)*. Sept. 2024. DOI: 10.13140/RG.2.2.18403.16166.

[156] Masocco, R. et al. “A dynamic optimization approach to steady-state input allocation for non-linear redundant systems”. In: *Proc. of the 10th International Conference on Control, Decision and Information Technologies (CoDIT)*. IEEE, July 2024, pp. 2833–2838.

[157] MathWorks. *MATLAB*. URL: https://www.mathworks.com/products/matlab.html (visited on 11/13/2024).

[158] MathWorks. *Simulink*. URL: https://www.mathworks.com/products/simulink.html (visited on 11/13/2024).

[159] Meier, L., Honegger, D., and Pollefeys, M. “PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms”. In: *Proc. of the 2015 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2015, pp. 6235–6240.

[160] Meier, L. et al. “PIXHAWK: A system for autonomous flight using onboard computer vision”. In: *2011 IEEE International Conference on Robotics and Automation*. May 2011, pp. 2992–2997.

[161] Meier, L. et al. “PIXHAWK: A micro aerial vehicle design for autonomous flight using onboard computer vision”. In: *Autonomous Robots* 33.1 (Aug. 2012), pp. 21–39.

[162] Mele, A. et al. “Design of a novel plasma shape controller for the TCV tokamak”. In: *2024 10th International Conference on Control, Decision and Information Technologies (CoDIT)*. July 2024, pp. 284–289.

[163] Merkel, D. “Docker: lightweight Linux containers for consistent development and deployment”. In: *Linux Journal* 2014.239 (May 2014), 2:2. URL: https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment (visited on 11/14/2024).

[164] Merle, A. et al. “Full discharge coil trajectory optimisation using a quasi-Newton method with the FBT code from the MEQ suite”. In: *50th EPS Conference on Controlled Fusion and Plasma Physics*. 2024.

[165] Merzlyakov, A. and Macenski, S. “A Comparison of Modern General-Purpose Visual SLAM Approaches”. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Sept. 2021, pp. 9190–9197.

[166] Microsoft. *Visual Studio Code*. URL: https://code.visualstudio.com/ (visited on 11/21/2024).

[167] Microsoft. *Windows Subsystem for Linux*. URL: https://learn.microsoft.com/en-us/windows/wsl/about (visited on 11/19/2024).

[168] Mizusawa, N., Nakazima, K., and Yamaguchi, S. “Performance Evaluation of File Operations on OverlayFS”. In: *2017 Fifth International Symposium on Computing and Networking (CANDAR)*. Nov. 2017, pp. 597–599.

[169] Moore, B. “Principal component analysis in linear systems: Controllability, observability, and model reduction”. In: *IEEE Transactions on Automatic Control* 26.1 (Feb. 1981), pp. 17–32.

[170] Moret, J. -M. et al. “Tokamak equilibrium reconstruction code LIUQE and its real time implementation”. In: *Fusion Engineering and Design* 91 (Feb. 2015), pp. 1–15.

[171] Morin, Y. *crosstool-NG*. URL: https://crosstool-ng.github.io/ (visited on 12/24/2024).

[172] Mur-Artal, R. and Tardós, Juan D. “ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras”. In: *IEEE Transactions on Robotics* 33.5 (Oct. 2017), pp. 1255–1262.

[173] Naderi, M., Sedigh, A., and Johansen, T. “Guaranteed feasible control allocation using model predictive control”. In: *Control Theory and Technology* 17 (2019), pp. 252–264.

[174] National Instruments. *LabVIEW*. URL: https://www.ni.com/en/shop/labview.html (visited on 11/13/2024).

[175] Naur, P. and Randell, B. *Software Engineering: Report of a conference sponsored by the NATO Science Committee, Garmisch, Germany, 7th-11th October 1968*. Tech. rep. NATO Scientific Affairs Division, 1969.

[176] Neto, A. C. et al. “MARTe: A Multiplatform Real-Time Framework”. In: *IEEE Transactions on Nuclear Science* 57.2 (Apr. 2010), pp. 479–486.

[177] Nvidia. *CUDA Deep Neural Network*. URL: https://developer.nvidia.com/cudnn (visited on 11/21/2024).

[178] Nvidia. *CUDA Toolkit*. URL: https://developer.nvidia.com/cuda-toolkit (visited on 11/21/2024).

[179] Nvidia. *JetPack SDK*. URL: https://developer.nvidia.com/embedded/jetpack (visited on 11/21/2024).

[180] Nvidia. *Jetson Modules*. URL: https://developer.nvidia.com/embedded/jetson-modules (visited on 11/21/2024).

[181] Nvidia. *NVIDIA L4T ML*. URL: https://catalog.ngc.nvidia.com/orgs/nvidia/containers/l4t-ml (visited on 11/21/2024).

[182] Nvidia. *Nvidia Video Codec SDK*. URL: https://developer.nvidia.com/video-codec-sdk (visited on 11/26/2024).

[183] Oliva, F. et al. “A Trajectory Based Optimization Approach for Hybrid Observer Design”. In: *Proc. of the 62nd IEEE Conference on Decision and Control (CDC)*. Dec. 2023, pp. 1873–1878.

[184] Open Navigation. *Nav2*. URL: https://docs.nav2.org/ (visited on 11/13/2024).

[185] Open Robotics. *ROS 2 Alternative middleware report*. Tech. rep. Sept. 2023. URL: https://discourse.ros.org/t/ros-2-alternative-middleware-report/33771 (visited on 11/13/2024).

[186] Open Robotics. *Launch — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Launch/Launch-Main.html (visited on 11/13/2024).

[187] Open Robotics. *Recording and playing back data — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html (visited on 11/13/2024).

[188] Open Robotics. *Robot Operating System*. URL: https://www.ros.org/ (visited on 11/16/2024).

[189] Open Robotics. *ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/index.html (visited on 11/13/2024).

[190] Open Robotics. *RViz*. URL: https://github.com/ros2/rviz (visited on 11/13/2024).

[191] Open Robotics. *Tutorials — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials.html (visited on 11/13/2024).

[192] Open Robotics. *Understanding actions — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html (visited on 11/13/2024).

[193] Open Robotics. *Understanding services — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services/Understanding-ROS2-Services.html (visited on 11/13/2024).

[194] Open Robotics. *Understanding topics — ROS 2 Documentation*. URL: https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html (visited on 11/13/2024).

[195] OpenCV team. *OpenCV*. URL: https://opencv.org/ (visited on 11/20/2024).

[196] Oracle. *Java*. URL: https://dev.java/ (visited on 11/20/2024).

[197] Pahl, C. et al. "Cloud Container Technologies: A State-of-the-Art Review". In: *IEEE Transactions on Cloud Computing* 7.3 (July 2019), pp. 677–692.

[198] Pardo-Castellote, G. "OMG Data-Distribution Service: architectural overview". In: *Proc. of the 23rd International Conference on Distributed Computing Systems*. May 2003, pp. 200–206.

[199] Parys, G. V., Merle, A., and Graves, J. P. "Modeling the effect of anisotropy on TCV equilibria using the MEQ suite of codes". In: *50th EPS Conference on Controlled Fusion and Plasma Physics*. 2024.

[200] PickNik Robotics. *MoveIt 2*. URL: https://moveit.picknik.ai/main/index.html (visited on 11/13/2024).

[201] Piras, F. et al. "Snowflake divertor experiments on TCV". In: *Plasma Physics and Controlled Fusion* 52.12 (Nov. 2010), p. 124010.

[202] Pola, G. et al. "Symbolic Models for Networks of Control Systems". In: *IEEE Transactions on Automatic Control* 61.11 (2016), pp. 3663–3668.

[203] Puck, L. et al. "Distributed and Synchronized Setup towards Real-Time Robotic Control using ROS2 on Linux". In: *2020 IEEE 16th International Conference on Automation Science and Engineering (CASE)*. Aug. 2020, pp. 1287–1293.

[204] PyTorch Foundation. *PyTorch*. URL: https://pytorch.org/ (visited on 11/21/2024).

[205] Quigley, M. et al. "ROS: an open-source Robot Operating System". In: *ICRA workshop on open source software*. Vol. 3. May 2009.

[206] Raspberry Pi Ltd. *Raspberry Pi*. URL: https://www.raspberrypi.com/ (visited on 11/21/2024).

[207] Raymond, E. S. *How To Become A Hacker*. 2001. URL: http://www.catb.org/esr/faqs/hacker-howto.html (visited on 11/04/2024).

[208] Reimerdes, H. et al. “Initial TCV operation with a baffled divertor”. In: *Nuclear Fusion* 61.2 (Jan. 2021), p. 024002.

[209] Robotics, Open. *Gazebo*. URL: https://gazebosim.org/home (visited on 11/13/2024).

[210] Robotics, Open. *Installation — ROS 2 Documentation: Rolling documentation*. URL: https://docs.ros.org/en/rolling/Installation.html (visited on 11/12/2024).

[211] Robotics, Open. *Open Robotics*. URL: https://www.openrobotics.org (visited on 11/12/2024).

[212] Rodríguez-Bustos, C. and Aponte, J. “How Distributed Version Control Systems impact open source software projects”. In: *2012 9th IEEE Working Conference on Mining Software Repositories (MSR)*. June 2012, pp. 36–39.

[213] Romero-Ramirez, F. J., Muñoz-Salinas, R., and Medina-Carnicer, R. “Speeded up detection of squared fiducial markers”. In: *Image and Vision Computing* 76 (Aug. 2018), pp. 38–47.

[214] Saxena, S., Farag, H. E. Z., and El-Taweel, N. “A distributed communication framework for smart Grid control applications based on data distribution service”. In: *Electric Power Systems Research* 201 (Dec. 2021), p. 107547.

[215] Scarciotti, G. and Astolfi, A. “Data-driven model reduction by moment matching for linear and nonlinear systems”. In: *Automatica* 79 (May 2017), pp. 340–351.

[216] Schlesselman, J. M., Pardo-Castellote, G., and Farabaugh, B. “OMG data-distribution service (DDS): architectural update”. In: *Proc. of the 2004 IEEE Military Communications Conference (MILCOM)*. Vol. 2. Oct. 2004, pp. 961–967.

[217] Schneider, F. B. “Implementing fault-tolerant services using the state machine approach: a tutorial”. In: *ACM Computing Surveys* 22.4 (1990), pp. 299–319.

[218] Serrani, A. “Output regulation for over-actuated linear systems via inverse model allocation”. In: *Proc. of the 51st IEEE Conference on Decision and Control (CDC)*. IEEE, 2012, pp. 4871–4876.

[219] Sigthorsson, D. et al. “Tracking control for an overactuated hypersonic air-breathing vehicle with steady state constraints”. In: *AIAA guidance, navigation, and control conference and exhibit*. 2006, p. 6558.

[220] Sklyarov, V. “Hierarchical finite-state machines and their use for digital control”. In: *IEEE Transactions on Very Large Scale Integration Systems* 7.2 (1999), pp. 222–228.

[221] Song, C. et al. “Hardware for Deep Learning Acceleration”. In: *Advanced Intelligent Systems* 6.10 (2024), p. 2300762.

[222] Staschulat, J., Lutkebohle, I., and Lange, R. “The rclc Executor: Domain-specific deterministic scheduling mechanisms for ROS applications on microcontrollers: work-in-progress”. In: *Proc. of the 2020 International Conference on Embedded Software (EMSOFT)*. IEEE, Sept. 2020, pp. 18–19.

[223] Stereolabs. *Stereolabs | AI perception for automation*. URL: https://www.stereolabs.com (visited on 11/26/2024).

[224] Stereolabs. *ZED SDK*. URL: https://www.stereolabs.com/developers/release (visited on 11/26/2024).

[225] Stillerman, J. A. et al. "MDSplus data acquisition system". In: *Review of Scientific Instruments* 68.1 (1997), pp. 939–942.

[226] STMicroelectronics. *STM32 Microcontrollers*. URL: https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html (visited on 11/13/2024).

[227] Tenaglia, A. et al. "A robust optimization approach for dynamic input allocation". In: *IFAC-PapersOnLine*. 22nd IFAC World Congress 56.2 (Jan. 2023), pp. 10396–10401.

[228] Tenaglia, A. et al. "A unified approach to plasma shape control with coil current allocation in magnetic confinement fusion". In: *50th EPS Conference on Controlled Fusion and Plasma Physics*. Vol. P2.085. 2024.

[229] Tenaglia, A. et al. "Dynamic steady-state coil current allocation for plasma shape control: a study on the TCV tokamak". In: *Proc. of the 10th International Conference on Control, Decision and Information Technologies (CoDIT)*. IEEE, July 2024, pp. 278–283.

[230] Thomas, D. *Changes betweek ROS 1 and ROS 2*. Sept. 2015. URL: https://design.ros2.org/articles/changes.html (visited on 11/13/2024).

[231] Thomas, D. *The build system "ament_cmake" and the meta build tool "ament_tools"*. July 2015. URL: https://design.ros2.org/articles/ament.html (visited on 11/13/2024).

[232] Thomas, D. *A universal build tool*. Mar. 2017. URL: https://design.ros2.org/articles/build_tool.html (visited on 11/13/2024).

[233] Thomas, D. *Interface definition using .msg / .srv / .action files*. Mar. 2019. URL: https://design.ros2.org/articles/legacy_interface_definition.html (visited on 11/13/2024).

[234] Thomas, D. *colcon - collective construction*. URL: https://colcon.readthedocs.io/en/released/ (visited on 11/13/2024).

[235] Tohidi, S. S., Yildiz, Y., and Kolmanovsky, I. "Adaptive Control Allocation for Over-Actuated Systems with Actuator Saturation *". In: *IFAC-PapersOnLine*. 20th IFAC World Congress 50.1 (July 2017), pp. 5492–5497.

[236] Torvalds, L. et al. *Git*. URL: https://git-scm.com/ (visited on 11/14/2024).

[237] Torvalds, L. et al. *Linux kernel*. URL: https://kernel.org/ (visited on 11/14/2024).

[238] Turnbull, J. *The Docker Book: Containerization Is the New Virtualization*. James Turnbull, July 2014.

[239] Ultralytics Inc. *Ultralytics YOLO*. URL: https://www.ultralytics.com/yolo (visited on 11/21/2024).

[240] Van Rossum, G. *Python*. URL: https://www.python.org/ (visited on 11/21/2024).

[241] Villone, F. et al. “Comparison of the CREATE-L plasma response model with TCV limited discharges”. In: *Nuclear Fusion* 37.10 (1997), p. 1395.

[242] Wagner, F. et al. *Modeling Software with Finite State Machines: A Practical Approach*. New York: Auerbach Publications, 2006.

[243] Wang, Y. H. et al. “First implementation of plasma shape GAP control method in EAST”. In: *Fusion Engineering and Design* 142 (May 2019), pp. 1–5.

[244] Wesson, J. and Campbell, D. J. *Tokamaks*. Vol. 149. Oxford university press, 2011.

[245] Woodall, W. *ROS on DDS*. June 2014. URL: https://design.ros2.org/articles/ros_on_dds.html (visited on 11/13/2024).

[246] Yarkoni, T. and Neumann, A. *transitions*. 2017. URL: https://github.com/pytransitions/transitions (visited on 11/25/2024).

[247] Zaccarian, L. “Dynamic allocation for input redundant control systems”. In: *Automatica* 45.6 (2009). Publisher: Elsevier, pp. 1431–1438.

[248] Zhou, K., Doyle, J. C., and Glover, K. *Robust and optimal control*. Prentice Hall, 1996.

[249] Zolkifli, N. N., Ngah, A., and Deraman, A. “Version Control System: A Review”. In: *Procedia Computer Science* 135 (Jan. 2018), pp. 408–415.

[250] Zúñiga Anaya, J. C. and Henrion, D. “An improved Toeplitz algorithm for polynomial matrix null-space computation”. In: *Applied Mathematics and Computation* 207.1 (Jan. 2009), pp. 256–272.